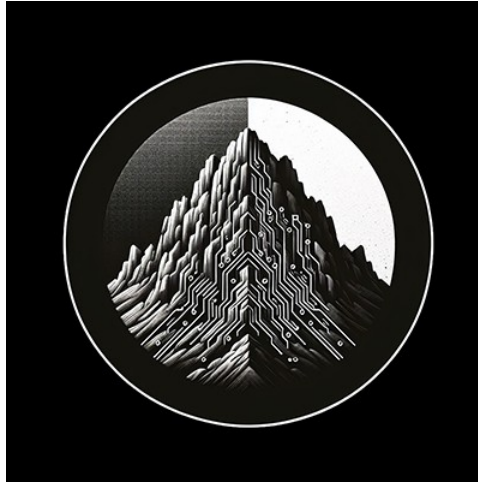




RTL Design Sherpa

RTL Common Library Module Reference — Rev 0.90

July 2026

Table of Contents

1 RTLCommon Library Overview.....	89
1.1 Library Philosophy.....	89
1.1.1 Design Principles.....	89
1.1.2 Quality Standards.....	89
1.2 Architecture Overview.....	90
1.2.1 Module Hierarchy.....	90
1.3 Functional Categories.....	90
1.3.1 1. Arithmetic and Mathematical Operations.....	90
1.3.2 2. Data Integrity and Error Management.....	91
1.3.3 3. Control Logic and Arbitration.....	92
1.3.4 4. Clock and Reset Management.....	92
1.3.5 5. Counting and Sequence Generation.....	92
1.3.6 6. Data Conversion and Encoding.....	93
1.3.7 7. Bit Manipulation and Searching.....	93
1.3.8 8. Shift Operations and LFSRs.....	93
1.3.9 9. Memory and Storage.....	94
1.3.10 10. Utility and Specialized Functions.....	94
1.4 Performance and Optimization.....	95
1.4.1 Synthesis Guidelines.....	95
1.4.2 Technology-Specific Considerations.....	95
1.5 Verification and Validation.....	95
1.5.1 Testbench Coverage.....	95
1.5.2 Compliance Testing.....	96

1.6 Usage Guidelines.....	96
1.6.1 Module Selection.....	96
1.6.2 Integration Best Practices.....	96
1.7 Future Enhancements.....	97
1.7.1 Planned Additions.....	97
1.7.2 Technology Evolution.....	97
2 Priority Encoder Arbiter.....	97
2.1 Overview.....	98
2.2 Module Declaration.....	98
2.3 Parameters.....	98
2.3.1 User-Settable Parameters.....	98
2.3.2 Derived Localparam (Computed Internally).....	98
2.4 Ports.....	99
2.4.1 Inputs.....	99
2.4.2 Outputs.....	99
2.5 Functionality.....	99
2.5.1 Priority Order.....	99
2.5.2 Request Selection.....	99
2.5.3 Combinational Operation.....	99
2.6 Implementation Details.....	100
2.6.1 Optimized Implementations.....	100
2.6.2 Priority Selection Logic.....	100
2.7 Performance Characteristics.....	101
2.7.1 Timing.....	101

2.7.2 Resource Utilization.....	101
2.8 Usage Examples.....	101
2.8.1 Standalone Priority Encoder.....	101
2.8.2 Dual-Input Priority Encoder.....	102
2.8.3 Integration in Round-Robin Arbiter.....	102
2.8.4 Multi-Level Priority Groups.....	102
2.9 Design Considerations.....	103
2.9.1 When to Use.....	103
2.9.2 Performance Optimization.....	103
2.9.3 Synthesis Considerations.....	103
2.10 Priority Arbitration Theory.....	104
2.10.1 Fixed Priority vs Round-Robin.....	104
2.10.2 Priority Inversion.....	104
2.11 Example Priority Scenarios.....	104
2.11.1 Scenario 1: Single Requester (Client 2).....	104
2.11.2 Scenario 2: Multiple Requesters (Clients 0, 2, 5).....	105
2.11.3 Scenario 3: Masked Priority (Clients 3, 6 masked).....	105
2.11.4 Scenario 4: No Requests.....	105
2.12 Verification Notes.....	105
2.13 Common Pitfalls.....	105
2.14 Related Modules.....	106
2.15 Navigation.....	106
3 Round Robin Arbiter.....	106
3.1 Purpose.....	106

3.2 Parameters.....	106
3.3 Ports.....	107
3.3.1 Inputs.....	107
3.3.2 Outputs.....	107
3.4 Key Internal Signals.....	107
3.5 Implementation Details.....	107
3.5.1 Priority Mechanism.....	107
3.5.2 Priority Encoder.....	108
3.5.3 Winner Selection Logic.....	108
3.5.4 Mask Update Logic.....	108
3.6 Special Features.....	108
3.6.1 Block Arbitration.....	108
3.6.2 Grant Acknowledgment Support.....	108
3.6.3 Fair Round-Robin Operation.....	109
3.7 Usage Notes.....	109
3.8 Navigation.....	109
4 arbiter_round_robin_simple.....	109
4.1 Overview.....	109
4.2 Module Declaration.....	109
4.3 Parameters.....	110
4.4 Ports.....	110
4.4.1 Inputs.....	110
4.4.2 Outputs.....	110
4.5 Functionality.....	110

4.5.1 Round-Robin Algorithm.....	110
4.5.2 Key Features.....	111
4.6 Implementation Details.....	111
4.6.1 Rotation Logic.....	111
4.6.2 Priority Selection.....	111
4.6.3 Grant Encoding.....	111
4.7 Timing Characteristics.....	112
4.8 Reset Behavior.....	112
4.9 Usage Examples.....	112
4.9.1 Basic 4-Agent Arbiter.....	112
4.9.2 8-Agent System.....	112
4.10 Arbitration Sequence Example.....	113
4.11 Design Considerations.....	113
4.11.1 Advantages.....	113
4.11.2 Limitations.....	113
4.12 Synthesis Considerations.....	113
4.13 Verification Notes.....	113
4.14 Related Modules.....	114
4.15 Navigation.....	114
5 Weighted Round Robin Arbiter.....	114
5.1 Overview.....	114
5.2 Module Declaration.....	114
5.3 Parameters.....	115
5.3.1 User-Settable Parameters.....	115

5.3.2 Derived Localparam (Computed Internally).....	115
5.4 Ports.....	116
5.4.1 Inputs.....	116
5.4.2 Outputs.....	116
5.5 Functionality.....	117
5.5.1 Credit-Based Weighting.....	117
5.5.2 Arbitration Stages.....	117
5.5.3 Weight Ratio Example.....	117
5.6 Dynamic Weight Changes.....	117
5.6.1 Weight Management FSM.....	117
5.6.2 Weight Update Sequence.....	118
5.6.3 Safety Features.....	118
5.7 Implementation Details.....	118
5.7.1 Credit Management.....	118
5.7.2 Global Replenishment Logic.....	119
5.7.3 Request Masking for Fairness.....	119
5.7.4 Sub-Module: arbiter_round_robin.....	119
5.8 Special Features.....	120
5.8.1 Zero-Weight Client Handling.....	120
5.8.2 ACK Protocol Support.....	120
5.8.3 Starvation Prevention.....	120
5.9 Usage Examples.....	120
5.9.1 Basic QoS Arbiter (No-ACK Mode).....	120
5.9.2 Dynamic Weight Adjustment.....	121

5.9.3 ACK Mode for Pipelined Masters.....	121
5.10 Timing Characteristics.....	122
5.10.1 Latency and Throughput.....	122
5.10.2 Critical Paths.....	122
5.11 Design Considerations.....	122
5.11.1 When to Use Each Mode.....	122
5.11.2 Weight Selection Guidelines.....	122
5.11.3 Dynamic Weight Change Best Practices.....	122
5.12 Performance Characteristics.....	123
5.12.1 Resource Utilization.....	123
5.12.2 Maximum Frequency.....	123
5.13 Verification Notes.....	123
5.14 Related Modules.....	123
5.15 Navigation.....	124
6 Single-Client Arbiter.....	124
6.1 Overview.....	124
6.1.1 Key Features.....	124
6.2 Module Purpose.....	125
6.3 Parameters.....	125
6.4 Port Groups.....	125
6.4.1 Clock and Reset.....	125
6.4.2 Control.....	126
6.4.3 Grant Outputs.....	126
6.5 Functional Description.....	126

6.5.1 Request Qualification.....	126
6.5.2 Grant-Eligibility Logic (ACK mode).....	126
6.5.3 Grant Lifecycle (ACK mode).....	127
6.5.4 No-ACK Fallback Mode.....	127
6.5.5 Output Mapping.....	128
6.6 Usage Example.....	128
6.7 Design Notes.....	128
6.8 Related Modules.....	129
6.8.1 Used By.....	129
6.8.2 Uses.....	129
6.8.3 See Also.....	129
6.9 References.....	129
6.9.1 Source Code.....	129
6.9.2 Documentation.....	129
6.10 Navigation.....	129
7 Binary to Gray Code Converter.....	129
7.1 Overview.....	129
7.2 Module Declaration.....	130
7.3 Parameters.....	130
7.3.1 WIDTH.....	130
7.4 Ports.....	130
7.4.1 Inputs.....	130
7.4.2 Outputs.....	130
7.5 Gray Code Theory.....	130

7.5.1 Mathematical Definition.....	130
7.5.2 Why Gray Code?.....	131
7.5.3 Gray Code Sequence Examples.....	131
7.6 Implementation Analysis.....	132
7.6.1 Core Logic.....	132
7.6.2 Logic Structure.....	132
7.6.3 Gate-Level Implementation.....	132
7.6.4 Timing Characteristics.....	133
7.7 Design Examples and Applications.....	133
7.7.1 1. Asynchronous FIFO Pointer.....	133
7.7.2 2. Clock Domain Crossing Counter.....	134
7.7.3 3. Rotary Encoder Interface.....	135
7.7.4 4. Address Scrambling for Memory Testing.....	136
7.7.5 5. Multi-Bit Synchronizer with Gray Code.....	137
7.8 Companion Gray-to-Binary Converter.....	138
7.8.1 Implementation.....	138
7.8.2 Gray-to-Binary Truth Table (4-bit).....	138
7.9 Advanced Implementations.....	139
7.9.1 1. Parameterized Converter with Validation.....	139
7.9.2 2. Pipelined Converter for High Speed.....	140
7.9.3 3. Bidirectional Converter.....	141
7.10 Verification and Testing.....	142
7.10.1 Comprehensive Test Bench.....	142
7.10.2 Property-Based Verification.....	143

7.10.3 Coverage Model.....	144
7.11 Synthesis Optimization.....	145
7.11.1 Resource Utilization.....	145
7.11.2 Timing Optimization.....	145
7.11.3 Power Optimization.....	146
7.12 Common Applications Summary.....	147
7.12.1 1. Asynchronous FIFOs: Prevent metastability in pointer comparisons.....	147
7.12.2 2. Clock Domain Crossing: Safe multi-bit signal transfer.....	147
7.12.3 3. Position Encoders: Mechanical/optical encoder interfaces.....	147
7.12.4 4. Memory Address Generation: Reduce EMI and power spikes.....	147
7.12.5 5. Test Pattern Generation: Controlled single-bit transitions.....	147
7.12.6 6. Error Detection: Single-bit error detection schemes.....	147
7.12.7 7. ADC/DAC Interfaces: Reduce glitches in conversion systems.....	147
7.13 Design Guidelines.....	147
7.13.1 1. Always Use for Async Boundaries: Gray code is essential for safe asynchronous transfers.....	147
7.13.2 2. Consider Timing: Purely combinational - no setup/hold concerns.....	147
7.13.3 3. Verify Single-Bit Property: Always verify adjacent values differ by one bit	147
7.13.4 4. Plan for Back-Conversion: Usually need Gray-to-Binary converter too.	147
7.13.5 5. Check Width Requirements: Ensure sufficient bits for application range	147
7.14 Navigation.....	147
8 Binary to BCD Converter - Comprehensive Documentation.....	148
8.1 Overview.....	148

8.2 Module Declaration.....	148
8.3 Parameters.....	148
8.3.1 WIDTH.....	148
8.3.2 DIGITS.....	148
8.3.3 Parameter Relationship Table.....	149
8.4 Ports.....	149
8.4.1 Inputs.....	149
8.4.2 Outputs.....	149
8.5 Double-Dabble Algorithm Theory.....	150
8.5.1 Algorithm Principle.....	150
8.5.2 Why Add 3?.....	150
8.5.3 Mathematical Proof.....	150
8.6 FSM Implementation.....	150
8.6.1 State Definitions.....	150
8.6.2 State Transition Diagram.....	150
8.6.3 Detailed State Descriptions.....	151
8.7 Implementation Details.....	152
8.7.1 Register Definitions.....	152
8.7.2 Core Processing Logic.....	153
8.8 Timing Analysis.....	155
8.8.1 Conversion Latency.....	155
8.8.2 Latency Breakdown.....	155
8.8.3 Latency Examples.....	155
8.8.4 Pipeline Considerations.....	155

8.9 BCD Format and Encoding.....	155
8.9.1 BCD Digit Encoding.....	155
8.9.2 Packed BCD Format.....	156
8.9.3 BCD Digit Extraction.....	156
8.10 Worked Examples.....	157
8.10.1 Example 1: Converting 8-bit Binary 156 (WIDTH=8, DIGITS=3).....	157
8.10.2 Example 2: Converting 4-bit Binary 9 (WIDTH=4, DIGITS=2).....	157
8.11 Design Examples and Applications.....	158
8.11.1 1. Seven-Segment Display Driver.....	158
8.11.2 2. LCD/UART Decimal Display.....	160
8.11.3 3. Multi-Channel BCD Converter.....	161
8.12 Optimization Techniques.....	163
8.12.1 1. Early Termination Optimization.....	163
8.12.2 2. Parallel Digit Processing.....	164
8.12.3 3. Pipelined Implementation.....	164
8.13 Verification and Testing.....	165
8.13.1 Comprehensive Test Strategy.....	165
8.13.2 Coverage Model.....	167
8.13.3 Formal Verification Properties.....	168
8.14 Performance and Resource Analysis.....	169
8.14.1 Synthesis Results (Typical FPGA).....	169
8.14.2 Power Consumption Analysis.....	169
8.15 Common Design Patterns.....	170
8.15.1 Pattern 1: Auto-Converting Display Interface.....	170

8.15.2 Pattern 2: Buffered Converter with FIFO.....	170
8.16 Troubleshooting Guide.....	172
8.16.1 Common Issues and Solutions.....	172
8.17 Navigation.....	173
9 CAM Tag Module.....	173
9.1 Purpose.....	173
9.2 Parameters.....	173
9.3 Ports.....	173
9.3.1 Inputs.....	173
9.3.2 Outputs.....	174
9.4 Key Internal Structures.....	174
9.4.1 Storage Arrays.....	174
9.4.2 Search Logic.....	174
9.5 Operations.....	175
9.5.1 Tag Insertion.....	175
9.5.2 Tag Invalidation.....	175
9.5.3 Tag Lookup.....	175
9.6 State Management.....	175
9.6.1 Update Logic.....	175
9.6.2 Status Signals.....	175
9.7 Special Features.....	176
9.7.1 Enable Control.....	176
9.7.2 Allocation Strategy.....	176
9.7.3 Debug Support.....	176

9.8 Usage Considerations.....	176
9.9 Navigation.....	176
10 Clock Divider Module - Comprehensive Documentation.....	177
10.1 Overview.....	177
10.2 Module Declaration.....	177
10.3 Parameters.....	177
10.3.1 N.....	177
10.3.2 PO_WIDTH.....	178
10.3.3 COUNTER_WIDTH.....	178
10.3.4 Parameter Relationships.....	178
10.4 Ports.....	178
10.4.1 Inputs.....	178
10.4.2 Outputs.....	179
10.5 Architecture Details.....	179
10.5.1 Internal Signals.....	179
10.5.2 Counter Operation.....	179
10.5.3 Pickoff Point Selection.....	179
10.5.4 Out-of-Range Handling.....	179
10.5.5 Output Registration.....	180
10.6 Operation Principles.....	180
10.6.1 Division Ratio Table (100MHz Input Clock).....	180
10.6.2 Duty Cycle.....	180
10.7 Timing Characteristics.....	180
10.7.1 Latency.....	180

10.7.2 Phase Relationships.....	181
10.7.3 Jitter.....	181
10.8 Design Examples.....	181
10.8.1 Example 1: Four Debug Clocks.....	181
10.8.2 Example 2: Baud Rate Generator (Approximate).....	181
10.8.3 Example 3: Runtime-Programmable Divider.....	182
10.8.4 Example 4: Test Clock Generation.....	182
10.9 Advanced Usage.....	183
10.9.1 Changing Pickoff Points at Runtime.....	183
10.10 Synthesis Considerations.....	183
10.10.1 Resource Utilization.....	183
10.10.2 Timing Optimization.....	183
10.10.3 Power Optimization.....	183
10.11 Important Warnings.....	184
10.11.1 ⚠ NOT for Functional Clocks.....	184
10.11.2 ⚠ Derived Clock Hazards.....	184
10.11.3 ⚠ Only Power-of-2 Divisions.....	184
10.11.4 ⚠ Glitches During Configuration Changes.....	184
10.12 Verification Strategy.....	184
10.12.1 Test Scenarios.....	184
10.12.2 Coverage Points.....	185
10.13 Performance Characteristics.....	185
10.13.1 Maximum Frequency.....	185
10.13.2 Resource Scaling.....	185

10.14 Related Modules.....	186
10.15 Navigation.....	186
11 Clock Gate Controller - Comprehensive Documentation.....	186
11.1 Overview.....	186
11.2 Module Declaration.....	186
11.3 Parameters.....	187
11.3.1 IDLE_CNTR_WIDTH.....	187
11.3.2 N.....	187
11.4 Ports.....	187
11.4.1 Inputs.....	187
11.4.2 Outputs.....	187
11.5 Clock Gating Theory.....	188
11.5.1 Why Clock Gating?.....	188
11.5.2 Clock Gating Safety.....	188
11.6 Architecture and Implementation.....	188
11.6.1 Idle Counter Logic.....	188
11.6.2 Gating Control Logic.....	188
11.6.3 State Machine Behavior.....	189
11.6.4 Timing Diagram.....	189
11.7 ICG (Integrated Clock Gate) Cell.....	189
11.7.1 Standard ICG Interface.....	189
11.7.2 ICG Implementation (Conceptual).....	189
11.8 Design Examples and Applications.....	190
11.8.1 1. CPU Core Clock Gating.....	190

11.8.2 2. Peripheral Interface Clock Management.....	191
11.8.3 3. Memory Subsystem Clock Gating.....	193
11.8.4 4. Adaptive Clock Gating with Activity Monitoring.....	195
11.8.5 5. Hierarchical Clock Gating System.....	196
11.9 Advanced Features.....	199
11.9.1 1. Glitch-Free Clock Switching.....	199
11.9.2 2. Multi-Level Clock Gating.....	200
11.10 Verification and Testing.....	201
11.10.1 Comprehensive Test Bench.....	201
11.10.2 Coverage Model.....	204
11.10.3 Formal Verification Properties.....	204
11.11 Power Analysis and Optimization.....	205
11.11.1 Power Savings Calculation.....	205
11.11.2 Technology-Specific ICG Implementation.....	206
11.12 Synthesis Considerations.....	208
11.12.1 Resource Utilization.....	208
11.12.2 Timing Constraints.....	208
11.12.3 Power Intent (UPF).....	208
11.13 Design Guidelines and Best Practices.....	209
11.13.1 1. Timeout Selection.....	209
11.13.2 2. Activity Signal Design.....	209
11.13.3 3. Hierarchical Gating Strategy.....	209
11.13.4 4. Verification Checklist.....	209
11.14 Common Applications Summary.....	210

11.15 Troubleshooting Guide.....	210
11.15.1 Common Issues.....	210
11.15.2 Debug Techniques.....	210
11.16 Navigation.....	211
12 Clock Pulse Generator - Comprehensive Documentation.....	211
12.1 Overview.....	211
12.2 Module Declaration.....	211
12.3 Parameters.....	211
12.3.1 WIDTH.....	211
12.4 Ports.....	212
12.4.1 Inputs.....	212
12.4.2 Outputs.....	212
12.5 Architecture and Implementation.....	212
12.5.1 Internal Counter.....	212
12.5.2 Core Logic.....	212
12.5.3 Operation Principles.....	212
12.5.4 Timing Characteristics.....	213
12.6 Timing Diagrams.....	213
12.6.1 Basic Operation (WIDTH=4).....	213
12.6.2 Reset Behavior.....	213
12.6.3 Different WIDTH Values.....	213
12.7 Design Examples and Applications.....	214
12.7.1 1. Heartbeat and Status Indicators.....	214
12.7.2 2. Sampling and Data Acquisition.....	215

12.7.3 3. Communication Protocol Timing.....	216
12.7.4 4. Memory Refresh Controller.....	217
12.7.5 5. Multi-Rate Pulse Generation.....	218
12.7.6 6. Test Pattern Generation.....	220
12.8 Advanced Features and Variations.....	221
12.8.1 1. Enable-Controlled Pulse Generator.....	221
12.8.2 2. Variable Width Pulse Generator.....	222
12.8.3 3. Multi-Phase Pulse Generator.....	223
12.8.4 4. Burst Pulse Generator.....	224
12.9 Verification and Testing.....	226
12.9.1 Comprehensive Test Bench.....	226
12.9.2 Coverage Model.....	228
12.9.3 Formal Properties.....	229
12.10 Synthesis Considerations.....	230
12.10.1 Resource Utilization.....	230
12.10.2 Timing Optimization.....	230
12.11 Common Applications Summary.....	231
12.12 Design Guidelines.....	231
12.13 Navigation.....	232
13 Binary-Gray Counter Module.....	232
13.1 Overview.....	232
13.2 Module Declaration.....	232
13.3 Parameters.....	232
13.3.1 WIDTH.....	232

13.4 Ports.....	233
13.4.1 Inputs.....	233
13.4.2 Outputs.....	233
13.5 Architecture Details.....	233
13.5.1 Internal Logic.....	233
13.5.2 Gray Code Conversion.....	233
13.6 Gray Code Theory.....	234
13.6.1 Why Gray Code?.....	234
13.6.2 Gray Code Sequence Example (4-bit).....	234
13.7 Implementation Analysis.....	234
13.7.1 Combinational Logic.....	234
13.7.2 Sequential Logic.....	235
13.8 Timing Characteristics.....	235
13.8.1 Critical Paths.....	235
13.8.2 Propagation Delays.....	235
13.9 Application: Asynchronous FIFO.....	235
13.9.1 FIFO Pointer Implementation.....	235
13.9.2 Cross-Domain Synchronization.....	236
13.9.3 FIFO Status Generation.....	236
13.10 Advanced Features.....	237
13.10.1 Almost Full/Empty Flags.....	237
13.10.2 Gray-to-Binary Conversion Function.....	237
13.11 Design Examples.....	237
13.11.1 1. 8-bit Asynchronous FIFO Pointer.....	237

13.11.2 2. Clock Domain Crossing Counter.....	237
13.12 Verification Strategy.....	238
13.12.1 Test Scenarios.....	238
13.12.2 Coverage Points.....	238
13.12.3 Assertions.....	239
13.13 Synthesis Considerations.....	239
13.13.1 Resource Utilization.....	239
13.13.2 Optimization Techniques.....	239
13.13.3 Tool-Specific Attributes.....	239
13.14 Performance Characteristics.....	239
13.14.1 Maximum Frequency.....	239
13.14.2 Power Consumption.....	240
13.15 Common Applications.....	240
13.16 Troubleshooting Guide.....	240
13.16.1 Common Issues.....	240
13.16.2 Debug Techniques.....	240
13.17 WaveDrom Visualization.....	240
13.17.1 Scenario 1: Binary vs Gray Code Comparison.....	241
13.17.2 Scenario 2: Single-Bit Transitions (CDC Safety) ★ KEY FEATURE.....	241
13.17.3 Scenario 3: Lookahead Signal (counter_bin_next).....	241
13.17.4 Scenario 4: Enable and Reset Control.....	242
13.18 Test and Verification.....	243
13.19 Navigation.....	243
14 Binary Counter with Load and Variable Increment.....	243

14.1 Overview.....	243
14.2 Module Declaration.....	244
14.3 Parameters.....	244
14.3.1 User-Settable Parameters.....	244
14.3.2 Derived Localparam (Computed Internally).....	245
14.4 Ports.....	245
14.4.1 Inputs.....	245
14.4.2 Outputs.....	245
14.5 Functionality.....	246
14.5.1 Operation Priority.....	246
14.5.2 FIFO-Optimized Wraparound.....	246
14.5.3 Full/Empty Detection Pattern.....	247
14.6 Implementation Details.....	247
14.6.1 Priority Logic.....	247
14.6.2 Registered Output.....	248
14.6.3 Combinational Next-Value Preview.....	248
14.7 Usage Examples.....	248
14.7.1 FIFO Read Pointer with Drop Support.....	248
14.7.2 FIFO Write Pointer.....	249
14.7.3 Full/Empty Status Logic.....	249
14.7.4 Packet Drop Controller.....	249
14.8 Timing Characteristics.....	250
14.8.1 Latency and Throughput.....	250
14.8.2 Operation Timing Diagrams.....	251

14.9 Design Considerations.....	251
14.9.1 When to Use counter_bin_load.....	251
14.9.2 When to Use Alternatives.....	251
14.9.3 Parameter Selection Guidelines.....	252
14.9.4 Critical Paths.....	252
14.10 Performance Characteristics.....	252
14.10.1 Resource Utilization.....	252
14.10.2 Maximum Frequency.....	253
14.11 Verification Notes.....	253
14.12 Common Pitfalls.....	253
14.13 Related Modules.....	254
14.14 Navigation.....	254
15 Binary Counter for FIFOs.....	254
15.1 Overview.....	254
15.2 Module Declaration.....	255
15.3 Parameters.....	255
15.3.1 WIDTH.....	255
15.3.2 MAX.....	255
15.4 Ports.....	255
15.4.1 Inputs.....	255
15.4.2 Outputs.....	256
15.5 Architecture Details.....	256
15.5.1 Internal Signals.....	256
15.5.2 Special Wrap Logic.....	256

15.6 Operation Principles.....	256
15.6.1 Normal Counting.....	256
15.6.2 Wrap Condition.....	257
15.6.3 Enable Control.....	257
15.7 Counting Sequence Example.....	257
15.7.1 Configuration: WIDTH=4, MAX=6.....	257
15.7.2 Pattern Analysis.....	257
15.8 FIFO Application Context.....	257
15.8.1 Address Generation.....	257
15.8.2 FIFO Pointer Comparison.....	258
15.9 Timing Characteristics.....	258
15.9.1 Combinational Paths.....	258
15.9.2 Sequential Behavior.....	258
15.10 Design Examples.....	258
15.10.1 1. Small FIFO Buffer (8 entries).....	258
15.10.2 2. Large FIFO Buffer (1024 entries).....	258
15.10.3 3. Asymmetric FIFO.....	259
15.11 Advanced Usage.....	259
15.11.1 Address Extraction.....	259
15.11.2 Gray Code Conversion.....	259
15.12 Synthesis Considerations.....	259
15.12.1 Resource Utilization.....	259
15.12.2 Timing Optimization.....	259
15.12.3 Parameter Validation.....	260

15.13 Verification Strategy.....	260
15.13.1 Test Scenarios.....	260
15.13.2 Coverage Points.....	260
15.14 Common Issues and Solutions.....	260
15.14.1 Issue 1: Incorrect FIFO Size.....	260
15.14.2 Issue 2: Width Mismatch.....	260
15.14.3 Issue 3: Timing Violations.....	261
15.15 Performance Characteristics.....	261
15.15.1 Maximum Frequency.....	261
15.15.2 Power Consumption.....	261
15.16 Related Modules.....	261
15.17 Navigation.....	261
16 Frequency Invariant Counter Module.....	261
16.1 Overview.....	261
16.2 Module Declaration.....	262
16.3 Parameters.....	262
16.3.1 COUNTER_WIDTH.....	262
16.3.2 PRESCALER_MAX.....	262
16.4 Ports.....	263
16.4.1 Inputs.....	263
16.4.2 Outputs.....	263
16.5 Frequency Selection Table.....	263
16.6 Architecture Overview.....	264
16.6.1 Block Diagram.....	264

16.6.2 Internal Signals.....	265
16.7 Implementation Details.....	265
16.7.1 Frequency Configuration Logic.....	265
16.7.2 Change Detection Logic.....	265
16.7.3 Prescaler Implementation.....	266
16.7.4 Main Counter and Tick Generation.....	266
16.8 Timing Analysis.....	267
16.8.1 Prescaler Operation.....	267
16.8.2 Main Counter Operation.....	267
16.8.3 Overall Timing.....	267
16.9 Design Examples.....	267
16.9.1 1. 1MHz Tick Generator (Any Input Clock).....	267
16.9.2 2. Configurable Baud Rate Generator.....	267
16.9.3 3. Multi-Rate Timer System.....	268
16.10 Advanced Features.....	268
16.10.1 Dynamic Frequency Switching.....	268
16.10.2 Synchronous Reset Applications.....	269
16.11 Verification Strategy.....	269
16.11.1 Test Scenarios.....	269
16.11.2 Testbench Example.....	269
16.11.3 Coverage Points.....	270
16.12 Performance Characteristics.....	271
16.12.1 Resource Utilization.....	271
16.12.2 Timing Performance.....	271

16.12.3 Power Consumption.....	271
16.13 Applications.....	271
16.13.1 Communication Systems.....	271
16.13.2 Display Controllers.....	271
16.13.3 ADC/DAC Timing.....	271
16.13.4 Power Management.....	272
16.14 Design Guidelines.....	272
16.14.1 Parameter Selection.....	272
16.14.2 Reset Strategy.....	272
16.14.3 Frequency Planning.....	272
16.15 Troubleshooting.....	272
16.15.1 Common Issues.....	272
16.15.2 Debug Techniques.....	273
16.16 Navigation.....	273
17 Johnson Counter Module.....	273
17.1 Overview.....	273
17.2 Module Declaration.....	273
17.3 Parameters.....	273
17.3.1 WIDTH.....	273
17.4 Ports.....	274
17.4.1 Inputs.....	274
17.4.2 Outputs.....	274
17.5 Theory of Operation.....	274
17.5.1 Johnson Counter Principle.....	274

17.5.2 Mathematical Representation.....	274
17.6 Implementation Details.....	274
17.6.1 Core Logic.....	274
17.6.2 Operation Breakdown.....	275
17.7 State Sequences.....	275
17.7.1 4-bit Johnson Counter (WIDTH=4).....	275
17.7.2 3-bit Johnson Counter (WIDTH=3).....	275
17.7.3 2-bit Johnson Counter (WIDTH=2).....	276
17.8 Unique Properties.....	276
17.8.1 Self-Starting.....	276
17.8.2 Symmetrical Patterns.....	276
17.8.3 Gray Code Properties.....	276
17.9 Design Examples.....	276
17.9.1 1. 4-Phase Clock Generator.....	276
17.9.2 2. LED Chaser Effect.....	277
17.9.3 3. State Machine Controller.....	277
17.9.4 4. Quadrature Signal Generation.....	277
17.10 Advantages.....	278
17.10.1 1. Simplicity.....	278
17.10.2 2. Reliability.....	278
17.10.3 3. Performance.....	278
17.10.4 4. Flexibility.....	278
17.11 Disadvantages.....	278
17.11.1 1. Limited States.....	278

17.11.2 2. Decode Complexity.....	279
17.12 Applications.....	279
17.12.1 Clock Generation.....	279
17.12.2 Sequential Control.....	279
17.12.3 Pattern Generation.....	279
17.13 Verification Strategy.....	280
17.13.1 Functional Tests.....	280
17.13.2 Coverage Points.....	280
17.13.3 Assertions.....	280
17.14 Synthesis Considerations.....	281
17.14.1 Resource Utilization.....	281
17.14.2 Optimization.....	281
17.14.3 Tool Attributes.....	281
17.15 Performance Characteristics.....	281
17.15.1 Maximum Frequency.....	281
17.15.2 Power Consumption.....	281
17.16 WaveDrom Visualization.....	282
17.16.1 Scenario 1: Walking Ones and Walking Zeros Pattern.....	282
17.16.2 Scenario 2: Single-Bit Transitions (CDC Safety) ★ KEY FEATURE.....	282
17.16.3 Scenario 3: Enable Control.....	282
17.16.4 Scenario 4: Reset Behavior.....	283
17.17 Related Modules.....	284
17.18 Test and Verification.....	284
17.19 Navigation.....	284

18 Counter Load Clear Module.....	284
18.1 Overview.....	284
18.2 Module Declaration.....	284
18.3 Parameters.....	285
18.3.1 MAX.....	285
18.4 Ports.....	285
18.4.1 Inputs.....	285
18.4.2 Outputs.....	285
18.5 Architecture Details.....	286
18.5.1 Internal Registers.....	286
18.5.2 Priority Logic.....	286
18.6 Implementation Details.....	286
18.6.1 Main Control Logic.....	286
18.6.2 Done Signal Generation.....	287
18.7 Operation Modes.....	287
18.7.1 1. Basic Counting Mode.....	287
18.7.2 2. Configurable Delay Mode.....	287
18.7.3 3. Retriggerable Timer Mode.....	288
18.8 Timing Diagrams.....	288
18.8.1 Basic Operation.....	288
18.8.2 Load During Count.....	288
18.9 Advanced Usage Examples.....	288
18.9.1 1. Dual-Rate Timer.....	288
18.9.2 2. PWM Generator.....	289

18.9.3 3. Burst Generator.....	289
18.9.4 4. Communication Timeout.....	290
18.10 Design Patterns.....	290
18.10.1 Pattern 1: Auto-Reloading Timer.....	290
18.10.2 Pattern 2: Conditional Counting.....	290
18.10.3 Pattern 3: Dynamic Period Adjustment.....	291
18.11 Verification Strategy.....	291
18.11.1 Test Scenarios.....	291
18.11.2 Coverage Model.....	291
18.11.3 Assertions.....	292
18.12 Synthesis Considerations.....	293
18.12.1 Resource Usage.....	293
18.12.2 Timing Optimization.....	293
18.12.3 Parameter Validation.....	293
18.13 Performance Characteristics.....	293
18.13.1 Maximum Frequency.....	293
18.13.2 Latency.....	293
18.14 Common Applications.....	293
18.15 Related Modules.....	294
18.16 Navigation.....	294
19 Basic Counter Module.....	294
19.1 Overview.....	294
19.2 Module Declaration.....	294
19.3 Parameters.....	294

19.3.1 MAX.....	294
19.4 Ports.....	295
19.4.1 Inputs.....	295
19.4.2 Outputs.....	295
19.5 Internal Architecture.....	295
19.5.1 Counter Register.....	295
19.5.2 State Machine Behavior.....	295
19.6 Implementation Details.....	296
19.6.1 Main Counter Logic.....	296
19.6.2 Tick Generation.....	296
19.6.3 Key Features.....	296
19.7 Timing Characteristics.....	296
19.7.1 Period Calculation.....	296
19.7.2 Examples.....	296
19.8 Usage Examples.....	297
19.8.1 1. Basic Timer (1ms tick at 100MHz).....	297
19.8.2 2. Heartbeat Generator (1Hz at 50MHz).....	297
19.8.3 3. Simple Divider (Divide by 8).....	297
19.9 Design Considerations.....	297
19.9.1 Resource Usage.....	297
19.9.2 Performance.....	297
19.9.3 Power Optimization.....	298
19.10 Synthesis Considerations.....	298
19.10.1 Parameter Constraints.....	298

19.10.2 Optimization Hints.....	298
19.11 Common Applications.....	298
19.12 Verification Considerations.....	298
19.12.1 Test Scenarios.....	298
19.12.2 Coverage Points.....	299
19.13 Limitations.....	299
19.14 Related Modules.....	299
19.15 Navigation.....	299
20 Ring Counter Module.....	299
20.1 Overview.....	299
20.2 Module Declaration.....	299
20.3 Parameters.....	300
20.3.1 WIDTH.....	300
20.4 Ports.....	300
20.4.1 Inputs.....	300
20.4.2 Outputs.....	300
20.5 Theory of Operation.....	301
20.5.1 Ring Counter Principle.....	301
20.5.2 Mathematical Representation.....	301
20.6 Implementation Details.....	301
20.6.1 Core Logic.....	301
20.6.2 Operation Breakdown.....	301
20.7 State Sequences.....	301
20.7.1 4-bit Ring Counter (WIDTH=4).....	301

20.7.2 3-bit Ring Counter (WIDTH=3).....	302
20.7.3 8-bit Ring Counter Example.....	302
20.8 Design Examples.....	302
20.8.1 1. Sequential Enable Generator.....	302
20.8.2 2. LED Chaser Display.....	303
20.8.3 3. Memory Bank Sequencer.....	303
20.8.4 4. State Machine Controller.....	304
20.8.5 5. Clock Domain Multiplexer.....	305
20.9 Properties and Characteristics.....	305
20.9.1 One-Hot Encoding.....	305
20.9.2 Self-Correcting Behavior.....	306
20.9.3 Comparison with Other Counters.....	306
20.10 Advanced Usage.....	306
20.10.1 1. Self-Correcting Ring Counter.....	306
20.10.2 2. Bidirectional Ring Counter.....	306
20.10.3 3. Programmable Ring Counter.....	307
20.11 Verification Strategy.....	307
20.11.1 Test Scenarios.....	307
20.11.2 Coverage Model.....	307
20.11.3 Assertions.....	308
20.12 Synthesis Considerations.....	308
20.12.1 Resource Utilization.....	308
20.12.2 Optimization Tips.....	308
20.12.3 Timing Considerations.....	309

20.13 Performance Characteristics.....	309
20.13.1 Maximum Frequency.....	309
20.13.2 Power Consumption.....	309
20.14 Common Pitfalls.....	309
20.14.1 1. Invalid Initial States.....	309
20.14.2 2. Stuck at Zero.....	309
20.14.3 3. Width Mismatch.....	309
20.15 Related Modules.....	310
20.16 Navigation.....	310
21 Count Leading Zeros Module.....	310
21.1 Overview.....	310
21.2 Module Declaration.....	310
21.3 Parameters.....	310
21.3.1 WIDTH.....	310
21.3.2 INSTANCE_NAME.....	311
21.4 Ports.....	311
21.4.1 Inputs.....	311
21.4.2 Outputs.....	311
21.4.3 Output Width Explanation.....	311
21.5 Algorithm Implementation.....	311
21.5.1 Function-Based Approach.....	311
21.5.2 Bit Scanning Process.....	312
21.5.3 Why LSB-to-MSB Scanning?.....	312
21.6 Examples and Truth Tables.....	312

21.6.1 8-bit Examples (WIDTH=8).....	312
21.6.2 32-bit Examples.....	313
21.7 Applications.....	313
21.7.1 1. Floating-Point Normalization.....	313
21.7.2 2. Priority Encoder.....	314
21.7.3 3. Bit Width Calculation.....	314
21.7.4 4. Log2 Calculation.....	314
21.7.5 5. Data Compression - Run Length Encoding.....	315
21.8 Advanced Implementations.....	315
21.8.1 1. Pipeline Version for High Speed.....	315
21.8.2 2. Hierarchical Implementation.....	316
21.8.3 3. LUT-Based Implementation (Small Widths).....	317
21.9 Performance Analysis.....	317
21.9.1 Resource Utilization.....	317
21.9.2 Timing Characteristics.....	318
21.10 Verification Strategy.....	318
21.10.1 Test Scenarios.....	318
21.10.2 Coverage Model.....	318
21.10.3 Assertions.....	318
21.11 Synthesis Optimization.....	319
21.11.1 Tool-Specific Attributes.....	319
21.11.2 Area vs. Speed Trade-offs.....	319
21.12 Common Use Cases Summary.....	319
21.13 Related Modules and Functions.....	320

21.14 Navigation.....	320
22 dataint_checksum Module Documentation.....	320
22.1 Purpose.....	320
22.2 Module Declaration.....	320
22.3 Parameters.....	320
22.4 Ports.....	321
22.5 Functionality.....	321
22.5.1 Operation.....	321
22.5.2 Reset Behavior.....	321
22.6 Implementation Details.....	321
22.6.1 Key Features.....	321
22.6.2 Special Considerations.....	322
22.7 State Machine.....	322
22.8 Usage Example.....	322
22.9 Applications.....	322
22.10 Navigation.....	322
23 dataint_crc.....	323
23.1 Overview.....	323
23.2 Module Declaration.....	323
23.3 Parameters.....	324
23.3.1 User-Settable Parameters.....	324
23.3.2 Derived Localparam (Computed Internally).....	324
23.4 Ports.....	324
23.4.1 Inputs.....	324

23.4.2 Outputs.....	325
23.5 Functionality.....	325
23.5.1 CRC Algorithm Implementation.....	325
23.5.2 Key Features.....	325
23.6 Implementation Details.....	326
23.6.1 Input Data Reflection.....	326
23.6.2 Cascade Processing Architecture.....	326
23.6.3 Cascade Selection Logic.....	326
23.7 CRC Standards Compatibility.....	327
23.7.1 Common CRC Configurations.....	327
23.8 Usage Examples.....	328
23.8.1 Basic CRC Calculation.....	328
23.8.2 Streaming CRC with Cascade.....	328
23.8.3 Multi-Stage CRC Pipeline.....	329
23.9 Performance Characteristics.....	329
23.9.1 Throughput Analysis.....	329
23.9.2 Latency Characteristics.....	329
23.10 Timing Considerations.....	330
23.10.1 Setup Requirements.....	330
23.10.2 Critical Paths.....	330
23.11 Design Considerations.....	330
23.11.1 Area Optimization.....	330
23.11.2 Power Optimization.....	330
23.11.3 Verification Notes.....	330

23.12 Related Modules.....	330
23.13 Navigation.....	331
24 dataint_crc_xor_shift_cascade Module Documentation.....	331
24.1 Purpose.....	331
24.2 Module Declaration.....	331
24.3 Parameters.....	331
24.4 Ports.....	331
24.5 Functionality.....	332
24.5.1 Cascade Operation.....	332
24.5.2 Data Bit Ordering.....	332
24.6 Implementation Details.....	332
24.6.1 Architecture Overview.....	332
24.6.2 Generate Block Structure.....	332
24.6.3 Intermediate Signals.....	333
24.7 Key Features.....	333
24.7.1 Parallel Byte Processing.....	333
24.7.2 Bit Order Handling.....	333
24.7.3 Scalable Architecture.....	333
24.8 Timing Characteristics.....	333
24.8.1 Combinational Delay.....	333
24.8.2 Critical Path.....	333
24.9 Usage Examples.....	334
24.9.1 Basic Usage.....	334
24.9.2 Multi-Byte Processing.....	334

24.10 Integration with CRC System.....	334
24.10.1 Role in CRC Module.....	334
24.10.2 Interface with Parent Module.....	334
24.11 Performance Optimization.....	335
24.11.1 Pipelining Considerations.....	335
24.11.2 Area Optimization.....	335
24.12 Applications.....	335
24.13 Related Modules.....	335
24.14 Navigation.....	335
25 dataint_crc_xor_shift Module Documentation.....	335
25.1 Purpose.....	335
25.2 Module Declaration.....	336
25.3 Parameters.....	336
25.4 Ports.....	336
25.5 Functionality.....	336
25.5.1 Core CRC Operation.....	336
25.5.2 Mathematical Foundation.....	336
25.6 Implementation Details.....	337
25.6.1 Key Features.....	337
25.6.2 Circular Dependency Solution.....	337
25.6.3 Bit-wise Operations.....	337
25.7 Special Implementation Features.....	337
25.7.1 Feedback Bit Logic.....	337
25.7.2 Polynomial Application.....	338

25.7.3 No State Elements.....	338
25.8 Usage in CRC Chain.....	338
25.8.1 Single Bit Processing.....	338
25.8.2 Cascaded for Multiple Bits.....	338
25.9 Mathematical Verification.....	339
25.10 Applications.....	339
25.11 Performance Characteristics.....	339
25.12 Navigation.....	339
26 dataint_ecc_hamming_decode_secded Module Documentation.....	339
26.1 Purpose.....	339
26.2 Module Declaration.....	339
26.3 Parameters.....	340
26.4 Calculated Parameters.....	340
26.5 Ports.....	340
26.6 Functionality.....	341
26.6.1 Decoding Process.....	341
26.6.2 Error Detection Logic.....	341
26.7 Implementation Details.....	341
26.7.1 Key Functions.....	341
26.7.2 Syndrome Generation.....	342
26.7.3 Overall Parity Check.....	343
26.8 State Machine Operation.....	343
26.8.1 Control States.....	343
26.8.2 Error Correction Logic.....	343

26.9 Key Features.....	344
26.9.1 Error Correction Capabilities.....	344
26.9.2 Performance Characteristics.....	344
26.9.3 Debug Support.....	344
26.10 Error Handling Details.....	344
26.10.1 Syndrome Interpretation.....	344
26.10.2 SECDED Bit Function.....	344
26.10.3 Error Correction Process.....	344
26.11 Usage Examples.....	345
26.11.1 Basic Decoder.....	345
26.11.2 Error Handling System.....	345
26.12 Data Extraction.....	345
26.12.1 Generate Block for Data Output.....	345
26.13 Applications.....	346
26.14 Related Modules.....	346
26.15 Performance Considerations.....	346
26.16 Navigation.....	346
27 dataint_ecc_hamming_encode_secDED Module Documentation.....	346
27.1 Purpose.....	346
27.2 Module Declaration.....	346
27.3 Parameters.....	347
27.4 Calculated Parameters.....	347
27.5 Ports.....	347
27.6 Functionality.....	347

27.6.1 Hamming Code Structure.....	347
27.6.2 Bit Position Layout.....	347
27.7 Implementation Details.....	348
27.7.1 Key Functions.....	348
27.7.2 Encoding Algorithm.....	348
27.8 Key Features.....	349
27.8.1 Combinational Design.....	349
27.8.2 Scalable Architecture.....	349
27.8.3 Debug Support.....	349
27.9 Example Configurations.....	350
27.9.1 4-bit Data (Common Example).....	350
27.9.2 8-bit Data.....	350
27.9.3 16-bit Data.....	350
27.10 Usage Example.....	350
27.10.1 Basic Encoding.....	350
27.10.2 With Debug.....	350
27.11 Error Correction Capabilities.....	351
27.11.1 Single Error Correction.....	351
27.11.2 Double Error Detection.....	351
27.11.3 Error Detection Matrix.....	351
27.12 Applications.....	351
27.13 Related Modules.....	351
27.14 Navigation.....	351
28 dataint_parity Module Documentation.....	352

28.1 Purpose.....	352
28.2 Module Declaration.....	352
28.3 Parameters.....	352
28.4 Calculated Parameters.....	352
28.5 Ports.....	352
28.6 Functionality.....	353
28.6.1 Dual Operation Modes.....	353
28.6.2 Parity Types.....	353
28.6.3 Data Chunking.....	353
28.7 Implementation Details.....	353
28.7.1 Generate Block Architecture.....	353
28.7.2 Chunk Boundary Calculation.....	354
28.7.3 Parity Calculation Logic.....	354
28.7.4 Error Detection.....	354
28.8 Key Features.....	354
28.8.1 Scalable Architecture.....	354
28.8.2 Simultaneous Operation.....	354
28.8.3 Parity Flexibility.....	354
28.9 Example Configurations.....	355
28.9.1 32-bit Data, 4 Chunks (8 bits each).....	355
28.9.2 30-bit Data, 4 Chunks (uneven division).....	355
28.10 Usage Examples.....	355
28.10.1 Basic Parity Generation.....	355
28.10.2 Parity Checking.....	355

28.10.3 Error Correction System.....	356
28.11 Performance Characteristics.....	356
28.11.1 Timing.....	356
28.11.2 Area.....	356
28.11.3 Power.....	356
28.12 Applications.....	357
28.12.1 Error Detection.....	357
28.12.2 Data Integrity.....	357
28.12.3 System Integration.....	357
28.13 Design Considerations.....	357
28.13.1 Chunk Size Selection.....	357
28.13.2 Parity Type Selection.....	357
28.13.3 Width Considerations.....	357
28.14 Related Applications.....	358
28.15 Navigation.....	358
29 Debounce Module (debounce.sv).....	358
29.1 Purpose.....	358
29.2 Ports.....	358
29.2.1 Input Ports.....	358
29.2.2 Output Ports.....	358
29.2.3 Parameters.....	358
29.3 Implementation Details.....	359
29.3.1 Core Algorithm.....	359
29.3.2 Key Features.....	359

29.4 Timing Characteristics.....	360
29.4.1 Debounce Delay.....	360
29.4.2 Response Time.....	360
29.5 Use Cases.....	360
29.6 Design Considerations.....	360
29.7 Navigation.....	360
30 Decoder Module (decoder.sv).....	361
30.1 Purpose.....	361
30.2 Ports.....	361
30.2.1 Input Ports.....	361
30.2.2 Output Ports.....	361
30.2.3 Parameters.....	361
30.3 Implementation Details.....	361
30.3.1 Core Logic.....	361
30.3.2 Operation Principle.....	361
30.4 Functional Behavior.....	362
30.4.1 Truth Table Example (M=2, N=4).....	362
30.4.2 Key Characteristics.....	362
30.5 Design Features.....	362
30.5.1 Parameterization.....	362
30.5.2 Synthesis Considerations.....	362
30.6 Use Cases.....	362
30.7 Common Applications.....	363
30.7.1 Memory Address Decoding.....	363

30.7.2 State Machine Output Decoding.....	363
30.8 Timing Characteristics.....	363
30.9 Related Modules.....	363
30.10 Design Notes.....	363
30.11 Navigation.....	363
31 Encoder Module (encoder.sv).....	364
31.1 Purpose.....	364
31.2 Ports.....	364
31.2.1 Input Ports.....	364
31.2.2 Output Ports.....	364
31.2.3 Parameters.....	364
31.3 Implementation Details.....	364
31.3.1 Core Algorithm.....	364
31.3.2 Operation Principle.....	364
31.4 Functional Behavior.....	365
31.4.1 Truth Table Example (N=8, output width=3).....	365
31.4.2 Key Characteristics.....	365
31.5 Design Features.....	365
31.5.1 Automatic Width Calculation.....	365
31.5.2 Priority Behavior.....	365
31.6 Use Cases.....	366
31.6.1 Valid One-Hot Inputs.....	366
31.6.2 Priority Encoding Applications.....	366
31.7 Timing Characteristics.....	366

31.8 Design Considerations.....	366
31.8.1 Input Validation.....	366
31.8.2 Synthesis Implications.....	366
31.9 Common Usage Patterns.....	366
31.9.1 Interrupt Controller.....	366
31.9.2 Arbiter Output Encoding.....	367
31.10 Related Modules.....	367
31.11 Behavioral Notes.....	367
31.12 Navigation.....	367
32 Priority Encoder with Enable (encoder_priority_enable.sv).....	367
32.1 Purpose.....	367
32.2 Ports.....	367
32.2.1 Input Ports.....	367
32.2.2 Output Ports.....	368
32.2.3 Parameters.....	368
32.3 Implementation Details.....	368
32.3.1 Core Algorithm.....	368
32.3.2 Operation Principle.....	368
32.4 Functional Behavior.....	368
32.4.1 Priority Resolution Example (WIDTH=8).....	368
32.4.2 Key Differences from Basic Encoder.....	369
32.5 Design Features.....	369
32.5.1 True Priority Encoding.....	369
32.5.2 Enable Control.....	369

32.5.3 Latch Prevention.....	370
32.6 Use Cases.....	370
32.6.1 Interrupt Controllers.....	370
32.6.2 Resource Arbitration.....	370
32.6.3 Error Reporting.....	370
32.7 Timing Characteristics.....	370
32.8 Design Considerations.....	370
32.8.1 Priority Assignment.....	370
32.8.2 Performance Scaling.....	371
32.8.3 Enable Usage Patterns.....	371
32.9 Common Applications.....	371
32.9.1 Multi-Level Interrupt Controllers.....	371
32.9.2 Bus Arbitration.....	371
32.9.3 Error and Exception Handling.....	371
32.10 Synthesis Considerations.....	371
32.11 Related Modules.....	372
32.12 Navigation.....	372
33 Asynchronous FIFO - Divisible by 2 (fifo_async_div2.sv).....	372
33.1 Purpose.....	372
33.2 Ports.....	372
33.2.1 Write Domain.....	372
33.2.2 Read Domain.....	372
33.2.3 Parameters.....	373
33.3 Architecture Overview.....	373

33.4 Johnson Counter Deep Dive.....	373
33.4.1 What are Johnson Counters?.....	373
33.4.2 Johnson vs. Gray Code Comparison.....	374
33.5 Implementation Details.....	374
33.5.1 Dual Counter Architecture.....	374
33.5.2 Why Both Counter Types?.....	375
33.6 Johnson-to-Binary Conversion.....	375
33.6.1 Conversion Challenge.....	375
33.6.2 grayj2bin Module Implementation.....	375
33.6.3 Leading/Trailing One Detection.....	376
33.7 Flexible Depth Support.....	376
33.7.1 Even Number Requirement.....	376
33.7.2 Example Depth Flexibility.....	376
33.7.3 vs. Power-of-2 Restriction.....	376
33.8 Memory Efficiency Trade-offs.....	376
33.8.1 Resource Usage Comparison.....	376
33.8.2 When to Choose Each Approach.....	377
33.9 Clock Domain Crossing Details.....	377
33.9.1 Synchronization Strategy.....	377
33.9.2 Metastability Protection.....	377
33.10 Use Cases.....	377
33.10.1 Ideal Applications.....	377
33.10.2 Application Examples.....	378
33.11 Performance Characteristics.....	378

33.11.1 Throughput.....	378
33.11.2 Resource Usage.....	378
33.11.3 Timing.....	378
33.12 Design Considerations.....	378
33.12.1 Depth Selection Guidelines.....	378
33.12.2 Resource Planning.....	378
33.13 WaveDrom Visualization.....	379
33.14 Error Detection.....	380
33.15 Related Modules.....	380
33.16 Test and Verification.....	380
33.17 Navigation.....	380
34 Asynchronous FIFO - Power of 2 (fifo_async.sv).....	381
34.1 Purpose.....	381
34.2 Ports.....	381
34.2.1 Write Domain.....	381
34.2.2 Read Domain.....	381
34.2.3 Parameters.....	381
34.3 Architecture Overview.....	382
34.3.1 Clock Domain Crossing Strategy.....	382
34.3.2 Core Components.....	382
34.4 Implementation Deep Dive.....	382
34.4.1 Gray Code Pointer System.....	382
34.4.2 Clock Domain Crossing.....	383
34.4.3 Memory Organization.....	383

34.4.4 Full/Empty Detection.....	384
34.5 Power-of-2 Requirement.....	384
34.5.1 Why Power-of-2 Only?.....	384
34.5.2 Limitation Impact.....	384
34.6 Timing Considerations.....	384
34.6.1 Clock Domain Crossing Latency.....	384
34.6.2 Metastability Protection.....	385
34.7 Use Cases.....	385
34.7.1 Typical Applications.....	385
34.7.2 When to Use vs. Alternatives.....	385
34.8 Design Guidelines.....	385
34.8.1 Depth Sizing.....	385
34.8.2 Clock Relationship.....	385
34.8.3 Almost Full/Empty Settings.....	385
34.9 Performance Characteristics.....	386
34.10 Error Detection Features.....	386
34.11 WaveDrom Visualization.....	386
34.12 Related Modules.....	387
34.13 Test and Verification.....	387
34.14 Navigation.....	387
35 FIFO Control Logic (fifo_control.sv).....	388
35.1 Purpose.....	388
35.2 Ports.....	388
35.2.1 Clock and Reset.....	388

35.2.2 Pointer Inputs.....	388
35.2.3 Status Outputs.....	388
35.2.4 Parameters.....	388
35.3 Architecture Overview.....	389
35.3.1 Dual-Domain Design.....	389
35.3.2 Pointer Arithmetic Foundation.....	389
35.4 Full Detection Logic.....	389
35.4.1 Basic Full Condition.....	389
35.4.2 Full Detection Algorithm.....	389
35.4.3 Visual Example (DEPTH=8, ADDR_WIDTH=3).....	389
35.5 Almost Full Logic.....	390
35.5.1 Occupancy Calculation.....	390
35.5.2 Two Cases Handled.....	390
35.5.3 Almost Full Threshold.....	390
35.6 Empty Detection Logic - Mode Aware.....	390
35.6.1 Critical Innovation: Mode-Aware Write Pointer Selection.....	390
35.6.2 Why Mode-Aware Detection?.....	391
35.6.3 Empty Detection Algorithm.....	391
35.7 Almost Empty Logic.....	391
35.7.1 Standard Timing (Mode-Independent).....	391
35.8 Count Generation.....	391
35.8.1 Occupancy Count Logic.....	391
35.8.2 Count Interpretation.....	391
35.9 Flag Registration.....	392

35.9.1 Write Domain Flags.....	392
35.9.2 Read Domain Flags.....	392
35.9.3 Reset Behavior.....	392
35.10 Width Casting Fix.....	392
35.10.1 Type Width Matching.....	392
35.11 Key Design Insights.....	393
35.11.1 Wraparound Handling.....	393
35.11.2 Timing Considerations.....	393
35.11.3 Conservative Design.....	393
35.12 Use Cases.....	393
35.13 Performance Impact.....	393
35.14 Navigation.....	393
36 Synchronous FIFO (fifo_sync.sv).....	394
36.1 Purpose.....	394
36.2 Ports.....	394
36.2.1 Input Ports.....	394
36.2.2 Output Ports.....	394
36.2.3 Parameters.....	394
36.3 Architecture Overview.....	394
36.3.1 Core Components.....	394
36.3.2 Memory Organization.....	395
36.4 Implementation Details.....	395
36.4.1 Pointer Management.....	395
36.4.2 Memory Operations.....	395

36.4.3 Status Flag Generation.....	396
36.5 Operating Modes.....	396
36.5.1 Mux Mode (REGISTERED = 0).....	396
36.5.2 Flop Mode (REGISTERED = 1).....	396
36.6 Functional Behavior.....	396
36.6.1 Write Operations.....	396
36.6.2 Read Operations.....	396
36.6.3 Reset Behavior.....	397
36.7 Timing Diagrams.....	397
36.7.1 Write Sequence.....	397
36.7.2 Read Sequence - Mux Mode.....	397
36.7.3 Read Sequence - Flop Mode.....	397
36.8 Use Cases.....	397
36.8.1 Low Latency Applications (Mux Mode).....	397
36.8.2 High Speed Applications (Flop Mode).....	397
36.9 Design Considerations.....	398
36.9.1 Mode Selection Guidelines.....	398
36.9.2 Sizing Considerations.....	398
36.9.3 Error Detection.....	398
36.10 Performance Characteristics.....	398
36.11 WaveDrom Visualization.....	398
36.12 Related Modules.....	399
36.13 Test and Verification.....	399
36.14 Navigation.....	400

37 Multi-Field Synchronous FIFO.....	400
37.1 Overview.....	400
37.1.1 Key Features.....	400
37.2 Module Purpose.....	400
37.3 Parameters.....	401
37.4 Port Groups.....	402
37.4.1 Clock and Reset.....	402
37.4.2 Write Interface.....	402
37.4.3 Read Interface.....	402
37.5 Functional Description.....	403
37.5.1 Field Packing.....	403
37.5.2 Field Unpacking.....	403
37.5.3 Storage and Flow Control.....	403
37.6 Usage Example.....	403
37.7 Design Notes.....	404
37.8 Related Modules.....	404
37.8.1 Used By.....	404
37.8.2 Uses.....	405
37.8.3 See Also.....	405
37.9 References.....	405
37.9.1 Source Code.....	405
37.9.2 Documentation.....	405
37.10 Navigation.....	405
38 Multi-Field Synchronous FIFO (Signal-Map Variant).....	405

38.1 Overview.....	405
38.1.1 Key Features.....	406
38.2 Module Purpose.....	406
38.3 Parameters.....	406
38.4 Port Groups.....	407
38.4.1 Clock and Reset.....	407
38.4.2 Write Interface.....	407
38.4.3 Read Interface.....	408
38.5 Functional Description.....	408
38.5.1 Field Packing.....	408
38.5.2 Field Unpacking.....	409
38.5.3 Storage and Flow Control.....	409
38.6 Usage Example.....	409
38.7 Design Notes.....	410
38.8 Related Modules.....	410
38.8.1 Used By.....	410
38.8.2 Uses.....	410
38.8.3 See Also.....	410
38.9 References.....	410
38.9.1 Source Code.....	410
38.9.2 Documentation.....	410
38.10 Navigation.....	411
39 Find First Set (find_first_set.sv).....	411
39.1 Purpose.....	411

39.2 Ports.....	411
39.2.1 Input Ports.....	411
39.2.2 Output Ports.....	411
39.2.3 Parameters.....	411
39.3 Implementation Details.....	411
39.3.1 Core Algorithm.....	411
39.3.2 Search Strategy.....	412
39.4 Functional Behavior.....	412
39.4.1 Truth Table Examples (WIDTH=8).....	412
39.4.2 Key Characteristics.....	412
39.5 Design Features.....	412
39.5.1 Width Parameterization.....	412
39.5.2 Found Flag Logic.....	413
39.5.3 Bit Width Handling.....	413
39.6 Use Cases.....	413
39.6.1 Interrupt Processing.....	413
39.6.2 Resource Allocation.....	413
39.6.3 Error Detection.....	414
39.6.4 Bit Manipulation.....	414
39.7 Timing Characteristics.....	414
39.7.1 Propagation Delay.....	414
39.7.2 Critical Path.....	414
39.8 Algorithm Comparison.....	414
39.8.1 vs. Find Last Set.....	414

39.8.2 vs. Priority Encoder.....	415
39.9 Design Considerations.....	415
39.9.1 Input Validation.....	415
39.9.2 Performance Optimization.....	415
39.9.3 Width Selection.....	415
39.10 Common Usage Patterns.....	415
39.10.1 Round-Robin Arbitration.....	415
39.10.2 Free List Management.....	415
39.10.3 Packet Classification.....	416
39.11 Synthesis Considerations.....	416
39.11.1 Resource Utilization.....	416
39.11.2 Timing Optimization.....	416
39.12 Verification Considerations.....	416
39.12.1 Test Scenarios.....	416
39.12.2 Coverage Metrics.....	416
39.13 Related Modules.....	416
39.14 Navigation.....	417
40 Find Last Set (find_last_set.sv).....	417
40.1 Purpose.....	417
40.2 Ports.....	417
40.2.1 Input Ports.....	417
40.2.2 Output Ports.....	417
40.2.3 Parameters.....	417
40.3 Implementation Details.....	417

40.3.1 Core Algorithm.....	417
40.3.2 Search Strategy.....	418
40.4 Functional Behavior.....	418
40.4.1 Truth Table Examples (WIDTH=8).....	418
40.4.2 Key Characteristics.....	418
40.5 Design Features.....	419
40.5.1 Reverse Loop Implementation.....	419
40.5.2 Width Parameterization.....	419
40.5.3 Found Flag Logic.....	419
40.6 Use Cases.....	419
40.6.1 Interrupt Priority Handling.....	419
40.6.2 Leading Zero Count.....	419
40.6.3 Bit Width Calculation.....	420
40.6.4 Cache Line Selection.....	420
40.7 Timing Characteristics.....	420
40.7.1 Propagation Delay.....	420
40.7.2 Critical Path Analysis.....	420
40.8 Algorithm Comparison.....	420
40.8.1 Find Last Set vs. Find First Set.....	420
40.8.2 Typical Use Case Patterns.....	421
40.9 Design Considerations.....	421
40.9.1 Zero Input Handling.....	421
40.9.2 Performance Scaling.....	421
40.9.3 Synthesis Optimization.....	421

40.10 Advanced Usage Patterns.....	421
40.10.1 Hierarchical Priority Encoding.....	421
40.10.2 Conditional Priority Selection.....	422
40.10.3 Dynamic Width Processing.....	422
40.11 Verification Strategies.....	422
40.11.1 Comprehensive Test Coverage.....	422
40.11.2 Random Testing.....	423
40.12 Related Modules.....	423
40.13 Navigation.....	423
41 Glitch-Free N-DFF Synchronizer (glitch_free_n_dff_arn.sv).....	423
41.1 Purpose.....	423
41.2 Ports.....	423
41.2.1 Input Ports.....	423
41.2.2 Output Ports.....	424
41.2.3 Parameters.....	424
41.3 Clock Domain Crossing Theory.....	424
41.3.1 Metastability Problem.....	424
41.3.2 Metastability Effects.....	424
41.4 Multi-Stage Synchronizer Solution.....	425
41.4.1 Stage-by-Stage Analysis.....	425
41.5 Implementation Details.....	425
41.5.1 Packed Array Structure.....	425
41.5.2 Shift Register Behavior.....	425
41.5.3 Reset Behavior.....	426

41.6 MTBF (Mean Time Between Failures) Analysis.....	426
41.6.1 MTBF Calculation.....	426
41.6.2 Stage Count Impact.....	426
41.6.3 Practical Guidelines.....	426
41.7 Latency vs. Reliability Trade-off.....	427
41.7.1 Synchronizer Latency.....	427
41.7.2 Performance Impact.....	427
41.8 Usage Guidelines.....	427
41.8.1 Appropriate Use Cases.....	427
41.8.2 Inappropriate Use Cases.....	427
41.8.3 Multi-Bit Considerations.....	427
41.9 Advanced Applications.....	428
41.9.1 Reset Synchronizer Pattern.....	428
41.9.2 Enable Signal Synchronization.....	428
41.9.3 Status Flag Collection.....	428
41.10 Synthesis Considerations.....	428
41.10.1 FPGA Implementation.....	428
41.10.2 Timing Constraints.....	429
41.10.3 Optimization Prevention.....	429
41.11 Verification Strategies.....	429
41.11.1 Functional Verification.....	429
41.11.2 Metastability Testing.....	429
41.11.3 Formal Verification.....	429
41.12 Debug Features.....	429

41.12.1 Waveform Analysis.....	429
41.12.2 Simulation Visibility.....	430
41.13 Related Modules.....	430
41.14 Navigation.....	430
42 Gray-to-Binary Converter (gray2bin.sv).....	430
42.1 Purpose.....	430
42.2 Ports.....	430
42.2.1 Input Ports.....	430
42.2.2 Output Ports.....	430
42.2.3 Parameters.....	430
42.3 Gray Code Theory.....	431
42.3.1 What is Gray Code?.....	431
42.3.2 Why Gray Codes Matter for CDC.....	431
42.4 Conversion Algorithm.....	431
42.4.1 Mathematical Foundation.....	431
42.4.2 Implementation.....	431
42.4.3 Bit-by-Bit Breakdown.....	431
42.5 Functional Examples.....	432
42.5.1 4-Bit Conversion Table.....	432
42.5.2 Step-by-Step Example (Gray 0110 → Binary).....	433
42.6 Implementation Analysis.....	433
42.6.1 XOR Reduction Technique.....	433
42.6.2 Generate Loop Benefits.....	433
42.7 Use Cases in Digital Design.....	433

42.7.1 Asynchronous FIFO Pointers.....	433
42.7.2 Clock Domain Crossing Counters.....	434
42.7.3 Position Encoding.....	434
42.8 Timing Characteristics.....	434
42.8.1 Propagation Delay.....	434
42.8.2 Delay Analysis by Width.....	434
42.8.3 Synthesis Optimization.....	434
42.9 Design Considerations.....	434
42.9.1 Width Scaling.....	434
42.9.2 Input Validation.....	435
42.9.3 Pipeline Considerations.....	435
42.10 Verification Strategies.....	435
42.10.1 Exhaustive Testing.....	435
42.10.2 Property-Based Verification.....	435
42.10.3 Random Testing.....	436
42.11 Common Mistakes and Pitfalls.....	436
42.11.1 Bit Order Confusion.....	436
42.11.2 Width Mismatches.....	436
42.11.3 Timing Assumptions.....	436
42.12 Related Modules.....	436
42.13 Navigation.....	437
43 Johnson-to-Binary Converter (grayj2bin.sv).....	437
43.1 Purpose.....	437
43.2 Ports.....	437

43.2.1 Input Ports.....	437
43.2.2 Output Ports.....	437
43.2.3 Parameters.....	437
43.3 Johnson Counter Sequence Review.....	437
43.3.1 Johnson Counter Characteristics.....	437
43.3.2 Key Properties.....	438
43.4 Conversion Algorithm.....	438
43.4.1 Strategy Overview.....	438
43.4.2 Position Detection Module.....	438
43.5 Detailed Conversion Examples.....	439
43.5.1 First Half Conversion (MSB = 0).....	439
43.5.2 Second Half Conversion (MSB = 1).....	439
43.5.3 Special Cases.....	439
43.6 Implementation Deep Dive.....	439
43.6.1 Three-Part Binary Construction.....	439
43.6.2 Width Calculations.....	439
43.6.3 Why This Algorithm Works.....	439
43.7 Use in Asynchronous FIFO.....	440
43.7.1 Context in FIFO Operation.....	440
43.7.2 Why Clock is Required.....	440
43.8 Comparison with Standard Gray2Bin.....	440
43.9 Performance Characteristics.....	441
43.9.1 Timing Analysis.....	441
43.9.2 Resource Utilization.....	441

43.10 Design Considerations.....	441
43.10.1 When to Use Johnson vs. Gray.....	441
43.10.2 Resource Planning.....	441
43.10.3 Verification Challenges.....	442
43.11 Error Conditions and Debug.....	442
43.11.1 Invalid Johnson Sequences.....	442
43.11.2 Debug Visibility.....	442
43.11.3 Common Issues.....	442
43.12 Related Modules.....	442
43.13 Advanced Topics.....	443
43.13.1 Hierarchical Johnson Counters.....	443
43.13.2 Alternative Position Detection.....	443
43.14 Navigation.....	443
44 Hex to 7-Segment Display (hex_to_7seg.sv).....	443
44.1 Purpose.....	443
44.2 Ports.....	443
44.2.1 Input Ports.....	443
44.2.2 Output Ports.....	443
44.2.3 Parameters.....	443
44.3 7-Segment Display Layout.....	444
44.3.1 Segment Naming Convention.....	444
44.3.2 Bit Assignment.....	444
44.4 Character Patterns.....	444
44.4.1 Complete Truth Table.....	444

44.4.2 Visual Representation.....	445
44.5 Implementation Details.....	445
44.5.1 Case Statement Structure.....	445
44.5.2 Design Choices.....	446
44.6 Display Technology Integration.....	447
44.6.1 Common Cathode Alternative.....	449
44.6.2 Current Limiting.....	449
44.7 Use Cases and Applications.....	449
44.7.1 Debug Displays.....	449
44.7.2 Multi-Digit Displays.....	449
44.7.3 Memory/Register Viewers.....	449
44.7.4 Error Code Display.....	449
44.8 Timing Characteristics.....	450
44.8.1 Propagation Delay.....	450
44.8.2 Display Update Rate.....	450
44.9 Design Variations.....	450
44.9.1 Extended Character Set.....	450
44.9.2 Alternative Fonts.....	450
44.9.3 Brightness Control.....	450
44.10 Multiplexed Display Support.....	450
44.10.1 Time-Division Multiplexing.....	450
44.10.2 Refresh Rate Calculation.....	451
44.11 Verification and Testing.....	451
44.11.1 Exhaustive Testing.....	451

44.11.2 Visual Verification.....	451
44.11.3 Pattern Validation.....	452
44.12 Common Issues and Debugging.....	452
44.12.1 Segment Mapping Errors.....	452
44.12.2 Display Quality Issues.....	452
44.12.3 Character Readability.....	452
44.13 Related Modules.....	452
44.14 Navigation.....	452
45 icg.....	453
45.1 Overview.....	453
45.2 Module Declaration.....	453
45.3 Ports.....	453
45.3.1 Inputs.....	453
45.3.2 Outputs.....	453
45.4 Functionality.....	453
45.4.1 Clock Gating Operation.....	453
45.4.2 Implementation Details.....	454
45.5 Timing Characteristics.....	454
45.5.1 Critical Timing Requirements.....	454
45.5.2 Timing Diagram.....	454
45.6 Power Optimization Benefits.....	455
45.6.1 Dynamic Power Reduction.....	455
45.6.2 Power Savings Calculation.....	455
45.7 Design Considerations.....	455

45.7.1 Safety Requirements.....	455
45.7.2 Enable Signal Guidelines.....	455
45.8 Usage Examples.....	456
45.8.1 Basic Register Bank Gating.....	456
45.8.2 CPU Pipeline Stage Gating.....	456
45.8.3 Memory Interface Gating.....	456
45.9 Implementation Alternatives.....	457
45.9.1 Standard Cell ICG.....	457
45.9.2 FPGA Implementation.....	457
45.10 Verification Considerations.....	457
45.10.1 Functional Tests.....	457
45.10.2 Timing Tests.....	457
45.10.3 Power Analysis.....	458
45.11 Synthesis Guidelines.....	458
45.11.1 Technology Mapping.....	458
45.11.2 Clock Tree Synthesis.....	458
45.12 Design Integration.....	458
45.12.1 Clock Tree Integration.....	458
45.12.2 Power Domain Considerations.....	458
45.13 Related Modules.....	459
45.14 Navigation.....	459
46 Leading One Trailing One Module.....	459
46.1 Purpose.....	459
46.2 Key Features.....	459

46.3 Port Description.....	459
46.3.1 Parameters.....	459
46.3.2 Inputs.....	460
46.3.3 Outputs.....	460
46.4 Implementation Details.....	460
46.4.1 Submodules.....	460
46.4.2 One-Hot Vector Generation.....	460
46.4.3 Status Flag Logic.....	461
46.5 Special Implementation Notes.....	461
46.6 Usage Examples.....	461
46.6.1 8-bit Example.....	461
46.6.2 Edge Cases.....	462
46.7 Applications.....	462
46.8 Navigation.....	462
47 Modulo-3 Compressor.....	462
47.1 Overview.....	462
47.1.1 Key Features.....	463
47.2 Module Purpose.....	463
47.3 Parameters.....	463
47.4 Port Groups.....	464
47.4.1 Data.....	464
47.5 Functional Description.....	464
47.5.1 Base-4 Digit-Sum Method.....	464
47.5.2 3:2 Compressor Tree.....	464

47.5.3 Final Fold and Conditional Subtract.....	465
47.6 Usage Example.....	465
47.7 Design Notes.....	465
47.8 Related Modules.....	466
47.8.1 Used By.....	466
47.8.2 Uses.....	466
47.8.3 See Also.....	466
47.9 References.....	466
47.9.1 Source Code.....	466
47.9.2 Documentation.....	466
47.10 Navigation.....	466
48 PWM (Pulse Width Modulation) Module.....	466
48.1 Purpose.....	466
48.2 Key Features.....	467
48.3 Port Description.....	467
48.3.1 Parameters.....	467
48.3.2 Inputs.....	467
48.3.3 Outputs.....	467
48.4 State Machine.....	468
48.4.1 States.....	468
48.4.2 State Transitions.....	468
48.5 Implementation Details.....	468
48.5.1 Per-Channel Parameter Extraction.....	468
48.5.2 Edge Detection for Start Signal.....	468

48.5.3 PWM Output Generation Logic.....	469
48.5.4 Counter Management.....	469
48.5.5 Critical Control Signals.....	469
48.6 Special Implementation Notes.....	470
48.6.1 1. Edge-Triggered Start Control.....	470
48.6.2 2. Duty Cycle Edge Cases.....	470
48.6.3 3. Infinite Repeat Mode.....	470
48.6.4 4. Simultaneous Multi-Channel Operation.....	470
48.6.5 5. Reset Behavior.....	470
48.7 Timing Examples.....	470
48.7.1 Single Period Example (WIDTH=8, duty=3, period=8).....	470
48.7.2 Multi-Repeat Example (repeat_count=2).....	470
48.8 Usage Scenarios.....	471
48.8.1 Motor Speed Control.....	471
48.8.2 LED Dimming Sequence.....	471
48.8.3 Servo Motor Control.....	471
48.9 Applications.....	471
48.10 Navigation.....	471
49 Reset Synchronizer Module.....	471
49.1 Purpose.....	471
49.2 Key Features.....	472
49.3 Port Description.....	472
49.3.1 Parameters.....	472
49.3.2 Inputs.....	472

49.3.3 Outputs.....	472
49.4 Reset Synchronizer Theory.....	472
49.4.1 The Reset Problem.....	472
49.4.2 Asynchronous Assert, Synchronous Deassert.....	472
49.5 Implementation Details.....	473
49.5.1 Synchronization Register Chain.....	473
49.5.2 Reset Logic (Corrected Implementation).....	473
49.5.3 Output Assignment.....	473
49.6 Timing Behavior.....	473
49.6.1 Reset Assertion (Asynchronous).....	473
49.6.2 Reset Deassertion (Synchronous, N=3).....	473
49.6.3 State Sequence for N=3.....	474
49.7 Special Implementation Notes.....	474
49.7.1 1. Inverted Logic Explanation.....	474
49.7.2 2. Metastability Resolution.....	474
49.7.3 3. Initialization.....	474
49.7.4 4. Parameterization Benefits.....	475
49.8 Parameter Selection Guidelines.....	475
49.8.1 Choosing N Value.....	475
49.9 Applications.....	476
49.9.1 Global Reset Distribution.....	476
49.9.2 Clock Domain Crossing.....	476
49.9.3 Power-On Reset.....	476
49.9.4 Reset Controller Integration.....	476

49.10 Design Considerations.....	477
49.10.1 Reset Tree Planning.....	477
49.10.2 Clock Quality Requirements.....	477
49.10.3 Reset Source Management.....	477
49.11 Common Design Mistakes.....	477
49.11.1 Incorrect Sensitivity List.....	477
49.11.2 Wrong Reset Polarity.....	478
49.11.3 Clock Domain Issues.....	478
49.12 Verification Considerations.....	478
49.12.1 Test Scenarios.....	478
49.12.2 Assertions.....	478
49.12.3 Coverage Points.....	478
49.13 Test Verification.....	479
49.13.1 Test Coverage.....	479
49.13.2 Test Scenarios.....	479
49.13.3 Running Tests.....	479
49.13.4 Test Results.....	479
49.14 Related Modules.....	479
49.14.1 Reset Synchronizer Variants.....	479
49.14.2 Integration with Other Modules.....	479
49.15 Navigation.....	480
50 Reverse Vector Module.....	480
50.1 Purpose.....	480
50.2 Key Features.....	480

50.3 Port Description.....	480
50.3.1 Parameters.....	480
50.3.2 Inputs.....	480
50.3.3 Outputs.....	480
50.4 Implementation Details.....	481
50.4.1 Bit Reversal Logic.....	481
50.5 Operation Examples.....	481
50.5.1 8-bit Vector Reversal.....	481
50.5.2 4-bit Vector Reversal.....	481
50.6 Special Implementation Notes.....	481
50.6.1 1. Combinational Operation.....	481
50.6.2 2. Parameterizable Width.....	482
50.6.3 3. For-Loop Synthesis.....	482
50.6.4 4. Bidirectional Property.....	482
50.7 Synthesis Considerations.....	482
50.7.1 Hardware Implementation.....	482
50.7.2 Optimization.....	482
50.8 Applications.....	482
50.8.1 Endianness Conversion.....	482
50.8.2 Serial Communication.....	482
50.8.3 Cryptographic Operations.....	483
50.8.4 Signal Processing.....	483
50.8.5 Data Format Conversion.....	483
50.8.6 Mirror Pattern Generation.....	483

50.9 Usage Examples.....	483
50.9.1 Instantiation with Different Widths.....	483
50.9.2 Pipeline Integration.....	483
50.9.3 Conditional Reversal.....	483
50.10 Design Considerations.....	484
50.10.1 Timing Analysis.....	484
50.10.2 Power Consumption.....	484
50.10.3 Area Utilization.....	484
50.11 Verification Considerations.....	484
50.11.1 Test Cases.....	484
50.11.2 Coverage Points.....	484
50.12 Related Modules and Alternatives.....	485
50.12.1 Byte Reversal.....	485
50.12.2 Configurable Reversal.....	485
50.12.3 Multi-Width Reversal.....	485
50.13 Performance Metrics.....	485
50.13.1 Speed.....	485
50.13.2 Area.....	485
50.13.3 Power.....	486
50.14 Common Pitfalls and Solutions.....	486
50.14.1 Simulation Warnings.....	486
50.14.2 Synthesis Issues.....	486
50.14.3 Timing Closure.....	486
50.15 Navigation.....	486

51 Barrel Shifter Module.....	486
51.1 Purpose.....	486
51.2 Key Features.....	486
51.3 Port Description.....	487
51.3.1 Parameters.....	487
51.3.2 Inputs.....	487
51.3.3 Outputs.....	487
51.4 Control Signal Encoding.....	487
51.5 Implementation Details.....	488
51.5.1 Shift Amount Modulation.....	488
51.5.2 Wrap-Around Implementation.....	488
51.5.3 Pre-computed Rotation Arrays.....	488
51.5.4 Main Shift Logic.....	488
51.6 Special Implementation Notes.....	489
51.6.1 1. Combinational Design.....	489
51.6.2 2. Arithmetic Right Shift.....	489
51.6.3 3. Zero Shift Optimization.....	489
51.6.4 4. Double-Width Concatenation.....	489
51.6.5 5. Generate Block Optimization.....	489
51.7 Timing Examples.....	490
51.7.1 8-bit Examples (WIDTH=8).....	490
51.8 Resource Utilization.....	490
51.8.1 Area Considerations.....	490
51.8.2 Speed Characteristics.....	490

51.9 Applications.....	490
51.10 Navigation.....	491
52 Beat-Packing Shifter.....	491
52.1 Overview.....	491
52.1.1 Key Features.....	491
52.2 Module Purpose.....	492
52.3 Parameters.....	492
52.4 Port Groups.....	493
52.4.1 Clock and Reset.....	493
52.4.2 Configuration.....	493
52.4.3 Push Interface (ingress).....	493
52.4.4 Pop Interface (egress).....	494
52.4.5 Status.....	494
52.5 Functional Description.....	494
52.5.1 Runtime Beat Width.....	494
52.5.2 Handshake and Status.....	494
52.5.3 Next-State: Pop First, Push Second.....	495
52.5.4 Elaboration-Time Contract.....	495
52.5.5 Sizing Guidance.....	496
52.6 Usage Example.....	496
52.7 Design Notes.....	496
52.8 Related Modules.....	497
52.8.1 Used By.....	497
52.8.2 Uses.....	497

52.8.3 See Also.....	497
52.9 References.....	497
52.9.1 Source Code.....	497
52.9.2 Documentation.....	497
52.10 Navigation.....	497
53 Fibonacci LFSR Module.....	498
53.1 Purpose.....	498
53.2 Key Features.....	498
53.3 Port Description.....	498
53.3.1 Parameters.....	498
53.3.2 Inputs.....	498
53.3.3 Outputs.....	499
53.4 Fibonacci vs Galois LFSR Architecture.....	499
53.4.1 Fibonacci LFSR Characteristics.....	499
53.5 Implementation Details.....	500
53.5.1 Tap Processing (Same as Standard LFSR).....	500
53.5.2 Fibonacci Feedback Calculation.....	500
53.5.3 Right-Shift with MSB Feedback.....	500
53.6 Special Implementation Notes.....	500
53.6.1 1. XOR vs XNOR Feedback.....	500
53.6.2 2. Non-Zero State Protection.....	501
53.6.3 3. Right-Shift Architecture.....	501
53.6.4 4. Polynomial Compatibility.....	501
53.6.5 5. Reset to All Zeros.....	501

53.7 Timing Example (4-bit Fibonacci LFSR).....	501
53.7.1 Configuration.....	501
53.7.2 Sequence Generation.....	501
53.8 Comparison with Galois LFSR.....	502
53.8.1 Advantages of Fibonacci LFSR.....	502
53.8.2 Disadvantages of Fibonacci LFSR.....	502
53.8.3 When to Use Fibonacci LFSR.....	502
53.9 Applications.....	502
53.10 Polynomial Examples for Fibonacci LFSR.....	502
53.10.1 Common Primitive Polynomials.....	502
53.10.2 Usage Note.....	503
53.11 Navigation.....	503
54 Galois LFSR Module.....	503
54.1 Purpose.....	503
54.2 Key Features.....	503
54.3 Port Description.....	503
54.3.1 Parameters.....	503
54.3.2 Inputs.....	504
54.3.3 Outputs.....	504
54.4 Galois LFSR Architecture.....	504
54.4.1 Key Architectural Differences.....	504
54.4.2 Advantages of Galois Architecture.....	504
54.5 Implementation Details.....	505
54.5.1 Tap Position Processing.....	505

54.5.2 LSB Feedback Generation.....	505
54.5.3 Galois Next State Calculation.....	505
54.5.4 Sequential Update.....	505
54.6 Special Implementation Notes.....	506
54.6.1 1. Reset to All Ones.....	506
54.6.2 2. Combinational Next State Logic.....	506
54.6.3 3. Conditional XOR Application.....	506
54.6.4 4. Bounds Checking.....	506
54.6.5 5. Right-Shift with Distributed Feedback.....	506
54.7 Timing Example (4-bit Galois LFSR).....	506
54.7.1 Configuration.....	506
54.7.2 Sequence Generation.....	507
54.8 Performance Characteristics.....	507
54.8.1 Critical Path Analysis.....	507
54.8.2 Timing Advantages.....	507
54.9 Polynomial Considerations.....	507
54.9.1 Tap Position Mapping.....	507
54.9.2 Example Polynomials.....	507
54.10 Design Trade-offs.....	508
54.10.1 Galois LFSR Advantages.....	508
54.10.2 Galois LFSR Disadvantages.....	508
54.11 Applications.....	508
54.11.1 High-Speed Systems.....	508
54.11.2 Communications.....	508

54.11.3 Cryptography.....	508
54.11.4 Testing.....	508
54.12 When to Choose Galois over Fibonacci.....	508
54.12.1 Use Galois LFSR When:.....	508
54.12.2 Use Fibonacci LFSR When:.....	509
54.13 Navigation.....	509
55 LFSR (Linear Feedback Shift Register) Module.....	509
55.1 Purpose.....	509
55.2 Key Features.....	509
55.3 Port Description.....	509
55.3.1 Parameters.....	509
55.3.2 Inputs.....	510
55.3.3 Outputs.....	510
55.4 LFSR Theory and Implementation.....	510
55.4.1 Feedback Polynomial.....	510
55.4.2 Tap Position Processing.....	510
55.4.3 Feedback Calculation.....	511
55.4.4 Shift Register Operation.....	511
55.5 LFSR Polynomial Reference.....	511
55.5.1 Popular LFSR Polynomials (XNOR taps).....	511
55.5.2 Example Tap Configuration for 8-bit LFSR.....	511
55.6 Special Implementation Notes.....	512
55.6.1 1. Tap Position Encoding.....	512
55.6.2 2. XNOR Feedback.....	512

55.6.3 3. Cycle Detection.....	512
55.6.4 4. Left-Shift Architecture.....	512
55.6.5 5. Zero State Handling.....	512
55.7 Timing Diagrams.....	512
55.7.1 Seed Loading and Operation.....	512
55.7.2 Sequence Example (3-bit LFSR, taps=[3,2]).....	513
55.8 Applications.....	513
55.8.1 Pseudo-Random Number Generation.....	513
55.8.2 Built-In Self-Test (BIST).....	513
55.8.3 Scrambling/Descrambling.....	513
55.9 Common Use Cases.....	513
55.10 Navigation.....	514
56 Universal Shifter Module.....	514
56.1 Purpose.....	514
56.2 Key Features.....	514
56.3 Port Description.....	514
56.3.1 Parameters.....	514
56.3.2 Inputs.....	514
56.3.3 Outputs.....	515
56.4 Operation Control.....	515
56.5 Implementation Details.....	515
56.5.1 Combinational Logic for Next State.....	515
56.5.2 Sequential Logic for State Update.....	516
56.6 Operation Details.....	516

56.6.1 Hold Operation (select = 2'b00).....	516
56.6.2 Right Shift Operation (select = 2'b01).....	517
56.6.3 Left Shift Operation (select = 2'b10).....	517
56.6.4 Parallel Load Operation (select = 2'b11).....	517
56.7 Special Implementation Notes.....	517
56.7.1 1. Bidirectional Serial Interface.....	517
56.7.2 2. Clean Serial Output Management.....	517
56.7.3 3. Asynchronous Reset.....	517
56.7.4 4. Combinational/Sequential Separation.....	518
56.8 Timing Diagrams.....	518
56.8.1 Right Shift Operation Example (WIDTH=4).....	518
56.8.2 Parallel Load followed by Left Shift.....	518
56.9 Applications.....	518
56.9.1 Serial Communication Interface.....	518
56.9.2 Multiplication/Division by Powers of 2.....	518
56.9.3 Data Delay Line.....	519
56.9.4 Pattern Generator.....	519
56.9.5 Scan Chain for Testing.....	519
56.10 Design Considerations.....	519
56.10.1 Chaining Multiple Shifters.....	519
56.10.2 Bidirectional Operation.....	519
56.10.3 Control Logic Integration.....	520
56.11 Common Use Cases.....	520
56.12 Navigation.....	520

57 Sort Module RTL Documentation.....	520
57.1 Purpose.....	520
57.2 Module Overview.....	520
57.3 Port Descriptions.....	521
57.3.1 Input Ports.....	521
57.3.2 Output Ports.....	521
57.3.3 Parameters.....	522
57.4 Data Format.....	522
57.5 Algorithm: Odd-Even Sort.....	522
57.5.1 Overview.....	522
57.5.2 Algorithm Description.....	522
57.5.3 Example Execution.....	522
57.5.4 Algorithm Comparison.....	523
57.6 Architecture.....	524
57.6.1 Pipeline Structure.....	524
57.6.2 Naming Conventions.....	525
57.7 Implementation Details.....	525
57.7.1 Compare-Swap Logic.....	525
57.7.2 Reset Behavior.....	526
57.7.3 Timing Characteristics.....	526
57.8 State Machines.....	526
57.8.1 Control Flow.....	526
57.8.2 Valid Signal Flow.....	527
57.9 Special Features.....	527

57.9.1 1. Parameterizable Design.....	527
57.9.2 2. High Throughput Pipeline.....	527
57.9.3 3. Synthesis-Friendly Design.....	527
57.9.4 4. Robust Data Handling.....	527
57.10 Verification Strategy.....	528
57.10.1 Test Levels.....	528
57.10.2 Test Coverage.....	528
57.11 Usage Guidelines.....	528
57.11.1 Typical Use Cases.....	528
57.11.2 Integration Considerations.....	528
57.11.3 Performance Optimization.....	528
57.12 Synthesis Results.....	529
57.13 Future Enhancements.....	529
57.14 Navigation.....	529
58 Pulse Synchronizer for Clock Domain Crossing.....	529
58.1 Overview.....	530
58.2 Module Declaration.....	530
58.3 Parameters.....	530
58.3.1 User-Settable Parameters.....	530
58.4 Ports.....	530
58.4.1 Source Clock Domain Ports.....	530
58.4.2 Destination Clock Domain Ports.....	531
58.5 Functionality.....	531
58.5.1 Toggle-Based Pulse Synchronization.....	531

58.5.2 Timing Characteristics.....	531
58.6 Implementation Details.....	532
58.6.1 Source Domain: Toggle Register.....	532
58.6.2 Destination Domain: Multi-Stage Synchronizer.....	532
58.6.3 Destination Domain: Edge Detection.....	532
58.7 Protocol Timing.....	533
58.7.1 Single Pulse Transfer.....	533
58.7.2 Multiple Pulse Transfer.....	533
58.8 Usage Examples.....	533
58.8.1 Basic Pulse Synchronization.....	533
58.8.2 Interrupt Synchronization.....	534
58.8.3 Event Counter Synchronization.....	534
58.8.4 With Edge Detector for Multi-Cycle Signals.....	535
58.9 Design Considerations.....	535
58.9.1 When to Use sync_pulse.....	535
58.9.2 Minimum Pulse Spacing.....	535
58.9.3 Reset Synchronization.....	536
58.10 Synthesis and Constraints.....	536
58.10.1 Synthesis Attributes.....	536
58.10.2 Timing Constraints (Xilinx XDC).....	536
58.10.3 Timing Constraints (Intel SDC).....	537
58.11 Verification and Assertions.....	537
58.11.1 Formal Assertions (Built-in).....	537
58.11.2 Simulation Checks.....	537

58.12 Performance Analysis.....	538
58.12.1 Latency Breakdown.....	538
58.12.2 Resource Utilization.....	538
58.13 Common Pitfalls.....	538
58.14 Related Modules.....	539
58.15 References.....	540
58.16 Navigation.....	540

[← Back to Main Index](#) | [RTLCommon Index](#)

1 RTLCommon Library Overview

The RTLCommon library provides a comprehensive collection of fundamental digital design building blocks, offering high-quality, parameterizable, and synthesis-friendly RTL modules for use across a wide range of digital systems.

1.1 Library Philosophy

1.1.1 Design Principles

The RTLCommon library is built on the following core principles:

Modularity: Each module focuses on a specific function and can be easily composed with others

Parameterizability: Configurable width and functionality parameters for maximum reusability

Synthesis Friendly: Optimized for modern synthesis tools with predictable area/timing results

Technology Independence: Clean RTL that works across ASIC and FPGA technologies

Comprehensive Coverage: From basic gates to complex arithmetic and control structures

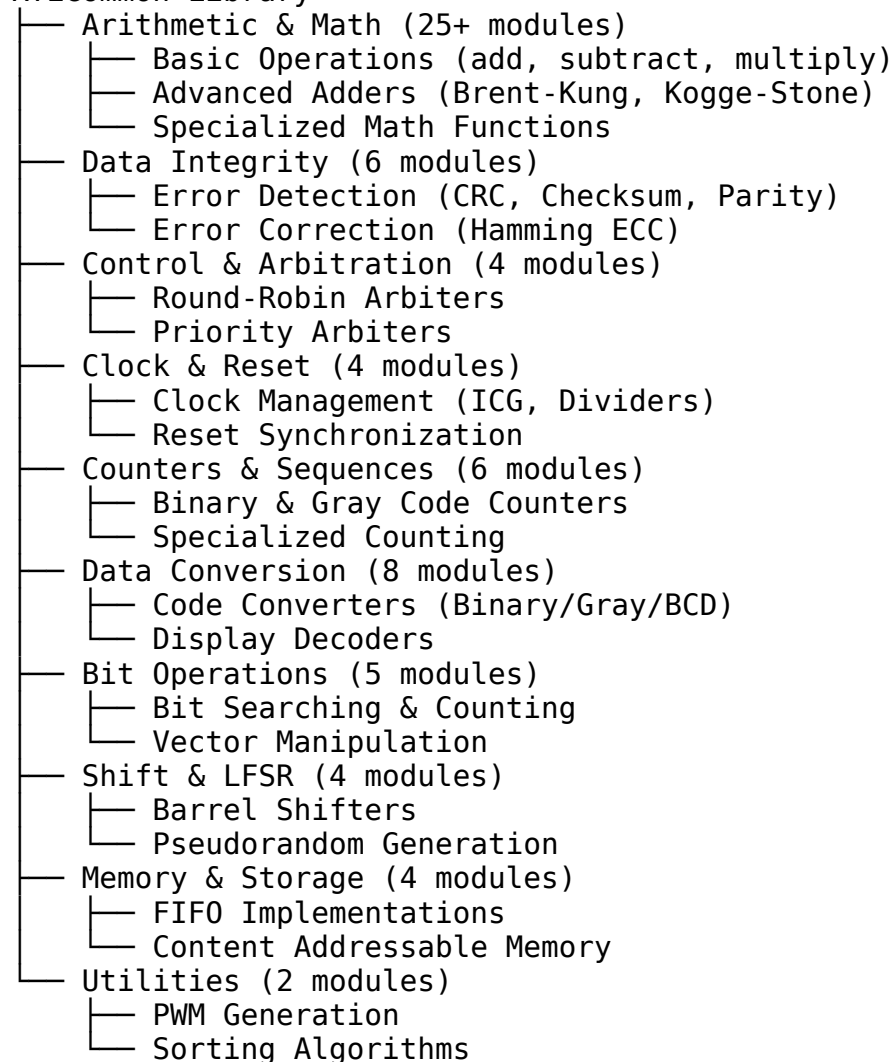
1.1.2 Quality Standards

- **Lint Clean:** All modules pass rigorous RTL linting checks
- **Simulation Verified:** Comprehensive testbenches validate functionality
- **Synthesis Proven:** Validated across multiple technology nodes and vendors
- **Documentation Complete:** Detailed documentation with usage examples

1.2 Architecture Overview

1.2.1 Module Hierarchy

RTLCommon Library



1.3 Functional Categories

1.3.1 1. Arithmetic and Mathematical Operations

The arithmetic category provides the foundation for digital signal processing and general computation.

1.3.1.1 Basic Arithmetic Building Blocks

- **Full Adders:** Single-bit and multi-bit addition with carry propagation
- **Half Adders:** Simplified 2-input addition without carry-in

- **Subtractors:** Complement-based subtraction with borrow handling
- **Add/Subtract Units:** Combined addition/subtraction functionality

1.3.1.2 High-Performance Parallel Adders

- **Kogge-Stone Adders:** Minimal depth parallel prefix adders for maximum speed
- **Brent-Kung Adders:** Area-efficient parallel prefix adders with reasonable speed
- **Carry-Lookahead:** Fast carry generation for moderate bit widths
- **Carry-Save:** Multiple operand addition with reduced carry propagation

1.3.1.3 Multiplication Architectures

- **Wallace Tree:** Fast parallel multiplier using carry-save addition
- **Dadda Tree:** Optimized multiplier with reduced partial product stages
- **Basic Cell:** Fundamental multiplication building blocks

Design Considerations: - Speed vs Area trade-offs clearly documented - Power consumption characteristics provided - Technology-specific optimization guidance included

1.3.2 2. Data Integrity and Error Management

Critical for reliable data storage and transmission in modern digital systems.

1.3.2.1 Cyclic Redundancy Check (CRC)

- **Generic CRC Engine:** Configurable polynomial, width, and initialization
- **Cascade Implementation:** High-throughput parallel CRC processing
- **XOR-Shift Variants:** Optimized for specific polynomial types

1.3.2.2 Error Correction Coding (ECC)

- **Hamming SECDED:** Single Error Correction, Double Error Detection
- **Configurable Width:** Support for various data widths
- **Encoder/Decoder Pairs:** Complete ECC implementation

1.3.2.3 Simple Integrity Checks

- **Checksum:** Software-compatible checksum algorithms

- **Parity:** Even/odd parity generation and checking

Key Features: - Standards-compliant implementations - Parameterizable for various data widths - Optimized for both latency and throughput scenarios

1.3.3 3. Control Logic and Arbitration

Essential components for managing shared resources and system coordination.

1.3.3.1 Round-Robin Arbitration

- **Simple Round-Robin:** Basic fair arbitration with minimal overhead
- **Weighted Round-Robin:** Priority-based fair arbitration
- **Advanced Features:** Configurable agent counts and weighting

1.3.3.2 Priority-Based Control

- **Priority Encoders:** Fixed priority arbitration schemes
- **Configurable Priority:** Runtime priority adjustment capability

Applications: - Bus arbitration in multi-master systems - Resource allocation in processors - Quality of Service (QoS) implementations

1.3.4 4. Clock and Reset Management

Power and timing management for robust digital system design.

1.3.4.1 Clock Control

- **Integrated Clock Gating (ICG):** Power optimization through controlled clock disabling
- **Clock Dividers:** Frequency synthesis and clock domain generation
- **Pulse Generators:** Controlled clock pulse generation

1.3.4.2 Reset and Synchronization

- **Reset Synchronizers:** Safe reset assertion/deassertion
- **Glitch-Free Logic:** Reliable signal conditioning
- **Debouncing:** Input signal stabilization

Power Management Benefits: - Up to 40% dynamic power reduction with clock gating - Proper reset sequencing for reliable startup - Signal integrity maintenance

1.3.5 5. Counting and Sequence Generation

Fundamental building blocks for timing, addressing, and control.

1.3.5.1 Binary Counters

- **Basic Counters:** Up/down counting with enable/reset
- **Gray Code Counters:** Glitch-free counting for async interfaces
- **Johnson Counters:** Shift-based sequence generation

1.3.5.2 Specialized Counting

- **Frequency-Invariant:** Clock-independent counting
- **Load/Clear:** Programmable counter initialization
- **Ring Counters:** One-hot sequence generation

Design Features: - Parameterizable width and functionality - Optional synchronous/asynchronous controls - Overflow/underflow detection

1.3.6 6. Data Conversion and Encoding

Code conversion and display interfacing utilities.

1.3.6.1 Number System Conversion

- **Binary ↔ Gray:** Safe async counter interfacing
- **Binary → BCD:** Decimal display conversion
- **Johnson Gray:** Specialized counting applications

1.3.6.2 Encoding/Decoding

- **Priority Encoders:** Multiple input to binary conversion
- **Binary Decoders:** Address decoding and control
- **7-Segment Display:** Hexadecimal character display

Applications: - Display interfaces - Address decoding - Safe async signal crossing

1.3.7 7. Bit Manipulation and Searching

Efficient bit-level operations for high-performance computing.

1.3.7.1 Bit Detection

- **Leading Zero Count:** Normalization and floating-point support

- **Find First/Last Set:** Efficient bit searching
- **Priority Detection:** Leading one and trailing one detection

1.3.7.2 Vector Operations

- **Bit Reversal:** Endianness conversion and FFT applications
- **Vector Manipulation:** Bit-level data reorganization

Performance Characteristics: - Logarithmic complexity algorithms where possible - Optimized for synthesis tool recognition - Support for various data widths

1.3.8 8. Shift Operations and LFSRs

Shift-based operations and pseudorandom sequence generation.

1.3.8.1 Shifters

- **Barrel Shifters:** Arbitrary shift/rotate in single cycle
- **Universal Shifters:** Combined left/right, logical/arithmetic shifting

1.3.8.2 Linear Feedback Shift Registers (LFSRs)

- **Fibonacci LFSRs:** External feedback topology
- **Galois LFSRs:** Internal feedback topology
- **Configurable Polynomials:** Various sequence lengths and properties

Applications: - Pseudorandom number generation - Built-in self-test (BIST) patterns - Encryption and scrambling

1.3.9 9. Memory and Storage

Buffering and storage solutions for data flow management.

1.3.9.1 FIFO Implementations

- **Synchronous FIFO:** Single clock domain buffering
- **Asynchronous FIFO:** Clock domain crossing with Gray code pointers
- **Specialized FIFOs:** Optimized for specific clock relationships

1.3.9.2 Associative Memory

- **Content Addressable Memory (CAM):** Tag matching and lookup

Features: - Configurable depth and width - Full/empty flag generation - Error handling and overflow protection

1.3.10 10. Utility and Specialized Functions

Specialized functions for specific application domains.

1.3.10.1 *Signal Generation*

- **PWM:** Pulse width modulation for analog control
- **Timing Control:** Precise pulse and timing generation

1.3.10.2 *Data Processing*

- **Hardware Sorting:** Parallel sorting implementations
- **Algorithmic Processing:** Specialized computational functions

1.4 Performance and Optimization

1.4.1 Synthesis Guidelines

1.4.1.1 *Area Optimization*

- Use ripple-carry arithmetic for low-area requirements
- Select appropriate FIFO depths based on data flow analysis
- Consider shared resources for infrequently used functions

1.4.1.2 *Speed Optimization*

- Use parallel prefix adders (Kogge-Stone) for critical paths
- Implement pipelining in high-throughput applications
- Utilize barrel shifters for single-cycle shift operations

1.4.1.3 *Power Optimization*

- Apply clock gating (ICG) to inactive functional blocks
- Use Gray code counters for reduced switching activity
- Implement frequency-invariant designs where applicable

1.4.2 Technology-Specific Considerations

1.4.2.1 ASIC Implementation

- Leverage dedicated adder and multiplier cells
- Use technology-specific clock gating cells
- Optimize for specific voltage and temperature ranges

1.4.2.2 FPGA Implementation

- Utilize DSP blocks for arithmetic operations
- Use BRAM for deep FIFOs
- Leverage dedicated clock management resources

1.5 Verification and Validation

1.5.1 Testbench Coverage

- **Functional Coverage:** All operational modes and edge cases
- **Directed Tests:** Specific functionality verification
- **Random Testing:** Stress testing with random inputs
- **Corner Cases:** Boundary conditions and error scenarios

1.5.2 Compliance Testing

- **Timing Analysis:** Setup/hold requirements validation
- **Power Analysis:** Dynamic and static power characterization
- **Area Analysis:** Resource utilization across technologies

1.6 Usage Guidelines

1.6.1 Module Selection

1. **Identify Requirements:** Performance, area, and power constraints
2. **Select Architecture:** Choose appropriate implementation (ripple vs. parallel, etc.)
3. **Configure Parameters:** Set widths and options for specific application

4. Verify Integration: Ensure proper interfacing and timing

1.6.2 Integration Best Practices

// Example: Building a complete arithmetic unit

```
module arithmetic_unit #(
    parameter int DATA_WIDTH = 32
) (
    input logic clk,
    input logic rst_n,
    input logic [DATA_WIDTH-1:0] operand_a,
    input logic [DATA_WIDTH-1:0] operand_b,
    input logic [1:0] operation,
    output logic [DATA_WIDTH-1:0] result,
    output logic overflow
);

    // Use appropriate math modules based on requirements
    logic [DATA_WIDTH-1:0] add_result, sub_result, mult_result;
    logic add_carry, sub_borrow;

    // High-speed addition using parallel prefix
    math_adder_kogge_stone_nbit #(
        .N(DATA_WIDTH)
    ) u_adder (
        .a(operand_a), .b(operand_b), .cin(1'b0),
        .sum(add_result), .cout(add_carry)
    );

    // Subtraction using complement method
    math_subtractor_full_nbit #(
        .N(DATA_WIDTH)
    ) u_subtractor (
        .a(operand_a), .b(operand_b), .bin(1'b0),
        .diff(sub_result), .bout(sub_borrow)
    );

    // Result selection based on operation
    always_comb begin
        case (operation)
            2'b00: begin result = add_result; overflow = add_carry;
end
            2'b01: begin result = sub_result; overflow = sub_borrow;
end
            default: begin result = '0; overflow = 1'b0; end
        endcase
    end
```

`endmodule`

1.7 Future Enhancements

1.7.1 Planned Additions

- **DSP Functions:** FFT, filtering, and signal processing primitives
- **Advanced Arithmetic:** Division, square root, and trigonometric functions
- **Security Primitives:** Hardware security modules and cryptographic functions
- **AI/ML Accelerators:** Matrix operations and neural network primitives

1.7.2 Technology Evolution

- **Emerging Technologies:** Support for new process nodes and device types
- **Tool Integration:** Enhanced synthesis and verification tool compatibility
- **Standards Compliance:** Updates for evolving industry standards

The RTLCommon library represents a comprehensive foundation for digital design, providing the building blocks necessary for implementing complex digital systems with confidence in their correctness, performance, and manufacturability.

2 Priority Encoder Arbiter

A high-speed combinational priority encoder optimized for arbitration applications, implementing fixed-priority arbitration where lower-indexed clients always have higher priority.

2.1 Overview

The `arbiter_priority_encoder` module provides single-cycle priority encoding with optimized implementations for common client counts (4, 8, 16, 32) using fully unrolled casez logic. For other client counts, it uses a synthesizable loop-based approach. The module supports dual request inputs (masked and unmasked) for use in sophisticated arbitration schemes.

2.2 Module Declaration

```
module arbiter_priority_encoder #(
    parameter int CLIENTS = 4,           // Number of requesting
clients
    parameter int N = $clog2(CLIENTS)    // Winner ID width (derived)
) (
```

```

    input logic [CLIENTS-1:0] requests_masked,      // Masked
request vector
    input logic [CLIENTS-1:0] requests_unmasked,    // Unmasked
request vector
    input logic                                any_masked_requests, // Select masked
vs unmasked
    output logic [N-1:0] winner,                    // Binary-
encoded winner ID
    output logic                                winner_valid      // Winner output
is valid
);

```

2.3 Parameters

2.3.1 User-Settable Parameters

Parameter	Type	Default	Description
CLIENTS	int	4	Number of requesting clients (range: 2-1024)

2.3.2 Derived Localparam (Computed Internally)

Localparam	Computation	Description
N	\$clog2(CLIENTS)	Winner ID width in bits

Note: N is computed automatically and cannot be overridden by users.

2.4 Ports

2.4.1 Inputs

Port	Width	Description
requests_masked	CLIENTS	Masked request vector (higher priority source)
requests_unmasked	CLIENTS	Unmasked request vector (fallback source)
any_masked_requests	1	Select masked (1) or unmasked (0) requests

2.4.2 Outputs

Port	Width	Description
winner	N	Binary-encoded winner client ID
winner_valid	1	Asserted when at least one request is active

2.5 Functionality

2.5.1 Priority Order

The module implements **fixed priority** arbitration: - **Client 0**: Highest priority (always wins if requesting) - **Client 1**: Second highest priority - ... - **Client CLIENTS-1**: Lowest priority

2.5.2 Request Selection

The module evaluates requests in priority order:

1. If **any_masked_requests** = 1: Use requests_masked input
2. If **any_masked_requests** = 0: Use requests_unmasked input
3. Find **lowest-indexed asserted bit** in selected request vector
4. If **no requests**: winner = 0 (don't care), winner_valid = 0

2.5.3 Combinational Operation

- **Zero cycles latency**: Purely combinational logic
- **No clock or reset**: Outputs update immediately with input changes
- **Single-cycle response**: Winner determined in one combinational path

2.6 Implementation Details

2.6.1 Optimized Implementations

The module uses different implementations based on client count:

Power-of-2 Counts (4, 8, 16, 32):

```
// Fully unrolled casez logic for maximum performance
always_comb begin
    casez (w_priority_requests)
```



```

        4'b???1: begin winner = 2'd0; winner_valid = 1'b1; end //
Client 0
        4'b??10: begin winner = 2'd1; winner_valid = 1'b1; end //
Client 1
        4'b?100: begin winner = 2'd2; winner_valid = 1'b1; end //
Client 2
        4'b1000: begin winner = 2'd3; winner_valid = 1'b1; end //
Client 3
        default: begin winner = 2'd0; winner_valid = 1'b0; end
    endcase
end

```

Other Client Counts:

```

// Generic loop-based implementation
always_comb begin
    winner = '0;
    winner_valid = 1'b0;

    // Find lowest-indexed requesting client
    for (int i = 0; i < CLIENTS; i++) begin
        if (w_priority_requests[i] && !winner_valid) begin
            winner = i[N-1:0];
            winner_valid = 1'b1;
        end
    end
end

```

2.6.2 Priority Selection Logic

```

// Internal priority request selection
logic [CLIENTS-1:0] w_priority_requests;
assign w_priority_requests = any_masked_requests ?
    requests_masked :
    requests_unmasked;

```

2.7 Performance Characteristics

2.7.1 Timing

Metric	Optimized (4,8,16,32)	Generic (<64)	Generic (>64)
Latency	0 cycles (combinational)	0 cycles	0 cycles
Propagation Delay	1-2 LUT levels	2-3 LUT levels	3-4 LUT levels
Max	>500 MHz (FPGA)	>400 MHz	>300 MHz

Metric	Optimized (4,8,16,32)	Generic (<64)	Generic (>64)
Frequency			

2.7.2 Resource Utilization

Client Count	Implementation	LUTs (Est.)	Area Complexity
4	Optimized	~8	O(CLIENTS)
8	Optimized	~16	O(CLIENTS)
16	Optimized	~32	O(CLIENTS)
32	Optimized	~64	O(CLIENTS)
Other	Generic	~CLIENTS×log(CLIENTS)	O(CLIENTS×log(CLIENTS))

2.8 Usage Examples

2.8.1 Standalone Priority Encoder

```
// Simple fixed-priority arbiter (single request input)
arbiter_priority_encoder #(
    .CLIENTS(8)
) u_pri_enc (
    .requests_masked    (8'b0),           // Unused
    .requests_unmasked (client_reqs),     // Primary request vector
    .any_masked_requests(1'b0),          // Always use unmasked
    .winner             (winner_id),
    .winner_valid       (grant_valid)
);

// Convert winner ID to one-hot grant signal
logic [7:0] grant_one_hot;
always_comb begin
    grant_one_hot = '0;
    if (grant_valid) begin
        grant_one_hot[winner_id] = 1'b1;
    end
end
```

2.8.2 Dual-Input Priority Encoder

```
// Masked + unmasked request inputs
arbiter_priority_encoder #(
    .CLIENTS(16)
) u_pri_enc_dual (
    .requests_masked    (masked_reqs),    // Higher priority requests
```

```

        .requests_unmasked (all_reqs),      // All requests
        .any_masked_requests(|masked_reqs), // Select masked if any
present
        .winner            (winner_id),
        .winner_valid      (grant_valid)
    );

```

2.8.3 Integration in Round-Robin Arbiter

```

// Used within arbiter_round_robin.sv
// Masked requests = clients above last winner
// Unmasked requests = all clients
arbiter_priority_encoder #(
    .CLIENTS(CLIENTS)
) u_priority_encoder (
    .requests_masked      (w_requests_masked),
    .requests_unmasked    (w_requests_unmasked),
    .any_masked_requests  (w_any_masked_requests),
    .winner               (w_winner),
    .winner_valid         (w_winner_valid)
);

```

2.8.4 Multi-Level Priority Groups

```

// Implement priority groups: High (C0-C3), Med (C4-C7), Low (C8-C11)
logic [3:0] high_reqs, med_reqs, low_reqs;
logic [1:0] high_winner, med_winner, low_winner;
logic high_valid, med_valid, low_valid;

// Separate encoders for each priority group
arbiter_priority_encoder #(.CLIENTS(4)) u_high_pri (
    .requests_masked(4'b0),
    .requests_unmasked(high_reqs),
    .any_masked_requests(1'b0),
    .winner(high_winner),
    .winner_valid(high_valid)
);

arbiter_priority_encoder #(.CLIENTS(4)) u_med_pri (
    .requests_masked(4'b0),
    .requests_unmasked(med_reqs),
    .any_masked_requests(1'b0),
    .winner(med_winner),
    .winner_valid(med_valid)
);

arbiter_priority_encoder #(.CLIENTS(4)) u_low_pri (
    .requests_masked(4'b0),
    .requests_unmasked(low_reqs),

```

```

        .any_masked_requests(1'b0),
        .winner(low_winner),
        .winner_valid(low_valid)
    );

    // Select winner from highest priority group with requests
    logic [3:0] final_winner;
    always_comb begin
        if (high_valid)      final_winner = {2'b00, high_winner};
        else if (med_valid)  final_winner = {2'b01, med_winner};
        else if (low_valid)  final_winner = {2'b10, low_winner};
        else                 final_winner = 4'b0;
    end

```

2.9 Design Considerations

2.9.1 When to Use

✓ **Appropriate Use Cases:** - Fixed-priority arbitration systems - Interrupt controllers - Bus arbitration with priority levels - Resource allocation with service levels - Component within round-robin or weighted arbiters

2.9.2 Performance Optimization

For High-Speed Designs: - Use power-of-2 client counts (4, 8, 16, 32) for optimized implementation - Pipeline outputs if critical path is too long - Consider hierarchical encoding for >32 clients

For Area Optimization: - Non-power-of-2 counts use smaller generic implementation - Share encoder between multiple arbitration paths - Use smallest practical CLIENTS parameter

2.9.3 Synthesis Considerations

Timing Optimization:

```

    // Pipeline option for high-speed designs
    logic [N-1:0] r_winner;
    logic r_winner_valid;

    always_ff @(posedge clk) begin
        r_winner <= winner;
        r_winner_valid <= winner_valid;
    end

```

Resource Optimization: - Optimized implementations use casez (efficient mux tree) - Generic implementation uses priority loop (more logic levels) - For CLIENTS > 32, consider hierarchical structure

2.10 Priority Arbitration Theory

2.10.1 Fixed Priority vs Round-Robin

Aspect	Fixed Priority	Round-Robin
Fairness	No (low-indexed clients favored)	Yes (all clients equal)
Latency	Deterministic for high-priority	Variable
Starvation	Possible for low-priority	Prevented
Simplicity	Very simple (this module)	More complex
Best Use	Interrupt handling, critical tasks	Data paths, fair scheduling

2.10.2 Priority Inversion

Issue: High-priority client blocked by low-priority client holding resource

Mitigation Strategies: 1. **Priority Inheritance:** Temporarily boost blocked client's priority 2. **Resource Preemption:** Force low-priority client to release resource 3. **Priority Ceiling:** Limit maximum priority of resource holders

2.11 Example Priority Scenarios

2.11.1 Scenario 1: Single Requester (Client 2)

```
requests_unmasked = 8'b0000_0100  
any_masked_requests = 0
```

```
→ winner = 3'd2  
→ winner_valid = 1
```

2.11.2 Scenario 2: Multiple Requesters (Clients 0, 2, 5)

```
requests_unmasked = 8'b0010_0101  
any_masked_requests = 0
```

```
→ winner = 3'd0 (lowest index wins)  
→ winner_valid = 1
```

2.11.3 Scenario 3: Masked Priority (Clients 3, 6 masked)

```
requests_masked    = 8'b0100_1000  
requests_unmasked = 8'b0110_1101  
any_masked_requests = 1
```

→ winner = 3'd3 (from masked, index 3 < 6)
→ winner_valid = 1

2.11.4 Scenario 4: No Requests

requests_unmasked = 8'b0000_0000
any_masked_requests = 0

→ winner = 3'd0 (don't care)
→ winner_valid = 0

2.12 Verification Notes

From comprehensive test suite (val/common/test_arbiter_priority_encoder.py):

- **Coverage:** 95%
- **Key Test Scenarios:**
 - Priority order verification (all bit patterns)
 - Masked vs unmasked selection
 - No requests (winner_valid = 0)
 - All clients requesting simultaneously
 - Single client at a time
 - Power-of-2 sizes (4, 8, 16, 32)
 - Non-power-of-2 sizes (5, 12)

Test Command: `pytest val/common/test_arbiter_priority_encoder.py -v`

2.13 Common Pitfalls

x Anti-Pattern 1: Assuming fairness

*// WRONG: Fixed priority is NOT fair
// Client 0 will always win if requesting*

x Anti-Pattern 2: Ignoring winner_valid

*// WRONG: Using winner when no requests
if (winner == 2'd0) ... // Ambiguous: valid client 0 or no requests?*

*// RIGHT: Check winner_valid first
if (winner_valid && winner == 2'd0) ...*

✗ **Anti-Pattern 3:** Mixing priority schemes

```
// WRONG: Trying to implement round-robin with priority encoder alone
// Use arbiter_round_robin.sv instead
```

2.14 Related Modules

- **arbiter_round_robin.sv** - Uses this module internally for winner selection
- **arbiter_round_robin_simple.sv** - Alternative simplified arbiter
- **arbiter_round_robin_weighted.sv** - Weighted arbitration with QoS
- **encoder.sv** - General-purpose binary encoder
- **priority_encoder.sv** - Alternative naming (may be same module)

2.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

3 Round Robin Arbiter

3.1 Purpose

The `arbiter_round_robin` module implements a fair round-robin arbitration scheme for multiple clients. It ensures that all requesting clients receive equal access to a shared resource by cycling through them in order, preventing starvation of any client.

3.2 Parameters

- **CLIENTS**: Number of clients (default: 4)
- **WAIT_GNT_ACK**: When set to 1, waits for grant acknowledgment before moving to next client (default: 0)
- **N**: Internal parameter for address width calculation ($\lceil \log_2(\text{CLIENTS}) \rceil$)

3.3 Ports

3.3.1 Inputs

- **clk**: System clock

- `rst_n`: Active-low asynchronous reset
- `block_arb`: When asserted, blocks all arbitration (forces no grants)
- `request[CLIENTS-1:0]`: Request signals from each client
- `grant_ack[CLIENTS-1:0]`: Grant acknowledgment signals (used when `WAIT_GNT_ACK = 1`)

3.3.2 Outputs

- `grant_valid`: Indicates when a valid grant is being issued
- `grant[CLIENTS-1:0]`: One-hot encoded grant signals
- `grant_id[N-1:0]`: Binary encoded ID of the granted client
- `last_grant[CLIENTS-1:0]`: Previous cycle's grant (for debugging/history)

3.4 Key Internal Signals

- `r_last_grant_id`: Tracks the last winner's client ID (smaller than full mask)
- `r_last_valid`: Indicates if last winner should be used for mask generation
- `r_pending_ack`: ACK mode state (indicates ACK pending)
- `r_pending_client`: Which client has pending ACK (only in ACK mode)
- `w_requests_masked`: Requests with priority mask applied
- `w_requests_unmasked`: Raw gated requests without masking

3.5 Implementation Details

3.5.1 Priority Mechanism

The arbiter uses a pre-computed mask lookup table approach:

1. **Mask Lookup Tables**: Pre-computed at elaboration time (no logic cost)
 - `w_mask_decode[i]`: Mask for clients 0 through i (give priority after i)
 - `w_win_mask_decode[i]`: Mask to give priority to clients above i+1
2. **Fast Request Preprocessing**: Single LUT level for request gating

- Block arbitration immediately gates all requests
- Masked and unmasked request vectors computed in parallel

3. Last Winner Tracking: Uses client ID instead of full one-hot mask

- `r_last_grant_id`: More efficient than storing full grant vector
- `r_last_valid`: Indicates if mask should be applied

3.5.2 Priority Encoder

Uses `arbiter_priority_encoder` submodule for winner selection: - Takes both masked and unmasked request vectors - Returns binary-encoded winner ID - Outputs validity signal

3.5.3 Winner Selection Logic

```
// Priority encoder selects highest priority requester
arbiter_priority_encoder #(.CLIENTS(CLIENTS), .N(N))
u_priority_encoder (
    .requests_masked      (w_requests_masked),
    .requests_unmasked    (w_requests_unmasked),
    .any_masked_requests  (w_any_masked_requests),
    .winner               (w_winner),
    .winner_valid         (w_winner_valid)
);

// Grant decision with ACK permission check
assign w_should_grant = w_winner_valid && w_any_requests &&
w_can_grant;
```

3.5.4 Mask Update Logic

- **No-ACK Mode:** Mask updates immediately when grant issued (1-cycle round-robin)
- **ACK Mode:** Mask updates only when ACK received (prevents premature rotation)

3.6 Special Features

3.6.1 Block Arbitration

When `block_arb` is asserted, all requests are masked to zero, effectively disabling arbitration.

3.6.2 Grant Acknowledgment Support

When `WAIT_GNT_ACK` = 1, the arbiter waits for the granted client to acknowledge receipt before updating internal state and moving to the next client.

3.6.3 Fair Round-Robin Operation

The algorithm ensures fairness by: 1. Serving lower-indexed clients first within each round 2. After serving a client, giving priority to all lower-indexed clients 3. When no lower-indexed clients are requesting, wrapping around to serve from the top

3.7 Usage Notes

- The arbiter prioritizes lower-indexed clients when multiple requests arrive simultaneously
- The round-robin nature ensures long-term fairness across all clients
- Grant acknowledgment feature is useful in systems where the granted client needs time to process the grant

3.8 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

4 arbiter_round_robin_simple

A generic rotating-priority arbiter with masking/rotation that provides fair arbitration among multiple requesting agents without using if/case ladders in the priority path.

4.1 Overview

The `arbiter_round_robin_simple` module implements a round-robin arbitration scheme that ensures fair access to a shared resource among N requesting agents. It maintains fairness by remembering the last granted agent and starting the next arbitration from the next agent in sequence.

4.2 Module Declaration

```
module arbiter_round_robin_simple #(
    parameter int unsigned N = 4
) (
    input logic clk,
    input logic rst_n,           // active-low reset
    input logic [N-1:0] request, // request bits [N-
1:0]
    output logic grant_valid,    // any grant
    output logic [N-1:0] grant,  // one-hot grant
    output logic [$clog2(N)-1:0] grant_id // encoded grant
)
```

```
(undef if grant_valid==0)
);
```

4.3 Parameters

Parameter	Type	Default	Description
N	int unsigned	4	Number of requesting agents (N >= 1)

4.4 Ports

4.4.1 Inputs

Port	Width	Description
clk	1	Clock signal
rst_n	1	Active-low asynchronous reset
request	N	Request bits from N agents [N-1:0]

4.4.2 Outputs

Port	Width	Description
grant_valid	1	Indicates if any grant is active
grant	N	One-hot grant output [N-1:0]
grant_id	\$clog2(N)	Binary encoded grant ID (undefined if grant_valid==0)

4.5 Functionality

4.5.1 Round-Robin Algorithm

The arbiter implements a rotating priority scheme using the following approach:

1. **State Memory:** Maintains `r_last_grant` register storing the last granted agent ID
2. **Rotation:** Rotates the request vector to prioritize agents after the last granted one

3. **Priority Selection:** Uses lowest-set-bit isolation ($x \& (\sim x + 1)$) for efficient priority selection
4. **Restoration:** Rotates the result back to original bit positions

4.5.2 Key Features

- **Fair Arbitration:** Ensures all requesting agents get equal opportunity over time
- **Efficient Logic:** Uses bit manipulation instead of complex if/case structures
- **Single-Cycle Response:** Combinational grant generation with registered state update
- **Scalable:** Parameterizable for any number of agents ($N \geq 1$)

4.6 Implementation Details

4.6.1 Rotation Logic

```
// Shift amount = last_grant + 1 (mod N)
logic [$clog2(N)-1:0] w_shift_amount;
assign w_shift_amount = (r_last_grant == ($clog2(N))'(N-1)) ? '0 :
(r_last_grant + 1);

// Rotate-left request by w_shift_amount
if (w_shift_amount == '0) begin
    w_rot_req = request;
end else begin
    w_rot_req = (request << w_shift_amount) | (request >>
(($clog2(N))'(N) - w_shift_amount));
end
```

4.6.2 Priority Selection

The module uses lowest-set-bit isolation for efficient one-hot selection:

```
// Isolate lowest set bit (one-hot)
w_rot_sel = w_rot_req & ((~w_rot_req) + {{(N-1){1'b0}}, 1'b1});
```

4.6.3 Grant Encoding

A compact one-hot to binary encoder generates the grant ID:

```
always_comb begin
    w_grant_id = r_last_grant; // default to last grant
    for (int i = 0; i < N; i++) begin
        if (w_nxt_grant[i]) w_grant_id = i[$clog2(N)-1:0];
    end
end
```

end
end

4.7 Timing Characteristics

Characteristic	Value
Setup Time	1 clock cycle for state update
Response Time	Combinational (0 cycles)
Reset Recovery	1 clock cycle

4.8 Reset Behavior

- **Power-on Reset:** `r_last_grant` initialized to N-1, causing first arbitration to start from agent 0
- **Active Reset:** All state registers cleared, grant outputs become invalid
- **Reset Release:** Next arbitration cycle begins with agent 0 having highest priority

4.9 Usage Examples

4.9.1 Basic 4-Agent Arbiter

```
arbiter_round_robin_simple #(
    .N(4)
) u_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .request       (req_bits),           // [3:0]
    .grant_valid   (grant_valid),
    .grant         (grant_oh),           // [3:0] one-hot
    .grant_id      (grant_bin)          // [1:0] binary
);
```

4.9.2 8-Agent System

```
arbiter_round_robin_simple #(
    .N(8)
) u_8way_arbiter (
    .clk          (clk),
    .rst_n        (rst_n),
    .request       (agent_requests),    // [7:0]
    .grant_valid   (resource_granted),
    .grant         (agent_grants),      // [7:0]
```

```
    .grant_id    (winning_agent)    // [2:0]
);
```

4.10 Arbitration Sequence Example

For N=4 with requests [3:0]:

Cycle	Last Grant	Requests	Rotated	Selected	Grant	Grant ID
0	3 (init)	1111	1111	0001	0001	0
1	0	1110	1101	0001	0010	1
2	1	1101	0111	0001	0100	2
3	2	1011	1101	0001	1000	3
4	3	1111	1111	0001	0001	0

4.11 Design Considerations

4.11.1 Advantages

- **Simple Logic:** Uses efficient bit manipulation instead of complex priority encoders
- **Scalable:** Works for any number of agents with minimal logic overhead
- **Fair:** Guarantees round-robin fairness over time
- **Fast:** Single combinational path from request to grant

4.11.2 Limitations

- **Grant Undefined:** grant_id is undefined when grant_valid is low
- **Request Requirement:** At least one request must be active for meaningful arbitration
- **Power Consumption:** Rotational logic may consume more power than simpler priority schemes

4.12 Synthesis Considerations

- **Area:** Scales approximately as $O(N \log N)$ due to rotation logic
- **Timing:** Critical path through rotation and bit isolation logic
- **Power:** Barrel shifter logic may have higher switching activity

- **Optimization:** Consider using dedicated arbiter primitives for large N

4.13 Verification Notes

- **Reset Testing:** Verify proper initialization and first-grant behavior
- **Fairness Testing:** Confirm round-robin sequence over multiple cycles
- **Edge Cases:** Test behavior with single request, all requests, and no requests
- **Timing:** Verify setup/hold requirements for state register updates

4.14 Related Modules

- `arbiter_round_robin`: Enhanced round-robin arbiter with additional features
- `arbiter_priority_encoder`: Fixed priority arbiter implementation
- `encoder_priority_enable`: Priority encoding utility module

The `arbiter_round_robin_simple` module provides an efficient and fair arbitration solution suitable for most round-robin arbitration needs in digital systems.

4.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

5 Weighted Round Robin Arbiter

A credit-based weighted round-robin arbiter with Quality of Service (QoS) support, providing proportional bandwidth allocation to multiple clients while maintaining starvation-free operation.

5.1 Overview

The `arbiter_round_robin_weighted` module combines fair round-robin arbitration with configurable per-client weights to provide proportional bandwidth allocation. Each client receives grants proportional to its assigned weight while preventing starvation. The module supports optional acknowledgment protocol and dynamic weight changes with atomic update semantics.

5.2 Module Declaration

```
module arbiter_round_robin_weighted #(
    parameter int MAX_LEVELS = 16,           // Maximum weight value per
client
    parameter int CLIENTS = 4,               // Number of requesting
clients
    parameter int WAIT_GNT_ACK = 0           // Enable ACK protocol (0=No-
ACK, 1=ACK)
) (
    // Derived localparam (computed internally - not user-settable):
    // localparam int MAX_LEVELS_WIDTH = $clog2(MAX_LEVELS); //
Credit counter width
    // localparam int N = $clog2(CLIENTS); //
Client ID width
    // localparam int C = CLIENTS; //
Convenience alias
    // localparam int MTW = MAX_LEVELS_WIDTH; //
Weight width alias
    // localparam int CXMTW = CLIENTS * MAX_LEVELS_WIDTH; //
Packed weight array width

    input logic clk, // Clock input
    input logic rst_n, // Active-low
asynchronous reset
    input logic block_arb, // Block all
arbitration
    input logic [CXMTW-1:0] max_thresh, // Packed weight
values for all clients
    input logic [C-1:0] request, // Request signals
from clients
    input logic [C-1:0] grant_ack, // Grant
acknowledgment (ACK mode only)
    output logic grant_valid, // Grant output valid
    output logic [C-1:0] grant, // Grant vector (one-
hot)
    output logic [N-1:0] grant_id // Grant client ID
(binary encoded)
);
```

5.3 Parameters

5.3.1 User-Settable Parameters

Parameter	Type	Default	Description
MAX_LEVELS	int	16	Maximum weight value per client (range: 1-256)

Parameter	Type	Default	Description
CLIENTS	int	4	Number of requesting clients (range: 2-32)
WAIT_GNT_ACK	int	0	Enable ACK protocol (0=No-ACK, 1=ACK required)

5.3.2 Derived Localparam (Computed Internally)

Localparam	Computation	Description
MAX_LEVELS_WIDT H	$\$clog2(\text{MAX_LEVELS})$	Credit counter width in bits
N	$\$clog2(\text{CLIENTS})$	Client ID width in bits
C	CLIENTS	Convenience alias for port widths
MTW	MAX_LEVELS_WIDT H	Convenience alias for weight width
CXMTW	CLIENTS * MAX_LEVELS_WIDT H	Total packed weight array width

Note: MAX_LEVELS_WIDTH, N, C, MTW, and CXMTW are computed internally and cannot be overridden by users.

5.4 Ports

5.4.1 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
block_arb	1	Block all arbitration (external gate)
max_thresh	CXMTW	Packed weight values: {weight[N-1], ..., weight[1], weight[0]}
request	C	Request signals from clients
grant_ack	C	Grant acknowledgment signals (ACK mode only)

5.4.2 Outputs

Port	Width	Description
grant_valid	1	Valid grant indicator
grant	C	Grant vector (one-hot encoded)
grant_id	N	Grant client ID (binary encoded)

5.5 Functionality

5.5.1 Credit-Based Weighting

The arbiter uses a credit-based system for weighted bandwidth allocation:

1. **Credit Initialization:** Each client's credit counter is initialized to its weight value
2. **Grant Completion:** When a grant completes, the client's credit counter decrements
3. **Credit Eligibility:** Only clients with non-zero credits are eligible for round-robin arbitration
4. **Global Replenishment:** When no requesting clients have credits, all credits are reloaded

5.5.2 Arbitration Stages

1. **Credit Eligibility:** Filter requests to only clients with credits > 0
2. **Round-Robin Selection:** Base arbiter (arbiter_round_robin) selects among eligible clients
3. **Grant Output:** Winning client receives one-hot grant signal
4. **Credit Update:** Winner's credit decrements on completion (immediate or ACK-based)

5.5.3 Weight Ratio Example

For weights [4, 2, 1, 1]: - **Client 0:** 4 credits → gets 4 consecutive grants - **Client 1:** 2 credits → gets 2 consecutive grants - **Client 2:** 1 credit → gets 1 grant - **Client 3:** 1 credit → gets 1 grant - **Pattern:** C0, C0, C0, C0, C1, C1, C2, C3, [replenish], repeat... - **Bandwidth:** C0=50%, C1=25%, C2=12.5%, C3=12.5%

5.6 Dynamic Weight Changes

5.6.1 Weight Management FSM

The module includes a finite state machine for atomic weight updates:

State	Description
WEIGHT_IDLE	Normal operation
WEIGHT_BLOCK	Detect weight change, block new grants
WEIGHT_DRAIN	Wait for pending ACK completion (2 cycles)
WEIGHT_UPDATE	Atomic weight update, reset credits
WEIGHT_STABILIZE	System stabilization (3 cycles)

5.6.2 Weight Update Sequence

1. **Detection:** FSM detects max_thresh change from shadow register
2. **Block:** New grants blocked, pending grants drain
3. **Drain:** Wait for ACK completion (ACK mode) or fixed delay
4. **Update:** Atomic update of weights, credit counters reset
5. **Stabilize:** System stabilization period
6. **Resume:** Return to normal operation

Total Latency: 5-15 cycles depending on pending ACKs

5.6.3 Safety Features

- **Shadow Register:** Active weights stored in r_safe_max_thresh for race-free updates
- **Timeout Protection:** 15-cycle timeout prevents FSM lockup
- **Credit Reset:** Credits reset to new weights during WEIGHT_STABILIZE state
- **ACK Awareness:** FSM waits for pending grant_ack before updating weights

5.7 Implementation Details

5.7.1 Credit Management

```
// Credit decrement on grant completion
if (w_grant_completed[i] && r_credit_counter[i] > 0) begin
    w_credit_counter[i] = r_credit_counter[i] - 1;
end
```

Grant Completion Modes: - **No-ACK Mode:** Completion when grant[i] && grant_valid (same cycle) - **ACK Mode:** Completion when grant[i] && grant_valid && grant_ack[i] (delayed)

5.7.2 Global Replenishment Logic

```
// Replenish when no requesting clients have credits
assign w_global_replenish = (w_normal_operation && !w_pending_grants
&&
                                (!request) && !
w_requesting_clients_with_credits);

// On replenish: reload all credit counters to their weights
if (w_global_replenish) begin
    w_credit_counter[i] = client_weight[i];
end
```

5.7.3 Request Masking for Fairness

```
// Only pass eligible clients to round-robin arbiter
assign w_requesting_eligible[i] = request[i] &&
                                ((w_has_crd[i]) ||
                                (w_global_replenish &&
client_weight[i] > 0));

// Apply masking for fairness among multiple eligible clients
assign w_mask_req[i] = (multiple_eligible) ?
                        (w_requesting_eligible[i] && !grant[i]) :
                        (w_requesting_eligible[i] &&
r_credit_counter[i] > 1);
```

5.7.4 Sub-Module: arbiter_round_robin

Uses base arbiter_round_robin module internally for fair selection among eligible clients:

```
arbiter_round_robin #(
    .CLIENTS      (CLIENTS),
    .WAIT_GNT_ACK (WAIT_GNT_ACK)
) u_base_arbiter (
    .clk      (clk),
    .rst_n    (rst_n),
```

```

        .block_arb    (w_sub_block_arb),
        .request      (w_mask_req),
        .grant_ack    (grant_ack),
        .grant_valid  (grant_valid),
        .grant        (grant),
        .grant_id     (grant_id),
        .last_grant   (r_last_grant)
    );

```

5.8 Special Features

5.8.1 Zero-Weight Client Handling

Clients with `max_thresh = 0` are effectively disabled: - Never accumulate credits - Never participate in arbitration - Credit counters remain at zero

5.8.2 ACK Protocol Support

When `WAIT_GNT_ACK = 1`: - Grant remains asserted until `grant_ack` received - Credit decrement waits for ACK - Weight change FSM waits for pending ACKs - Enables pipelined/posted transaction masters

When `WAIT_GNT_ACK = 0`: - Grant completes immediately (1 cycle) - Credit decrements same cycle as grant - Optimal for simple masters (no pipeline)

5.8.3 Starvation Prevention

The arbiter guarantees starvation-free operation: - All non-zero weight clients eventually served - Maximum starvation time: Sum of all other clients' weights - Global replenishment ensures fairness

5.9 Usage Examples

5.9.1 Basic QoS Arbiter (No-ACK Mode)

```

// 4-client arbiter with QoS weights: 8:4:2:1
localparam int NUM_CLIENTS = 4;
localparam int MAX_WEIGHT = 16;
localparam int WEIGHT_W = $clog2(MAX_WEIGHT);

logic [NUM_CLIENTS-1:0] client_req, client_grant;
logic grant_vld;
logic [$clog2(NUM_CLIENTS)-1:0] grant_idx;
logic [NUM_CLIENTS*WEIGHT_W-1:0] qos_weights;

// Assign QoS weights (high priority = higher weight)
assign qos_weights = {4'd1, 4'd2, 4'd4, 4'd8}; // {C3, C2, C1, C0}

```

```

arbiter_round_robin_weighted #(
    .CLIENTS(NUM_CLIENTS),
    .MAX_LEVELS(MAX_WEIGHT),
    .WAIT_GNT_ACK(0)           // No-ACK mode
) u_qos_arbiter (
    .clk            (clk),
    .rst_n          (rst_n),
    .block_arb      (1'b0),
    .max_thresh     (qos_weights),
    .request        (client_req),
    .grant_ack      ('0),      // Unused in No-ACK mode
    .grant_valid    (grant_vld),
    .grant          (client_grant),
    .grant_id       (grant_idx)
);

```

5.9.2 Dynamic Weight Adjustment

```

// Adjust weights based on priority events
logic [NUM_CLIENTS*WEIGHT_W-1:0] r_qos_weights;

always_ff @(posedge clk) begin
    if (high_priority_event) begin
        r_qos_weights[7:0] <= 4'd15; // Boost Client 0 weight
    end else if (normal_priority) begin
        r_qos_weights[7:0] <= 4'd4;  // Reset Client 0 weight
    end
end

```

```

arbiter_round_robin_weighted #(
    .CLIENTS(NUM_CLIENTS),
    .MAX_LEVELS(MAX_WEIGHT),
    .WAIT_GNT_ACK(0)
) u_dynamic_arbiter (
    .clk            (clk),
    .rst_n          (rst_n),
    .block_arb      (1'b0),
    .max_thresh     (r_qos_weights), // Dynamic weights
    .request        (client_req),
    .grant_ack      ('0),
    .grant_valid    (grant_vld),
    .grant          (client_grant),
    .grant_id       (grant_idx)
);

```

5.9.3 ACK Mode for Pipelined Masters

```

// ACK mode arbiter for pipelined masters
arbiter_round_robin_weighted #(

```

```

        .CLIENTS(2),
        .MAX_LEVELS(8),
        .WAIT_GNT_ACK(1)           // ACK required
    ) u_ack_arbiter (
        .clk          (clk),
        .rst_n        (rst_n),
        .block_arb     (bus_busy),
        .max_thresh    ({4'd3, 4'd5}), // Weights [5, 3]
        .request       (m_req),
        .grant_ack     (m_done),       // Master completion signal
        .grant_valid   (m_grant_vld),
        .grant         (m_grant),
        .grant_id      (m_id)
    );

```

5.10 Timing Characteristics

5.10.1 Latency and Throughput

- **Latency:** 2 cycles (credit calculation + round-robin arbitration)
- **Throughput:** 1 grant per cycle (maximum)
- **Grant Hold:** No-ACK=1 cycle, ACK=Until grant_ack asserted
- **Weight Update:** 5-15 cycles (FSM: BLOCK→DRAIN→UPDATE→STABILIZE)
- **Reset:** Asynchronous (all credits→1, weights→default)

5.10.2 Critical Paths

1. **Credit Comparison:** Credit counter → eligibility logic
2. **Request Filtering:** Eligible clients → masked requests
3. **Round-Robin:** Base arbiter selection logic

5.11 Design Considerations

5.11.1 When to Use Each Mode

No-ACK Mode (WAIT_GNT_ACK=0): - ✓ Simple masters without pipelining - ✓ Combinational logic masters - ✓ Maximum throughput (1 grant/cycle) - ⚠ Grant completes same cycle (no hold)

ACK Mode (WAIT_GNT_ACK=1): - ✓ Pipelined masters (e.g., AXI) - ✓ Posted transaction buses - ✓ Multi-cycle operations - ⚠ Reduced throughput (grant held until ACK)

5.11.2 Weight Selection Guidelines

- **Finer QoS Control:** Increase MAX_LEVELS (but more credit counter bits)
- **Typical Range:** MAX_LEVELS=8 to 32 for most applications
- **Weight Ratio:** Should reflect desired bandwidth ratios
- **Zero Weights:** Effectively disable clients (never granted)

5.11.3 Dynamic Weight Change Best Practices

-  **DO NOT** change weights every cycle (causes FSM thrashing)
- ✓ **Recommended:** Change weights on policy updates (millisecond timescale)
- ✓ **Safe:** FSM ensures atomic updates without race conditions
- ✓ **Timeout:** 15-cycle timeout prevents lockup

5.12 Performance Characteristics

5.12.1 Resource Utilization

CLIENTS	MAX_LEVELS	FFs (Est.)	LUTs (Est.)
4	16	~40	~50
8	16	~70	~90
4	32	~45	~55
8	32	~80	~110

5.12.2 Maximum Frequency

- **Typical:** 300-500 MHz (modern FPGAs)
- **Limiting Factor:** Credit comparison and request filtering logic

5.13 Verification Notes

From the comprehensive test suite (val/common/test_arbiter_round_robin_weighted.py):

- **Coverage:** 94%
- **Key Test Scenarios:**
 - Weight ratio verification ([4,2,1,1] patterns)

- Global credit replenishment
- Dynamic weight changes (atomic updates)
- ACK mode timing and weight updates
- Zero weight handling (client disable)
- Single client optimization
- FSM timeout protection
- Starvation prevention

Test Command: `pytest val/common/test_arbiter_round_robin_weighted.py -v`

5.14 Related Modules

- **arbiter_round_robin.sv** - Base round-robin arbiter (used internally)
- **arbiter_round_robin_simple.sv** - Lightweight RR arbiter (no weights)
- **arbiter_priority_encoder.sv** - Fixed priority arbiter

5.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

6 Single-Client Arbiter

Module: `arbiter_single_client.sv` **Location:** `rtl/common/` **Status:** Production Ready

6.1 Overview

`arbiter_single_client` is the degenerate, one-requester member of the round-robin arbiter family. It exists to give a single-channel datapath the **exact same registered, ack-held grant lifecycle** as `arbiter_round_robin` running in `WAIT_GNT_ACK` mode, without the parameter degeneracy that breaks the multi-client arbiter when `CLIENTS == 1`.

`arbiter_round_robin` underflows at `CLIENTS == 1` because `$clog2(1) = 0` produces a `grant_id[-1:0]` slice. Historically single-channel builds worked around this by substituting a purely *combinational* passthrough (`grant = request`). That passthrough does **not** reproduce the arbiter's registered grant/ack handshake, so it injects a bubble beat at every burst boundary in the

AXI read/write engines that depend on the “request, grant, hold-for-ack” timing. This module is the faithful single-client reduction of those rules.

6.1.1 Key Features

- **Timing-faithful:** Mirrors `arbiter_round_robin WAIT_GNT_ACK always_ff` rules exactly for one client
- **Registered grant:** Grant is a flip-flop output, held until acknowledged (no combinational passthrough bubble)
- **Ack-held lifecycle:** Grant persists until `grant_ack`, then clears and can re-grant the next cycle
- **Bubble-free bursts:** Preserves the grant timing the AXI read/write engines rely on for back-to-back burst beats
- **Optional ack mode:** `WAIT_GNT_ACK = 0` falls back to a simple registered request-to-grant with no ack hold
- **Trivial decode:** `grant` is a one-hot of width 1 (equal to `grant_valid`); `grant_id` is always 0

6.2 Module Purpose

The module provides a drop-in single-client arbiter so that a design parameterized down to one channel behaves identically to the same design with two or more channels. Rather than special-casing the `NUM_CHANNELS == 1` build with a passthrough, the datapath instantiates `arbiter_single_client` and gets the identical grant/ack sequencing the general arbiter would have produced for one requester.

Use Cases:

- Single-channel builds of a datapath that is otherwise parameterized for N channels
- AXI read/write engines that require the registered, ack-held grant handshake for correct burst sequencing
- Any place a `arbiter_round_robin` instance would be used, but with `CLIENTS == 1`
- Replacing a combinational `grant = request` passthrough that was injecting per-burst bubble beats

Key Benefit: A one-channel build behaves bit-for-bit like the multi-channel arbiter path, eliminating the burst-boundary bubble that a combinational passthrough introduced.

6.3 Parameters

Parameter	Type	Default	Description
WAIT_GNT_ACK	int	1	1 = registered grant is held until grant_ack is received (round-robin ACK lifecycle). 0 = simple registered request-to-grant with no ack hold.

6.4 Port Groups

6.4.1 Clock and Reset

Port	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Active-low asynchronous reset

6.4.2 Control

Port	Direction	Width	Description
block_arb	Input	1	Block arbitration; when high the request is masked and no grant is issued
request	Input	1	The single client's request
grant_ack	Input	1	The single client's grant acknowledgment (used only when WAIT_GNT_ACK = 1)

6.4.3 Grant Outputs

Port	Direction	Width	Description
grant_val_id	Output	1	Grant valid (registered)

Port	Direction	Width	Description
grant	Output	1	One-hot grant of width 1; equal to grant_valid
grant_id	Output	1	Granted client index; always 0 for a single client

6.5 Functional Description

6.5.1 Request Qualification

The incoming request is first masked by block_arb:

```
assign w_req = request && !block_arb;
```

A blocked arbiter therefore behaves as if the client were not requesting.

6.5.2 Grant-Eligibility Logic (ACK mode)

When WAIT_GNT_ACK = 1, three combinational signals gate whether a new grant may be issued:

```
assign w_ack_received = r_pending_ack && grant_ack;           // ack
// seen for the outstanding grant
assign w_can_grant     = !r_pending_ack || w_ack_received;    // free
// to grant, or ack just closed the last grant
assign w_should_grant = w_req && w_can_grant;
```

r_pending_ack records that a grant has been issued and is awaiting its acknowledgment. A new grant can only be launched when there is no pending ack, or when the ack for the current grant arrives this cycle.

6.5.3 Grant Lifecycle (ACK mode)

The registered state machine is the single-client reduction of the arbiter_round_robin WAIT_GNT_ACK rules. Because there is only one client, w_other_requests is always 0, so the arbiter's Rule 4 (switch to another requester) never applies and Rule 3 simply clears:

State	Condition	Action	grant_valid next	r_pending_ack next
Rule 1	grant_val id == 0	Not granted; grant if request ing	w_should_grant	w_should_grant
Rule 2	granted, ! w_ack_rec	Hold the	1	1

State	Condition	Action	grant_valid next	r_pending_ack next
Rule 3	eived	grant		
	granted, w_ack_rec eived	Ack receive d; clear (re- grant next cycle)	0	0

The grant is thus **registered and held** across cycles until the client acknowledges, then it drops for one cycle before it can be re-issued.

6.5.4 No-ACK Fallback Mode

When WAIT_GNT_ACK = 0, the ack machinery is bypassed. grant_valid simply follows a registered version of the qualified request and r_pending_ack stays 0:

```
grant_valid    <= w_should_grant;    // w_can_grant is constant 1 in
this mode
r_pending_ack  <= 1'b0;
```

This gives a one-cycle registered request-to-grant with no hold.

6.5.5 Output Mapping

```
assign grant    = grant_valid;    // one-hot of width 1
assign grant_id = 1'b0;           // single client
```

Because there is exactly one client, grant and grant_valid are identical, and grant_id is a constant 0.

6.6 Usage Example

```
// Single-channel build: use arbiter_single_client where a >1-channel
// build would instantiate arbiter_round_robin in WAIT_GNT_ACK mode.
arbiter_single_client #(
    .WAIT_GNT_ACK(1)
) u_arb (
    .clk          (aclk),
    .rst_n        (aresetn),
    .block_arb    (arb_blocked),
    .request       (engine_request),
    .grant_ack     (engine_grant_ack),
    .grant_valid   (engine_grant_valid),
    .grant         (engine_grant),    // width-1 one-hot
    .grant_id      ()                // always 0, usually unconnected
);
```

6.7 Design Notes

- **Why not a combinational passthrough?** `grant = request` looks correct for one client but loses the registered, ack-held timing. The AXI read/write engines interpret the missing hold as an early grant drop and insert a bubble beat at every burst boundary. This module preserves the exact cycle-by-cycle behavior.
- **grant_id degeneracy.** The general arbiter's `grant_id` is $\$clog2(CLIENTS)$ bits wide, which collapses to an illegal `[-1:0]` slice at `CLIENTS == 1`. Here `grant_id` is simply tied to 0, sidestepping the elaboration problem.
- **Block behavior.** `block_arb` masks the request combinationaly, so raising it prevents new grants; an already-pending ack-held grant continues to follow the state machine.
- **Reset.** Active-low asynchronous reset clears both `grant_valid` and `r_pending_ack`.

6.8 Related Modules

6.8.1 Used By

- Single-channel builds of AXI read/write datapath engines that otherwise instantiate `arbiter_round_robin`

6.8.2 Uses

- None (self-contained; no submodule instantiations)

6.8.3 See Also

- [arbiter_round_robin](#) - The general N-client arbiter whose `WAIT_GNT_ACK` timing this module reproduces for one client
- [arbiter_round_robin_simple](#) - Minimal-area round-robin variant
- [arbiter_round_robin_weighted](#) - Weighted / QoS round-robin variant
- [arbiter_priority_encoder](#) - Fixed-priority arbiter

6.9 References

6.9.1 Source Code

- `rtl/common/arbiter_single_client.sv`
- `rtl/common/arbiter_round_robin.sv` (timing reference)

6.9.2 Documentation

- `rtl/common/PRD.md`

Last Updated: 2026-07-15

6.10 Navigation

- [Back to RTLCommon Index](#)

7 Binary to Gray Code Converter

7.1 Overview

The `bin2gray` module implements a purely combinational binary-to-Gray code converter. Gray code (also known as reflected binary code or unit distance code) is a binary numeral system where two successive values differ in only one bit. This property makes Gray code essential for reducing glitches and metastability in digital systems, particularly in asynchronous designs and clock domain crossings.

7.2 Module Declaration

```
module bin2gray #(
    parameter int WIDTH = 4
) (
    input wire [WIDTH-1:0] binary,
    output wire [WIDTH-1:0] gray
);
```

7.3 Parameters

7.3.1 WIDTH

- **Type:** `int`
- **Default:** 4
- **Description:** Bit width of both input and output

- **Range:** Any positive integer ≥ 1
- **Common Values:** 4, 8, 16, 32 for address counters and data buses
- **Impact:** Determines the number of XOR gates required

7.4 Ports

7.4.1 Inputs

Port	Width	Type	Description
binary	WIDTH	wire	Input binary value

7.4.2 Outputs

Port	Width	Type	Description
gray	WIDTH	wire	Output Gray code value

7.5 Gray Code Theory

7.5.1 Mathematical Definition

Gray code conversion follows a simple mathematical relationship: - **MSB:** $\text{gray}[\text{WIDTH}-1] = \text{binary}[\text{WIDTH}-1]$ (MSB unchanged) - **Other bits:** $\text{gray}[i] = \text{binary}[i] \oplus \text{binary}[i+1]$ for $i = 0$ to $\text{WIDTH}-2$

7.5.2 Why Gray Code?

Gray code solves several problems in digital systems:

1. **Single Bit Transitions:** Adjacent values differ by exactly one bit
2. **Glitch Reduction:** Eliminates intermediate states during transitions
3. **Metastability Prevention:** Safer for asynchronous clock domain crossings
4. **Mechanical Encoders:** Natural for optical/magnetic position encoders

7.5.3 Gray Code Sequence Examples

7.5.3.1 2-bit Gray Code

Decimal	Binary	Gray	Transition
0	00	00	-

Decimal	Binary	Gray	Transition
1	01	01	bit 0
2	10	11	bit 1
3	11	10	bit 0

7.5.3.2 3-bit Gray Code

Decimal	Binary	Gray	Changed Bit
0	000	000	-
1	001	001	0
2	010	011	1
3	011	010	0
4	100	110	2
5	101	111	0
6	110	101	1
7	111	100	0

7.5.3.3 4-bit Gray Code (Complete Table)

Dec	Binary	Gray	Dec	Binary	Gray
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

7.6 Implementation Analysis

7.6.1 Core Logic

```

genvar i;
generate
    for (i = 0; i < WIDTH - 1; i++) begin : gen_gray
        assign gray[i] = binary[i] ^ binary[i+1];
    end
end

```

endgenerate

```
assign gray[WIDTH-1] = binary[WIDTH-1];
```

7.6.2 Logic Structure

The implementation creates a chain of XOR gates: - **Lower bits**: Each gray bit is XOR of current and next binary bits - **MSB**: Directly assigned from binary MSB - **Propagation**: Single level of logic (all XORs in parallel)

7.6.3 Gate-Level Implementation

For WIDTH = 4:

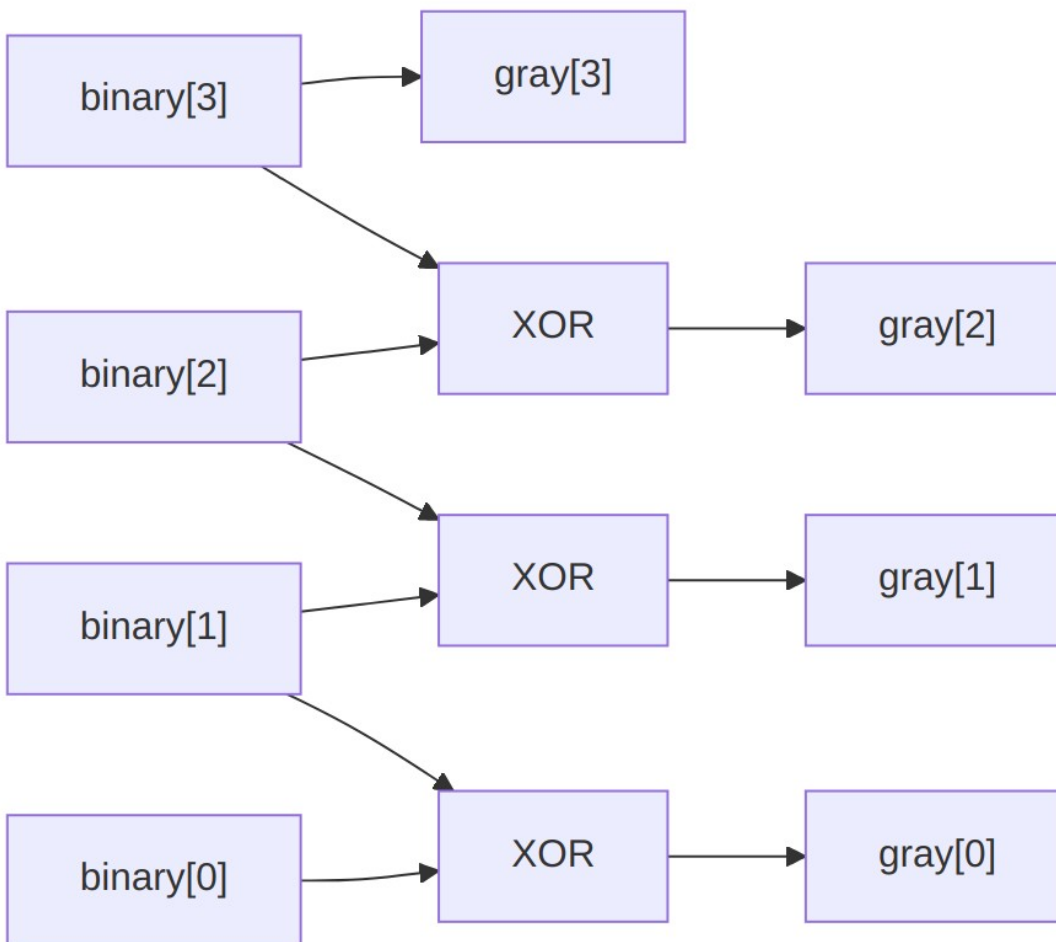


Diagram 1

7.6.4 Timing Characteristics

- **Propagation Delay:** Single XOR gate delay (typically 0.1-0.3ns in modern processes)
- **Critical Path:** Through one XOR gate only
- **Fan-out:** Each binary bit drives at most 2 XOR gates
- **Scalability:** Constant delay regardless of WIDTH

7.7 Design Examples and Applications

7.7.1 1. Asynchronous FIFO Pointer

```
module async_fifo_ptr #(
    parameter int ADDR_WIDTH = 4
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    output logic [ADDR_WIDTH:0] gray_ptr,
    output logic [ADDR_WIDTH-1:0] addr
);

    logic [ADDR_WIDTH:0] binary_ptr, binary_ptr_next;

    // Binary counter
    assign binary_ptr_next = enable ? (binary_ptr + 1) : binary_ptr;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            binary_ptr <= 'b0;
        else
            binary_ptr <= binary_ptr_next;
    end

    // Binary to Gray conversion
    bin2gray #(
        .WIDTH(ADDR_WIDTH + 1)
    ) ptr_converter (
        .binary(binary_ptr),
        .gray(gray_ptr)
    );

    // Extract address (lower bits of binary counter)
    assign addr = binary_ptr[ADDR_WIDTH-1:0];
```

endmodule

7.7.2 2. Clock Domain Crossing Counter

```
module cross_domain_counter #(
    parameter int WIDTH = 8
) (
    // Source domain
    input    logic          src_clk,
    input    logic          src_rst_n,
    input    logic          src_enable,

    // Destination domain
    input    logic          dst_clk,
    input    logic          dst_rst_n,
    output   logic [WIDTH-1:0] dst_count_binary,
    output   logic [WIDTH-1:0] dst_count_gray
);

    // Source domain counter
    logic [WIDTH-1:0] src_binary, src_gray;

    always_ff @(posedge src_clk or negedge src_rst_n) begin
        if (!src_rst_n)
            src_binary <= 'b0;
        else if (src_enable)
            src_binary <= src_binary + 1;
    end

    // Convert to Gray in source domain
    bin2gray #(.WIDTH(WIDTH)) src_converter (
        .binary(src_binary),
        .gray(src_gray)
    );

    // Synchronize Gray code to destination domain
    logic [WIDTH-1:0] dst_gray_sync;

    synchronizer #(.WIDTH(WIDTH)) gray_sync (
        .clk(dst_clk),
        .rst_n(dst_rst_n),
        .data_in(src_gray),
        .data_out(dst_gray_sync)
    );

    // Convert back to binary in destination domain
    gray2bin #(.WIDTH(WIDTH)) dst_converter (
```

```

        .gray(dst_gray_sync),
        .binary(dst_count_binary)
    );

    assign dst_count_gray = dst_gray_sync;

```

endmodule

7.7.3 3. Rotary Encoder Interface

```

module rotary_encoder_interface #(
    parameter int POSITION_WIDTH = 12
) (
    input logic                clk,
    input logic                rst_n,
    input logic [POSITION_WIDTH-1:0] encoder_binary,
    output logic [POSITION_WIDTH-1:0] encoder_gray,
    output logic [POSITION_WIDTH-1:0] position_filtered,
    output logic                position_changed
);

    // Convert encoder binary position to Gray
    bin2gray #(
        .WIDTH(POSITION_WIDTH)
    ) encoder_conv (
        .binary(encoder_binary),
        .gray(encoder_gray)
    );

    // Synchronize and filter
    logic [POSITION_WIDTH-1:0] gray_sync, gray_prev;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            gray_sync <= 'b0;
            gray_prev <= 'b0;
        end else begin
            gray_sync <= encoder_gray;
            gray_prev <= gray_sync;
        end
    end

    // Detect changes (only one bit should change in Gray code)
    logic valid_transition;
    assign valid_transition = ($countones(gray_sync ^ gray_prev) <=
1);

    // Convert back to binary for position output

```

```

        gray2bin #(.WIDTH(POSITION_WIDTH)) pos_converter (
            .gray(gray_sync),
            .binary(position_filtered)
        );

        assign position_changed = valid_transition && (gray_sync !=
gray_prev);

endmodule

```

7.7.4 4. Address Scrambling for Memory Testing

```

module memory_address_scrambler #(
    parameter int ADDR_WIDTH = 16
) (
    input  logic [ADDR_WIDTH-1:0] linear_addr,
    output logic [ADDR_WIDTH-1:0] scrambled_addr
);

    logic [ADDR_WIDTH-1:0] gray_addr;

    // Convert linear address to Gray code for scrambling
    bin2gray #(.WIDTH(ADDR_WIDTH)) scrambler (
        .binary(linear_addr),
        .gray(gray_addr)
    );

    // Additional scrambling (optional)
    assign scrambled_addr = {gray_addr[0], gray_addr[ADDR_WIDTH-1:1]};

endmodule

```

7.7.5 5. Multi-Bit Synchronizer with Gray Code

```

module gray_code_synchronizer #(
    parameter int WIDTH = 8,
    parameter int SYNC_STAGES = 2
) (
    input  logic          src_clk,
    input  logic          src_rst_n,
    input  logic [WIDTH-1:0] src_data,

    input  logic          dst_clk,
    input  logic          dst_rst_n,
    output logic [WIDTH-1:0] dst_data
);

    // Convert to Gray in source domain
    logic [WIDTH-1:0] src_gray;

```

```

bin2gray #(.WIDTH(WIDTH)) src_conv (
    .binary(src_data),
    .gray(src_gray)
);

// Multi-stage synchronizer for Gray code
logic [WIDTH-1:0] sync_regs [SYNC_STAGES];

always_ff @(posedge dst_clk or negedge dst_rst_n) begin
    if (!dst_rst_n) begin
        for (int i = 0; i < SYNC_STAGES; i++) begin
            sync_regs[i] <= 'b0;
        end
    end else begin
        sync_regs[0] <= src_gray;
        for (int i = 1; i < SYNC_STAGES; i++) begin
            sync_regs[i] <= sync_regs[i-1];
        end
    end
end

// Convert back to binary in destination domain
gray2bin #(.WIDTH(WIDTH)) dst_conv (
    .gray(sync_regs[SYNC_STAGES-1]),
    .binary(dst_data)
);

endmodule

```

7.8 Companion Gray-to-Binary Converter

7.8.1 Implementation

```

module gray2bin #(
    parameter int WIDTH = 4
) (
    input wire [WIDTH-1:0] gray,
    output wire [WIDTH-1:0] binary
);

    genvar i;

    // MSB is unchanged
    assign binary[WIDTH-1] = gray[WIDTH-1];

    // Other bits: XOR accumulation from MSB down
    generate

```

```

        for (i = WIDTH-2; i >= 0; i--) begin : gen_binary
            assign binary[i] = binary[i+1] ^ gray[i];
        end
    endgenerate

```

```
endmodule
```

7.8.2 Gray-to-Binary Truth Table (4-bit)

Gray	Binary	Conversion Process
0000	0000	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 0=0, bin[0]=0 $\oplus$ 0=0
0001	0001	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 0=0, bin[0]=0 $\oplus$ 1=1
0011	0010	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 1=1, bin[0]=1 $\oplus$ 1=0
0010	0011	bin[3]=0, bin[2]=0 $\oplus$ 0=0, bin[1]=0 $\oplus$ 1=1, bin[0]=1 $\oplus$ 0=1

7.9 Advanced Implementations

7.9.1 1. Parameterized Converter with Validation

```

module bin2gray_validated #(
    parameter int WIDTH = 4,
    parameter bit ENABLE_CHECKS = 1
) (
    input logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray,
    output logic valid
);

    // Basic conversion
    bin2gray #(.WIDTH(WIDTH)) converter (

```



```

        .binary(binary),
        .gray(gray)
    );

    // Optional validation
    generate
        if (ENABLE_CHECKS) begin : validation
            logic [WIDTH-1:0] binary_check;

            // Round-trip conversion check
            gray2bin #(.WIDTH(WIDTH)) check_converter (
                .gray(gray),
                .binary(binary_check)
            );

            assign valid = (binary == binary_check);
        end else begin : no_validation
            assign valid = 1'b1;
        end
    endgenerate

endmodule

```

7.9.2 2. Pipelined Converter for High Speed

```

module bin2gray_pipelined #(
    parameter int WIDTH = 32,
    parameter int PIPELINE_STAGES = 2
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH-1:0] binary_in,
    input logic valid_in,
    output logic [WIDTH-1:0] gray_out,
    output logic valid_out
);

    // Pipeline registers
    logic [WIDTH-1:0] binary_pipe [PIPELINE_STAGES];
    logic valid_pipe [PIPELINE_STAGES];
    logic [WIDTH-1:0] gray_pipe [PIPELINE_STAGES];

    // Stage 0: Input registration
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            binary_pipe[0] <= 'b0;
            valid_pipe[0] <= 1'b0;
        end else begin

```

```

        binary_pipe[0] <= binary_in;
        valid_pipe[0] <= valid_in;
    end
end

// Conversion in stage 0
bin2gray #(.WIDTH(WIDTH)) stage0_conv (
    .binary(binary_pipe[0]),
    .gray(gray_pipe[0])
);

// Additional pipeline stages
generate
    for (genvar s = 1; s < PIPELINE_STAGES; s++) begin :
pipeline_stages
        always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n) begin
                gray_pipe[s] <= 'b0;
                valid_pipe[s] <= 1'b0;
            end else begin
                gray_pipe[s] <= gray_pipe[s-1];
                valid_pipe[s] <= valid_pipe[s-1];
            end
        end
    end
endgenerate

// Output assignment
assign gray_out = gray_pipe[PIPELINE_STAGES-1];
assign valid_out = valid_pipe[PIPELINE_STAGES-1];

endmodule

```

7.9.3 3. Bidirectional Converter

```

module bidirectional_gray_converter #(
    parameter int WIDTH = 8
) (
    input logic direction, // 0: bin→gray, 1: gray→bin
    input logic [WIDTH-1:0] data_in,
    output logic [WIDTH-1:0] data_out
);

    logic [WIDTH-1:0] bin_to_gray_out, gray_to_bin_out;

    // Both converters always active
    bin2gray #(.WIDTH(WIDTH)) b2g (
        .binary(data_in),

```

```

        .gray(bin_to_gray_out)
    );

    gray2bin #(.WIDTH(WIDTH)) g2b (
        .gray(data_in),
        .binary(gray_to_bin_out)
    );

    // Mux output based on direction
    assign data_out = direction ? gray_to_bin_out : bin_to_gray_out;

endmodule

```

7.10 Verification and Testing

7.10.1 Comprehensive Test Bench

```

module tb_bin2gray;

    parameter int WIDTH = 4;
    parameter int MAX_VAL = (1 << WIDTH) - 1;

    logic [WIDTH-1:0] binary, gray;
    logic [WIDTH-1:0] expected_gray;
    logic [WIDTH-1:0] binary_check;

    // DUT
    bin2gray #(.WIDTH(WIDTH)) dut (
        .binary(binary),
        .gray(gray)
    );

    // Reference converter for checking
    gray2bin #(.WIDTH(WIDTH)) check_conv (
        .gray(gray),
        .binary(binary_check)
    );

    // Test sequence
    initial begin
        $display("Testing %d-bit Binary to Gray converter", WIDTH);

        // Test all possible values
        for (int i = 0; i <= MAX_VAL; i++) begin
            binary = i;
            expected_gray = compute_gray_reference(i);

            #1; // Allow propagation

```

```

        // Check conversion correctness
        if (gray !== expected_gray) begin
            $error("Mismatch: binary=%b, expected_gray=%b,
actual_gray=%b",
                binary, expected_gray, gray);
        end

        // Check round-trip conversion
        if (binary_check !== binary) begin
            $error("Round-trip failed: original=%b, recovered=%b",
                binary, binary_check);
        end

        // Check single-bit change property
        if (i > 0) begin
            logic [WIDTH-1:0] prev_gray =
compute_gray_reference(i-1);
            int bit_changes = $countones(gray ^ prev_gray);
            if (bit_changes != 1) begin
                $error("Multiple bit change: %d→%d, gray %b→%b,
changes=%d",
                    i-1, i, prev_gray, gray, bit_changes);
            end
        end

        $display("PASS: %d → binary:%b → gray:%b", i, binary,
gray);
    end

    $display("All tests passed!");
    $finish;
end

// Reference Gray code computation
function [WIDTH-1:0] compute_gray_reference(input [WIDTH-1:0]
bin);
    compute_gray_reference[WIDTH-1] = bin[WIDTH-1];
    for (int i = WIDTH-2; i >= 0; i--) begin
        compute_gray_reference[i] = bin[i] ^ bin[i+1];
    end
endfunction

endmodule

```

7.10.2 Property-Based Verification

```

module bin2gray_properties #(
    parameter int WIDTH = 4

```

```

);

logic [WIDTH-1:0] binary, gray;

// Bind to DUT
bind bin2gray bin2gray_properties #(WIDTH) props_inst (.*);

// Property: MSB preservation
property msb_unchanged;
    gray[WIDTH-1] == binary[WIDTH-1];
endproperty

// Property: Single bit change between consecutive values
property single_bit_change;
    logic [WIDTH-1:0] bin_plus_one = binary + 1;
    logic [WIDTH-1:0] gray_plus_one;

    // Compute Gray code for binary+1
    assign gray_plus_one[WIDTH-1] = bin_plus_one[WIDTH-1];
    for (genvar i = 0; i < WIDTH-1; i++) begin
        assign gray_plus_one[i] = bin_plus_one[i] ^
bin_plus_one[i+1];
    end

    // Check single bit difference
    (binary < (2**WIDTH - 1)) |-> ($countones(gray ^
gray_plus_one) == 1);
endproperty

// Property: Round-trip conversion
property round_trip_conversion;
    logic [WIDTH-1:0] recovered_binary;

    // Convert back to binary
    assign recovered_binary[WIDTH-1] = gray[WIDTH-1];
    for (genvar i = WIDTH-2; i >= 0; i--) begin
        assign recovered_binary[i] = recovered_binary[i+1] ^
gray[i];
    end

    recovered_binary == binary;
endproperty

// Assertions
assert property (msb_unchanged);
assert property (single_bit_change);
assert property (round_trip_conversion);

```

```
endmodule
```

7.10.3 Coverage Model

```
covergroup bin2gray_cg;
```

```
    cp_binary_values: coverpoint binary {
        bins zero = {0};
        bins powers_of_two[] = {1, 2, 4, 8, 16}; // For appropriate
WIDTH
        bins max_value = {2**WIDTH - 1};
        bins mid_range[] = {[1:2**(WIDTH-1)-1]};
        bins upper_range[] = {[2**(WIDTH-1):2**WIDTH-2]};
    }

    cp_gray_values: coverpoint gray {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_bit_patterns: coverpoint binary {
        bins alternating_01 = {8'b01010101}; // For WIDTH=8
        bins alternating_10 = {8'b10101010};
        bins all_ones = {'1'};
        bins all_zeros = {'0'};
    }

    // Cross coverage between input patterns and outputs
    cross_binary_gray: cross cp_binary_values, cp_gray_values;
```

```
endcovergroup
```

7.11 Synthesis Optimization

7.11.1 Resource Utilization

For different widths, typical FPGA resource usage:

WIDTH	LUTs	Delay (ns)	Max Freq (MHz)
4	3	0.2	800+
8	7	0.2	800+
16	15	0.3	600+
32	31	0.4	500+

7.11.2 Timing Optimization

```
// For critical timing paths, add pipeline register
module bin2gray_registered #(
    parameter int WIDTH = 16
) (
    input logic          clk,
    input logic          rst_n,
    input logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray
);

    logic [WIDTH-1:0] gray_comb;

    // Combinational conversion
    bin2gray #(.WIDTH(WIDTH)) conv (
        .binary(binary),
        .gray(gray_comb)
    );

    // Output register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            gray <= 'b0;
        else
            gray <= gray_comb;
    end

endmodule
```

7.11.3 Power Optimization

```
// Clock gating for power savings
module bin2gray_gated #(
    parameter int WIDTH = 8
) (
    input logic          clk,
    input logic          rst_n,
    input logic          enable,
    input logic [WIDTH-1:0] binary,
    output logic [WIDTH-1:0] gray
);

    logic gated_clk;

    // Clock gate
    clock_gate cg_inst (
        .clk(clk),
        .enable(enable),
```

```

        .gated_clk(gated_clk)
    );

    // Registered converter
    bin2gray_registered #(.WIDTH(WIDTH)) conv (
        .clk(gated_clk),
        .rst_n(rst_n),
        .binary(binary),
        .gray(gray)
    );

endmodule

```


7.12 Common Applications Summary

7.12.11. Asynchronous FIFOs: Prevent metastability in pointer comparisons

7.12.22. Clock Domain Crossing: Safe multi-bit signal transfer

7.12.33. Position Encoders: Mechanical/optical encoder interfaces

7.12.44. Memory Address Generation: Reduce EMI and power spikes

7.12.55. Test Pattern Generation: Controlled single-bit transitions

7.12.66. Error Detection: Single-bit error detection schemes

7.12.77. ADC/DAC Interfaces: Reduce glitches in conversion systems

7.13 Design Guidelines

7.13.11. Always Use for Async Boundaries: Gray code is essential for safe asynchronous transfers

7.13.22. Consider Timing: Purely combinational - no setup/hold concerns

7.13.33. Verify Single-Bit Property: Always verify adjacent values differ by one bit

7.13.44. Plan for Back-Conversion: Usually need Gray-to-Binary converter too

7.13.55. Check Width Requirements: Ensure sufficient bits for application range

The Binary-to-Gray converter is a fundamental building block in digital design, providing robust solutions for asynchronous interfaces and glitch-free operations. Its simplicity and reliability make it indispensable for safe digital system design.

7.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

8 Binary to BCD Converter - Comprehensive Documentation

8.1 Overview

The `bin_to_bcd` module implements the Double-Dabble algorithm (also known as the Add-3-Then-Shift algorithm) to convert binary numbers to Binary Coded Decimal (BCD) format. This is a fundamental operation in digital systems for displaying numeric values on seven-segment displays, LCD panels, and other decimal-based output devices.

8.2 Module Declaration

```
module bin_to_bcd #(
    parameter int WIDTH  = 8,
    parameter int DIGITS = 3
) (
    input logic          clk,          // Clock signal
    input logic          rst_n,        // Active-low reset signal
    input logic          start,
    input logic [WIDTH-1:0] binary,
    output logic [DIGITS*4-1:0] bcd,
    output logic         done
);
```

8.3 Parameters

8.3.1 WIDTH

- **Type:** int
- **Default:** 8
- **Description:** Bit width of the input binary number
- **Range:** 1 to 32 (practical range)
- **Impact:** Determines conversion complexity and timing
- **Sizing Rule:** For n-bit binary input, maximum decimal value is $2^n - 1$

8.3.2 DIGITS

- **Type:** int
- **Default:** 3

- **Description:** Number of BCD digits in the output
- **Range:** Must be sufficient for maximum input value
- **Calculation:** $\text{DIGITS} \geq \text{ceil}(\log_{10}(2^{\text{WIDTH}}))$
- **Output Width:** $\text{DIGITS} \times 4$ bits (4 bits per BCD digit)

8.3.3 Parameter Relationship Table

WIDTH	Max Binary Value	Max Decimal	Min DIGITS Required	BCD Width
4	15	15	2	8 bits
8	255	255	3	12 bits
10	1023	1023	4	16 bits
12	4095	4095	4	16 bits
16	65535	65535	5	20 bits
20	1048575	1048575	7	28 bits
24	16777215	16777215	8	32 bits

8.4 Ports

8.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock
rst_n	1	logic	Active-low asynchronous reset
start	1	logic	Initiate conversion (pulse)
binary	WIDTH	logic	Binary number to convert

8.4.2 Outputs

Port	Width	Type	Description
bcd	DIGITS×4	logic	BCD result (packed)

Port	Width	Type	Description
done	1	logic	format) Conversion complete flag

8.5 Double-Dabble Algorithm Theory

8.5.1 Algorithm Principle

The Double-Dabble algorithm converts binary to BCD through iterative shifting and adjustment:

1. **Initialize:** BCD accumulator = 0, binary shift register = input value
2. **Check:** For each BCD digit, if value ≥ 5 , add 3
3. **Shift:** Shift entire register (BCD + binary) left by 1 bit
4. **Repeat:** Continue until all binary bits are processed
5. **Result:** BCD accumulator contains the decimal equivalent

8.5.2 Why Add 3?

The “add 3” operation prevents BCD overflow during shifting: - **BCD Range:** Each digit must stay within 0-9 (4 bits) - **Problem:** Shifting a BCD digit ≥ 5 creates values > 9 - **Solution:** Adding 3 before shifting ensures proper BCD after shift

8.5.3 Mathematical Proof

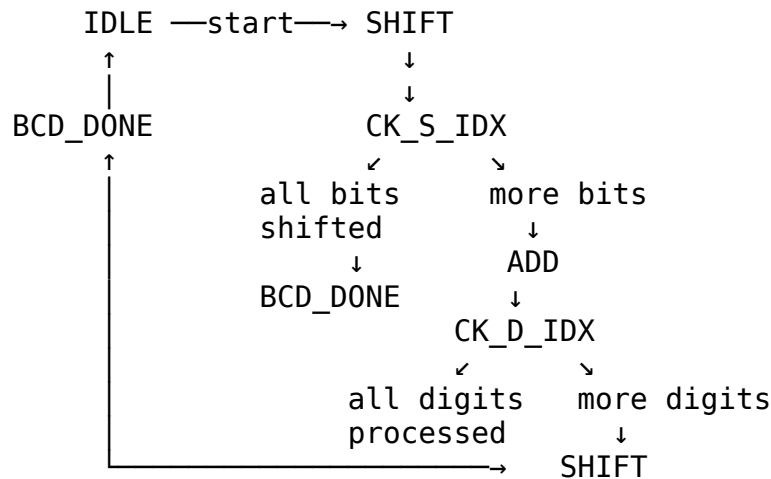
For BCD digit D where $D \geq 5$: - Before shift: $D + 3$ - After shift: $2(D + 3) = 2D + 6$ - For D = 5: $2(5) + 6 = 16 = 0x10$ (BCD: 1,6 - carry generated) - For D = 9: $2(9) + 6 = 24 = 0x18$ (BCD: 2,4 - carry generated)

8.6 FSM Implementation

8.6.1 State Definitions

```
typedef enum logic [5:0] {
    IDLE      = 6'b000001, // Wait for start signal
    SHIFT     = 6'b000010, // Shift binary bit into BCD
    CK_S_IDX  = 6'b000100, // Check if all bits shifted
    ADD       = 6'b001000, // Add 3 to digits > 4
    CK_D_IDX  = 6'b010000, // Check if all digits processed
    BCD_DONE  = 6'b100000 // Conversion complete
} fsm_state_t;
```

8.6.2 State Transition Diagram



8.6.3 Detailed State Descriptions

8.6.3.1 IDLE State

- **Function:** Wait for start signal
- **Duration:** Variable (until start asserted)
- **Actions:**
 - Monitor start input
 - Initialize counters and registers when start detected
- **Next State:** SHIFT when start == 1'b1

8.6.3.2 SHIFT State

- **Function:** Shift one bit from binary to BCD
- **Duration:** 1 clock cycle
- **Actions:**
 - Left-shift BCD accumulator by 1 bit
 - Move MSB of binary into LSB of BCD
 - Left-shift binary register by 1 bit
 - Increment loop counter

- **Next State:** Always CK_S_IDX

8.6.3.3 CK_S_IDX State

- **Function:** Check if all binary bits processed
- **Duration:** 1 clock cycle
- **Actions:**
 - Compare loop counter with WIDTH
 - Reset digit index for ADD phase
- **Next State:**
 - BCD_DONE if loop_count == WIDTH-1
 - ADD if more bits to process

8.6.3.4 ADD State

- **Function:** Add 3 to BCD digits ≥ 5
- **Duration:** 1 clock cycle
- **Actions:**
 - Extract current BCD digit
 - Add 3 if digit > 4
 - Update BCD accumulator
- **Next State:** Always CK_D_IDX

8.6.3.5 CK_D_IDX State

- **Function:** Check if all digits processed
- **Duration:** 1 clock cycle
- **Actions:**
 - Compare digit index with DIGITS
 - Increment digit index

- **Next State:**
 - SHIFT if `digit_index == DIGITS-1`
 - ADD if more digits to process

8.6.3.6 BCD_DONE State

- **Function:** Signal completion
- **Duration:** 1 clock cycle
- **Actions:**
 - Assert done flag
 - Prepare for next conversion
- **Next State:** Always IDLE

8.7 Implementation Details

8.7.1 Register Definitions

```
// FSM state register
fsm_state_t r_fsm_main;

// BCD accumulator - stores the converted result
logic [DIGITS*4-1:0] r_bcd;

// Binary shift register - input value being processed
logic [WIDTH-1:0] r_binary;

// Loop counter - tracks processed bits
localparam int LOOP_COUNT_WIDTH = $clog2(WIDTH + 1);
logic [LOOP_COUNT_WIDTH-1:0] r_loop_count;

// Digit index - tracks current BCD digit being processed
localparam int DIGIT_INDEX_WIDTH = $clog2(DIGITS);
logic [DIGIT_INDEX_WIDTH-1:0] r_digit_index;

// Data valid flag
logic r_dv;
```

8.7.2 Core Processing Logic

```
// Extract current BCD digit being processed
logic [3:0] w_bcd_digit;
assign w_bcd_digit = r_bcd[r_digit_index*4+:4];

// Main FSM
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_fsm_main    <= IDLE;
        r_bcd         <= '0;
        r_binary      <= '0;
        r_digit_index <= '0;
        r_loop_count  <= '0;
        r_dv          <= '0;
    end else begin
        r_dv <= 1'b0; // Default: done is low

        casez (r_fsm_main)
            IDLE: begin
                if (start == 1'b1) begin
                    r_binary    <= binary;
                    r_fsm_main  <= SHIFT;
                    r_bcd       <= '0;
                    r_loop_count <= '0;
                    r_digit_index <= '0;
                end else begin
                    r_fsm_main <= IDLE;
                end
            end

            SHIFT: begin
                // Shift BCD left and insert MSB of binary
                r_bcd    <= r_bcd << 1;
                r_bcd[0] <= r_binary[WIDTH-1];
                r_binary  <= r_binary << 1;
                r_fsm_main <= CK_S_IDX;
            end

            CK_S_IDX: begin
                if (r_loop_count == LOOP_COUNT_WIDTH'(WIDTH - 1))
begin
                    r_loop_count <= '0;
                    r_fsm_main  <= BCD_DONE;
                end else begin
                    r_loop_count <= r_loop_count + 1'b1;
                    r_digit_index <= '0;
                    r_fsm_main  <= ADD;
                end
            end
        end casez
    end
end
```



```

        end
    end

    ADD: begin
        if (w_bcd_digit > 4'd4) begin
            r_bcd[(r_digit_index*4)+:4] <= w_bcd_digit + 4'd3;
        end
        r_fsm_main <= CK_D_IDX;
    end

    CK_D_IDX: begin
        if (r_digit_index == DIGIT_INDEX_WIDTH'(DIGITS - 1))
            r_digit_index <= '0;
            r_fsm_main <= SHIFT;
        end else begin
            r_digit_index <= r_digit_index + 1'b1;
            r_fsm_main <= ADD;
        end
    end

    BCD_DONE: begin
        r_dv <= 1'b1;
        r_fsm_main <= IDLE;
    end

    default: begin
        r_fsm_main <= IDLE;
    end
endcase
end
end

```

8.8 Timing Analysis

8.8.1 Conversion Latency

The total conversion time depends on the input width and number of digits:

Formula: Latency = WIDTH + (WIDTH-1) × DIGITS clock cycles

8.8.2 Latency Breakdown

- **SHIFT operations:** WIDTH cycles (one per input bit)
- **ADD operations:** (WIDTH-1) × DIGITS cycles (check each digit after each shift except the last)

- **State transitions:** CK_S_IDX and CK_D_IDX states (included in above counts)
- **Completion:** 1 cycle for BCD_DONE state

8.8.3 Latency Examples

WIDTH	DIGITS	Total Cycles	@ 100MHz	@ 50MHz
4	2	$4 + 3 \times 2 = 10$	100ns	200ns
8	3	$8 + 7 \times 3 = 29$	290ns	580ns
12	4	$12 + 11 \times 4 = 56$	560ns	1.12μs
16	5	$16 + 15 \times 5 = 91$	910ns	1.82μs
20	7	$20 + 19 \times 7 = 153$	1.53μs	3.06μs

8.8.4 Pipeline Considerations

The conversion is inherently sequential, making pipelining challenging: - **Data Dependency:** Each iteration depends on previous results - **State-Based:** FSM prevents overlapping conversions - **Optimization:** Consider parallel converters for high throughput

8.9 BCD Format and Encoding

8.9.1 BCD Digit Encoding

Each BCD digit uses 4 bits to represent decimal values 0-9:

Decimal	BCD (4-bit)	Hex
0	0000	0x0
1	0001	0x1
2	0010	0x2
3	0011	0x3
4	0100	0x4
5	0101	0x5
6	0110	0x6
7	0111	0x7
8	1000	0x8

Decimal	BCD (4-bit)	Hex
9	1001	0x9
Invalid	1010-1111	0xA-0xF

8.9.2 Packed BCD Format

The output bcd uses packed BCD format where multiple digits are concatenated:

```
// For DIGITS = 3, bcd[11:0] contains:
// bcd[11:8] = hundreds digit
// bcd[7:4] = tens digit
// bcd[3:0] = ones digit

// Example: decimal 123 → bcd = 12'h123
```

8.9.3 BCD Digit Extraction

```
// Extract individual digits from packed BCD
logic [3:0] ones, tens, hundreds;

assign ones      = bcd[3:0];
assign tens      = bcd[7:4];
assign hundreds = bcd[11:8];

// Convert to ASCII for display
logic [7:0] ascii_ones, ascii_tens, ascii_hundreds;

assign ascii_ones      = 8'h30 + ones;    // Add ASCII '0'
assign ascii_tens      = 8'h30 + tens;
assign ascii_hundreds = 8'h30 + hundreds;
```

8.10 Worked Examples

8.10.1 Example 1: Converting 8-bit Binary 156 (WIDTH=8, DIGITS=3)

Initial State: - Binary: 8'b10011100 (156 decimal) - BCD: 12'b000000000000 - Loop count: 0

Iteration 1: 1. SHIFT: BCD = 000000000001, Binary = 00111000, Loop = 0 2. CK_S_IDX: More bits to process 3. ADD: Check digits [000, 000, 001] - none ≥ 5 4. CK_D_IDX: Check next digit 5. ADD: Check digits [000, 000, 001] - none ≥ 5 6. CK_D_IDX: Check next digit 7. ADD: Check digits [000, 000, 001] - none ≥ 5 8. CK_D_IDX: All digits checked, go to SHIFT

Iteration 2: 1. SHIFT: BCD = 000000000010, Binary = 01110000, Loop = 1 2. ADD phase: No digits ≥ 5

Iteration 3: 1. SHIFT: BCD = 000000000101, Binary = 11100000, Loop = 2

2. ADD phase: Ones digit = 5 $\geq$ 5, add 3 $\rightarrow$ BCD = 000000001000

Iteration 4: 1. SHIFT: BCD = 000000010001, Binary = 11000000, Loop = 3 2. ADD phase: No digits $\geq$ 5

Iteration 5: 1. SHIFT: BCD = 000000100011, Binary = 10000000, Loop = 4 2. ADD phase: No digits $\geq$ 5

Iteration 6: 1. SHIFT: BCD = 000001000111, Binary = 00000000, Loop = 5 2. ADD phase: Ones = 7 $\geq$ 5, add 3 $\rightarrow$ BCD = 000001001010 (ones = 10, invalid!) Actually: 7 + 3 = 10 = 1010₂, but this represents BCD “10” which will be handled in next shift

Iteration 7: 1. SHIFT: BCD = 000010010100, Loop = 6 2. ADD phase: Tens = 9, Ones = 4 - none $\geq$ 5

Iteration 8: 1. SHIFT: BCD = 000100101000, Loop = 7 2. CK_S_IDX: Loop count = 7 = WIDTH-1, conversion done

Final Result: BCD = 000100101000 = 4'h1, 4'h5, 4'h6 = 156 decimal $\checkmark$

8.10.2 Example 2: Converting 4-bit Binary 9 (WIDTH=4, DIGITS=2)

Step-by-step conversion of binary 1001₂ (9₁₀):

Cycle	State	Action	BCD	Binary	Notes
0	IDLE	Start	00000000	1001	Initialize
1	SHIFT	Shift bit	00000001	0010	MSB $\rightarrow$ BCD LSB
2	ADD	Check digits	00000001	0010	Ones=1 <5, no add
3	SHIFT	Shift bit	00000010	0100	Next bit
4	ADD	Check digits	00000010	0100	Ones=2 <5, no add
5	SHIFT	Shift bit	00000100	1000	Next bit
6	ADD	Check digits	00000100	1000	Ones=4 <5, no add
7	SHIFT	Shift bit	00001001	0000	Final bit
8	DONE	Complete	00001001	0000	Result: BCD=09

Result: BCD = 8'b00001001 = 09₁₆ = 9₁₀ ✓

8.11 Design Examples and Applications

8.11.11. Seven-Segment Display Driver

```
module display_controller #(
    parameter int WIDTH = 8
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH-1:0] binary_value,
    input logic update,
    output logic [6:0] seg_hundreds,
    output logic [6:0] seg_tens,
    output logic [6:0] seg_ones,
    output logic display_ready
);

    // BCD converter
    logic [11:0] bcd_result;
    logic conversion_done;

    bin_to_bcd #(
        .WIDTH(WIDTH),
        .DIGITS(3)
    ) converter (
        .clk(clk),
        .rst_n(rst_n),
        .start(update),
        .binary(binary_value),
        .bcd(bcd_result),
        .done(conversion_done)
    );

    // Extract digits
    logic [3:0] digit_hundreds, digit_tens, digit_ones;
    assign digit_hundreds = bcd_result[11:8];
    assign digit_tens      = bcd_result[7:4];
    assign digit_ones      = bcd_result[3:0];

    // Seven-segment decoders
    seven_seg_decoder dec_hundreds
    (.digit(digit_hundreds), .segments(seg_hundreds));
    seven_seg_decoder dec_tens
    (.digit(digit_tens), .segments(seg_tens));
    seven_seg_decoder dec_ones
    (.digit(digit_ones), .segments(seg_ones));
```

```

    assign display_ready = conversion_done;

endmodule

// Seven-segment decoder
module seven_seg_decoder (
    input logic [3:0] digit,
    output logic [6:0] segments // {g,f,e,d,c,b,a}
);
    always_comb begin
        case (digit)
            4'd0: segments = 7'b0111111; // 0
            4'd1: segments = 7'b0000110; // 1
            4'd2: segments = 7'b1011011; // 2
            4'd3: segments = 7'b1001111; // 3
            4'd4: segments = 7'b1100110; // 4
            4'd5: segments = 7'b1101101; // 5
            4'd6: segments = 7'b1111101; // 6
            4'd7: segments = 7'b0000111; // 7
            4'd8: segments = 7'b1111111; // 8
            4'd9: segments = 7'b1101111; // 9
            default: segments = 7'b1000000; // Error (top segment
only)
        endcase
    end
endmodule

```

8.11.22. LCD/UART Decimal Display

```

module decimal_uart_tx #(
    parameter int WIDTH = 16,
    parameter int DIGITS = 5
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH-1:0] value,
    input logic send_value,
    output logic uart_tx,
    output logic tx_busy
);

// BCD conversion
logic [DIGITS*4-1:0] bcd_value;
logic bcd_done;

bin_to_bcd #(
    .WIDTH(WIDTH),

```

```

        .DIGITS(DIGITS)
    ) bcd_conv (
        .clk(clk),
        .rst_n(rst_n),
        .start(send_value),
        .binary(value),
        .bcd(bcd_value),
        .done(bcd_done)
    );

    // ASCII conversion and transmission
    typedef enum {IDLE, CONVERT, TRANSMIT} state_t;
    state_t state;

    logic [2:0] digit_idx;
    logic [3:0] current_digit;
    logic [7:0] ascii_char;
    logic uart_start, uart_done;

    // Extract current digit (MSB first for display)
    assign current_digit = bcd_value[(DIGITS-1-digit_idx)*4+:4];
    assign ascii_char = 8'h30 + current_digit; // Convert to ASCII

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state <= IDLE;
            digit_idx <= 0;
            uart_start <= 0;
        end else begin
            uart_start <= 0; // Default

            case (state)
                IDLE: begin
                    if (bcd_done) begin
                        state <= TRANSMIT;
                        digit_idx <= 0;
                    end
                end

                TRANSMIT: begin
                    if (!uart_busy && !uart_start) begin
                        uart_start <= 1;
                        if (digit_idx == DIGITS-1) begin
                            state <= IDLE;
                            digit_idx <= 0;
                        end else begin
                            digit_idx <= digit_idx + 1;
                        end
                    end
                end
            endcase
        end
    end

```

```

        end
    endcase
end
end

// UART transmitter instance
uart_tx_module uart_tx_inst (
    .clk(clk),
    .rst_n(rst_n),
    .data(ascii_char),
    .start(uart_start),
    .tx(uart_tx),
    .busy(uart_busy),
    .done(uart_done)
);

assign tx_busy = (state != IDLE);

endmodule

```

8.11.33. Multi-Channel BCD Converter

```

module multi_channel_bcd #(
    parameter int CHANNELS = 4,
    parameter int WIDTH = 12,
    parameter int DIGITS = 4
) (
    input logic clk,
    input logic rst_n,
    input logic [CHANNELS-1:0] convert_request,
    input logic [WIDTH-1:0] binary_data [CHANNELS],
    output logic [DIGITS*4-1:0] bcd_results [CHANNELS],
    output logic conversion_done,
    output logic converter_busy
);

// Round-robin arbitration for converter access
logic [$clog2(CHANNELS)-1:0] current_channel;
logic [CHANNELS-1:0] pending_requests;
logic conversion_active;

// Single BCD converter instance
logic [WIDTH-1:0] conv_binary;
logic conv_start, conv_done;
logic [DIGITS*4-1:0] conv_bcd;

bin_to_bcd #(

```



```

        .WIDTH(WIDTH),
        .DIGITS(DIGITS)
    ) converter (
        .clk(clk),
        .rst_n(rst_n),
        .start(conv_start),
        .binary(conv_binary),
        .bcd(conv_bcd),
        .done(conv_done)
    );

    // Request management
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pending_requests <= 0;
            current_channel <= 0;
            conversion_active <= 0;
            conv_start <= 0;
            conversion_done <= 0;
        end else begin
            conv_start <= 0;
            conversion_done <= 0;

            // Latch new requests
            pending_requests <= (pending_requests | convert_request) &
                                ~(conversion_done);

            if (!conversion_active && |pending_requests) begin
                // Find next channel to service
                for (int i = 0; i < CHANNELS; i++) begin
                    if (pending_requests[(current_channel + i) %
CHANNELS]) begin
                        current_channel <= (current_channel + i) %
CHANNELS;
                        conv_binary <= binary_data[(current_channel +
i) % CHANNELS];
                        conv_start <= 1;
                        conversion_active <= 1;
                        break;
                    end
                end
            end

            if (conv_done) begin
                bcd_results[current_channel] <= conv_bcd;
                conversion_done[current_channel] <= 1;
                pending_requests[current_channel] <= 0;
                conversion_active <= 0;
            end
        end
    end

```

```

        current_channel <= (current_channel + 1) % CHANNELS;
    end
end
end

assign converter_busy = conversion_active || |pending_requests;

endmodule

```

8.12 Optimization Techniques

8.12.11. Early Termination Optimization

```

// Optimize for small numbers by detecting leading zeros
logic [WIDTH-1:0] leading_zero_mask;
logic [$clog2(WIDTH):0] significant_bits;

// Count significant bits
count_leading_zeros #(.WIDTH(WIDTH)) clz_inst (
    .data(binary),
    .clz(leading_zeros)
);

assign significant_bits = WIDTH - leading_zeros;

// Modify loop termination condition
always_ff @(posedge clk) begin
    // ... other logic ...

    CK_S_IDX: begin
        if (r_loop_count == significant_bits - 1) begin
            r_fsm_main <= BCD_DONE; // Early termination
        end else begin
            // Continue normal processing
        end
    end
end

```

8.12.22. Parallel Digit Processing

```

// Process all digits in parallel during ADD state
logic [3:0] digit_array [DIGITS];
logic [3:0] adjusted_digits [DIGITS];

// Extract all digits
genvar i;
generate
    for (i = 0; i < DIGITS; i++) begin : extract_digits

```

```

        assign digit_array[i] = r_bcd[i*4+:4];
        assign adjusted_digits[i] = (digit_array[i] > 4) ?
                                   (digit_array[i] + 3) :
digit_array[i];
    end
endgenerate

// Update all digits simultaneously
always_ff @(posedge clk) begin
    // ... other states ...

    ADD: begin
        for (int j = 0; j < DIGITS; j++) begin
            r_bcd[j*4+:4] <= adjusted_digits[j];
        end
        r_fsm_main <= CK_S_IDX; // Skip CK_D_IDX state
    end
end

```

8.12.33. Pipelined Implementation

```

// Multi-stage pipeline for high throughput
module bin_to_bcd_pipelined #(
    parameter int WIDTH = 8,
    parameter int DIGITS = 3,
    parameter int STAGES = 4 // Pipeline depth
) (
    input logic clk,
    input logic rst_n,
    input logic valid_in,
    input logic [WIDTH-1:0] binary_in,
    output logic valid_out,
    output logic [DIGITS*4-1:0] bcd_out
);

// Pipeline stages
logic [STAGES-1:0] valid_pipe;
logic [WIDTH-1:0] binary_pipe [STAGES];
logic [DIGITS*4-1:0] bcd_pipe [STAGES];

// Stage processing
generate
    for (genvar s = 0; s < STAGES; s++) begin : pipeline_stages
        logic [7:0] bits_to_process;
        assign bits_to_process = WIDTH * s / STAGES;

        // Process subset of bits in each stage
        // Implementation depends on specific algorithm
    end
endgenerate

```

```
partitioning
    end
endgenerate
```

```
endmodule
```

8.13 Verification and Testing

8.13.1 Comprehensive Test Strategy

```
module tb_bin_to_bcd;

    // Test parameters
    parameter int WIDTH = 8;
    parameter int DIGITS = 3;
    parameter int MAX_VALUE = (1 << WIDTH) - 1;

    // DUT signals
    logic clk, rst_n, start, done;
    logic [WIDTH-1:0] binary;
    logic [DIGITS*4-1:0] bcd;

    // Test control
    logic [WIDTH-1:0] test_vector;
    logic [31:0] expected_decimal;
    logic [DIGITS*4-1:0] expected_bcd;

    // DUT instantiation
    bin_to_bcd #(
        .WIDTH(WIDTH),
        .DIGITS(DIGITS)
    ) dut (.*);

    // Clock generation
    initial clk = 0;
    always #5 clk = ~clk;

    // Test sequence
    initial begin
        rst_n = 0;
        start = 0;
        binary = 0;

        #100 rst_n = 1;

        // Test all possible values
        for (int i = 0; i <= MAX_VALUE; i++) begin
            test_conversion(i);
        end
    end
endmodule
```

```

    end

    // Boundary tests
    test_conversion(0);
    test_conversion(1);
    test_conversion(MAX_VALUE);

    // Random tests
    repeat (1000) begin
        test_conversion($random % (MAX_VALUE + 1));
    end

    $display("All tests completed successfully!");
    $finish;
end

// Test task
task test_conversion(input [WIDTH-1:0] value);
    begin
        binary = value;
        expected_decimal = value;
        expected_bcd = decimal_to_bcd(expected_decimal);

        @(posedge clk);
        start = 1;
        @(posedge clk);
        start = 0;

        wait(done);
        @(posedge clk);

        if (bcd !== expected_bcd) begin
            $error("Mismatch: binary=%d, expected_bcd=%h,
actual_bcd=%h",
                value, expected_bcd, bcd);
        end else begin
            $display("PASS: binary=%d → bcd=%h", value, bcd);
        end
    end
endtask

// Reference function
function [DIGITS*4-1:0] decimal_to_bcd(input [31:0] decimal);
    logic [31:0] temp;
    logic [DIGITS*4-1:0] result;
    begin
        temp = decimal;
        for (int i = 0; i < DIGITS; i++) begin

```

```

        result[i*4+:4] = temp % 10;
        temp = temp / 10;
    end
    decimal_to_bcd = result;
end
endfunction

```

```
endmodule
```

8.13.2 Coverage Model

```

covergroup bcd_conversion_cg;

    // Input value coverage
    cp_binary_value: coverpoint binary {
        bins zero = {0};
        bins small[] = {[1:9]};
        bins medium[] = {[10:99]};
        bins large[] = {[100:255]}; // For 8-bit
        bins max_val = {2**WIDTH - 1};
    }

    // State coverage
    cp_fsm_states: coverpoint dut.r_fsm_main {
        bins all_states[] = {IDLE, SHIFT, CK_S_IDX, ADD, CK_D_IDX,
BCD_DONE};
        bins state_transitions[] = (IDLE => SHIFT),
                                   (SHIFT => CK_S_IDX),
                                   (CK_S_IDX => ADD),
                                   (CK_S_IDX => BCD_DONE),
                                   (ADD => CK_D_IDX),
                                   (CK_D_IDX => ADD),
                                   (CK_D_IDX => SHIFT),
                                   (BCD_DONE => IDLE);
    }

    // BCD digit coverage
    cp_bcd_digits: coverpoint bcd {
        bins valid_bcd[] = {[12'h000:12'h999]}; // Valid BCD range
    }

    // Loop iteration coverage
    cp_loop_count: coverpoint dut.r_loop_count {
        bins all_counts[] = {[0:WIDTH-1]};
    }

    // Cross coverage
    cross_value_states: cross cp_binary_value, cp_fsm_states;

```

```
endcovergroup
```

8.13.3 Formal Verification Properties

```
// Properties for formal verification
module bin_to_bcd_properties #(
    parameter int WIDTH = 8,
    parameter int DIGITS = 3
);

    // Bind to DUT
    bind bin_to_bcd bin_to_bcd_properties #(WIDTH, DIGITS) props_inst
    (.*);

    // Basic properties
    property conversion_eventually_completes;
        @(posedge clk) disable iff (!rst_n)
            start |-> ##[1:200] done; // Adjust bound based on max latency
    endproperty

    property bcd_output_valid;
        @(posedge clk) disable iff (!rst_n)
            done |-> valid_bcd(bcd);
    endproperty

    property conversion_correctness;
        logic [31:0] expected_decimal;
        @(posedge clk) disable iff (!rst_n)
            (start, expected_decimal = binary) |->
            ##[1:200] (done && (bcd_to_decimal(bcd) == expected_decimal));
    endproperty

    // Helper functions
    function bit valid_bcd(logic [DIGITS*4-1:0] bcd_val);
        for (int i = 0; i < DIGITS; i++) begin
            if (bcd_val[i*4+:4] > 9) return 0;
        end
        return 1;
    endfunction

    function [31:0] bcd_to_decimal(logic [DIGITS*4-1:0] bcd_val);
        logic [31:0] result = 0;
        for (int i = 0; i < DIGITS; i++) begin
            result += bcd_val[i*4+:4] * (10**i);
        end
        return result;
    endfunction
```

```
// Assertions
assert property (conversion_eventually_completes);
assert property (bcd_output_valid);
assert property (conversion_correctness);
```

```
endmodule
```

8.14 Performance and Resource Analysis

8.14.1 Synthesis Results (Typical FPGA)

Configurat ion	LUTs	FFs	BRAM	Max Freq	Notes
8-bit, 3- digit	85	45	0	250 MHz	Basic config
12-bit, 4- digit	145	65	0	220 MHz	Medium size
16-bit, 5- digit	220	85	0	200 MHz	Larger design
20-bit, 7- digit	350	125	0	180 MHz	Complex design

8.14.2 Power Consumption Analysis

- **Dynamic Power:** Proportional to conversion frequency
- **Static Power:** Minimal (standard CMOS)
- **Optimization:** Clock gating during IDLE state
- **Trade-off:** Performance vs. power efficiency

8.15 Common Design Patterns

8.15.1 Pattern 1: Auto-Converting Display Interface

```
module auto_display_interface #(
    parameter int WIDTH = 10
) (
    input logic [WIDTH-1:0] sensor_data,
    input logic             data_valid,
    output logic [6:0]      display_segments[4],
    output logic            display_valid
)
```



```

);

// Auto-trigger conversion on new data
logic prev_data_valid;
logic start_conversion;

always_ff @(posedge clk) begin
    prev_data_valid <= data_valid;
end

assign start_conversion = data_valid && !prev_data_valid;

// BCD converter and display driver
// ... (implementation similar to previous examples)

endmodule

```

8.15.2 Pattern 2: Buffered Converter with FIFO

```

module buffered_bcd_converter #(
    parameter int WIDTH = 12,
    parameter int DIGITS = 4,
    parameter int BUFFER_DEPTH = 8
) (
    input logic clk,
    input logic rst_n,
    input logic [WIDTH-1:0] binary_in,
    input logic write_enable,
    output logic [DIGITS*4-1:0] bcd_out,
    output logic bcd_valid,
    output logic buffer_full,
    output logic buffer_empty
);

// Input FIFO
logic [WIDTH-1:0] fifo_data_out;
logic fifo_read_enable, fifo_empty_int;

sync_fifo #(
    .WIDTH(WIDTH),
    .DEPTH(BUFFER_DEPTH)
) input_buffer (
    .clk(clk),
    .rst_n(rst_n),
    .write_data(binary_in),
    .write_enable(write_enable),
    .read_data(fifo_data_out),
    .read_enable(fifo_read_enable),

```

```

        .full(buffer_full),
        .empty(fifo_empty_int)
    );

    // BCD converter control
    logic conversion_idle;

    assign fifo_read_enable = !fifo_empty_int && conversion_idle;
    assign buffer_empty = fifo_empty_int;

    // BCD converter instance
    bin_to_bcd #(
        .WIDTH(WIDTH),
        .DIGITS(DIGITS)
    ) converter (
        .clk(clk),
        .rst_n(rst_n),
        .start(fifo_read_enable),
        .binary(fifo_data_out),
        .bcd(bcd_out),
        .done(bcd_valid)
    );

    // Track converter state
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            conversion_idle <= 1;
        end else begin
            if (fifo_read_enable) begin
                conversion_idle <= 0;
            end else if (bcd_valid) begin
                conversion_idle <= 1;
            end
        end
    end
end
endmodule

```

8.16 Troubleshooting Guide

8.16.1 Common Issues and Solutions

8.16.1.1 *Issue 1: Incorrect BCD Output*

Symptoms: Output doesn't match expected decimal value **Causes:** - Insufficient DIGITS parameter
 - Width mismatch between binary input and internal registers - BCD digit corruption during processing

Debug Steps:

```
// Add internal signal monitoring
always_ff @(posedge clk) begin
    if (dut.r_fsm_main == ADD && dut.w_bcd_digit > 4) begin
        $display("ADD operation: digit=%d, new_value=%d",
            dut.w_bcd_digit, dut.w_bcd_digit + 3);
    end
end
```

8.16.1.2 Issue 2: Conversion Never Completes

Symptoms: done signal never asserts **Causes:** - FSM stuck in loop - Incorrect loop termination condition - Clock or reset issues

Debug Steps:

```
// Monitor FSM state transitions
always_ff @(posedge clk) begin
    $display("State: %s, Loop: %d, Digit: %d",
        dut.r_fsm_main.name(), dut.r_loop_count,
        dut.r_digit_index);
end
```

8.16.1.3 Issue 3: Timing Violations

Symptoms: Synthesis reports timing failures **Causes:** - Long combinational paths in ADD logic - High fan-out on BCD register - Insufficient pipeline stages

Solutions:

```
// Add pipeline registers
logic [DIGITS*4-1:0] bcd_pipe;
always_ff @(posedge clk) begin
    bcd_pipe <= r_bcd;
end

// Use parallel digit processing
// (Shown in optimization section)
```

This comprehensive documentation provides everything needed to understand, implement, verify, and optimize the Binary to BCD converter module. The Double-Dabble algorithm, while sequential in nature, provides reliable and predictable conversion suitable for a wide range of digital display applications.

8.17 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

9 CAM Tag Module

9.1 Purpose

The `cam_tag` module implements a Content Addressable Memory (CAM) for tag storage and lookup. It provides fast associative lookup capabilities where data can be found based on content rather than address, commonly used in cache implementations, translation lookaside buffers (TLBs), and routing tables.

9.2 Parameters

- `ENABLE`: Enable/disable CAM functionality (default: 1)
- `N`: Width of each tag in bits (default: 8)
- `DEPTH`: Number of tag entries in the CAM (default: 16)

9.3 Ports

9.3.1 Inputs

- `clk`: System clock
- `rst_n`: Active-low asynchronous reset
- `tag_in_status[N-1:0]`: Tag to search for (lookup operation)
- `mark_valid`: Signal to add a new valid tag
- `tag_in_valid[N-1:0]`: Tag value to mark as valid
- `mark_invalid`: Signal to invalidate an existing tag
- `tag_in_invalid[N-1:0]`: Tag value to mark as invalid

9.3.2 Outputs

- `tags_empty`: Indicates when CAM contains no valid entries
- `tags_full`: Indicates when CAM is completely full
- `tag_status`: Result of lookup operation (1 if tag found, 0 if not found)

9.4 Key Internal Structures

9.4.1 Storage Arrays

```
logic [N-1:0]    r_tag_array [0:DEPTH-1]; // Tag storage array
logic [DEPTH-1:0] r_valid;               // Valid bit for each
entry
```

9.4.2 Search Logic

Three parallel search operations are implemented:

9.4.2.1 1. Next Available Location Search

```
always_comb begin
    w_next_valid_loc = -1;
    for (int i=DEPTH-1; i >= 0; i--)
        if (r_valid[i] == 1'b0)
            w_next_valid_loc = i;
end
```

- Searches from highest to lowest index
- Returns the highest-indexed free location
- Returns -1 if no free locations available

9.4.2.2 2. Valid Tag Match Search

```
always_comb begin
    w_match_loc = -1;
    for (int i = 0; i < DEPTH; i++)
        if (r_valid[i] == 1'b1 && tag_in_status == r_tag_array[i])
            w_match_loc = i;
end
```

- Searches for exact tag match among valid entries
- Returns index of matching entry or -1 if no match

9.4.2.3 3. Invalid Tag Match Search

```
always_comb begin
    w_match_invalid_loc = -1;
    for (int i = 0; i < DEPTH; i++)
        if (r_valid[i] == 1'b1 && tag_in_invalid == r_tag_array[i])
            w_match_invalid_loc = i;
end
```

- Used for invalidation operations

- Finds valid entries that match the invalidation tag

9.5 Operations

9.5.1 Tag Insertion

When mark_valid is asserted and CAM is not full: 1. Store tag_in_valid at w_next_valid_loc
2. Set corresponding valid bit 3. Only occurs when ENABLE != 0

9.5.2 Tag Invalidation

When mark_invalid is asserted and matching tag found: 1. Clear the tag value at matching location
2. Clear corresponding valid bit

9.5.3 Tag Lookup

Continuously compares tag_in_status against all valid entries: - Returns 1 if tag found among valid entries - Returns 0 if tag not found or entry is invalid

9.6 State Management

9.6.1 Update Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_valid <= 'b0;
        for (int i = 0; i < DEPTH; i++) begin
            r_tag_array[i] <= 'b0;
        end
    end else begin
        if (mark_valid && !tags_full && (ENABLE != 0)) begin
            r_tag_array[w_next_valid_loc] <= tag_in_valid;
            r_valid[w_next_valid_loc] <= 1'b1;
        end else if (mark_invalid && w_match_invalid_loc >= 0) begin
            r_tag_array[w_match_invalid_loc] <= 'b0;
            r_valid[w_match_invalid_loc] <= 1'b0;
        end
    end
end
```

9.6.2 Status Signals

- tags_empty = ~|r_valid: NOR of all valid bits
- tags_full = &r_valid: AND of all valid bits
- tag_status: Result of tag lookup operation

9.7 Special Features

9.7.1 Enable Control

When `ENABLE = 0`: - Tag insertion is disabled - Tag lookup and invalidation still function - Useful for disabling CAM without losing existing data

9.7.2 Allocation Strategy

- **First Available:** Uses highest-indexed free location
- **Replacement:** No automatic replacement policy (must manually invalidate)
- **Overflow Protection:** Prevents insertion when full

9.7.3 Debug Support

The module includes simulation-only logic for waveform viewing:

```
// synopsys translate_off
logic [(N*DEPTH)-1:0] flat_r_tag_array;
generate
    for (j = 0; j < DEPTH; j++) begin : gen_flatten_memory
        assign flat_r_tag_array[j*N+:N] = r_tag_array[j];
    end
endgenerate
// synopsys translate_on
```

9.8 Usage Considerations

- **Timing:** All lookups are combinational (single cycle)
- **Capacity:** Monitor `tags_full` before insertions
- **Conflicts:** No protection against duplicate tag insertion
- **Performance:** Search time is constant regardless of occupancy
- **Power:** All entries are searched simultaneously (high power for large depths)

9.9 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

10 Clock Divider Module - Comprehensive Documentation

10.1 Overview

The `clock_divider` module generates multiple divided clock signals from a single input clock using configurable pick-off points from a shared master counter. This approach provides synchronized, phase-aligned divided clocks that are essential for multi-rate digital signal processing, communication systems, and power management applications.

⚠ WARNING: NOT recommended for functional clocks! Use PLL/MMCM/clock manager primitives instead. This module is intended for testbenches, debug outputs, and non-critical timing applications.

10.2 Module Declaration

```
module clock_divider #(
    parameter int N                = 4, // Number of output clocks
    parameter int PO_WIDTH        = 8, // Width of the Pick off
    registers
    parameter int COUNTER_WIDTH = 64 // Width of the counter
) (
    input logic          clk,          // Input clock
    signal
    input logic          rst_n,        // Active low
    reset signal
    input logic [(N*PO_WIDTH-1):0] pickoff_points, // the pick off
    point config registers
    output logic [      N-1:0] divided_clk // Divided clock
    signals
);
```

10.3 Parameters

10.3.1 N

- **Type:** int
- **Default:** 4
- **Description:** Number of independently configurable output clocks
- **Range:** 1 to 16 (practical limit)
- **Impact:** Determines number of output clock domains and resource usage (N registers + N muxes)

10.3.2 PO_WIDTH

- **Type:** int
- **Default:** 8
- **Description:** Bit width of each pick-off point configuration register
- **Range:** 4 to 8 bits
- **Constraints:** Must be $> \lceil \log_2(\text{COUNTER_WIDTH}) \rceil$ to avoid truncation
 - Equivalently: $\text{PO_WIDTH} \geq \lceil \log_2(\text{COUNTER_WIDTH} + 1) \rceil$
 - This ensures PO_WIDTH can hold the value COUNTER_WIDTH without truncation
 - Examples: CW=16 needs $\text{PO} \geq 5$, CW=32 needs $\text{PO} \geq 6$, CW=64 needs $\text{PO} \geq 7$
- **Trade-off:** Larger width allows finer counter bit selection but uses more configuration storage
- **Typical:** PO_WIDTH=8 for COUNTER_WIDTH up to 128

10.3.3 COUNTER_WIDTH

- **Type:** int
- **Default:** 64
- **Description:** Bit width of the master binary counter
- **Range:** 2 to 64 bits (practical range)
- **Impact:** Determines maximum division ratio ($2^{\text{COUNTER_WIDTH}}$)
- **Example:** WIDTH=16 $\rightarrow$ max division ratio = 65536
- **Trade-off:** Larger counters enable slower frequencies but consume more resources

10.3.4 Parameter Relationships

- **Addressing Constraint:** $\text{PO_WIDTH} \geq \lceil \log_2(\text{COUNTER_WIDTH}) \rceil$
- **Division Range:** Each output can divide by 2^1 to $2^{\text{COUNTER_WIDTH}}$

- **Configuration Width:** Total configuration bits = $N \times PO_WIDTH$

10.4 Ports

10.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	Master input clock (source to be divided)
rst_n	1	logic	Active-low asynchronous reset
pickoff_points	$N \times PO_WIDTH$	logic	Packed pickoff point configuration array

pickoff_points Format:

Bit Packing: {po[N-1], ..., po[1], po[0]}

Each po[i] (PO_WIDTH bits): Counter bit index to sample for output i

10.4.2 Outputs

Port	Width	Type	Description
divided_clk	N	logic	Divided clock outputs (registered)

10.5 Architecture Details

10.5.1 Internal Signals

```

logic [COUNTER_WIDTH-1:0] r_divider_counters; // Free-running binary counter
localparam int ADDR_WIDTH = $clog2(COUNTER_WIDTH); // Address width

```

10.5.2 Counter Operation

- Free-running binary counter increments every input clock cycle
- Wraps at $2^{COUNTER_WIDTH}$ (no overflow flag)
- Each bit toggles at half the frequency of the previous bit
 - Bit 0: $clk/2$ (toggles every cycle)
 - Bit 1: $clk/4$ (toggles every 2 cycles)
 - Bit N: $clk/2^{(N+1)}$

10.5.3 Pickoff Point Selection

- Each output i samples `counter[pickoff_points[i]]`
- Division ratio = $2^{(\text{pickoff_point}+1)}$
- Example: `pickoff=3` → samples `counter[3]` → divides by 16

10.5.4 Out-of-Range Handling

- If `pickoff_points[i] ≥ COUNTER_WIDTH`, clamps to `COUNTER_WIDTH-1` (MSB)
- Prevents illegal bit indexing
- MSB provides slowest possible division

10.5.5 Output Registration

- All `divided_clk` outputs are registered for glitch-free operation
- Adds 1 cycle latency but prevents combinational glitches
- Reset clears all outputs to 0 (low phase)

10.6 Operation Principles

10.6.1 Division Ratio Table (100MHz Input Clock)

<code>pickoff_point</code>	Counter Bit	Division Ratio	Output Freq	Period
0	[0]	2	50MHz	20ns
1	[1]	4	25MHz	40ns
2	[2]	8	12.5MHz	80ns
3	[3]	16	6.25MHz	160ns
4	[4]	32	3.125MHz	320ns
5	[5]	64	1.5625MHz	640ns
10	[10]	2048	48.8kHz	20.5μs
15	[15]	65536	1.5kHz	655μs
19	[19]	1048576	95Hz	10.5ms
23	[23]	16777216	6Hz	168ms

10.6.2 Duty Cycle

- All divided clocks have 50% duty cycle
- Result of sampling counter bit (toggle behavior)
- High for $2^{(\text{pickoff_point})}$ input cycles
- Low for $2^{(\text{pickoff_point})}$ input cycles

10.7 Timing Characteristics

10.7.1 Latency

- **Output Latency:** 1 cycle (registered output)
- **Clock-to-Q:** Standard flip-flop delay
- **Propagation:** Counter increment → mux → output register

10.7.2 Phase Relationships

- All divided clocks share same counter → phase-locked to each other
- Divided clocks have phase offset relative to input clock
- Phase offset depends on counter reset value (0)

10.7.3 Jitter

- Input clock jitter propagates to outputs
- Additional quantization jitter: ± 1 input clock cycle
- For low-jitter clocks: Use dedicated PLL/clock synthesis

10.8 Design Examples

10.8.1 Example 1: Four Debug Clocks

```
// Generate clk/4, clk/16, clk/256, clk/65536
logic [31:0] pickoff_cfg;
assign pickoff_cfg = {8'd15, 8'd7, 8'd3, 8'd1}; // {po3, po2, po1, po0}
```

```
clock_divider #(
    .N(4),                // 4 outputs
    .PO_WIDTH(8),         // 8-bit pickoff point
```

```

        .COUNTER_WIDTH(16) // 16-bit counter (max div = 65536)
    ) u_clk_div (
        .clk            (clk_100mhz),
        .rst_n          (rst_n),
        .pickoff_points(pickoff_cfg),
        .divided_clk     ({clk_1p5khz, clk_390khz, clk_6p25mhz, clk_25mhz})
    );

```

10.8.2 Example 2: Baud Rate Generator (Approximate)

```

// For 9600 baud from 100MHz:
// Ideal: 100MHz / 9600 ≈ 10417 (not power-of-2)
// Closest: 213 = 8192 → 100MHz/8192 = 12.2kHz (≈9600 baud)

```

```

logic [7:0] baud_pickoff = 8'd12; // 213 = 8192 division

clock_divider #(
    .N(1),
    .PO_WIDTH(8),
    .COUNTER_WIDTH(16)
) u_baud_clk (
    .clk            (sys_clk),
    .rst_n          (rst_n),
    .pickoff_points(baud_pickoff),
    .divided_clk    (uart_baud_x16) // 16× oversampling clock
);

```

Note: For precise baud rates (not power-of-2), use counter_load_clear instead.

10.8.3 Example 3: Runtime-Programmable Divider

```

// APB-configurable clock divider
logic [7:0] cfg_pickoff_0, cfg_pickoff_1;

always_ff @(posedge pclk) begin
    if (pwrite && paddr == ADDR_PICKOFF_0) cfg_pickoff_0 <=
pdata[7:0];
    if (pwrite && paddr == ADDR_PICKOFF_1) cfg_pickoff_1 <=
pdata[7:0];
end

clock_divider #(
    .N(2),
    .PO_WIDTH(8),
    .COUNTER_WIDTH(24)
) u_cfg_clk_div (
    .clk            (core_clk),
    .rst_n          (core_rst_n),
    .pickoff_points({cfg_pickoff_1, cfg_pickoff_0}),

```

```

        .divided_clk    ({clk_out_1, clk_out_0})
    );

```

10.8.4 Example 4: Test Clock Generation

```

// Generate 8 test clocks for debug
clock_divider #(
    .N(8),                      // 8 debug clocks
    .PO_WIDTH(8),
    .COUNTER_WIDTH(32) // Support ultra-slow clocks
) u_test_clks (
    .clk            (ref_clk),
    .rst_n          (test_rst_n),
    .pickoff_points({8'd20, 8'd18, 8'd16, 8'd14, 8'd12, 8'd10, 8'd8,
8'd6}),
    .divided_clk    (test_clk_bus)
);

```

10.9 Advanced Usage

10.9.1 Changing Pickoff Points at Runtime

```

// To avoid glitches when changing pickoff configuration:
// 1. Assert rst_n (reset)
// 2. Change pickoff_points
// 3. Wait for configuration to settle
// 4. Deassert rst_n
// 5. Outputs start with new division ratios

```

```

always_ff @(posedge cfg_clk) begin
    if (cfg_write) begin
        rst_n <= 1'b0;                      // Assert reset
        new_pickoff <= cfg_data;

        @(posedge cfg_clk);
        pickoff_points <= new_pickoff;      // Update configuration

        @(posedge cfg_clk);
        rst_n <= 1'b1;                      // Release reset
    end
end

```

10.10 Synthesis Considerations

10.10.1 Resource Utilization

- **Flip-Flops:** COUNTER_WIDTH (counter) + N (output registers)

- **LUTs:** N muxes (pickoff selection) + clamping logic
- **Example (N=4, COUNTER_WIDTH=16):** ~20 FFs, ~8 LUTs

10.10.2 Timing Optimization

- **Critical Path:** Counter increment → mux → output register
- **For high-frequency operation:**
 - Reduce COUNTER_WIDTH (if possible)
 - Pipeline counter output (add register stage)
 - Use dedicated clock resources (PLL/MMCM)

10.10.3 Power Optimization

- **Clock Gating:** Gate counter when no divided clocks needed
- **Selective Outputs:** Only generate divided clocks actually used
- **Example:**

```

logic counter_enable;
assign counter_enable = |divided_clk_enable;  // Only run when needed

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) r_divider_counters <= 0;
    else if (counter_enable) r_divider_counters <= r_divider_counters
+ 1;
end

```

10.11 Important Warnings

10.11.1 NOT for Functional Clocks

Do NOT use for: - Primary system clocks - Clocking synchronous logic - Production designs requiring precise timing

Use PLL/MMCM/Clock Manager instead: - Dedicated clock synthesis primitives - Clean clock tree - Proper static timing analysis (STA) - Phase-locked loops for low jitter - Precise frequency generation

10.11.2 Derived Clock Hazards

Using divided_clk as a clock creates a **derived clock**: - Complicates timing analysis - STA tools may not properly constrain - Potential setup/hold violations - Clock domain crossing issues

10.11.3 ⚠️ Only Power-of-2 Divisions

- This module ONLY supports power-of-2 division ratios (2, 4, 8, 16, ...)
- For arbitrary ratios (e.g., divide by 7, 100, 1000): Use counter_load_clear

10.11.4 ⚠️ Glitches During Configuration Changes

- Changing pickoff_points at runtime can cause glitches
- Always reset module when changing configuration
- Do NOT change configuration while outputs are actively used

10.12 Verification Strategy

10.12.1 Test Scenarios

1. **Single Output:** N=1 with various pickoff points
2. **Multiple Outputs:** N=4 with different divisions
3. **Out-of-Range Clamping:** pickoff $\geq$ COUNTER_WIDTH
4. **Runtime Changes:** Modify pickoff_points dynamically
5. **Reset Behavior:** All outputs low after reset
6. **Division Verification:** Measure output frequencies
7. **Edge Cases:**
 - pickoff=0 (divide by 2)
 - pickoff=COUNTER_WIDTH-1 (slowest division)
 - pickoff > COUNTER_WIDTH (clamping)

10.12.2 Coverage Points

```
covergroup clock_divider_cg @(posedge clk);
  cp_pickoff_range: coverpoint pickoff_points[7:0] {
    bins low = {[0:3]};
    bins mid = {[4:11]};
    bins high = {[12:15]};
    bins oob = {[16:255]}; // Out-of-bounds (will clamp)
  }
```



```

        cp_outputs: coverpoint divided_clk {
            bins all_outputs[] = {[1:$]};
        }
    endgroup

```

10.13 Performance Characteristics

10.13.1 Maximum Frequency

- **Limiting Factor:** Counter width and mux depth
- **Typical Performance:**
 - COUNTER_WIDTH=16: ~500 MHz (modern FPGAs)
 - COUNTER_WIDTH=32: ~400 MHz
 - COUNTER_WIDTH=64: ~300 MHz

10.13.2 Resource Scaling

COUNTER_W IDTH	N Outputs	FFs	LUTs	fmax (Est.)
16	4	20	8	500 MHz
32	4	36	10	400 MHz
64	4	68	12	300 MHz
16	8	24	16	500 MHz

10.14 Related Modules

- counter_bin.sv - Simple binary counter (used internally)
- counter_freq_invariant.sv - Time-based counter with 1MHz tick
- counter_load_clear.sv - Arbitrary division ratios (not power-of-2)
- clock_pulse.sv - Configurable pulse generator
- clock_gate_ctrl.sv - Clock gating for power management

10.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

11 Clock Gate Controller - Comprehensive Documentation

11.1 Overview

The `clock_gate_ctrl` module implements an intelligent clock gating controller that automatically stops the clock when a circuit block becomes idle, significantly reducing dynamic power consumption. It features configurable idle timeout, immediate wake-up capability, and safe clock gating using integrated clock gate (ICG) cells.

11.2 Module Declaration

```
module clock_gate_ctrl #(
    parameter int IDLE_CNTR_WIDTH = 4,
    parameter int N = IDLE_CNTR_WIDTH
) (
    // Inputs
    input logic      clk_in,
    input logic      aresetn,
    input logic      cfg_cg_enable,      // Global clock gate
enable
    input logic [N-1:0] cfg_cg_idle_count, // Idle countdown value
    input logic      wakeup,             // Signal to wake up the
block

    // Outputs
    output logic      clk_out,
    output logic      gating             // clock gating indicator
);
```

11.3 Parameters

11.3.1 IDLE_CNTR_WIDTH

- **Type:** int
- **Default:** 4
- **Description:** Bit width of the idle timeout counter
- **Range:** 2 to 16 bits (practical range)
- **Impact:** Determines maximum idle timeout ($2^{\text{IDLE_CNTR_WIDTH}} - 1$ cycles)

- **Power Trade-off:** Larger counters allow longer timeouts but consume more power

11.3.2 N

- **Type:** int
- **Default:** IDLE_CNTR_WIDTH
- **Description:** Alias for counter width (convenience parameter)
- **Usage:** Internal consistency check and code readability

11.4 Ports

11.4.1 Inputs

Port	Width	Type	Description
clk_in	1	logic	Input clock to be gated
aresetn	1	logic	Active-low asynchronous reset
cfg_cg_enable	1	logic	Global clock gating enable control
cfg_cg_idle_count	N	logic	Idle timeout value (configuration)
wakeup	1	logic	Immediate wake-up signal

11.4.2 Outputs

Port	Width	Type	Description
clk_out	1	logic	Gated output clock
gating	1	logic	Clock gating status indicator

11.5 Clock Gating Theory

11.5.1 Why Clock Gating?

Clock gating is one of the most effective power reduction techniques: - **Dynamic Power:** $P = C \times V^2 \times f \times \alpha$ (α = switching activity) - **Clock Networks:** Typically consume 20-40% of total chip power -

Gating Benefits: Can reduce power by 50-90% in idle blocks - **Area Overhead:** Minimal (< 5% for typical designs)

11.5.2 Clock Gating Safety

Proper clock gating requires: 1. **Glitch-Free:** No spurious clock edges during gating transitions 2. **Setup/Hold:** Adequate timing margins around gating decisions 3. **Reset Handling:** Proper behavior during reset conditions 4. **Tool Support:** Use of qualified ICG (Integrated Clock Gate) cells

11.6 Architecture and Implementation

11.6.1 Idle Counter Logic

```
logic [N-1:0] r_idle_counter;

always_ff @(posedge clk_in or negedge aresetn) begin
    if (!aresetn) begin
        r_idle_counter <= cfg_cg_idle_count;
    end else begin
        if (wakeup || !cfg_cg_enable) begin
            // On wakeup or global disable, reset counter
            r_idle_counter <= cfg_cg_idle_count;
        end else if (r_idle_counter != 'h0) begin
            // Normal counting operation - decrement when not zero
            r_idle_counter <= r_idle_counter - 1'b1;
        end
        // When counter reaches zero, it stays at zero
    end
end
```

11.6.2 Gating Control Logic

```
wire w_gate_enable = cfg_cg_enable && !wakeup && (r_idle_counter == 'h0);

// Instantiate the ICG cell
icg u_icg (
    .clk(clk_in),
    .en(~w_gate_enable), // Enable is active high in ICG
    .gclk(clk_out)
);

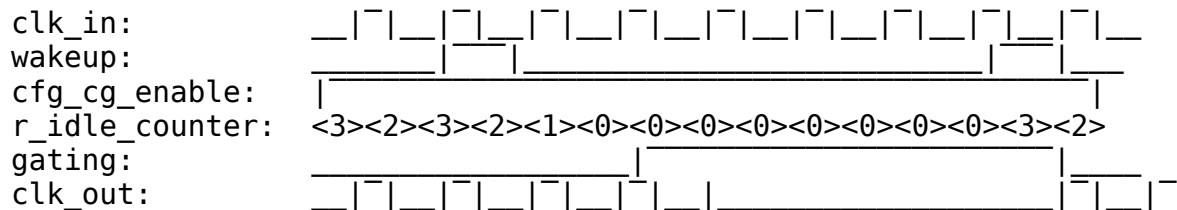
assign gating = w_gate_enable;
```

11.6.3 State Machine Behavior

The controller operates as a countdown timer with three states:

1. **ACTIVE State:** Counter loaded with `cfg_cg_idle_count`
2. **COUNTDOWN State:** Counter decrements each cycle when no activity
3. **GATED State:** Counter at zero, clock is gated off

11.6.4 Timing Diagram



11.7 ICG (Integrated Clock Gate) Cell

11.7.1 Standard ICG Interface

```

module icg (
    input logic clk,      // Input clock
    input logic en,       // Enable (active high)
    output logic gclk     // Gated clock output
);

    // Implementation varies by technology library
    // Typically includes:
    // - Latch to avoid glitches
    // - AND gate for gating
    // - Proper setup/hold timing

endmodule

```

11.7.2 ICG Implementation (Conceptual)

```

// Simplified ICG implementation (technology-specific)
module icg_generic (
    input logic clk,
    input logic en,
    output logic gclk
);

    logic en_latched;

    // Latch enable on falling edge to avoid glitches
    always_latch begin
        if (~clk)
            en_latched = en;
    end

endmodule

```

```

// Gate clock with latched enable
assign gclk = clk & en_latched;

```

```
endmodule
```

11.8 Design Examples and Applications

11.8.11. CPU Core Clock Gating

```

module cpu_clock_gating (
    input logic clk_cpu,
    input logic rst_n,

    // CPU control signals
    input logic cpu_active,
    input logic interrupt_pending,
    input logic debug_mode,

    // Power management
    input logic [3:0] idle_threshold,
    input logic power_save_enable,

    // Output clocks
    output logic clk_cpu_gated,
    output logic clk_debug,
    output logic cpu_sleeping
);

    logic cpu_wakeup;
    logic debug_wakeup;

    // CPU wake-up conditions
    assign cpu_wakeup = cpu_active || interrupt_pending;

    // Debug interface always active in debug mode
    assign debug_wakeup = debug_mode || cpu_wakeup;

    // Main CPU clock gating
    clock_gate_ctrl #(
        .IDLE_CNTR_WIDTH(4)
    ) cpu_gate (
        .clk_in(clk_cpu),
        .aresetn(rst_n),
        .cfg_cg_enable(power_save_enable),
        .cfg_cg_idle_count(idle_threshold),
        .wakeup(cpu_wakeup),
        .clk_out(clk_cpu_gated),

```

```

        .gating(cpu_sleeping)
    );

    // Debug clock gating (shorter timeout)
    clock_gate_ctrl #(
        .IDLE_CNTR_WIDTH(3)
    ) debug_gate (
        .clk_in(clk_cpu),
        .aresetn(rst_n),
        .cfg_cg_enable(power_save_enable && !debug_mode),
        .cfg_cg_idle_count(3'd2), // Short timeout for debug
        .wakeup(debug_wakeup),
        .clk_out(clk_debug),
        .gating() // Unused
    );

```

endmodule

11.8.22. Peripheral Interface Clock Management

```

module peripheral_clock_manager (
    input logic      clk_system,
    input logic      rst_n,

    // Peripheral activity signals
    input logic      uart_active,
    input logic      spi_active,
    input logic      i2c_active,
    input logic      timer_active,

    // Configuration
    input logic [7:0] power_config, // [7:4] = enable bits, [3:0]
    = timeout

    // Gated clocks
    output logic      clk_uart,
    output logic      clk_spi,
    output logic      clk_i2c,
    output logic      clk_timer,

    // Status
    output logic [3:0] peripheral_sleeping
);

logic [3:0] idle_timeout;
logic [3:0] gate_enables;

assign idle_timeout = power_config[3:0];

```

```

assign gate_enables = power_config[7:4];

// UART clock gating
clock_gate_ctrl #(4) uart_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[0]),
    .cfg_cg_idle_count(idle_timeout),
    .wakeup(uart_active),
    .clk_out(clk_uart),
    .gating(peripheral_sleeping[0])
);

// SPI clock gating
clock_gate_ctrl #(4) spi_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[1]),
    .cfg_cg_idle_count(idle_timeout),
    .wakeup(spi_active),
    .clk_out(clk_spi),
    .gating(peripheral_sleeping[1])
);

// I2C clock gating
clock_gate_ctrl #(4) i2c_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[2]),
    .cfg_cg_idle_count(idle_timeout),
    .wakeup(i2c_active),
    .clk_out(clk_i2c),
    .gating(peripheral_sleeping[2])
);

// Timer clock gating (different timeout)
clock_gate_ctrl #(4) timer_cg (
    .clk_in(clk_system),
    .aresetn(rst_n),
    .cfg_cg_enable(gate_enables[3]),
    .cfg_cg_idle_count(4'd1), // Quick timeout for timers
    .wakeup(timer_active),
    .clk_out(clk_timer),
    .gating(peripheral_sleeping[3])
);

```

endmodule

11.8.33. Memory Subsystem Clock Gating

```
module memory_clock_gating (
    input logic      clk_memory,
    input logic      rst_n,

    // Memory access signals
    input logic      mem_read,
    input logic      mem_write,
    input logic      mem_refresh_req,
    input logic      cache_miss,

    // Configuration
    input logic [2:0] mem_power_mode, // 0=always on, 7=aggressive
gating

    // Gated clocks
    output logic      clk_mem_ctrl,    // Memory controller
    output logic      clk_mem_data,    // Data path
    output logic      clk_mem_refresh, // Refresh logic

    // Status
    output logic [2:0] mem_gating_status
);

    logic [3:0] timeout_ctrl, timeout_data, timeout_refresh;
    logic mem_active, data_active, refresh_active;

    // Activity detection
    assign mem_active = mem_read || mem_write || cache_miss;
    assign data_active = mem_read || mem_write;
    assign refresh_active = mem_refresh_req;

    // Timeout configuration based on power mode
    always_comb begin
        case (mem_power_mode)
            3'd0: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd15, 4'd15, 4'd15}; // Always on
            3'd1: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd10, 4'd8, 4'd12}; // Conservative
            3'd2: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd8, 4'd6, 4'd10}; // Moderate
            3'd3: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd6, 4'd4, 4'd8}; // Balanced
            3'd4: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd4, 4'd2, 4'd6}; // Aggressive
            3'd5: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd2, 4'd1, 4'd4}; // Very aggressive
            3'd6: {timeout_ctrl, timeout_data, timeout_refresh} =
```

```

{4'd1, 4'd0, 4'd2};    // Maximum power save
    3'd7: {timeout_ctrl, timeout_data, timeout_refresh} =
{4'd0, 4'd0, 4'd1};    // Ultra power save
    endcase
end

// Memory controller clock gating
clock_gate_ctrl #(4) ctrl_cg (
    .clk_in(clk_memory),
    .aresetn(rst_n),
    .cfg_cg_enable(mem_power_mode != 0),
    .cfg_cg_idle_count(timeout_ctrl),
    .wakeup(mem_active),
    .clk_out(clk_mem_ctrl),
    .gating(mem_gating_status[0])
);

// Data path clock gating
clock_gate_ctrl #(4) data_cg (
    .clk_in(clk_memory),
    .aresetn(rst_n),
    .cfg_cg_enable(mem_power_mode != 0),
    .cfg_cg_idle_count(timeout_data),
    .wakeup(data_active),
    .clk_out(clk_mem_data),
    .gating(mem_gating_status[1])
);

// Refresh logic clock gating
clock_gate_ctrl #(4) refresh_cg (
    .clk_in(clk_memory),
    .aresetn(rst_n),
    .cfg_cg_enable(mem_power_mode > 2), // Only gate in aggressive
modes
    .cfg_cg_idle_count(timeout_refresh),
    .wakeup(refresh_active),
    .clk_out(clk_mem_refresh),
    .gating(mem_gating_status[2])
);

endmodule

```

11.8.44. Adaptive Clock Gating with Activity Monitoring

```

module adaptive_clock_gating #(
    parameter int ACTIVITY_WINDOW = 256 // Activity monitoring window
) (
    input logic      clk_in,
    input logic      rst_n,

```

```

// Activity signals
input logic [7:0] activity_vector,

// Configuration
input logic adaptive_enable,
input logic [3:0] base_timeout,

// Outputs
output logic clk_out,
output logic gating_active,
output logic [7:0] activity_level // Activity percentage
);

logic [15:0] activity_counter;
logic [7:0] activity_window_counter;
logic [3:0] adaptive_timeout;
logic block_active;

// Activity monitoring
always_ff @(posedge clk_in or negedge rst_n) begin
    if (!rst_n) begin
        activity_counter <= 0;
        activity_window_counter <= 0;
        activity_level <= 0;
    end else begin
        // Count activity in current window
        if (|activity_vector)
            activity_counter <= activity_counter + 1;

        activity_window_counter <= activity_window_counter + 1;

        // Update activity level at end of window
        if (activity_window_counter == ACTIVITY_WINDOW - 1) begin
            activity_level <= activity_counter[15:8]; // Scale to
0-255

            activity_counter <= 0;
            activity_window_counter <= 0;
        end
    end
end

// Adaptive timeout based on activity level
always_comb begin
    if (!adaptive_enable) begin
        adaptive_timeout = base_timeout;
    end else begin
        case (activity_level)

```

```

        8'h00: adaptive_timeout = base_timeout + 4'd8; // Very
low activity
        8'h01: adaptive_timeout = base_timeout + 4'd6; // Low
activity
        8'h02: adaptive_timeout = base_timeout + 4'd4; //
Medium-low activity
        8'h04: adaptive_timeout = base_timeout + 4'd2; //
Medium activity
        8'h08: adaptive_timeout = base_timeout; //
Normal activity
        default: adaptive_timeout = (base_timeout > 2) ?
base_timeout - 2 : 0; // High activity
    endcase
end
end

// Activity detection
assign block_active = |activity_vector;

// Clock gating controller
clock_gate_ctrl #(4) adaptive_cg (
    .clk_in(clk_in),
    .aresetn(rst_n),
    .cfg_cg_enable(1'b1),
    .cfg_cg_idle_count(adaptive_timeout),
    .wakeup(block_active),
    .clk_out(clk_out),
    .gating(gating_active)
);

endmodule

```

11.8.55. Hierarchical Clock Gating System

```

module hierarchical_clock_gating (
    input logic clk_root,
    input logic rst_n,

    // Global control
    input logic global_cg_enable,
    input logic [3:0] global_timeout,

    // Subsystem activity
    input logic cpu_active,
    input logic dsp_active,
    input logic peripheral_active,
    input logic memory_active,

    // Hierarchical clocks

```

```

output logic      clk_cpu_domain,
output logic      clk_dsp_domain,
output logic      clk_peripheral_domain,
output logic      clk_memory_domain,

// Power monitoring
output logic [3:0] domain_gating_status,
output logic [15:0] power_savings_estimate
);

logic system_active;
logic clk_system_gated;
logic system_gated;

// System-level activity detection
assign system_active = cpu_active || dsp_active ||
peripheral_active || memory_active;

// Top-level system clock gating
clock_gate_ctrl #(4) system_cg (
    .clk_in(clk_root),
    .aresetn(rst_n),
    .cfg_cg_enable(global_cg_enable),
    .cfg_cg_idle_count(global_timeout + 4'd4), // Longer timeout
for system level
    .wakeup(system_active),
    .clk_out(clk_system_gated),
    .gating(system_gated)
);

// CPU domain gating
clock_gate_ctrl #(4) cpu_cg (
    .clk_in(clk_system_gated),
    .aresetn(rst_n),
    .cfg_cg_enable(global_cg_enable && !system_gated),
    .cfg_cg_idle_count(global_timeout),
    .wakeup(cpu_active),
    .clk_out(clk_cpu_domain),
    .gating(domain_gating_status[0])
);

// DSP domain gating
clock_gate_ctrl #(4) dsp_cg (
    .clk_in(clk_system_gated),
    .aresetn(rst_n),
    .cfg_cg_enable(global_cg_enable && !system_gated),
    .cfg_cg_idle_count(global_timeout - 1), // Aggressive for DSP
    .wakeup(dsp_active),

```

```

        .clk_out(clk_dsp_domain),
        .gating(domain_gating_status[1])
    );

    // Peripheral domain gating
    clock_gate_ctrl #(4) peripheral_cg (
        .clk_in(clk_system_gated),
        .aresetn(rst_n),
        .cfg_cg_enable(global_cg_enable && !system_gated),
        .cfg_cg_idle_count(global_timeout + 1), // Conservative for
peripherals
        .wakeup(peripheral_active),
        .clk_out(clk_peripheral_domain),
        .gating(domain_gating_status[2])
    );

    // Memory domain gating
    clock_gate_ctrl #(4) memory_cg (
        .clk_in(clk_system_gated),
        .aresetn(rst_n),
        .cfg_cg_enable(global_cg_enable && !system_gated),
        .cfg_cg_idle_count(global_timeout + 2), // Conservative for
memory
        .wakeup(memory_active),
        .clk_out(clk_memory_domain),
        .gating(domain_gating_status[3])
    );

    // Power savings estimation (simplified)
    logic [3:0] gated_domains;
    assign gated_domains = domain_gating_status[3:0];

    always_ff @(posedge clk_root or negedge rst_n) begin
        if (!rst_n) begin
            power_savings_estimate <= 0;
        end else begin
            // Rough power savings estimate (percentage)
            case ({system_gated, gated_domains})
                5'b1_0000: power_savings_estimate <= 16'd9000; // 90%
savings
                5'b0_1111: power_savings_estimate <= 16'd7000; // 70%
savings
                5'b0_1110: power_savings_estimate <= 16'd6000; // 60%
savings
                5'b0_1100: power_savings_estimate <= 16'd5000; // 50%
savings
                5'b0_1000: power_savings_estimate <= 16'd3000; // 30%
savings
            end
        end
    end

```

```

savings        5'b0_0100: power_savings_estimate <= 16'd2000; // 20%
savings        5'b0_0010: power_savings_estimate <= 16'd1500; // 15%
savings        5'b0_0001: power_savings_estimate <= 16'd1000; // 10%
savings        default:   power_savings_estimate <= 16'd0000; // 0%
savings
    endcase
end
end
endmodule

```

11.9 Advanced Features

11.9.11. Glitch-Free Clock Switching

```

module glitch_free_clock_gate (
    input  logic clk_in,
    input  logic rst_n,
    input  logic gate_enable,
    input  logic wakeup,
    output logic clk_out
);

    logic gate_enable_sync;
    logic gate_request;

    // Synchronize gate enable to avoid glitches
    always_ff @(posedge clk_in or negedge rst_n) begin
        if (!rst_n) begin
            gate_enable_sync <= 0;
            gate_request <= 0;
        end else begin
            gate_enable_sync <= gate_enable;
            gate_request <= gate_enable_sync && !wakeup;
        end
    end

    // Use ICG with synchronized control
    icg u_icg (
        .clk(clk_in),
        .en(~gate_request),
        .gclk(clk_out)
    );

```

endmodule

11.9.22. Multi-Level Clock Gating

```
module multi_level_clock_gate #(
    parameter int LEVELS = 3
) (
    input logic clk_in,
    input logic rst_n,
    input logic [LEVELS-1:0] level_enables,
    input logic [LEVELS-1:0] level_wakeups,
    input logic [4*LEVELS-1:0] level_timeouts,
    output logic clk_out,
    output logic [LEVELS-1:0] level_gating_status
);

    logic [LEVELS:0] clk_levels; // Include input clock
    assign clk_levels[0] = clk_in;

    genvar i;
    generate
        for (i = 0; i < LEVELS; i++) begin : level_gates
            logic [3:0] timeout;
            assign timeout = level_timeouts[i*4+:4];

            clock_gate_ctrl #(4) level_cg (
                .clk_in(clk_levels[i]),
                .aresetn(rst_n),
                .cfg_cg_enable(level_enables[i]),
                .cfg_cg_idle_count(timeout),
                .wakeup(|level_wakeups[LEVELS-1:i]), // Wake if any
                higher level active
                .clk_out(clk_levels[i+1]),
                .gating(level_gating_status[i])
            );
        end
    endgenerate

    assign clk_out = clk_levels[LEVELS];

endmodule
```


11.10 Verification and Testing

11.10.1 Comprehensive Test Bench

```
module tb_clock_gate_ctrl;

    parameter IDLE_CNTR_WIDTH = 4;

    logic clk_in, aresetn;
    logic cfg_cg_enable;
    logic [IDLE_CNTR_WIDTH-1:0] cfg_cg_idle_count;
    logic wakeup;
    logic clk_out, gating;

    // Clock generation
    initial clk_in = 0;
    always #5ns clk_in = ~clk_in; // 100MHz

    // DUT instantiation
    clock_gate_ctrl #(
        .IDLE_CNTR_WIDTH(IDLE_CNTR_WIDTH)
    ) dut (.*);

    // Test sequence
    initial begin
        // Initialize
        aresetn = 0;
        cfg_cg_enable = 0;
        cfg_cg_idle_count = 4'd5;
        wakeup = 0;

        #100ns aresetn = 1;

        // Test basic gating
        test_basic_gating();

        // Test immediate wakeup
        test_immediate_wakeup();

        // Test global disable
        test_global_disable();

        // Test different timeouts
        test_timeout_values();

        $display("All tests completed!");
        $finish;
    end
end
```

```

// Test basic clock gating functionality
task test_basic_gating();
    begin
        $display("Testing basic clock gating...");

        cfg_cg_enable = 1;
        cfg_cg_idle_count = 4'd3;

        // Wait for gating to occur
        wait_for_gating();

        // Verify clock is gated
        check_clock_gated();

        // Wake up and verify clock restoration
        wakeup = 1;
        #20ns wakeup = 0;
        check_clock_active();
    end
endtask

// Wait for clock gating to occur
task wait_for_gating();
    int timeout_cycles;
    begin
        timeout_cycles = 0;
        while (!gating && timeout_cycles < 100) begin
            @(posedge clk_in);
            timeout_cycles++;
        end

        if (!gating) begin
            $error("Clock gating did not occur within timeout");
        end else begin
            $display("Clock gating occurred after %d cycles",
timeout_cycles);
        end
    end
endtask

// Check that clock is properly gated
task check_clock_gated();
    logic prev_clk_out;
    int stable_cycles;
    begin
        prev_clk_out = clk_out;
        stable_cycles = 0;

```

```

        repeat (10) begin
            @(posedge clk_in);
            if (clk_out == prev_clk_out) begin
                stable_cycles++;
            end else begin
                stable_cycles = 0;
                prev_clk_out = clk_out;
            end
        end

        if (stable_cycles >= 5) begin
            $display("PASS: Clock is properly gated");
        end else begin
            $error("FAIL: Clock is not properly gated");
        end
    end
endtask

// Check that clock is active
task check_clock_active();
    int edge_count;
    begin
        edge_count = 0;

        repeat (20) begin
            @(posedge clk_in);
            if ($rose(clk_out)) edge_count++;
        end

        if (edge_count >= 8) begin
            $display("PASS: Clock is active after wakeup");
        end else begin
            $error("FAIL: Clock not properly restored after
wakeup");
        end
    end
endtask

endmodule

```

11.10.2 Coverage Model

```

covergroup clock_gate_cg @(posedge clk_in);

    cp_idle_count: coverpoint cfg_cg_idle_count {
        bins short_timeout[] = {[0:3]};
        bins medium_timeout[] = {[4:10]};
        bins long_timeout[] = {[11:15]};
    }
endgroup

```

```

}

cp_counter_values: coverpoint dut.r_idle_counter {
    bins countdown[] = {[0:15]};
    bins zero = {0};
}

cp_gating_enable: coverpoint cfg_cg_enable {
    bins enabled = {1};
    bins disabled = {0};
}

cp_wakeup: coverpoint wakeup {
    bins active = {1};
    bins inactive = {0};
}

cp_gating_status: coverpoint gating {
    bins gated = {1};
    bins ungated = {0};
}

// Cross coverage
cross_timeout_gating: cross cp_idle_count, cp_gating_status;
cross_wakeup_gating: cross cp_wakeup, cp_gating_status;

```

endcovergroup

11.10.3 Formal Verification Properties

module clock_gate_properties;

```

// Bind to DUT
bind clock_gate_ctrl clock_gate_properties props_inst (.*);

// Property: Immediate wakeup
property immediate_wakeup;
    @(posedge clk_in) disable iff (!aresetn)
    wakeup |-> ##1 !gating;
endproperty

// Property: Gating after timeout
property gating_after_timeout;
    @(posedge clk_in) disable iff (!aresetn)
    cfg_cg_enable && !wakeup && (dut.r_idle_counter == 0) |->
gating;
endproperty

```

```

// Property: Counter behavior
property counter_countdown;
    @(posedge clk_in) disable iff (!aresetn)
        cfg_cg_enable && !wakeup && (dut.r_idle_counter > 0) | =>
            (dut.r_idle_counter == $past(dut.r_idle_counter) - 1);
endproperty

// Property: Global disable
property global_disable;
    @(posedge clk_in) disable iff (!aresetn)
        !cfg_cg_enable | -> !gating;
endproperty

// Assertions
assert property (immediate_wakeup);
assert property (gating_after_timeout);
assert property (counter_countdown);
assert property (global_disable);

endmodule

```

11.11 Power Analysis and Optimization

11.11.1 Power Savings Calculation

```

module power_analysis_wrapper (
    input logic      clk_in,
    input logic      rst_n,
    input logic      cfg_cg_enable,
    input logic [3:0] cfg_cg_idle_count,
    input logic      wakeup,
    output logic      clk_out,
    output logic      gating,
    output logic [15:0] power_savings_percent
);

// Clock gate controller
clock_gate_ctrl #(4) cg_ctrl (
    .clk_in(clk_in),
    .aresetn(rst_n),
    .cfg_cg_enable(cfg_cg_enable),
    .cfg_cg_idle_count(cfg_cg_idle_count),
    .wakeup(wakeup),
    .clk_out(clk_out),
    .gating(gating)
);

// Power monitoring

```

```

logic [31:0] total_cycles;
logic [31:0] gated_cycles;
logic [31:0] ungated_cycles;

always_ff @(posedge clk_in or negedge rst_n) begin
    if (!rst_n) begin
        total_cycles <= 0;
        gated_cycles <= 0;
        ungated_cycles <= 0;
    end else begin
        total_cycles <= total_cycles + 1;
        if (gating)
            gated_cycles <= gated_cycles + 1;
        else
            ungated_cycles <= ungated_cycles + 1;
    end
end

// Calculate power savings percentage
always_ff @(posedge clk_in) begin
    if (total_cycles > 1000) begin // Avoid division by small
numbers
        power_savings_percent <= (gated_cycles * 100) /
total_cycles;
    end else begin
        power_savings_percent <= 0;
    end
end

endmodule

```

11.11.2 Technology-Specific ICG Implementation

```

// Example for TSMC technology
module icg_tsmc (
    input logic clk,
    input logic en,
    output logic gclk
);

    // Technology-specific implementation
    CKGTPBUF_X1M_A9TH u_icg (
        .CK(clk),
        .E(en),
        .GCK(gclk)
    );

endmodule

```

```

// Example for Intel/Altera technology
module icg_intel (
    input logic clk,
    input logic en,
    output logic gclk
);

    // Intel FPGA implementation
    logic en_reg;

    always_ff @(negedge clk) begin
        en_reg <= en;
    end

    assign gclk = clk & en_reg;

endmodule

// Generic behavioral model
module icg_behavioral (
    input logic clk,
    input logic en,
    output logic gclk
);

    logic en_latch;

    // Latch enable on falling edge to prevent glitches
    always_latch begin
        if (!clk)
            en_latch = en;
    end

    // Gate the clock
    assign gclk = clk & en_latch;

endmodule

```

11.12 Synthesis Considerations

11.12.1 Resource Utilization

Configuration	LUTs	FFs	ICG Cells	Area Overhead
4-bit	8	4	1	< 2%

Configuration	LUTs	FFs	ICG Cells	Area Overhead
counter				
8-bit	15	8	1	< 3%
counter				
16-bit	30	16	1	< 5%
counter				

11.12.2 Timing Constraints

SDC constraints for clock gating

```
create_clock -name clk_in -period 10.0 [get_ports clk_in]
```

Clock gating timing

```
set_case_analysis 0 [get_pins u_icg/en]
```

```
set_case_analysis 1 [get_pins u_icg/en]
```

Setup/hold for gating control

```
set_input_delay -clock clk_in -max 2.0 [get_ports wakeup]
```

```
set_input_delay -clock clk_in -min 0.5 [get_ports wakeup]
```

Clock gating cell timing

```
set_clock_gating_check -setup 0.2 -hold 0.1 [get_pins u_icg/en]
```

11.12.3 Power Intent (UPF)

UPF power intent for clock gating

```
create_power_domain PD_MAIN
```

```
create_power_domain PD_GATED
```

Clock gating specification

```
set_domain_supply_net PD_MAIN -primary_power_net VDD -  
primary_ground_net VSS
```

```
set_domain_supply_net PD_GATED -primary_power_net VDD -  
primary_ground_net VSS
```

Clock gating control

```
create_power_switch PS_GATED \  
-domain PD_GATED \  
-input_supply_port {vin VDD} \  
-output_supply_port {vout VDD_GATED} \  
-control_port {ctrl gating} \  
-on_state {on vin {!ctrl}}
```


11.13 Design Guidelines and Best Practices

11.13.1 1. Timeout Selection

```
// Guidelines for timeout values
parameter TIMEOUT_GUIDELINES = {
    "CPU cores: 4-16 cycles",
    "Memory controllers: 8-32 cycles",
    "Peripherals: 16-64 cycles",
    "Debug interfaces: 2-8 cycles",
    "Always-on domains: Disable gating"
};
```

11.13.2 2. Activity Signal Design

```
// Good activity signal examples
logic cpu_active;
assign cpu_active = instruction_fetch ||
                    data_access ||
                    interrupt_processing ||
                    debug_access;

// Avoid overly broad activity signals
logic bad_activity;
assign bad_activity = |all_system_signals; // Too broad

// Avoid overly narrow activity signals
logic narrow_activity;
assign narrow_activity = specific_register_write; // Too narrow
```

11.13.3 3. Hierarchical Gating Strategy

```
System Level (Longest timeout)
├── CPU Domain (Medium timeout)
│   ├── ALU (Short timeout)
│   ├── Cache (Medium timeout)
│   └── Debug (Very short timeout)
├── DSP Domain (Short timeout)
├── Peripheral Domain (Long timeout)
└── Memory Domain (Medium timeout)
```

11.13.4 4. Verification Checklist

- ☐ Verify gating occurs after correct timeout
- ☐ Verify immediate wakeup functionality
- ☐ Test global enable/disable

- ☐ Check reset behavior
- ☐ Validate power savings measurements
- ☐ Test all timeout configurations
- ☐ Verify clock integrity (no glitches)
- ☐ Check setup/hold timing margins

11.14 Common Applications Summary

1. **CPU Power Management:** Core, cache, and pipeline clock gating
2. **Peripheral Controllers:** UART, SPI, I2C, timer clock gating
3. **Memory Subsystems:** Controller, data path, refresh logic gating
4. **Graphics/Video:** Pixel processing, display controller gating
5. **Communication Blocks:** Baseband, RF, protocol stack gating
6. **Test and Debug:** Scan chains, JTAG, debug interface gating
7. **AI/ML Accelerators:** Compute engine, memory interface gating

11.15 Troubleshooting Guide

11.15.1 Common Issues

1. **Clock Glitches:** Use qualified ICG cells, proper setup/hold margins
2. **Timing Violations:** Ensure adequate timing margins for gating signals
3. **Unexpected Gating:** Check activity signal connectivity and logic
4. **Power Not Saved:** Verify clock tree reaches gated logic
5. **Reset Issues:** Ensure proper reset sequencing and distribution

11.15.2 Debug Techniques

1. **Simulation:** Monitor gating signals and clock activity
2. **Power Analysis:** Use tools to measure actual power savings
3. **Timing Analysis:** Check setup/hold violations on gating paths

4. **Logic Analyzer:** Capture real-time gating behavior in hardware
5. **Power Profiling:** Measure current consumption with/without gating

The clock gate controller provides an essential power management capability, enabling significant dynamic power reduction with minimal area overhead and design complexity. Proper implementation requires careful attention to timing, safety, and verification to ensure reliable operation while maximizing power savings.

11.16 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

12 Clock Pulse Generator - Comprehensive Documentation

12.1 Overview

The `clock_pulse` module generates periodic pulse signals with configurable width (period). It creates a single-cycle pulse every `WIDTH` clock cycles, making it ideal for timing generation, heartbeat signals, sampling triggers, and periodic event generation in digital systems.

12.2 Module Declaration

```
module clock_pulse #(
    parameter int WIDTH = 10 // Width of the generated pulse in clock
    cycles
) (
    input logic clk, // Input clock signal
    input logic rst_n, // Input reset signal
    output logic pulse // Output pulse signal
);
```

12.3 Parameters

12.3.1 WIDTH

- **Type:** int
- **Default:** 10
- **Description:** Period of the pulse generation in clock cycles

- **Range:** 2 to $2^{32}-1$ (practical range)
- **Impact:** Determines pulse frequency ($f_{\text{clk}} / \text{WIDTH}$)
- **Pulse Width:** Always exactly 1 clock cycle
- **Duty Cycle:** $1/\text{WIDTH}$ (e.g., 10% for $\text{WIDTH}=10$)

12.4 Ports

12.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low asynchronous reset

12.4.2 Outputs

Port	Width	Type	Description
pulse	1	logic	Periodic pulse output

12.5 Architecture and Implementation

12.5.1 Internal Counter

```

logic [WIDTH-1:0] r_counter;
logic [WIDTH-1:0] w_width_minus_one;

// Create a properly sized constant
assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

```

12.5.2 Core Logic

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse      <= 'b0;
    end else begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1'b1;
        else

```

```

        r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one);
    end
end

```

12.5.3 Operation Principles

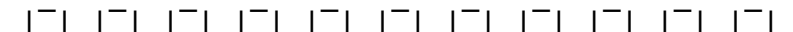
1. **Counter:** Free-running counter from 0 to WIDTH-1
2. **Pulse Generation:** Pulse asserted when counter reaches WIDTH-1
3. **Auto-Reset:** Counter wraps to 0 after reaching maximum
4. **Synchronous:** All operations synchronized to input clock
5. **Single Cycle:** Pulse duration is exactly one clock cycle

12.5.4 Timing Characteristics

- **Period:** WIDTH clock cycles
- **Frequency:** $f_{\text{clk}} / \text{WIDTH}$
- **Pulse Width:** 1 clock cycle
- **Duty Cycle:** $1/\text{WIDTH}$
- **Phase:** Pulse occurs on last count (WIDTH-1)

12.6 Timing Diagrams

12.6.1 Basic Operation (WIDTH=4)

Clock: 
 Counter: < 0 > 1 > 2 > 3 > 0 > 1 > 2 > 3 > 0 >
 Pulse: 

12.6.2 Reset Behavior

Clock: 
 Reset_n: 
 Counter: < 0 > 1 > 2 > 3 > 0 > 0 > 1 > 2 > 3 >
 Pulse: 

12.6.3 Different WIDTH Values

WIDTH	Period	Duty Cycle	Use Case
2	2 cycles	50%	Clock divider

WIDTH	Period	Duty Cycle	Use Case
4	4 cycles	25%	Sampling
10	10 cycles	10%	Timing reference
100	100 cycles	1%	Heartbeat
1000	1000 cycles	0.1%	Slow events

12.7 Design Examples and Applications

12.7.1 1. Heartbeat and Status Indicators

```

module system_heartbeat #(
    parameter int CLOCK_FREQ_HZ = 100_000_000, // 100MHz
    parameter int HEARTBEAT_FREQ_HZ = 1 // 1Hz heartbeat
) (
    input logic clk,
    input logic rst_n,
    input logic system_active,
    output logic heartbeat_led,
    output logic status_ok
);

    localparam int HEARTBEAT_WIDTH = CLOCK_FREQ_HZ /
HEARTBEAT_FREQ_HZ;

    logic heartbeat_pulse;

    // Generate 1Hz heartbeat pulse
    clock_pulse #(
        .WIDTH(HEARTBEAT_WIDTH)
    ) heartbeat_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(heartbeat_pulse)
    );

    // LED control with heartbeat
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            heartbeat_led <= 1'b0;
        end else if (heartbeat_pulse) begin
            heartbeat_led <= ~heartbeat_led; // Toggle LED
        end
    end

```

```

// System status based on heartbeat activity
logic [7:0] heartbeat_monitor;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        heartbeat_monitor <= 8'h00;
    end else begin
        heartbeat_monitor <= {heartbeat_monitor[6:0],
heartbeat_pulse};
    end
end

assign status_ok = system_active && |heartbeat_monitor;

endmodule

```

12.7.22. Sampling and Data Acquisition

```

module data_sampler #(
    parameter int SAMPLE_RATE_KHZ = 48,
    parameter int CLOCK_FREQ_MHZ = 100,
    parameter int DATA_WIDTH = 16
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    input logic [DATA_WIDTH-1:0] analog_data,

    output logic [DATA_WIDTH-1:0] sampled_data,
    output logic sample_valid,
    output logic buffer_full
);

    localparam int SAMPLE_WIDTH = (CLOCK_FREQ_MHZ * 1000) /
SAMPLE_RATE_KHZ;

    logic sample_trigger;

    // Generate sampling trigger
    clock_pulse #(
        .WIDTH(SAMPLE_WIDTH)
    ) sample_clock (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(sample_trigger)
    );

    // Sample data on trigger

```

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        sampled_data <= '0;
        sample_valid <= 1'b0;
    end else if (enable && sample_trigger) begin
        sampled_data <= analog_data;
        sample_valid <= 1'b1;
    end else begin
        sample_valid <= 1'b0;
    end
end

// Buffer management (simplified)
logic [7:0] buffer_count;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        buffer_count <= 8'h00;
    end else if (sample_valid) begin
        buffer_count <= buffer_count + 1;
    end
end

assign buffer_full = &buffer_count; // Full at 255 samples

endmodule

```

12.7.33. Communication Protocol Timing

```

module uart_baud_generator #(
    parameter int CLOCK_FREQ_HZ = 100_000_000,
    parameter int BAUD_RATE = 115200
) (
    input logic clk,
    input logic rst_n,
    input logic enable,

    output logic baud_tick, // 1x baud rate
    output logic baud_tick_16x // 16x baud rate for oversampling
);

localparam int BAUD_WIDTH = CLOCK_FREQ_HZ / BAUD_RATE;
localparam int BAUD_16X_WIDTH = CLOCK_FREQ_HZ / (BAUD_RATE * 16);

logic baud_pulse, baud_16x_pulse;

// 1x baud rate generator
clock_pulse #(

```



```

        .WIDTH(BAUD_WIDTH)
    ) baud_1x_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(baud_pulse)
    );

    // 16x baud rate generator
    clock_pulse #(
        .WIDTH(BAUD_16X_WIDTH)
    ) baud_16x_gen (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(baud_16x_pulse)
    );

    // Gate with enable
    assign baud_tick = enable ? baud_pulse : 1'b0;
    assign baud_tick_16x = enable ? baud_16x_pulse : 1'b0;

```

endmodule

12.7.44. Memory Refresh Controller

```

module dram_refresh_controller #(
    parameter int REFRESH_PERIOD_US = 64,           // 64µs refresh period
    parameter int CLOCK_FREQ_MHZ = 100,           // 100MHz system clock
    parameter int REFRESH_ROWS = 8192              // Number of rows to
    refresh
) (
    input   logic      clk,
    input   logic      rst_n,
    input   logic      refresh_enable,

    output  logic      refresh_request,
    output  logic [12:0] refresh_row_addr,
    output  logic      refresh_active
);

    localparam int REFRESH_WIDTH = (REFRESH_PERIOD_US *
    CLOCK_FREQ_MHZ) / REFRESH_ROWS;

    logic refresh_pulse;
    logic [12:0] row_counter;

    // Generate refresh timing
    clock_pulse #(
        .WIDTH(REFRESH_WIDTH)

```

```

) refresh_timer (
    .clk(clk),
    .rst_n(rst_n),
    .pulse(refresh_pulse)
);

// Row address counter
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        row_counter <= 13'h0000;
    end else if (refresh_pulse && refresh_enable) begin
        if (row_counter >= REFRESH_ROWS - 1)
            row_counter <= 13'h0000;
        else
            row_counter <= row_counter + 1;
    end
end

// Output assignments
assign refresh_request = refresh_pulse && refresh_enable;
assign refresh_row_addr = row_counter;
assign refresh_active = refresh_enable;

```

endmodule

12.7.55. Multi-Rate Pulse Generation

```

module multi_rate_pulse_generator (
    input logic      clk,
    input logic      rst_n,
    input logic [2:0] rate_select,    // Select pulse rate

    output logic      pulse_fast,     // Fast pulse output
    output logic      pulse_medium,   // Medium pulse output
    output logic      pulse_slow,     // Slow pulse output
    output logic      pulse_config    // Configurable pulse output
);

    logic [31:0] config_width;

    // Rate selection for configurable output
    always_comb begin
        case (rate_select)
            3'b000: config_width = 32'd10;    // Very fast
            3'b001: config_width = 32'd100;    // Fast
            3'b010: config_width = 32'd1000;    // Medium
            3'b011: config_width = 32'd10000;    // Slow
            3'b100: config_width = 32'd100000;    // Very slow
        end
    end

```

```

        default: config_width = 32'd1000; // Default medium
    endcase
end

// Fixed rate pulse generators
clock_pulse #(.WIDTH(50)) fast_gen (
    .clk(clk), .rst_n(rst_n), .pulse(pulse_fast)
);

clock_pulse #(.WIDTH(500)) medium_gen (
    .clk(clk), .rst_n(rst_n), .pulse(pulse_medium)
);

clock_pulse #(.WIDTH(5000)) slow_gen (
    .clk(clk), .rst_n(rst_n), .pulse(pulse_slow)
);

// Configurable rate generator (requires parameterizable module)
configurable_pulse_gen config_gen (
    .clk(clk),
    .rst_n(rst_n),
    .width_config(config_width),
    .pulse(pulse_config)
);

endmodule

// Configurable pulse generator module
module configurable_pulse_gen #(
    parameter int MAX_WIDTH = 1000000
) (
    input logic [31:0] width_config,
    input logic clk,
    input logic rst_n,
    output logic pulse
);

    logic [31:0] r_counter;
    logic [31:0] w_width_minus_one;

    assign w_width_minus_one = (width_config > 0) ? (width_config - 1)
: 0;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            r_counter <= 32'h00000000;
            pulse <= 1'b0;
        end else begin

```

```

        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1;
        else
            r_counter <= 32'h00000000;

        pulse <= (r_counter == w_width_minus_one) && (width_config
> 0);
    end
end

endmodule

```

12.7.66. Test Pattern Generation

```

module test_pattern_generator (
    input logic clk,
    input logic rst_n,
    input logic test_enable,
    input logic [3:0] pattern_select,

    output logic [7:0] test_data,
    output logic test_valid,
    output logic pattern_complete
);

    logic update_pulse;
    logic [7:0] pattern_counter;

    // Generate test data update rate
    clock_pulse #(.WIDTH(16)) pattern_clock (
        .clk(clk),
        .rst_n(rst_n),
        .pulse(update_pulse)
    );

    // Pattern counter
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            pattern_counter <= 8'h00;
        end else if (test_enable && update_pulse) begin
            pattern_counter <= pattern_counter + 1;
        end
    end

    // Test pattern generation
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            test_data <= 8'h00;

```

```

        test_valid <= 1'b0;
    end else if (test_enable && update_pulse) begin
        case (pattern_select)
            4'h0: test_data <= pattern_counter;
// Incrementing
            4'h1: test_data <= ~pattern_counter;
// Decrementing
            4'h2: test_data <= {pattern_counter[0], 7'b00000000};
// Walking 1
            4'h3: test_data <= ~{pattern_counter[0], 7'b00000000};
// Walking 0
            4'h4: test_data <= 8'hAA;
// Alternating
            4'h5: test_data <= 8'h55;
// Alternating inv
            4'h6: test_data <= 8'hFF;
// All ones
            4'h7: test_data <= 8'h00;
// All zeros
            default: test_data <= pattern_counter;
// Default
        endcase
        test_valid <= 1'b1;
    end else begin
        test_valid <= 1'b0;
    end
end

assign pattern_complete = test_enable && (&pattern_counter) &&
update_pulse;

endmodule

```

12.8 Advanced Features and Variations

12.8.11. Enable-Controlled Pulse Generator

```

module pulse_generator_with_enable #(
    parameter int WIDTH = 10
) (
    input logic clk,
    input logic rst_n,
    input logic enable,          // Pulse generation enable
    input logic pause,          // Pause counting (hold current state)
    output logic pulse,
    output logic [WIDTH-1:0] count_value // Current counter value
);

```

```

logic [WIDTH-1:0] r_counter;
logic [WIDTH-1:0] w_width_minus_one;

assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;
assign count_value = r_counter;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse <= 1'b0;
    end else if (enable && !pause) begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1'b1;
        else
            r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one);
    end else begin
        pulse <= 1'b0; // No pulse when disabled or paused
    end
end

endmodule

```

12.8.22. Variable Width Pulse Generator

```

module variable_pulse_generator #(
    parameter int MAX_WIDTH = 65536
) (
    input logic                clk,
    input logic                rst_n,
    input logic [$clog2(MAX_WIDTH):0] pulse_width, // Configurable
width
    input logic                load_width, // Load new
width
    output logic                pulse,
    output logic                width_loaded
);

logic [$clog2(MAX_WIDTH):0] r_counter;
logic [$clog2(MAX_WIDTH):0] r_width;
logic [$clog2(MAX_WIDTH):0] w_width_minus_one;

assign w_width_minus_one = (r_width > 0) ? (r_width - 1) : 0;

// Width register
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin

```

```

        r_width <= MAX_WIDTH[$clog2(MAX_WIDTH):0]; // Default
width
        width_loaded <= 1'b0;
    end else if (load_width) begin
        r_width <= (pulse_width > 0) ? pulse_width : 1; //
Minimum width = 1
        width_loaded <= 1'b1;
    end else begin
        width_loaded <= 1'b0;
    end
end

// Counter and pulse generation
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse <= 1'b0;
    end else begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1;
        else
            r_counter <= 'b0;

        pulse <= (r_counter == w_width_minus_one) && (r_width >
0);
    end
end

endmodule

```

12.8.33. Multi-Phase Pulse Generator

```

module multi_phase_pulse_generator #(
    parameter int WIDTH = 12,
    parameter int PHASES = 4
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    output logic [PHASES-1:0] phase_pulses
);

    logic [WIDTH-1:0] r_counter;
    logic [WIDTH-1:0] w_width_minus_one;
    logic [WIDTH-1:0] phase_thresholds [PHASES];

    assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

```

```

// Calculate phase thresholds
genvar i;
generate
    for (i = 0; i < PHASES; i++) begin : calc_phases
        assign phase_thresholds[i] = (WIDTH * i) / PHASES;
    end
endgenerate

// Counter
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
    end else if (enable) begin
        if (r_counter < w_width_minus_one)
            r_counter <= r_counter + 1;
        else
            r_counter <= 'b0;
    end
end

// Phase pulse generation
generate
    for (i = 0; i < PHASES; i++) begin : gen_phases
        always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n) begin
                phase_pulses[i] <= 1'b0;
            end else begin
                phase_pulses[i] <= enable && (r_counter ==
phase_thresholds[i]);
            end
        end
    end
endgenerate

endmodule

```

12.8.44. Burst Pulse Generator

```

module burst_pulse_generator #(
    parameter int BURST_LENGTH = 8,
    parameter int BURST_PERIOD = 100,
    parameter int INTER_BURST_DELAY = 1000
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    input logic trigger,           // Start burst sequence

    output logic burst_pulse,     // Individual pulses in burst

```



```

    output logic burst_active,           // Burst sequence active
    output logic burst_complete         // Burst sequence completed
);

typedef enum logic [2:0] {
    IDLE,
    BURST_ACTIVE,
    INTER_PULSE_DELAY,
    INTER_BURST_DELAY_ST
} burst_state_t;

burst_state_t state;

logic [$clog2(BURST_LENGTH):0] pulse_count;
logic [$clog2(BURST_PERIOD):0] period_counter;
logic [$clog2(INTER_BURST_DELAY):0] delay_counter;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        pulse_count <= 0;
        period_counter <= 0;
        delay_counter <= 0;
        burst_pulse <= 1'b0;
        burst_complete <= 1'b0;
    end else begin
        burst_complete <= 1'b0; // Default

        case (state)
            IDLE: begin
                burst_pulse <= 1'b0;
                if (enable && trigger) begin
                    state <= BURST_ACTIVE;
                    pulse_count <= 0;
                    period_counter <= 0;
                    burst_pulse <= 1'b1; // First pulse
                end
            end

            BURST_ACTIVE: begin
                burst_pulse <= 1'b0;
                pulse_count <= pulse_count + 1;

                if (pulse_count >= BURST_LENGTH - 1) begin
                    state <= INTER_BURST_DELAY_ST;
                    delay_counter <= 0;
                    burst_complete <= 1'b1;
                end else begin

```

```

        state <= INTER_PULSE_DELAY;
        period_counter <= 0;
    end
end

INTER_PULSE_DELAY: begin
    period_counter <= period_counter + 1;
    if (period_counter >= BURST_PERIOD - 1) begin
        state <= BURST_ACTIVE;
        burst_pulse <= 1'b1;
    end
end

INTER_BURST_DELAY_ST: begin
    delay_counter <= delay_counter + 1;
    if (delay_counter >= INTER_BURST_DELAY - 1) begin
        state <= IDLE;
    end
end
endcase
end
end

assign burst_active = (state != IDLE);

endmodule

```

12.9 Verification and Testing

12.9.1 Comprehensive Test Bench

```

module tb_clock_pulse;

    parameter WIDTH = 10;

    logic clk, rst_n, pulse;

    // Clock generation
    initial clk = 0;
    always #5ns clk = ~clk; // 100MHz

    // DUT instantiation
    clock_pulse #(.WIDTH(WIDTH)) dut (.*);

    // Test sequence
    initial begin
        rst_n = 0;
        #100ns rst_n = 1;
    end
endmodule

```

```

    // Test basic pulse generation
    test_basic_pulse_generation();

    // Test reset behavior
    test_reset_behavior();

    // Test timing accuracy
    test_timing_accuracy();

    $display("All tests completed!");
    $finish;
end

// Test basic pulse generation
task test_basic_pulse_generation();
    int pulse_count = 0;
    int cycle_count = 0;
    begin
        $display("Testing basic pulse generation (WIDTH=%0d)...",
WIDTH);

        // Monitor for several pulse periods
        repeat (WIDTH * 3) begin
            @(posedge clk);
            cycle_count++;
            if (pulse) pulse_count++;
        end

        // Verify pulse count
        if (pulse_count == 3) begin
            $display("PASS: Correct number of pulses generated");
        end else begin
            $error("FAIL: Expected 3 pulses, got %0d",
pulse_count);
        end
    end
endtask

// Test timing accuracy
task test_timing_accuracy();
    time pulse_times[10];
    time pulse_periods[9];
    int i;
    real avg_period, expected_period;
    begin
        $display("Testing timing accuracy...");

```

```

    expected_period = WIDTH * 10.0; // 10ns clock period

    // Capture pulse timing
    for (i = 0; i < 10; i++) begin
        @(posedge pulse);
        pulse_times[i] = $time;
    end

    // Calculate periods
    for (i = 0; i < 9; i++) begin
        pulse_periods[i] = pulse_times[i+1] - pulse_times[i];
    end

    // Calculate average period
    avg_period = 0;
    for (i = 0; i < 9; i++) begin
        avg_period += pulse_periods[i];
    end
    avg_period = avg_period / 9.0;

    // Check accuracy (within 1% tolerance)
    if (abs(avg_period - expected_period) < expected_period *
0.01) begin
        $display("PASS: Average period = %.1f ns (expected
%.1f ns)",
                    avg_period, expected_period);
    end else begin
        $error("FAIL: Average period = %.1f ns (expected %.1f
ns)",
                    avg_period, expected_period);
    end
end
endtask

function real abs(real value);
    abs = (value >= 0) ? value : -value;
endfunction

endmodule

```

12.9.2 Coverage Model

```

covergroup clock_pulse_cg @(posedge clk);

    cp_counter_values: coverpoint dut.r_counter {
        bins zero = {0};
        bins low[] = {[1:WIDTH/4]};
        bins mid[] = {[WIDTH/4+1:3*WIDTH/4]};
        bins high[] = {[3*WIDTH/4+1:WIDTH-2]};
    }
end

```

```

    bins max = {WIDTH-1};
}

cp_pulse: coverpoint pulse {
    bins asserted = {1};
    bins deasserted = {0};
}

cp_reset: coverpoint rst_n {
    bins reset = {0};
    bins normal = {1};
}

// Transition coverage
cp_pulse_edges: coverpoint pulse {
    bins rising = (0 ==> 1);
    bins falling = (1 ==> 0);
    bins stable_low = (0 ==> 0);
    bins stable_high = (1 ==> 1);
}

// Cross coverage
cross_counter_pulse: cross cp_counter_values, cp_pulse;

```

endcovergroup

12.9.3 Formal Properties

module clock_pulse_properties;

```

// Bind to DUT
bind clock_pulse clock_pulse_properties props_inst (.*)

// Property: Pulse occurs at correct count
property pulse_at_max_count;
    @(posedge clk) disable iff (!rst_n)
        (dut.r_counter == WIDTH-1) |-> pulse;
endproperty

// Property: Pulse only occurs at max count
property pulse_only_at_max;
    @(posedge clk) disable iff (!rst_n)
        pulse |-> (dut.r_counter == WIDTH-1);
endproperty

// Property: Counter wraps correctly
property counter_wrap;
    @(posedge clk) disable iff (!rst_n)

```

```

        (dut.r_counter == WIDTH-1) |>= (dut.r_counter == 0);
    endproperty

    // Property: Counter increments
    property counter_increment;
        @(posedge clk) disable iff (!rst_n)
            (dut.r_counter < WIDTH-1) |>=
                (dut.r_counter == $past(dut.r_counter) + 1);
    endproperty

    // Property: Reset behavior
    property reset_behavior;
        @(posedge clk)
            !rst_n |>= (dut.r_counter == 0) && !pulse;
    endproperty

    // Assertions
    assert property (pulse_at_max_count);
    assert property (pulse_only_at_max);
    assert property (counter_wrap);
    assert property (counter_increment);
    assert property (reset_behavior);

endmodule

```

12.10 Synthesis Considerations

12.10.1 Resource Utilization

WIDTH	Counter Bits	LUTs	FFs	Max Freq
8	3	8	4	500MHz
16	4	12	5	450MHz
32	5	18	6	400MHz
1024	10	28	11	350MHz

12.10.2 Timing Optimization

```

// For high-frequency applications, pipeline the comparison
module clock_pulse_pipelined #(
    parameter int WIDTH = 1000
) (
    input logic clk,
    input logic rst_n,
    output logic pulse
);

```

```

logic [WIDTH-1:0] r_counter;
logic [WIDTH-1:0] w_width_minus_one;
logic compare_result;

assign w_width_minus_one = WIDTH[WIDTH-1:0] - 1'b1;

// Pipeline the comparison
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        compare_result <= 1'b0;
    end else begin
        compare_result <= (r_counter == w_width_minus_one);
    end
end

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_counter <= 'b0;
        pulse <= 1'b0;
    end else begin
        if (compare_result)
            r_counter <= 'b0;
        else
            r_counter <= r_counter + 1;

        pulse <= compare_result;
    end
end

endmodule

```

12.11 Common Applications Summary

1. **Timing References:** System heartbeats, watchdog timers
2. **Sampling Systems:** ADC triggers, data acquisition timing
3. **Communication:** Baud rate generation, protocol timing
4. **Memory Systems:** Refresh timing, access scheduling
5. **Test Equipment:** Pattern generation, stimulus timing
6. **Power Management:** Activity monitoring, timeout generation
7. **Display Systems:** Refresh rates, sync generation

12.12 Design Guidelines

1. **Choose Appropriate WIDTH:** Balance between resolution and resource usage
2. **Consider Reset Timing:** Ensure clean startup behavior
3. **Validate Timing:** Verify pulse frequency matches requirements
4. **Plan for Variation:** Consider process, voltage, temperature effects
5. **Monitor Resource Usage:** Large counters can impact timing and area
6. **Use Enables:** Add enable signals for conditional operation
7. **Document Timing:** Clearly specify pulse rates and relationships

The clock pulse generator provides a simple, reliable solution for periodic timing generation, essential for many digital system functions requiring precise, regular timing events.

12.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

13 Binary-Gray Counter Module

13.1 Overview

The `counter_bingray` module is a dual-output counter that simultaneously provides both binary and Gray code representations of the same count value. This is essential for asynchronous FIFO implementations where Gray code is required to prevent metastability when crossing clock domains, while binary values are needed for arithmetic operations.

13.2 Module Declaration

```
module counter_bingray #(
    parameter int WIDTH = 4
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    output logic [WIDTH-1:0] counter_bin,
    output logic [WIDTH-1:0] counter_bin_next,
```



```

        output logic [WIDTH-1:0] counter_gray
    );

```

13.3 Parameters

13.3.1 WIDTH

- **Type:** int
- **Default:** 4
- **Description:** Bit width of both binary and Gray code outputs
- **Range:** Any positive integer ≥ 1
- **Impact:** Determines maximum count value ($2^{\text{WIDTH}} - 1$)

13.4 Ports

13.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low asynchronous reset
enable	1	logic	Counter enable control

13.4.2 Outputs

Port	Width	Type	Description
counter_bin	WIDTH	logic	Binary counter output (registered)
counter_bin_next	WIDTH	logic	Next binary value (combinational)
counter_gray	WIDTH	logic	Gray code output (registered)

13.5 Architecture Details

13.5.1 Internal Logic

```
logic [WIDTH-1:0] w_counter_bin, w_counter_gray;
```

```
assign w_counter_bin = enable ? (counter_bin + 1) : counter_bin;
```

```
assign w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
```

```
assign counter_bin_next = w_counter_bin;
```

13.5.2 Gray Code Conversion

The module implements the standard binary-to-Gray conversion: - **MSB**: `gray[WIDTH-1] = binary[WIDTH-1]` - **Other bits**: `gray[i] = binary[i] ^ binary[i+1]` for `i = 0` to `WIDTH-2`

13.6 Gray Code Theory

13.6.1 Why Gray Code?

Gray code (reflected binary code) has the property that adjacent values differ by exactly one bit. This is crucial for:

1. **Metastability Prevention:** When crossing clock domains, only one bit changes at a time
2. **Glitch Elimination:** Reduces intermediate states during transitions
3. **Asynchronous Safety:** Safe for use in asynchronous circuits

13.6.2 Gray Code Sequence Example (4-bit)

Decimal	Binary	Gray	Changes
0	0000	0000	-
1	0001	0001	bit 0
2	0010	0011	bit 1
3	0011	0010	bit 0
4	0100	0110	bit 2
5	0101	0111	bit 0
6	0110	0101	bit 1
7	0111	0100	bit 0
8	1000	1100	bit 3
9	1001	1101	bit 0

Decimal	Binary	Gray	Changes
10	1010	1111	bit 1
11	1011	1110	bit 0
12	1100	1010	bit 2
13	1101	1011	bit 0
14	1110	1001	bit 1
15	1111	1000	bit 0

13.7 Implementation Analysis

13.7.1 Combinational Logic

```
// Next binary calculation
w_counter_bin = enable ? (counter_bin + 1) : counter_bin;

// Gray code conversion
w_counter_gray = w_counter_bin ^ (w_counter_bin >> 1);
```

13.7.2 Sequential Logic

```
always_ff @(posedge clk, negedge rst_n) begin
    if (!rst_n) begin
        counter_bin <= 'b0;
        counter_gray <= 'b0;
    end else begin
        counter_bin <= w_counter_bin;
        counter_gray <= w_counter_gray;
    end
end
```

13.8 Timing Characteristics

13.8.1 Critical Paths

1. **Binary Increment:** Standard binary addition timing
2. **Gray Conversion:** XOR tree with $\log(\text{WIDTH})$ levels
3. **Combined Path:** Increment + conversion in same cycle

13.8.2 Propagation Delays

- **Clock-to-Q:** Standard flip-flop delay
- **Combinational:** Depends on addition and XOR logic depth

- **Setup Time:** Must account for longest combinational path

13.9 Application: Asynchronous FIFO

13.9.1 FIFO Pointer Implementation

```
// Write domain counter
counter_bingray #(.WIDTH(ADDR_WIDTH+1)) wr_counter (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(wr_enable),
    .counter_bin(wr_bin),
    .counter_bin_next(wr_bin_next),
    .counter_gray(wr_gray)
);

// Read domain counter
counter_bingray #(.WIDTH(ADDR_WIDTH+1)) rd_counter (
    .clk(rd_clk),
    .rst_n(rd_rst_n),
    .enable(rd_enable),
    .counter_bin(rd_bin),
    .counter_bin_next(rd_bin_next),
    .counter_gray(rd_gray)
);
```

13.9.2 Cross-Domain Synchronization

```
// Synchronize Gray code pointers across domains
logic [ADDR_WIDTH:0] wr_gray_sync, rd_gray_sync;

// Write domain: synchronize read Gray pointer
synchronizer #(.WIDTH(ADDR_WIDTH+1)) rd_sync (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .data_in(rd_gray),
    .data_out(rd_gray_sync)
);

// Read domain: synchronize write Gray pointer
synchronizer #(.WIDTH(ADDR_WIDTH+1)) wr_sync (
    .clk(rd_clk),
    .rst_n(rd_rst_n),
    .data_in(wr_gray),
    .data_out(wr_gray_sync)
);
```

13.9.3 FIFO Status Generation

```
// FIFO empty: Gray pointers equal
assign fifo_empty = (rd_gray == wr_gray_sync);

// FIFO full: Binary addresses equal, MSBs different
wire [ADDR_WIDTH-1:0] wr_addr = wr_bin[ADDR_WIDTH-1:0];
wire [ADDR_WIDTH-1:0] rd_addr_sync = gray2bin(rd_gray_sync)
[ADDR_WIDTH-1:0];
wire wr_msb = wr_bin[ADDR_WIDTH];
wire rd_msb_sync = gray2bin(rd_gray_sync)[ADDR_WIDTH];

assign fifo_full = (wr_addr == rd_addr_sync) && (wr_msb !=
rd_msb_sync);
```

13.10 Advanced Features

13.10.1 Almost Full/Empty Flags

```
// Calculate occupancy using binary values
wire [ADDR_WIDTH:0] occupancy = wr_bin - gray2bin(rd_gray_sync);

// Generate status flags
assign almost_full = (occupancy >= ALMOST_FULL_THRESH);
assign almost_empty = (occupancy <= ALMOST_EMPTY_THRESH);
```

13.10.2 Gray-to-Binary Conversion Function

```
function automatic [WIDTH-1:0] gray2bin;
  input [WIDTH-1:0] gray;
  integer i;
  begin
    gray2bin[WIDTH-1] = gray[WIDTH-1];
    for (i = WIDTH-2; i >= 0; i--) begin
      gray2bin[i] = gray2bin[i+1] ^ gray[i];
    end
  end
endfunction
```

13.11 Design Examples

13.11.1 1. 8-bit Asynchronous FIFO Pointer

```
parameter FIFO_DEPTH = 256;
parameter ADDR_WIDTH = $clog2(FIFO_DEPTH);
parameter PTR_WIDTH = ADDR_WIDTH + 1; // Extra bit for full detection
```

```
counter_bingray #(
```

```

        .WIDTH(PTR_WIDTH)
    ) fifo_wr_ptr (
        .clk(wr_clk),
        .rst_n(async_rst_n),
        .enable(wr_enable && !fifo_full),
        .counter_bin(wr_ptr_bin),
        .counter_bin_next(wr_ptr_bin_next),
        .counter_gray(wr_ptr_gray)
    );

```

13.11.2 2. Clock Domain Crossing Counter

// Source domain

```

counter_bingray #(.WIDTH(8)) src_counter (
    .clk(src_clk),
    .rst_n(src_rst_n),
    .enable(src_enable),
    .counter_bin(src_bin),
    .counter_bin_next(),
    .counter_gray(src_gray)
);

```

// Destination domain receives synchronized Gray value

```

logic [7:0] dest_gray_sync;
synchronizer #(.WIDTH(8)) sync_inst (
    .clk(dest_clk),
    .rst_n(dest_rst_n),
    .data_in(src_gray),
    .data_out(dest_gray_sync)
);

```

13.12 Verification Strategy

13.12.1 Test Scenarios

1. **Sequential Counting:** Verify both outputs increment correctly
2. **Gray Code Properties:** Ensure single-bit changes between adjacent values
3. **Reset Behavior:** Confirm both outputs reset to zero
4. **Enable Control:** Test hold behavior when disabled
5. **Rollover:** Verify proper wrap-around from maximum value

13.12.2 Coverage Points

```

covergroup counter_bingray_cg @(posedge clk);
    cp_binary: coverpoint counter_bin {

```

```

        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_gray: coverpoint counter_gray {
        bins all_values[] = {[0:2**WIDTH-1]};
    }

    cp_enable: coverpoint enable {
        bins enabled = {1};
        bins disabled = {0};
    }

    cp_transitions: coverpoint counter_bin {
        bins transitions[] = ([0:2**WIDTH-2] => [1:2**WIDTH-1]);
        bins rollover = (2**WIDTH-1 => 0);
    }
endgroup

```

13.12.3 Assertions

```

// Verify Gray code has single bit changes
property gray_single_bit_change;
    @(posedge clk) disable iff (!rst_n)
    enable && !$isunknown(counter_gray) |->
    $countones(counter_gray ^ $past(counter_gray)) <= 1;
endproperty

```

```

assert property (gray_single_bit_change);

```

```

// Verify binary and Gray relationship
property bin_gray_relationship;
    @(posedge clk) disable iff (!rst_n)
    counter_gray == (counter_bin ^ (counter_bin >> 1));
endproperty

```

```

assert property (bin_gray_relationship);

```

13.13 Synthesis Considerations

13.13.1 Resource Utilization

- **Flip-Flops:** 2×WIDTH (separate for binary and Gray)
- **LUTs:** Increment logic + XOR tree for Gray conversion
- **Critical Path:** Through binary increment and Gray conversion

13.13.2 Optimization Techniques

```
// Optional: Pipeline Gray conversion for high speed  
logic [WIDTH-1:0] counter_gray_pipe;  
always_ff @(posedge clk) begin  
    counter_gray_pipe <= w_counter_gray;  
end
```

13.13.3 Tool-Specific Attributes

```
(* ASYNC_REG = "TRUE" *) logic [WIDTH-1:0] counter_gray; // Xilinx  
// synthesis attribute ASYNC_REG of counter_gray is "TRUE" //  
Altera/Intel
```

13.14 Performance Characteristics

13.14.1 Maximum Frequency

- **Typical:** 200-400 MHz in modern FPGAs
- **Limitation:** Binary increment + Gray conversion logic depth
- **Optimization:** Pipeline for higher frequencies

13.14.2 Power Consumption

- **Dynamic:** Proportional to switching activity
- **Static:** Minimal for CMOS technology
- **Optimization:** Clock gating when enable is inactive

13.15 Common Applications

1. **Asynchronous FIFO Pointers:** Primary use case
2. **Clock Domain Crossing Counters:** Safe multi-domain counting
3. **Position Encoders:** Mechanical/optical encoder interfaces
4. **State Machine Counters:** When glitch-free operation required
5. **Memory Address Generation:** For dual-port memory systems

13.16 Troubleshooting Guide

13.16.1 Common Issues

1. **Metastability:** Ensure proper synchronization of Gray values
2. **Timing Violations:** Pipeline Gray conversion if needed
3. **Incorrect FIFO Status:** Verify Gray-to-binary conversion
4. **Reset Skew:** Use proper reset synchronization

13.16.2 Debug Techniques

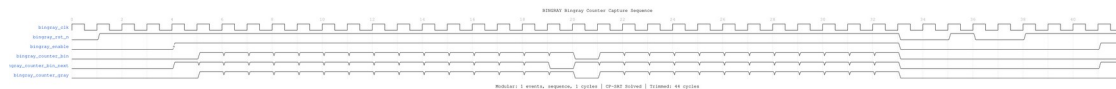
1. **Simulation:** Verify Gray code properties with assertions
2. **Logic Analyzer:** Capture actual transitions in hardware
3. **Timing Analysis:** Check setup/hold requirements
4. **Cross-Domain Checks:** Verify synchronizer timing

13.17 WaveDrom Visualization

High-quality waveforms showcasing Binary-Gray counter operation are available!

The following timing diagrams demonstrate the dual-output counter design across 4 key scenarios:

13.17.1 Scenario 1: Binary vs Gray Code Comparison

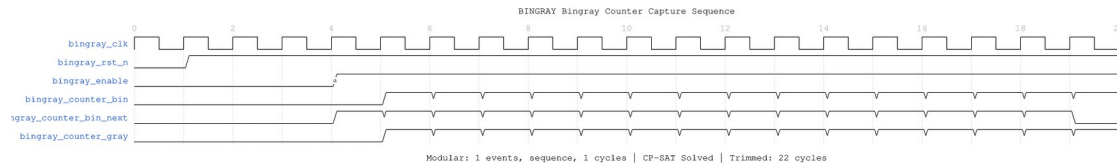


BinGray Comparison

WaveJSON: [bingray_counter_binary_vs_gray.json](#)

Side-by-side comparison of both outputs: - Binary: Normal sequential counting (0→1→2→3→...) - Gray: Single-bit transitions between values - Shows full cycle demonstrating encoding differences - Illustrates why Gray code is CDC-safe

13.17.2 Scenario 2: Single-Bit Transitions (CDC Safety) ★ KEY FEATURE

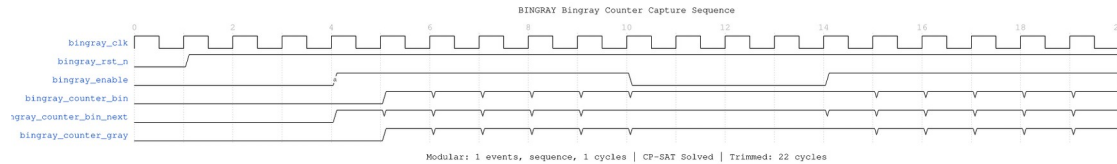


BinGray Single-Bit Transitions

WaveJSON: [bingray_counter_single_bit_transitions.json](#)

Gray code CDC safety property: - Each Gray transition changes EXACTLY one bit - Hamming distance = 1 between adjacent values - **Critical for fifo_async CDC mechanism** - Prevents metastability in clock domain crossing

13.17.3 Scenario 3: Lookahead Signal (counter_bin_next)

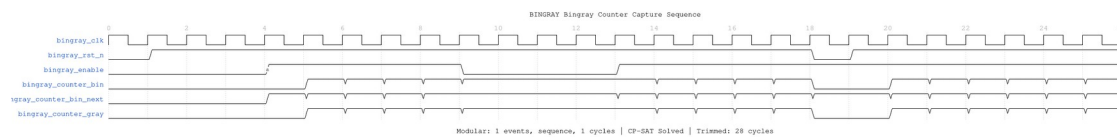


BinGray Lookahead

WaveJSON: [bingray_counter_lookahead.json](#)

Combinational lookahead feature: - Predicts next binary value one cycle ahead - Useful for FIFO full/empty prediction - Shows enable gating behavior - Enables early decision-making

13.17.4 Scenario 4: Enable and Reset Control



BinGray Enable and Reset

WaveJSON: [bingray_counter_enable_reset.json](#)

Control signal behavior: - Both outputs hold when enable=0 - Asynchronous reset to zero - Clean state transitions - Reset during counting demonstration

To regenerate these waveforms:

```

pytest val/common/test_counter_bingray_wavedrom.py -v
# Then convert JSON to PNG:
cd docs/markdown/assets/WAVES/counter_bingray
for f in *.json; do wavedrom-cli -i "$f" -p "${f%.json}.png"; done

```

What Makes Binary-Gray Counters Special:

The waveforms highlight the unique dual-output design: - **Dual Encoding:** Binary for arithmetic, Gray for CDC safety - **Single-Bit Transitions:** Gray code changes one bit at a time - **Lookahead:** counter_bin_next provides one-cycle prediction - **CDC-Safe:** Safe for asynchronous clock domain crossing

Relationship to fifo_async:

Binary-Gray counters are the foundation of the standard fifo_async module: - **fifo_async** uses this counter for read/write pointers - Gray outputs cross clock domains safely - Binary outputs used for arithmetic (occupancy, address generation) - Logarithmic width ($\lceil \log_2(\text{DEPTH}) \rceil + 1$) for resource efficiency

Comparison Table:

Feature	BinGray Counter	Johnson Counter
Output Width	$\lceil \log_2(\text{DEPTH}) \rceil + 1$ bits	DEPTH bits
CDC Method	Standard Gray code	Johnson code
Depth Support	Power-of-2 only	Any even number
Resource Efficiency	Logarithmic (better for large depths)	Linear
Conversion	XOR tree (simple)	Position detection (complex)
Used In	fifo_async (standard)	fifo_async_div2 (flexible)

Comparison with Other Modules:

- test_counter_johnson_wavedrom.py - Johnson counter (even depths, linear width)
- test_fifo_async_wavedrom.py - BinGray counter in action (async FIFO, power-of-2)
- test_fifo_async_div2_wavedrom.py - Johnson counter in action (async FIFO, even depths)

13.18 Test and Verification

Comprehensive Test Suite: - val/common/test_counter_bingray.py - Full functional verification - val/common/test_counter_bingray_wavedrom.py - WaveDrom timing diagrams



Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_counter_bingray.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_counter_bingray_wavedrom.py -v
```

13.19 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

14 Binary Counter with Load and Variable Increment

A FIFO-optimized binary counter supporting standard increment, variable increment, and direct load operations with configurable wraparound behavior for efficient FIFO pointer management.

14.1 Overview

The counter_bin_load module extends basic binary counting with three distinct operating modes: standard +1 increment, variable-amount increment, and direct load operations. It is specifically optimized for FIFO pointer management where drop/flush operations require jumping the read pointer by arbitrary amounts. The module implements FIFO-style wraparound at $2 \times \text{MAX}$ with MSB toggle for full/empty detection.

14.2 Module Declaration

```
module counter_bin_load #(
    parameter int WIDTH = 5,           // Counter bit width
    parameter int MAX = 10             // Maximum count value (wraps at MAX-1)
) (
    // Derived localparam (computed internally - not user-settable):
    // localparam int WRAP_BOUNDARY = 2 * MAX; // Wraparound boundary

    input logic clk,                    // Clock input
    input logic rst_n,                  // Active-low
    asynchronous reset
    input logic enable,                 // Count +1 enable
```

```

    input logic          add_enable,          // Variable increment
enable
    input logic [WIDTH-1:0] add_value,        // Value to add
    input logic          load,                // Load enable
(highest priority)
    input logic [WIDTH-1:0] load_value,       // Value to load
    output logic [WIDTH-1:0] counter_bin_curr, // Current count
(register)
    output logic [WIDTH-1:0] counter_bin_next // Next count
(combination)
);

```

14.3 Parameters

14.3.1 User-Settable Parameters

Parameter	Type	Default	Description
WIDTH	int	5	Counter bit width (range: 2-64)
MAX	int	10	Maximum count value before wraparound (range: 2 to 2 ^(WIDTH-1))

WIDTH Guidelines: - **Minimum:** 2 bits (required for MSB inversion behavior) - **Typical:** FIFO address width + 1 (e.g., 4 for 8-entry FIFO: 3 addr + 1 MSB) - **Maximum:** 64 bits (practical limit for synthesis)

MAX Constraints: - Must fit within WIDTH-1 bits (excludes MSB) - Example: WIDTH=4 → MAX ≤ 8 (3 bits for addresses 0-7, 1 MSB for full/empty) - FIFO depth is typically MAX (e.g., MAX=8 → 8-entry FIFO)

14.3.2 Derived Localparam (Computed Internally)

Localparam	Computation	Description
WRAP_BOUNDARY	2 × MAX	Wraparound boundary for add operations

Note: WRAP_BOUNDARY is computed internally and cannot be overridden by users.

14.4 Ports

14.4.1 Inputs

Port	Width	Description
clk	1	System clock (rising edge active)
rst_n	1	Active-low asynchronous reset
enable	1	Standard +1 increment enable
add_enable	1	Variable increment enable
add_value	WIDTH	Value to add when add_enable=1
load	1	Load enable (highest priority operation)
load_value	WIDTH	Value to load when load=1

14.4.2 Outputs

Port	Width	Description
counter_bin_cur r	WIDTH	Current counter value (registered output)
counter_bin_next t	WIDTH	Next counter value (combinational preview)

14.5 Functionality

14.5.1 Operation Priority

The module implements a three-level priority hierarchy:

1. **Load Operation** (Highest Priority): load = 1
 - Directly sets counter_bin_next = load_value
 - Used for: Flush all, jump to specific position

2. **Variable Increment:** $\text{add_enable} = 1$ (when $\text{load} = 0$)

- Adds add_value to current count: $\text{counter_bin_next} = \text{counter_bin_curr} + \text{add_value}$
- Wraps at $2 \times \text{MAX}$: Subtracts $2 \times \text{MAX}$ if $\text{sum} \geq 2 \times \text{MAX}$
- Used for: Drop multiple entries, bulk operations

3. **Standard Increment:** $\text{enable} = 1$ (when $\text{load} = 0$ and $\text{add_enable} = 0$)

- Adds 1 to current count: $\text{counter_bin_next} = \text{counter_bin_curr} + 1$
- Special FIFO wraparound: MSB toggles at $\text{MAX}-1$, lower bits clear
- Used for: Normal FIFO read/write pointer increment

4. **Hold:** All enables = 0

- Maintains current value: $\text{counter_bin_next} = \text{counter_bin_curr}$

14.5.2 FIFO-Optimized Wraparound

14.5.2.1 *Standard Increment Mode (enable=1):*

For $\text{MAX}=8$, $\text{WIDTH}=4$:

Count sequence: 0, 1, 2, 3, 4, 5, 6, 7, 8 (toggle MSB, clear lower)

Binary: 0000 → 0001 → ... → 0111 → 1000 (MSB inverts, lower bits = 0)

0000 → 0001 → ... → 0111 → 1000 → 1001 → ... → 1111 → 0000

Key Feature: MSB toggles when lower bits reach $\text{MAX}-1$ - This creates a $2 \times \text{MAX}$ count range: 0 to $(2 \times \text{MAX} - 1)$ - MSB different between write/read pointers → FIFO full or empty

14.5.2.2 *Variable Increment Mode (add_enable=1):*

Wraparound at $2 \times \text{MAX}$:

Example: $\text{counter_bin_curr} = 14$, $\text{add_value} = 5$, $\text{MAX} = 8$, $\text{WRAP_BOUNDARY} = 16$

$\text{Sum} = 14 + 5 = 19 \geq 16 \rightarrow \text{counter_bin_next} = 19 - 16 = 3$

Key Feature: Full 0 to $(2 \times \text{MAX} - 1)$ range with modulo arithmetic

14.5.3 Full/Empty Detection Pattern

// In FIFO controller:

logic full, empty;

assign full = ($\text{wr_ptr}[\text{WIDTH}-2:0] == \text{rd_ptr}[\text{WIDTH}-2:0]$) &&

```

        (wr_ptr[WIDTH-1] != rd_ptr[WIDTH-1]); // MSB different

assign empty = (wr_ptr[WIDTH-2:0] == rd_ptr[WIDTH-2:0]) &&
               (wr_ptr[WIDTH-1] == rd_ptr[WIDTH-1]); // MSB same

```

Why This Works: - Normal operation: wr_ptr advances, MSBs eventually differ → full - After wraparound: rd_ptr catches up, MSBs match again → empty - MSB acts as “lap counter” for circular buffer

14.6 Implementation Details

14.6.1 Priority Logic

```

always_comb begin
    counter_bin_next = counter_bin_curr; // Default: hold

    if (load) begin
        // Highest priority: Direct load
        counter_bin_next = load_value;

    end else if (add_enable) begin
        // Variable increment with wraparound at 2*MAX
        w_sum_ext = {1'b0, counter_bin_curr} + {1'b0, add_value};

        if (w_sum_ext >= WRAP_BOUNDARY) begin
            counter_bin_next = w_sum_ext[WIDTH-1:0] -
WIDTH'(WRAP_BOUNDARY);
        end else begin
            counter_bin_next = w_sum_ext[WIDTH-1:0];
        end

    end else if (enable) begin
        // Standard +1 with MSB toggle at MAX-1
        if (counter_bin_curr[WIDTH-2:0] == (MAX - 1)) begin
            counter_bin_next = {~counter_bin_curr[WIDTH-1], {(WIDTH-1)
{1'b0}}};
        end else begin
            counter_bin_next = counter_bin_curr + 1;
        end

    end
    // else: hold (default assignment)
end

```

14.6.2 Registered Output

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        counter_bin_curr <= 'b0; // Reset to 0

```



```

        else
            counter_bin_curr <= counter_bin_next;
    end

```

Reset Behavior: Counter initializes to 0 on rst_n = 0

14.6.3 Combinational Next-Value Preview

The counter_bin_next output provides a one-cycle lookahead of the counter value, useful for: -
FIFO full/empty prediction - Pre-computing next address - Pipelined operations

14.7 Usage Examples

14.7.1 FIFO Read Pointer with Drop Support

```

// 8-entry FIFO with drop capability
localparam int FIFO_DEPTH = 8;
localparam int PTR_WIDTH = $clog2(FIFO_DEPTH) + 1; // +1 for MSB

logic [PTR_WIDTH-1:0] rd_ptr, rd_ptr_next;
logic rd_en, drop_valid;
logic [PTR_WIDTH-1:0] drop_count;

counter_bin_load #(
    .WIDTH(PTR_WIDTH),           // 4 bits: 3 for addr (0-7) + 1 MSB
    .MAX(FIFO_DEPTH)            // Wrap at 8
) u_rd_ptr (
    .clk                         (clk),
    .rst_n                      (rst_n),
    .enable                     (rd_en & !fifo_empty),           // Normal read
    .add_enable                 (drop_valid),                     // Drop multiple
    .add_value                  (drop_count),                     // How many to
drop
    .load                       (flush_all),                       // Flush operation
    .load_value                 (wr_ptr),                           // Jump to write
ptr
    .counter_bin_curr           (rd_ptr),
    .counter_bin_next           (rd_ptr_next)
);

// Use lower bits for memory address
assign rd_addr = rd_ptr[PTR_WIDTH-2:0];

```

14.7.2 FIFO Write Pointer

```

logic [PTR_WIDTH-1:0] wr_ptr, wr_ptr_next;
logic wr_en;

```

```

counter_bin_load #(
    .WIDTH(PTR_WIDTH),
    .MAX(FIFO_DEPTH)
) u_wr_ptr (
    .clk            (clk),
    .rst_n          (rst_n),
    .enable         (wr_en & !fifo_full),
    .add_enable     (1'b0),           // No variable increment
    for write
    .add_value      ('0),
    .load           (flush_all),      // Reset to read pointer
    .load_value     (rd_ptr),
    .counter_bin_curr (wr_ptr),
    .counter_bin_next (wr_ptr_next)
);

```

```

assign wr_addr = wr_ptr[PTR_WIDTH-2:0];

```

14.7.3 Full/Empty Status Logic

// Full: Lower bits match, MSB different

```

assign fifo_full = (wr_ptr[PTR_WIDTH-2:0] == rd_ptr[PTR_WIDTH-2:0]) &&
    (wr_ptr[PTR_WIDTH-1] != rd_ptr[PTR_WIDTH-1]);

```

// Empty: All bits match

```

assign fifo_empty = (wr_ptr == rd_ptr);

```

// Count calculation (handles wraparound)

```

logic [PTR_WIDTH-1:0] fifo_count;
assign fifo_count = (wr_ptr >= rd_ptr) ? (wr_ptr - rd_ptr) :
    (2*FIFO_DEPTH - rd_ptr +
wr_ptr);

```

14.7.4 Packet Drop Controller

// Drop N packets from FIFO

```

logic [7:0] packets_to_drop;
logic drop_start;

```

// State machine for multi-cycle drop

```

typedef enum logic [1:0] {
    IDLE,
    DROPPING
} drop_state_t;
drop_state_t drop_state;

```

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin

```

```

        drop_state <= IDLE;
        drop_valid <= 1'b0;
        drop_count <= '0;
    end else begin
        case (drop_state)
            IDLE: begin
                if (drop_start && packets_to_drop > 0) begin
                    drop_valid <= 1'b1;
                    drop_count <= packets_to_drop;
                    drop_state <= DROPPING;
                end
            end
            DROPPING: begin
                drop_valid <= 1'b0; // Single-cycle pulse
                drop_state <= IDLE;
            end
        endcase
    end
end

// Connect to counter
counter_bin_load #(
    .WIDTH(PTR_WIDTH),
    .MAX(FIFO_DEPTH)
) u_rd_ptr (
    .clk          (clk),
    .rst_n        (rst_n),
    .enable        (rd_en & !fifo_empty),
    .add_enable    (drop_valid),
    .add_value     (drop_count),
    .load          (1'b0),
    .load_value    ('0),
    .counter_bin_curr(rd_ptr),
    .counter_bin_next(rd_ptr_next)
);

```

14.8 Timing Characteristics

14.8.1 Latency and Throughput

- **Increment Latency:** 1 cycle (registered output)
- **Next-Value Availability:** 0 cycles (combinational counter_bin_next)
- **Throughput:** 1 operation per cycle

- **Reset Recovery:** 1 cycle (asynchronous reset)

14.8.2 Operation Timing Diagrams

14.8.2.1 Standard Increment (enable=1):

Cycle:	0	1	2	3	4	5	6	7	8	9
clk:										
enable:										
curr:	0000	0000	0001	0010	0011	0100	0101	0110	0111	0111
next:	0000	0001	0010	0011	0100	0101	0110	0111	0111	0111

14.8.2.2 Variable Increment (add_enable=1, add_value=3):

Cycle:	0	1	2	3
clk:				
add_en:				
add_val:	-----	[3]	-----	-----
curr:	0010	0010	0101	0110
next:	0010	0101	0110	0110

↑ +3

14.8.2.3 Load Operation (load=1, load_value=12):

Cycle:	0	1	2
clk:			
load:			
load_val:	-----	[C]	---
curr:	0011	0011	1100
next:	0011	1100	1101

↑ direct load

14.9 Design Considerations

14.9.1 When to Use counter_bin_load

✓ **Appropriate Use Cases:** - FIFO read/write pointers with drop/flush - Circular buffer management with skip operations - Packet queue pointers with bulk discard - Ring buffer pointers with reset capability - Any counter needing jump-to-position capability

14.9.2 When to Use Alternatives

Use counter_bin.sv if: - Only need standard +1 increment - No load or variable increment required - Simpler interface preferred

Use counter_load_clear.sv if: - Need load/clear but not FIFO wraparound - Don't need MSB toggle behavior - Variable increment not required

14.9.3 Parameter Selection Guidelines

WIDTH Calculation:

```
// For FIFO with N entries:
localparam int FIFO_DEPTH = 16;
localparam int PTR_WIDTH = $clog2(FIFO_DEPTH) + 1; // +1 for MSB

// Example: 16-entry FIFO
// FIFO_DEPTH = 16
// PTR_WIDTH = 4 + 1 = 5 bits
// Lower 4 bits = address (0-15)
// MSB = full/empty indicator
```

MAX Selection: - Set MAX equal to FIFO depth - MAX must be power of 2 for efficient address decoding - MAX must fit in WIDTH-1 bits

14.9.4 Critical Paths

1. **Add Operation:** Extended addition + wraparound comparison
 - Longest path: counter_bin_curr + add_value → overflow detection → modulo
2. **Standard Increment:** Comparison + MSB toggle
 - Path: Lower bits comparison → MSB invert + clear
3. **Load Operation:** Direct assignment
 - Shortest path: Mux select

Optimization Tip: If add operation is critical path, consider pipelining:

```
// Pipeline add operation
logic [WIDTH-1:0] r_add_result;
always_ff @(posedge clk) begin
    r_add_result <= counter_bin_curr + add_value;
end
```

14.10 Performance Characteristics

14.10.1 Resource Utilization

Configuration	FFs	LUTs	Description
WIDTH=4, MAX=8	4	~12	Minimal (8-entry FIFO)
WIDTH=5, MAX=16	5	~16	Small (16-entry FIFO)
WIDTH=9,	9	~30	Medium (256-

Configuration	FFs	LUTs	Description
MAX=256			entry FIFO)
WIDTH=13, MAX=4096	13	~45	Large (4K- entry FIFO)

Area Scaling: Approximately linear with WIDTH due to adder and comparator

14.10.2 Maximum Frequency

- **Typical:** 300-500 MHz (modern FPGAs)
- **Limiting Factor:** Add operation with wraparound detection
- **Optimization:** Pipeline add path for >500 MHz

14.11 Verification Notes

Test suite location: `val/common/test_counter_bin_load.py`

Key Test Scenarios: - Standard increment (enable=1) - Variable increment with various add_value - Load operation priority verification - Wraparound behavior at 2×MAX - MSB toggle at MAX-1 for standard increment - Full/empty condition verification - Priority hierarchy (load > add > enable) - Reset behavior

Test Command:

```
pytest val/common/test_counter_bin_load.py -v
```

14.12 Common Pitfalls

✗ **Anti-Pattern 1:** Incorrect WIDTH for FIFO depth

```
// WRONG: WIDTH = $clog2(FIFO_DEPTH)
localparam int WIDTH = $clog2(16); // = 4 bits
// Problem: No MSB for full/empty detection!

// RIGHT: WIDTH = $clog2(FIFO_DEPTH) + 1
localparam int WIDTH = $clog2(16) + 1; // = 5 bits
// Lower 4 bits = address, MSB = full/empty indicator
```

✗ **Anti-Pattern 2:** Ignoring operation priority

```
// WRONG: Trying to increment while load is active
load <= 1'b1;
enable <= 1'b1; // This will be IGNORED!
// Result: Counter loads load_value, enable has no effect

// RIGHT: Ensure only one operation active
```

```
load <= drop_all;
enable <= rd_en & !drop_all; // Mutually exclusive
```

✗ **Anti-Pattern 3:** add_value exceeds 2×MAX

```
// WRONG: add_value larger than valid range
// MAX = 8, add_value = 20
// Result: Multiple wraparounds, unpredictable behavior
```

```
// RIGHT: Constrain add_value
assert property (@(posedge clk) add_enable |-> add_value < 2*MAX);
```

✗ **Anti-Pattern 4:** Using wrong bits for memory address

```
// WRONG: Using full counter including MSB
assign mem_addr = wr_ptr; // Includes MSB!
// Result: Addresses 0-15 appear as 0-15 and 16-31
```

```
// RIGHT: Use only lower bits
assign mem_addr = wr_ptr[WIDTH-2:0]; // Exclude MSB
```

14.13 Related Modules

- **counter_bin.sv** - Simple binary counter (no load or variable increment)
- **counter_load_clear.sv** - Counter with load/clear (no FIFO wraparound)
- **counter_freq_invariant.sv** - Time-based counter (millisecond/microsecond timing)
- **gaxi_drop_fifo_sync.sv** - Uses counter_bin_load for drop-capable FIFO
- **gaxi_fifo_sync.sv** - Standard synchronous FIFO with this counter

14.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

15 Binary Counter for FIFOs

15.1 Overview

The counter_bin module is a specialized binary counter designed specifically for FIFO (First-In-First-Out) buffer applications. Unlike traditional counters, it implements a unique wrap-around

behavior where the MSB is inverted and lower bits are cleared when reaching the maximum value, creating a pattern suitable for circular buffer management.

15.2 Module Declaration

```
module counter_bin #(
    parameter int WIDTH = 5,
    parameter int MAX    = 10
) (
    input logic          clk,
    input logic          rst_n,
    input logic          enable,
    output logic [WIDTH-1:0] counter_bin_curr,
    output logic [WIDTH-1:0] counter_bin_next
);
```

15.3 Parameters

15.3.1 WIDTH

- **Type:** int
- **Default:** 5
- **Description:** Total width of the counter in bits
- **Range:** Must be ≥ 2 for proper MSB inversion operation
- **Usage:** Determines the counter's bit width and maximum representable value

15.3.2 MAX

- **Type:** int
- **Default:** 10
- **Description:** Maximum count value before special wrap behavior
- **Range:** Should be $< 2^{(WIDTH-1)}$ for proper operation
- **Constraint:** Must fit within the lower WIDTH-1 bits

15.4 Ports

15.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low asynchronous reset
enable	1	logic	Counter enable control

15.4.2 Outputs

Port	Width	Type	Description
counter_bin_curr	WIDTH	logic	Current counter value (registered)
counter_bin_next	WIDTH	logic	Next counter value (combinational)

15.5 Architecture Details

15.5.1 Internal Signals

```
logic [WIDTH-2:0] w_max_val;  
assign w_max_val = (WIDTH-1)'(MAX - 1);
```

15.5.2 Special Wrap Logic

The counter implements a unique wrap-around behavior:

```
always_comb begin  
    if (enable) begin  
        if (counter_bin_curr[WIDTH-2:0] == w_max_val)  
            counter_bin_next = {~counter_bin_curr[WIDTH-1], {(WIDTH -  
1)}{1'b0}}};  
        else  
            counter_bin_next = counter_bin_curr + 1;  
        end else begin  
            counter_bin_next = counter_bin_curr;  
        end  
    end  
end
```

15.6 Operation Principles

15.6.1 Normal Counting

When `counter_bin_curr[WIDTH-2:0] < MAX-1`: - Counter increments normally by 1 - MSB remains unchanged - Standard binary counting behavior

15.6.2 Wrap Condition

When `counter_bin_curr[WIDTH-2:0] == MAX-1`: - MSB is inverted:
`~counter_bin_curr[WIDTH-1]` - Lower bits are cleared: `{(WIDTH-1){1'b0}}` - Creates a “folded” counting pattern

15.6.3 Enable Control

- When `enable = 1`: Counter operates normally
- When `enable = 0`: Counter holds current value

15.7 Counting Sequence Example

15.7.1 Configuration: WIDTH=4, MAX=6

```
Initial: 0000 (MSB=0, lower=000)
          0001
          0010
          0011
          0100
          0101 <- MAX-1 reached
Wrap:    1000 <- MSB inverted (0→1), lower bits cleared
          1001
          1010
          1011
          1100
          1101 <- MAX-1 reached again
Wrap:    0000 <- MSB inverted (1→0), lower bits cleared
```

15.7.2 Pattern Analysis

- **First Half:** MSB=0, counts 0 to MAX-1
- **Second Half:** MSB=1, counts 0 to MAX-1
- **Full Cycle:** 2×MAX states total

15.8 FIFO Application Context

15.8.1 Address Generation

This counter pattern is ideal for FIFO addressing because: 1. **Circular Addressing**: Lower bits provide circular buffer addresses 2. **Overflow Detection**: MSB toggles indicate buffer wrap-around 3. **Full/Empty Logic**: Comparing MSBs between read/write pointers

15.8.2 FIFO Pointer Comparison

```
// FIFO empty when pointers exactly equal  
assign fifo_empty = (rd_ptr == wr_ptr);
```

```
// FIFO full when addresses equal but MSBs differ  
assign fifo_full = (rd_ptr[WIDTH-2:0] == wr_ptr[WIDTH-2:0]) &&  
                  (rd_ptr[WIDTH-1] != wr_ptr[WIDTH-1]);
```

15.9 Timing Characteristics

15.9.1 Combinational Paths

- **Enable to Next**: Single logic level
- **Current to Next**: Increment/wrap logic depth
- **Critical Path**: Wrap detection and MSB inversion

15.9.2 Sequential Behavior

- **Setup Time**: Standard flip-flop requirements
- **Clock-to-Q**: Standard flip-flop propagation
- **Reset Recovery**: One cycle after reset deassertion

15.10 Design Examples

15.10.1 1. Small FIFO Buffer (8 entries)

```
counter_bin #(  
    .WIDTH(4),      // 4-bit counter  
    .MAX(8)         // 8 entries (0-7)  
) wr_ptr_counter (  
    .clk(clk),  
    .rst_n(rst_n),  
    .enable(wr_enable),  
    .counter_bin_curr(wr_ptr),
```

```
        .counter_bin_next(wr_ptr_next)
    );
```

15.10.2 2. Large FIFO Buffer (1024 entries)

```
counter_bin #(
    .WIDTH(11),    // 11-bit counter (10 bits address + 1 MSB)
    .MAX(1024)     // 1024 entries (0-1023)
) rd_ptr_counter (
    .clk(clk),
    .rst_n(rst_n),
    .enable(rd_enable),
    .counter_bin_curr(rd_ptr),
    .counter_bin_next(rd_ptr_next)
);
```

15.10.3 3. Asymmetric FIFO

```
// Write pointer - 16 entries
counter_bin #(.WIDTH(5), .MAX(16)) wr_cnt (...);
```

```
// Read pointer - different size for width conversion
counter_bin #(.WIDTH(6), .MAX(32)) rd_cnt (...);
```

15.11 Advanced Usage

15.11.1 Address Extraction

```
// Extract memory address (lower bits only)
assign mem_addr = counter_bin_curr[WIDTH-2:0];
```

```
// Extract wrap flag (MSB only)
assign wrap_flag = counter_bin_curr[WIDTH-1];
```

15.11.2 Gray Code Conversion

For asynchronous FIFOs, convert to Gray code:

```
logic [WIDTH-1:0] gray_ptr;
bin2gray #(.WIDTH(WIDTH)) b2g (
    .binary(counter_bin_curr),
    .gray(gray_ptr)
);
```

15.12 Synthesis Considerations

15.12.1 Resource Utilization

- **Flip-Flops:** WIDTH registers for current counter

- **LUTs:** Increment, wrap detection, and MSB inversion logic
- **Optimization:** Synthesis tools typically optimize the wrap logic efficiently

15.12.2 Timing Optimization

```
// Optional: Pipeline for high-frequency operation
logic [WIDTH-1:0] counter_bin_pipe;
always_ff @(posedge clk) begin
    counter_bin_pipe <= counter_bin_next;
end
```

15.12.3 Parameter Validation

```
initial begin
    assert (WIDTH >= 2)
        else $error("WIDTH must be at least 2 for MSB operation");
    assert (MAX <= (2**(WIDTH-1)))
        else $error("MAX too large for given WIDTH");
    assert (MAX > 0)
        else $error("MAX must be positive");
end
```

15.13 Verification Strategy

15.13.1 Test Scenarios

1. **Basic Increment:** Verify normal counting behavior
2. **Wrap Boundary:** Test MSB inversion at MAX-1
3. **Enable Control:** Verify hold behavior when disabled
4. **Reset Operation:** Confirm proper reset to zero
5. **Full Sequence:** Complete cycle through all states

15.13.2 Coverage Points

```
covergroup counter_bin_cg @(posedge clk);
    cp_lower_bits: coverpoint counter_bin_curr[WIDTH-2:0] {
        bins all_values[] = {[0:MAX-1]};
    }
    cp_msb: coverpoint counter_bin_curr[WIDTH-1] {
        bins low = {0};
        bins high = {1};
    }
    cp_wrap: coverpoint (counter_bin_curr[WIDTH-2:0] == w_max_val);
```

```
    cp_enable: coverpoint enable;  
endgroup
```

15.14 Common Issues and Solutions

15.14.1 Issue 1: Incorrect FIFO Size

Problem: MAX parameter doesn't match actual FIFO depth **Solution:** Ensure MAX equals the number of FIFO entries

15.14.2 Issue 2: Width Mismatch

Problem: WIDTH too small for MAX value **Solution:** Set $WIDTH \geq \lceil \log_2(2*MAX) \rceil$ for full range

15.14.3 Issue 3: Timing Violations

Problem: Wrap logic creates long combinational paths **Solution:** Pipeline the next value calculation or reduce MAX

15.15 Performance Characteristics

15.15.1 Maximum Frequency

- **Limiting Factor:** Wrap detection and MSB inversion logic
- **Typical Performance:** 300-500 MHz in modern FPGAs
- **Optimization:** Consider registering intermediate signals for high speed

15.15.2 Power Consumption

- **Dynamic Power:** Proportional to toggle rate
- **Optimization:** Use enable signal to reduce unnecessary toggles
- **Clock Gating:** Gate clock when FIFO operations are idle

15.16 Related Modules

- `counter_bingray`: Provides both binary and Gray code outputs
- `counter`: Basic counter without wrap behavior
- `counter_load_clear`: Loadable counter with different wrap behavior
- Standard FIFO controllers that use this addressing scheme

15.17 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

16 Frequency Invariant Counter Module

16.1 Overview

The `counter_freq_invariant` module is an advanced timing counter that maintains consistent time-based counting regardless of the input clock frequency. It uses a configurable prescaler to divide the input clock and provides a frequency-independent output counter, making it ideal for applications requiring consistent timing across different clock domains or frequencies.

16.2 Module Declaration

```
module counter_freq_invariant #(
    parameter int COUNTER_WIDTH = 5,          // Width of the output
    counter
    parameter int PRESCALER_MAX = 65536       // Maximum value of the pre-
    scaler
) (
    input logic clk,                          // Input clock
    input logic rst_n,                        // Active low
    reset
    input logic sync_reset_n,                // Synchronous
    reset signal
    input logic [3:0] freq_sel,              // Frequency
    selection (configurable)
    output logic [COUNTER_WIDTH-1:0] counter, // 5-bit output
    counter
    output logic tick                        // Pulse every
    time counter increments
);
```

16.3 Parameters

16.3.1 COUNTER_WIDTH

- **Type:** int
- **Default:** 5
- **Description:** Bit width of the main output counter

- **Range:** 1 to 32 (practical range)
- **Impact:** Determines maximum count value ($2^{\text{COUNTER_WIDTH}} - 1$)

16.3.2 PRESCALER_MAX

- **Type:** int
- **Default:** 65536
- **Description:** Maximum division factor for the prescaler
- **Range:** Must be $\geq$ largest division factor (10000)
- **Usage:** Determines internal prescaler counter width

16.4 Ports

16.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	Input clock signal (any frequency)
rst_n	1	logic	Active-low asynchronous reset
sync_reset_n	1	logic	Synchronous reset control
freq_sel	4	logic	Frequency selection (0-15)

16.4.2 Outputs

Port	Width	Type	Description
counter	COUNTER_WIDTH	logic	Main counter output
tick	1	logic	Pulse when counter increments

16.5 Frequency Selection Table

The module provides 16 predefined frequency divisions based on a 1GHz reference:

freq_sel	Division Factor	Target Frequency	Period	Use Case
4'b0000	1	1000MHz (1GHz)	1ns	High-speed processing
4'b0001	10	100MHz	10ns	Standard FPGA operations
4'b0010	20	50MHz	20ns	Memory interfaces
4'b0011	25	40MHz	25ns	Display controllers
4'b0100	40	25MHz	40ns	Audio sampling
4'b0101	50	20MHz	50ns	Serial communications
4'b0110	80	12.5MHz	80ns	Sensor interfaces
4'b0111	100	10MHz	100ns	ADC/DAC timing
4'b1000	125	8MHz	125ns	Microcontroller speeds
4'b1001	200	5MHz	200ns	Slow peripherals
4'b1010	250	4MHz	250ns	Power-optimized
4'b1011	500	2MHz	500ns	Ultra-low power
4'b1100	1000	1MHz	1μs	Timing references
4'b1101	2000	500kHz	2μs	Human interface


```

        4'b1011: w_division_factor = 16'd500;      // 2MHz
        4'b1100: w_division_factor = 16'd1000;    // 1MHz
        4'b1101: w_division_factor = 16'd2000;    // 500kHz
        4'b1110: w_division_factor = 16'd5000;    // 200kHz
        4'b1111: w_division_factor = 16'd10000;   // 100kHz
        default: w_division_factor = 16'd1;       // Safe default
    endcase
end

```

16.7.2 Change Detection Logic

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_prev_freq_sel <= 4'b0;
        r_clear_pulse <= 1'b0;
    end else begin
        r_prev_freq_sel <= freq_sel;

        // Generate clear pulse when freq_sel changes or sync reset
        r_clear_pulse <= (freq_sel != r_prev_freq_sel) || !
sync_reset_n;
    end
end

```

16.7.3 Prescaler Implementation

The prescaler uses the counter_load_clear module to implement configurable division:

```

counter_load_clear #(
    .MAX(PRESCALER_MAX)
) prescaler_counter(
    .clk(clk),
    .rst_n(rst_n),
    .clear(r_clear_pulse),           // Clear on frequency change
    .increment(1'b1),               // Always increment
    .load(1'b1),                    // Always load new value
    .loadval(w_division_factor[$clog2(PRESCALER_MAX)-1:0] - 1'b1),
    .done(w_prescaler_done),
    .count()                         // Unused
);

```

16.7.4 Main Counter and Tick Generation

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        counter <= 'b0;
        tick <= 'b0;
    end else begin
        // Generate tick when prescaler completes and counter at max
        if (w_prescaler_done && counter)

```

```

        tick <= 'b1;
    else
        tick <= 'b0;

    // Update main counter
    if (r_clear_pulse)
        counter <= 'b0;
    else if (w_prescaler_done)
        counter <= counter + 1'b1; // Natural rollover
end
end

```

16.8 Timing Analysis

16.8.1 Prescaler Operation

- **Load Value:** $\text{division_factor} - 1$
- **Count Cycles:** division_factor clock cycles per prescaler tick
- **Frequency:** $f_{\text{input}} / \text{division_factor}$

16.8.2 Main Counter Operation

- **Increment Rate:** Prescaler tick rate
- **Roll-over:** At $2^{\text{COUNTER_WIDTH}}$
- **Tick Generation:** One cycle when counter is all 1's and prescaler completes

16.8.3 Overall Timing

- **Counter Period:** $\text{division_factor} \times 2^{\text{COUNTER_WIDTH}}$ input clock cycles
- **Tick Period:** division_factor input clock cycles
- **Frequency Independence:** Same tick rate regardless of input frequency

16.9 Design Examples

16.9.1 1. 1MHz Tick Generator (Any Input Clock)

```

counter_freq_invariant #(
    .COUNTER_WIDTH(8),           // 8-bit counter (0-255)
    .PRESCALER_MAX(65536)       // Support up to 65536 division
) tick_1mhz (
    .clk(clk_any_freq),         // Could be 100MHz, 200MHz, etc.

```

```

        .rst_n(rst_n),
        .sync_reset_n(1'b1),
        .freq_sel(4'b1100),      // Select 1MHz (1000 division)
        .counter(timer_count),
        .tick(tick_1mhz)
    );

```

16.9.22. Configurable Baud Rate Generator

```

logic [3:0] baud_sel;
logic baud_tick;

// Baud rate selection mapping
// 4'b0111 = 10MHz → 115200 baud (10MHz/87 ≈ 115200)
// 4'b1001 = 5MHz → 57600 baud (5MHz/87 ≈ 57600)
// 4'b1100 = 1MHz → 9600 baud (1MHz/104 ≈ 9600)

counter_freq_invariant #(
    .COUNTER_WIDTH(4),
    .PRESCALER_MAX(10000)
) baud_gen (
    .clk(sys_clk),
    .rst_n(rst_n),
    .sync_reset_n(1'b1),
    .freq_sel(baud_sel),
    .counter(),           // Unused
    .tick(baud_tick)
);

```

16.9.33. Multi-Rate Timer System

```

// Different timers for different purposes
counter_freq_invariant #(.COUNTER_WIDTH(6)) fast_timer (
    .freq_sel(4'b0001),      // 100MHz rate
    .tick(fast_tick),        // Every 10ns
    ...
);

counter_freq_invariant #(.COUNTER_WIDTH(8)) med_timer (
    .freq_sel(4'b0111),      // 10MHz rate
    .tick(med_tick),         // Every 100ns
    ...
);

counter_freq_invariant #(.COUNTER_WIDTH(10)) slow_timer (
    .freq_sel(4'b1100),      // 1MHz rate
    .tick(slow_tick),        // Every 1μs
    ...
);

```

16.10 Advanced Features

16.10.1 Dynamic Frequency Switching

// Example: Switch between fast and slow modes

```
logic power_save_mode;
logic [3:0] dynamic_freq_sel;

always_comb begin
    if (power_save_mode)
        dynamic_freq_sel = 4'b1111; // 100kHz for power save
    else
        dynamic_freq_sel = 4'b0001; // 100MHz for performance
end

counter_freq_invariant adaptive_timer (
    .freq_sel(dynamic_freq_sel),
    .sync_reset_n(!power_save_mode), // Reset when switching modes
    ...
);
```

16.10.2 Synchronous Reset Applications

// Reset timer when entering specific system state

```
logic enter_calibration_mode;

counter_freq_invariant cal_timer (
    .sync_reset_n(!enter_calibration_mode),
    .freq_sel(4'b1010), // 4MHz for calibration
    ...
);
```

16.11 Verification Strategy

16.11.1 Test Scenarios

1. **Frequency Selection:** Test all 16 freq_sel values
2. **Dynamic Switching:** Change freq_sel during operation
3. **Reset Behavior:** Test both async and sync reset
4. **Tick Generation:** Verify tick timing at various frequencies
5. **Counter Rollover:** Test main counter wrap-around
6. **Prescaler Boundary:** Test prescaler terminal count

16.11.2 Testbench Example

```
module tb_counter_freq_invariant;
    logic clk, rst_n, sync_reset_n;
    logic [3:0] freq_sel;
    logic [4:0] counter;
    logic tick;

    // Clock generation - 100MHz
    initial clk = 0;
    always #5ns clk = ~clk;

    // Test sequence
    initial begin
        rst_n = 0;
        sync_reset_n = 1;
        freq_sel = 4'b0000;

        #100ns rst_n = 1;

        // Test different frequencies
        #1000ns freq_sel = 4'b0001; // 100MHz → 10MHz
        #1000ns freq_sel = 4'b0111; // 10MHz
        #1000ns freq_sel = 4'b1100; // 1MHz

        // Test synchronous reset
        #1000ns sync_reset_n = 0;
        #50ns sync_reset_n = 1;

        #10000ns $finish;
    end

    // Monitor tick frequency
    real tick_period, last_tick_time;
    always @(posedge tick) begin
        tick_period = $realtime - last_tick_time;
        last_tick_time = $realtime;
        $display("Tick period: %.2f ns (freq_sel=%b)", tick_period,
freq_sel);
    end
endmodule
```

16.11.3 Coverage Points

```
covergroup freq_invariant_cg @(posedge clk);
    cp_freq_sel: coverpoint freq_sel {
        bins all_freqs[] = {[0:15]};
    }
}
```

```

cp_counter: coverpoint counter {
    bins low = {[0:7]};
    bins mid = {[8:23]};
    bins high = {[24:31]};
}

cp_freq_change: coverpoint (freq_sel != $past(freq_sel)) {
    bins change = {1};
    bins stable = {0};
}

cp_sync_reset: coverpoint sync_reset_n {
    bins active = {0};
    bins inactive = {1};
}
endgroup

```

16.12 Performance Characteristics

16.12.1 Resource Utilization

- **Prescaler Counter:** $\lceil \log_2(\text{PRESCALER_MAX}) \rceil$ bits
- **Main Counter:** COUNTER_WIDTH bits
- **Logic:** Division factor lookup table + change detection
- **Total FFs:** ~20-25 flip-flops for typical configuration

16.12.2 Timing Performance

- **Maximum Frequency:** Limited by prescaler counter increment
- **Typical:** 200-400 MHz in modern FPGAs
- **Critical Path:** Prescaler increment + done detection

16.12.3 Power Consumption

- **Dynamic:** Proportional to clock frequency and toggle rate
- **Optimization:** Higher division factors reduce overall power
- **Clock Gating:** Consider gating prescaler when not needed

16.13 Applications

16.13.1 Communication Systems

```
// UART baud rate generation
counter_freq_invariant uart_baud (
    .freq_sel(baud_rate_sel), // From configuration register
    .tick(baud_x16_tick),      // 16x oversampling
    ...
);
```

16.13.2 Display Controllers

```
// Pixel clock generation for different video modes
counter_freq_invariant pixel_clk (
    .freq_sel(video_mode_sel), // Different pixel rates
    .tick(pixel_enable),
    ...
);
```

16.13.3 ADC/DAC Timing

```
// Sample rate control independent of system clock
counter_freq_invariant sample_rate (
    .freq_sel(sample_rate_sel), // Audio sample rates
    .tick(conversion_trigger),
    ...
);
```

16.13.4 Power Management

```
// Frequency scaling for power optimization
counter_freq_invariant power_timer (
    .freq_sel(power_mode_sel), // Fast/medium/slow/sleep
    .tick(power_event),
    ...
);
```

16.14 Design Guidelines

16.14.1 Parameter Selection

1. **COUNTER_WIDTH**: Choose based on maximum count needed
2. **PRESCALER_MAX**: Must be $\geq$ largest division factor (10000)
3. **freq_sel Mapping**: Customize division table for your application

16.14.2 Reset Strategy

1. **Asynchronous Reset:** For system-wide reset
2. **Synchronous Reset:** For mode changes and state machine control
3. **Clear Pulse:** Automatically generated on frequency changes

16.14.3 Frequency Planning

1. **Input Clock:** Can be any stable frequency
2. **Division Factors:** Choose based on target frequencies
3. **Jitter:** Higher division factors may accumulate jitter

16.15 Troubleshooting

16.15.1 Common Issues

1. **Incorrect Tick Rate:** Verify division factor calculation
2. **Missing Ticks:** Check for proper reset sequencing
3. **Frequency Switching Glitches:** Ensure clean freq_sel transitions
4. **Timing Violations:** Pipeline if prescaler counter is too wide

16.15.2 Debug Techniques

1. **Simulation:** Use time-based measurements to verify tick rates
2. **Logic Analyzer:** Capture actual tick timing in hardware
3. **Frequency Counter:** Measure actual output frequency
4. **Reset Analysis:** Check reset propagation and synchronization

16.16 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

17 Johnson Counter Module

17.1 Overview

The `counter_johnson` module implements a Johnson counter (also known as a twisted-ring counter or switch-tail counter). This is a special type of shift register where the inverted output of the last stage is fed back to the first stage, creating a sequence with $2 \times \text{WIDTH}$ unique states. Johnson counters are widely used for generating multi-phase clock signals, state machine control, and sequential timing applications.

17.2 Module Declaration

```
module counter_johnson #(
    parameter int WIDTH = 4
) (
    input logic          clk,
    input logic          rst_n,
    input logic          enable,
    output logic [WIDTH - 1:0] counter_gray
);
```

17.3 Parameters

17.3.1 WIDTH

- **Type:** int
- **Default:** 4
- **Description:** Number of stages in the Johnson counter
- **Range:** Any positive integer ≥ 1
- **States Generated:** $2 \times \text{WIDTH}$ unique states
- **Impact:** Determines sequence length and number of output phases

17.4 Ports

17.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low

Port	Width	Type	Description
enable	1	logic	asynchronous reset Counter enable control

17.4.2 Outputs

Port	Width	Type	Description
counter_gray	WIDTH	logic	Johnson counter output register

17.5 Theory of Operation

17.5.1 Johnson Counter Principle

A Johnson counter is a shift register with inverted feedback: - **Normal Operation:** Bits shift left (or right) each clock cycle - **Feedback:** The complement of the MSB feeds back to the LSB - **Sequence Length:** $2 \times \text{WIDTH}$ states before repeating

17.5.2 Mathematical Representation

For a WIDTH-bit Johnson counter:

Next_State[i] = Current_State[i-1] for i = 1 to WIDTH-1
Next_State[0] = ~Current_State[WIDTH-1]

17.6 Implementation Details

17.6.1 Core Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        counter_gray <= {WIDTH{1'b0}}; // Reset to all zeros
    else if (enable) begin
        counter_gray <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]};
    end
end
```

17.6.2 Operation Breakdown

1. **Shift Operation:** counter_gray[WIDTH-2:0] → lower bits of next state

2. **Feedback:** $\sim\text{counter_gray}[\text{WIDTH}-1] \rightarrow$ MSB complement becomes new LSB
3. **Enable Control:** Only advance when enable is asserted
4. **Reset:** All bits cleared on reset

17.7 State Sequences

17.7.1 4-bit Johnson Counter (WIDTH=4)

Step	counter_gray	Decimal	Description
0	0000	0	Reset state
1	0001	1	First '1' enters
2	0011	3	'1' propagates
3	0111	7	'1' propagates
4	1111	15	All '1's
5	1110	14	First '0' enters
6	1100	12	'0' propagates
7	1000	8	'0' propagates
8	0000	0	Back to start

17.7.2 3-bit Johnson Counter (WIDTH=3)

Step	counter_gray	Binary Pattern
0	000	All zeros
1	001	One bit set
2	011	Two bits set
3	111	All ones
4	110	Two bits set
5	100	One bit set
6	000	Cycle repeats

17.7.3 2-bit Johnson Counter (WIDTH=2)

Step	counter_gray	State Name
0	00	State A
1	01	State B

Step	counter_gray	State Name
2	11	State C
3	10	State D
4	00	Back to A

17.8 Unique Properties

17.8.1 Self-Starting

Johnson counters are self-starting from most invalid states: - **Valid States:** $2 \times \text{WIDTH}$ states in the normal sequence - **Invalid States:** $(2^{\text{WIDTH}} - 2 \times \text{WIDTH})$ states not in sequence - **Recovery:** Most invalid states naturally converge to valid sequence

17.8.2 Symmetrical Patterns

The sequence has inherent symmetry: - **First Half:** Filling with '1's (0→1→11→111...) - **Second Half:** Filling with '0's (111→110→100→000...) - **Phase Shift:** Each bit represents a different phase

17.8.3 Gray Code Properties

Johnson counter outputs have some Gray code characteristics: - **Adjacent Differences:** States differ by more than one bit - **Predictable Transitions:** Well-defined state progression - **Decode Simplicity:** Easy to decode specific states

17.9 Design Examples

17.9.1 1. 4-Phase Clock Generator

```
counter_johnson #(
    .WIDTH(4)
) phase_gen (
    .clk(master_clk),
    .rst_n(rst_n),
    .enable(1'b1),
    .counter_gray(phases)
);
```

```
// Decode individual phases
assign phase_0 = (phases == 4'b0001); // Step 1
assign phase_1 = (phases == 4'b0011); // Step 2
assign phase_2 = (phases == 4'b0111); // Step 3
assign phase_3 = (phases == 4'b1111); // Step 4
```

17.9.2. LED Chaser Effect

```
counter_johnson #(
    .WIDTH(8)
```

```

) led_chaser (
    .clk(slow_clk),           // ~1Hz clock
    .rst_n(rst_n),
    .enable(chase_enable),
    .counter_gray(led_pattern)
);

```

```

// LEDs show walking pattern
assign leds[7:0] = led_pattern;

```

17.9.33. State Machine Controller

```

counter_johnson #(
    .WIDTH(3)
) state_ctrl (
    .clk(clk),
    .rst_n(rst_n),
    .enable(state_advance),
    .counter_gray(state_code)
);

```

```

// Decode states for control logic
always_comb begin
    case (state_code)
        3'b001: current_state = INIT;
        3'b011: current_state = LOAD;
        3'b111: current_state = PROCESS;
        3'b110: current_state = STORE;
        3'b100: current_state = CLEANUP;
        3'b000: current_state = IDLE;
        default: current_state = ERROR;
    endcase
end

```

17.9.44. Quadrature Signal Generation

```

// Generate quadrature encoder-like signals
counter_johnson #(
    .WIDTH(4)
) quad_gen (
    .clk(position_clk),
    .rst_n(rst_n),
    .enable(motor_moving),
    .counter_gray(quad_state)
);

```

```

// Extract A and B channels with 90° phase shift
assign quad_a = quad_state[0] ^ quad_state[1];
assign quad_b = quad_state[1] ^ quad_state[2];

```

17.10 Advantages

17.10.1 1. Simplicity

- **Minimal Logic:** Just a shift register with inverted feedback
- **No Decode Logic:** States can be used directly
- **Self-Clocking:** No complex timing requirements

17.10.2 2. Reliability

- **Self-Starting:** Recovers from most error states
- **No Invalid States:** All reachable states are functional
- **Glitch-Free:** Clean transitions between states

17.10.3 3. Performance

- **High Speed:** Simple logic allows high clock frequencies
- **Low Power:** Minimal switching activity
- **Predictable Timing:** Regular state progression

17.10.4 4. Flexibility

- **Scalable:** Any width supported
- **Multiple Outputs:** Each bit provides different phase
- **Easy Modification:** Simple parameter changes

17.11 Disadvantages

17.11.1 1. Limited States

- **Count Efficiency:** Only $2 \times \text{WIDTH}$ states vs 2^{WIDTH} for binary
- **Sequence Constraint:** Fixed progression pattern
- **No Random Access:** Must step through sequence

17.11.2 2. Decode Complexity

- **State Detection:** May need comparators for specific states
- **Non-Binary:** Not directly compatible with binary logic

- **Overlap Prevention:** Care needed to avoid state conflicts

17.12 Applications

17.12.1 Clock Generation

```
// Multi-phase clock for pipeline stages
counter_johnson #(.WIDTH(4)) pipeline_phases (
    .clk(master_clk),
    .enable(pipeline_enable),
    .counter_gray(phase_vector)
);
```

```
// Generate phase-shifted clocks
assign clk_ph0 = master_clk & phase_vector[0];
assign clk_ph1 = master_clk & phase_vector[1];
assign clk_ph2 = master_clk & phase_vector[2];
assign clk_ph3 = master_clk & phase_vector[3];
```

17.12.2 Sequential Control

```
// Memory refresh controller
counter_johnson #(.WIDTH(3)) refresh_ctrl (
    .clk(clk),
    .enable(refresh_request),
    .counter_gray(refresh_phase)
);
```

```
// Control refresh operations
assign precharge = (refresh_phase == 3'b001);
assign activate  = (refresh_phase == 3'b011);
assign refresh   = (refresh_phase == 3'b111);
assign restore   = (refresh_phase == 3'b110);
```

17.12.3 Pattern Generation

```
// Test pattern generator
counter_johnson #(.WIDTH(8)) pattern_gen (
    .clk(test_clk),
    .enable(pattern_enable),
    .counter_gray(test_pattern)
);
```

```
// Use for built-in self-test (BIST)
assign test_data = test_pattern;
```

17.13 Verification Strategy

17.13.1 Functional Tests

1. **Sequence Verification:** Check complete $2 \times \text{WIDTH}$ state cycle
2. **Reset Behavior:** Verify initialization to all zeros
3. **Enable Control:** Test hold behavior when disabled
4. **Self-Starting:** Verify recovery from invalid states

17.13.2 Coverage Points

```
covergroup johnson_cg @(posedge clk);
  cp_states: coverpoint counter_gray {
    // Valid Johnson counter states for WIDTH=4
    bins valid_states[] = {4'b0000, 4'b0001, 4'b0011, 4'b0111,
                          4'b1111, 4'b1110, 4'b1100, 4'b1000};
    bins invalid_states[] = default;
  }

  cp_transitions: coverpoint counter_gray {
    bins valid_trans[] = (4'b0000 => 4'b0001),
                        (4'b0001 => 4'b0011),
                        (4'b0011 => 4'b0111),
                        (4'b0111 => 4'b1111),
                        (4'b1111 => 4'b1110),
                        (4'b1110 => 4'b1100),
                        (4'b1100 => 4'b1000),
                        (4'b1000 => 4'b0000);
  }

  cp_enable: coverpoint enable {
    bins enabled = {1};
    bins disabled = {0};
  }
endgroup
```

17.13.3 Assertions

```
// Verify valid state progression
property johnson_sequence;
  @(posedge clk) disable iff (!rst_n)
  enable && (counter_gray == 4'b0000) | =>
  counter_gray == 4'b0001;
endproperty

assert property (johnson_sequence);
```

```
// Check for invalid states
property no_invalid_states;
    @(posedge clk) disable iff (!rst_n)
        counter_gray inside {4'b0000, 4'b0001, 4'b0011, 4'b0111,
                             4'b1111, 4'b1110, 4'b1100, 4'b1000};
endproperty

assert property (no_invalid_states);
```

17.14 Synthesis Considerations

17.14.1 Resource Utilization

- **Flip-Flops:** WIDTH registers
- **LUTs:** Minimal - just for shift and invert logic
- **Routing:** Simple connections between adjacent stages

17.14.2 Optimization

```
// For high-speed applications, consider pipeline stages
logic [WIDTH-1:0] johnson_pipe;
always_ff @(posedge clk) begin
    johnson_pipe <= {counter_gray[WIDTH-2:0], ~counter_gray[WIDTH-1]};
end
```

17.14.3 Tool Attributes

```
(* SHREG_EXTRACT = "NO" *) logic [WIDTH-1:0] counter_gray; // Prevent
SRL inference
```

17.15 Performance Characteristics

17.15.1 Maximum Frequency

- **Typical:** 400-600 MHz in modern FPGAs
- **Limitation:** Shift register timing
- **Optimization:** Pipeline for extreme speeds

17.15.2 Power Consumption

- **Low Dynamic Power:** Only one bit changes per cycle
- **Efficient:** Minimal logic switching

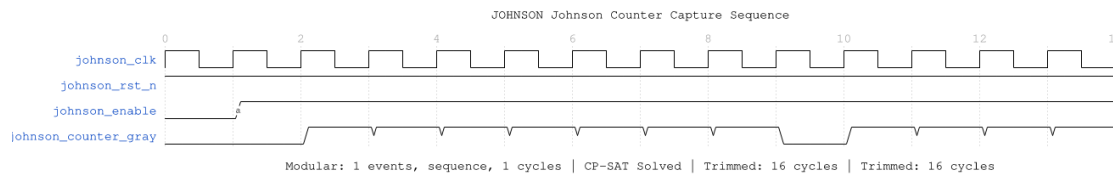
- **Clock Gating:** Use enable for power savings

17.16 WaveDrom Visualization

High-quality waveforms showcasing Johnson counter operation are available!

The following timing diagrams demonstrate the unique properties of Johnson counters across 4 key scenarios:

17.16.1 Scenario 1: Walking Ones and Walking Zeros Pattern

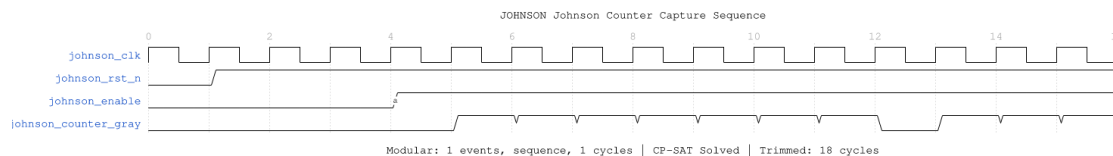


Johnson Walking Pattern

WaveJSON: [johnson_counter_walking_pattern.json](#)

Complete $2 \times \text{WIDTH}$ state cycle (8 states for $\text{WIDTH}=4$): - Walking ones: 0000 → 0001 → 0011 → 0111 → 1111 - Walking zeros: 1111 → 1110 → 1100 → 1000 → 0000 - Demonstrates predictable sequential progression

17.16.2 Scenario 2: Single-Bit Transitions (CDC Safety) ★ KEY FEATURE

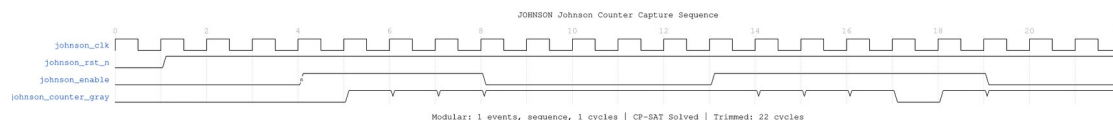


Johnson Single-Bit Transitions

WaveJSON: [johnson_counter_single_bit_transitions.json](#)

Each transition changes only ONE bit: - CDC-safe like Gray codes - **Critical for `fifo_async_div2` CDC mechanism** - Prevents metastability in clock domain crossing - Hamming distance = 1 between all adjacent states

17.16.3 Scenario 3: Enable Control

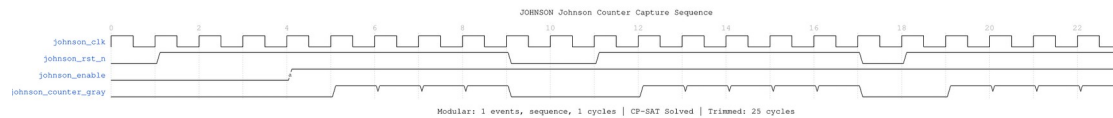


Johnson Enable Control

WaveJSON: [johnson_counter_enable_control.json](#)

Enable control and state holding: - Counter advances when enable=1 - Counter holds state when enable=0 - Clean enable/disable transitions - Demonstrates conditional counting

17.16.4 Scenario 4: Reset Behavior



Johnson Reset Behavior

WaveJSON: [johnson_counter_reset_behavior.json](#)

Reset and initialization: - Asynchronous reset to all zeros (0000) - Immediate reset effect - Clean restart from reset state - Reset during counting operation

To regenerate these waveforms:

```
pytest val/common/test_counter_johnson_wavedrom.py -v
# Then convert JSON to PNG:
cd docs/markdown/assets/WAVES/counter_johnson
for f in *.json; do wavedrom-cli -i "$f" -p "${f%.json}.png"; done
```

What Makes Johnson Counters Special:

The waveforms highlight the unique properties that make Johnson counters useful: - **Single-Bit Transitions**: Only one bit changes per state, making them CDC-safe - **2×WIDTH States**: More efficient than one-hot (N states with N/2 flip-flops) - **Walking Pattern**: Natural “fill then empty” sequence useful for visual effects - **Self-Starting**: Recovers from invalid states automatically

Relationship to fifo_async_div2:

Johnson counters are the foundation of the `fifo_async_div2` CDC mechanism: - **fifo_async_div2** uses Johnson counters for pointer synchronization - Single-bit transitions enable safe clock domain crossing - Linear width scaling (DEPTH bits) allows flexible even depths - See `test_fifo_async_div2_wavedrom.py` for CDC application

Comparison with Other Counters:

- `test_counter_bingray_wavedrom.py` - Binary-Gray counter (power-of-2 depths, logarithmic width)
- `test_fifo_async_wavedrom.py` - Gray code in action (async FIFO)

- `test_fifo_async_div2_wavedrom.py` - Johnson counter in action (async FIFO, even depths)

17.17 Related Modules

- `counter_ring`: Different shift register pattern
- `counter_bingray`: Binary and Gray code counter
- `counter`: Simple binary counter
- Standard shift register implementations

17.18 Test and Verification

Comprehensive Test Suite: - `val/common/test_counter_johnson.py` - Full functional verification - `val/common/test_counter_johnson_wavedrom.py` - WaveDrom timing diagrams



Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_counter_johnson.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_counter_johnson_wavedrom.py -v
```

17.19 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

18 Counter Load Clear Module

18.1 Overview

The `counter_load_clear` module is a versatile counter with loadable terminal count and clear functionality. It provides precise control over counting behavior with configurable match values, making it ideal for timing applications, protocol implementations, and configurable delay generation.

18.2 Module Declaration

```
module counter_load_clear #(
    parameter int MAX = 32'd32
```

```

) (
    input logic          clk,
    input logic          rst_n,
    input logic          clear,
    input logic          increment,
    input logic          load,
    input logic [$clog2(MAX)-1:0] loadval,
    output logic [$clog2(MAX)-1:0] count,
    output logic         done
);

```

18.3 Parameters

18.3.1 MAX

- **Type:** int
- **Default:** 32'd32
- **Description:** Maximum possible count value
- **Range:** Any positive 32-bit integer
- **Usage:** Determines counter width and maximum loadable value
- **Width Calculation:** Counter width = $\$clog2(MAX)$

18.4 Ports

18.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock
rst_n	1	logic	Active-low asynchronous reset
clear	1	logic	Synchronous clear (highest priority)
increment	1	logic	Enable counter increment
load	1	logic	Load new match value

Port	Width	Type	Description
loadval	$\$clog2(MAX)$	logic	New terminal count value

18.4.2 Outputs

Port	Width	Type	Description
count	$\$clog2(MAX)$	logic	Current counter value
done	1	logic	Terminal count reached flag

18.5 Architecture Details

18.5.1 Internal Registers

```

logic [ $\$clog2(MAX)-1:0$ ] count;           // Current count value
logic [ $\$clog2(MAX)-1:0$ ] r_match_val;    // Terminal count register

```

18.5.2 Priority Logic

The module implements a priority-based control scheme:

1. **Asynchronous Reset** (Highest Priority)
 - Resets both count and r_match_val to 0
 - Independent of clock edge
2. **Load Operation** (High Priority)
 - Updates r_match_val with loadval
 - Can occur simultaneously with other operations
3. **Clear Operation** (Medium Priority)
 - Immediately sets count to 0
 - Overrides increment operation
4. **Increment Operation** (Lowest Priority)
 - Advances counter or wraps to 0 at terminal count

- Only when increment is asserted

18.6 Implementation Details

18.6.1 Main Control Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 'b0;
        r_match_val <= 'b0;
    end else begin
        // Load has independent priority
        if (load) r_match_val <= loadval;

        // Clear has highest operational priority
        if (clear) begin
            count <= 'b0;
        end else if (increment) begin
            count <= (count == r_match_val) ? 'b0 : count + 'b1;
        end
    end
end
```

18.6.2 Done Signal Generation

```
assign done = (count == r_match_val);
```

18.7 Operation Modes

18.7.1 1. Basic Counting Mode

```
// Simple timer: count to 100
counter_load_clear #(.MAX(1000)) timer (
    .clk(clk),
    .rst_n(rst_n),
    .clear(1'b0),
    .increment(1'b1),
    .load(1'b1),
    .loadval(8'd99),    // Count 0-99 (100 cycles)
    .count(timer_count),
    .done(timer_done)
);
```

18.7.2 2. Configurable Delay Mode

```
// Variable delay generator
logic [7:0] delay_setting;
logic start_delay, delay_complete;
```

```

counter_load_clear #(.MAX(256)) delay_gen (
    .clk(clk),
    .rst_n(rst_n),
    .clear(start_delay),           // Start new delay
    .increment(1'b1),
    .load(start_delay),           // Load new value when starting
    .loadval(delay_setting),
    .count(),
    .done(delay_complete)
);

```

18.7.33. Retriggerable Timer Mode

// Watchdog timer that resets on activity

logic activity_detected, timeout;

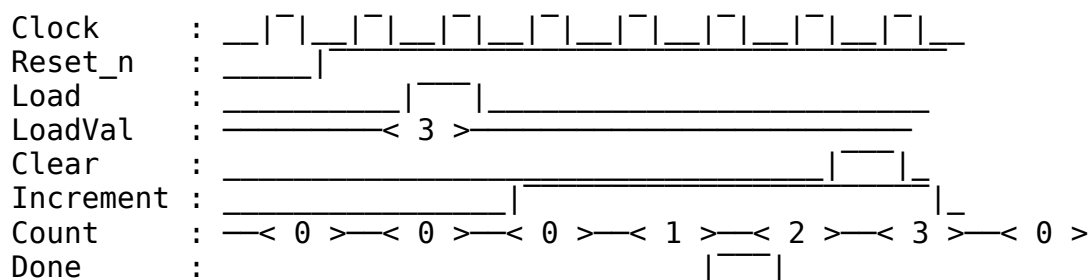
```

counter_load_clear #(.MAX(10000)) watchdog (
    .clk(clk),
    .rst_n(rst_n),
    .clear(activity_detected),    // Reset on activity
    .increment(1'b1),
    .load(1'b1),
    .loadval(16'd9999),         // 10000 cycle timeout
    .count(),
    .done(timeout)
);

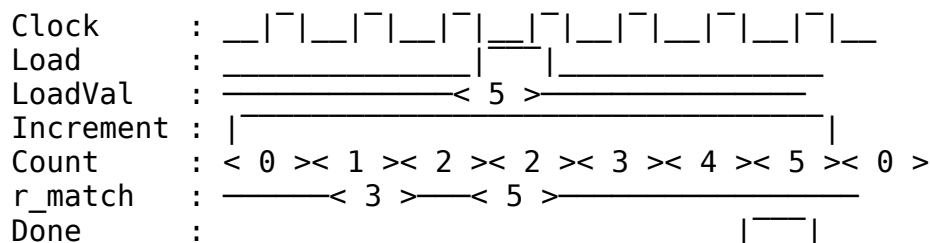
```

18.8 Timing Diagrams

18.8.1 Basic Operation



18.8.2 Load During Count



18.9 Advanced Usage Examples

18.9.11. Dual-Rate Timer

// Timer with fast/slow modes

```
logic fast_mode;
logic [15:0] timer_period;

always_comb begin
    timer_period = fast_mode ? 16'd99 : 16'd999; // 100 vs 1000
cycles
end
```

```
counter_load_clear #(.MAX(65536)) dual_timer (
    .clk(clk),
    .rst_n(rst_n),
    .clear(timer_done),           // Auto-restart
    .increment(1'b1),
    .load(timer_done),           // Reload on completion
    .loadval(timer_period),
    .count(timer_value),
    .done(timer_done)
);
```

18.9.22. PWM Generator

// Simple PWM with configurable duty cycle

```
logic [7:0] duty_cycle; // 0-255
logic pwm_out;

counter_load_clear #(.MAX(256)) pwm_counter (
    .clk(clk),
    .rst_n(rst_n),
    .clear(1'b0),
    .increment(1'b1),
    .load(1'b1),
    .loadval(8'd255),           // Always count 0-255
    .count(pwm_count),
    .done()
);
```

```
assign pwm_out = (pwm_count < duty_cycle);
```

18.9.33. Burst Generator

// Generate bursts of N pulses

```
logic [7:0] burst_length;
logic start_burst, burst_active, burst_complete;
```

```

counter_load_clear #(.MAX(256)) burst_counter (
    .clk(clk),
    .rst_n(rst_n),
    .clear(start_burst),
    .increment(burst_active),
    .load(start_burst),
    .loadval(burst_length - 1),
    .count(burst_count),
    .done(burst_complete)
);

```

```

// State machine for burst control
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        burst_active <= 1'b0;
    end else begin
        if (start_burst)
            burst_active <= 1'b1;
        else if (burst_complete)
            burst_active <= 1'b0;
    end
end

```

18.9.44. Communication Timeout

```

// UART receive timeout
logic rx_active, rx_timeout, char_received;
parameter RX_TIMEOUT_CYCLES = 1000;

counter_load_clear #(.MAX(2048)) rx_timer (
    .clk(clk),
    .rst_n(rst_n),
    .clear(char_received),           // Reset on new character
    .increment(rx_active),          // Only count when receiving
    .load(rx_active && !char_received),
    .loadval(RX_TIMEOUT_CYCLES-1),
    .count(),
    .done(rx_timeout)
);

```

18.10 Design Patterns

18.10.1 Pattern 1: Auto-Reloading Timer

```

wire auto_reload = timer_done;
counter_load_clear timer (
    .clear(auto_reload),

```

```

        .load(auto_reload),
        .loadval(PERIOD-1),
        .done(timer_done),
        ...
    );

```

18.10.2 Pattern 2: Conditional Counting

```

wire count_enable = condition_met && !paused;
counter_load_clear conditional_timer (
    .increment(count_enable),
    ...
);

```

18.10.3 Pattern 3: Dynamic Period Adjustment

```

logic period_changed;
wire reload_needed = timer_done || period_changed;

counter_load_clear adaptive_timer (
    .clear(reload_needed),
    .load(reload_needed),
    .loadval(current_period),
    ...
);

```

18.11 Verification Strategy

18.11.1 Test Scenarios

1. **Basic Counting:** Verify normal increment operation
2. **Terminal Count:** Test done signal generation
3. **Load Operation:** Verify match value updates
4. **Clear Priority:** Test clear overrides increment
5. **Simultaneous Operations:** Load + clear + increment
6. **Boundary Conditions:** Count = 0, count = match_val
7. **Reset Behavior:** Async reset during various operations

18.11.2 Coverage Model

```

covergroup load_clear_cg @(posedge clk);
    cp_count: coverpoint count {
        bins zero = {0};
    }

```

```

    bins low = {[1:MAX/4]};
    bins mid = {[MAX/4+1:3*MAX/4]};
    bins high = {[3*MAX/4+1:MAX-1]};
}

cp_match_val: coverpoint r_match_val {
    bins zero = {0};
    bins low = {[1:MAX/4]};
    bins mid = {[MAX/4+1:3*MAX/4]};
    bins high = {[3*MAX/4+1:MAX-1]};
}

cp_operations: coverpoint {clear, increment, load} {
    bins clear_only = {3'b100};
    bins inc_only = {3'b010};
    bins load_only = {3'b001};
    bins clear_inc = {3'b110};
    bins clear_load = {3'b101};
    bins inc_load = {3'b011};
    bins all_ops = {3'b111};
    bins no_ops = {3'b000};
}

cp_done: coverpoint done;

// Cross coverage
cross_done_ops: cross cp_operations, done;
endgroup

```

18.11.3 Assertions

```

// Count should never exceed match value
property count_bounds;
    @(posedge clk) disable iff (!rst_n)
        count <= r_match_val;
endproperty

// Done should be true when count equals match
property done_when_match;
    @(posedge clk) disable iff (!rst_n)
        (count == r_match_val) |-> done;
endproperty

// Clear should reset count to 0
property clear_resets_count;
    @(posedge clk) disable iff (!rst_n)
        clear |=> (count == 0);
endproperty

```

```
// Load should update match value
property load_updates_match;
    @(posedge clk) disable iff (!rst_n)
        load | => (r_match_val == $past(loadval));
endproperty

assert property (count_bounds);
assert property (done_when_match);
assert property (clear_resets_count);
assert property (load_updates_match);
```

18.12 Synthesis Considerations

18.12.1 Resource Usage

- **Flip-Flops:** $\$clog2(\text{MAX})$ for count + $\$clog2(\text{MAX})$ for match = $2 \times \$clog2(\text{MAX})$
- **LUTs:** Increment logic, comparator, and control logic
- **Typical:** 15-25 LUTs for 16-bit counter

18.12.2 Timing Optimization

```
// For high-speed operation, pipeline the comparison
logic done_pipe;
always_ff @(posedge clk) begin
    done_pipe <= (count == r_match_val);
end
```

18.12.3 Parameter Validation

```
initial begin
    assert (MAX > 0) else $error("MAX must be positive");
    assert (MAX <= 2**30) else $warning("Large MAX may impact
synthesis");
end
```

18.13 Performance Characteristics

18.13.1 Maximum Frequency

- **Typical:** 300-500 MHz in modern FPGAs
- **Critical Path:** Increment + comparison logic
- **Optimization:** Consider pipelining for extreme speeds

18.13.2 Latency

- **Load Operation:** 1 cycle to update match value
- **Clear Operation:** 1 cycle to reset count
- **Done Signal:** Combinational (0 cycles)

18.14 Common Applications

1. **Timer Modules:** Configurable timing generation
2. **Protocol Engines:** Timeout and retry mechanisms
3. **PWM Controllers:** Variable duty cycle generation
4. **Burst Generators:** Programmable pulse trains
5. **Watchdog Timers:** System monitoring and reset
6. **Clock Dividers:** Variable frequency division
7. **Test Pattern Generators:** Configurable test sequences

18.15 Related Modules

- counter: Basic counter without load/clear
- counter_freq_invariant: Uses this module for prescaling
- Standard timer and PWM controller implementations

18.16 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

19 Basic Counter Module

19.1 Overview

The counter module is a simple, configurable up-counter that generates a tick pulse when reaching its maximum value. It's designed for basic timing applications where a periodic signal is needed.

19.2 Module Declaration

```
module counter #(
    parameter int MAX = 32767
) (
    input logic clk,
    input logic rst_n,
    output logic tick
);
```

19.3 Parameters

19.3.1 MAX

- **Type:** int
- **Default:** 32767
- **Description:** Maximum count value before wrapping to zero
- **Range:** Any positive integer
- **Width Calculation:** Counter width is automatically calculated as $\lceil \log_2(\text{MAX}+1) \rceil$

19.4 Ports

19.4.1 Inputs

Port	Width	Type	Description
clk	1	logic	System clock input
rst_n	1	logic	Active-low asynchronous reset

19.4.2 Outputs

Port	Width	Type	Description
tick	1	logic	Pulse output when counter reaches MAX

19.5 Internal Architecture

19.5.1 Counter Register

logic [**\$clog2**(MAX+1)-1:0] r_count;

- **Width:** Automatically sized to accommodate MAX value
- **Reset Value:** '0 (zero)
- **Operation:** Increments every clock cycle until reaching MAX

19.5.2 State Machine Behavior

The counter operates as a simple state machine:

1. **Reset State:** r_count = 0, tick = 0
2. **Counting State:** r_count increments each cycle
3. **Maximum State:** r_count = MAX, tick = 1 for one cycle
4. **Wrap State:** r_count returns to 0, cycle repeats

19.6 Implementation Details

19.6.1 Main Counter Logic

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_count <= '0;
    end else begin
        if (r_count == MAX[$clog2(MAX+1)-1:0]) begin
            r_count <= '0;
        end else begin
            r_count <= r_count + 1'b1;
        end
    end
end
```

19.6.2 Tick Generation

```
assign tick = (r_count == MAX[$clog2(MAX+1)-1:0]);
```

19.6.3 Key Features

- **Automatic Width Sizing:** Counter width adapts to MAX parameter
- **Wrap-around Operation:** Automatically returns to zero after MAX

- **Single Cycle Tick:** Tick pulse lasts exactly one clock cycle
- **Asynchronous Reset:** Immediate reset capability

19.7 Timing Characteristics

19.7.1 Period Calculation

- **Total Period:** $\text{MAX} + 1$ clock cycles
- **Tick Frequency:** $f_{\text{clk}} / (\text{MAX} + 1)$
- **Duty Cycle:** $1 / (\text{MAX} + 1)$

19.7.2 Examples

MAX Value	Counter Width	Period (cycles)	Tick Frequency
7	3 bits	8	$f_{\text{clk}}/8$
15	4 bits	16	$f_{\text{clk}}/16$
99	7 bits	100	$f_{\text{clk}}/100$
999	10 bits	1000	$f_{\text{clk}}/1000$

19.8 Usage Examples

19.8.11. Basic Timer (1ms tick at 100MHz)

```
counter #(
    .MAX(99999) // 100,000 cycles = 1ms at 100MHz
) timer_1ms (
    .clk(clk_100mhz),
    .rst_n(rst_n),
    .tick(tick_1ms)
);
```

19.8.22. Heartbeat Generator (1Hz at 50MHz)

```
counter #(
    .MAX(49999999) // 50M cycles = 1 second at 50MHz
) heartbeat (
    .clk(clk_50mhz),
    .rst_n(rst_n),
    .tick(heartbeat_1hz)
);
```

19.8.33. Simple Divider (Divide by 8)

```
counter #(
    .MAX(7) // 8 cycles total
) div8 (
    .clk(clk_in),
    .rst_n(rst_n),
    .tick(clk_div8)
);
```

19.9 Design Considerations

19.9.1 Resource Usage

- **LUTs:** Approximately $\lceil \log_2(\text{MAX}+1) \rceil$ LUTs for the counter
- **FFs:** $\lceil \log_2(\text{MAX}+1) \rceil$ flip-flops for the counter register
- **Logic Levels:** Single level for tick generation

19.9.2 Performance

- **Maximum Frequency:** Limited by counter increment logic
- **Reset Recovery:** One clock cycle after reset deassertion
- **Propagation Delay:** Tick output has minimal combinational delay

19.9.3 Power Optimization

- **Clock Gating:** Consider gating when tick output not needed
- **Reset Strategy:** Asynchronous reset reduces reset tree loading
- **Parameter Selection:** Smaller MAX values reduce power consumption

19.10 Synthesis Considerations

19.10.1 Parameter Constraints

```
initial begin
    assert (MAX > 0) else $error("MAX must be positive");
    assert (MAX < 2**31) else $warning("Large MAX may impact
synthesis");
end
```

19.10.2 Optimization Hints

- **Binary MAX Values:** Powers of 2 minus 1 optimize better (7, 15, 31, etc.)

- **Tool Directives:** Consider synthesis attributes for timing-critical paths
- **Reset Inference:** Ensure reset is properly inferred as asynchronous

19.11 Common Applications

1. **Periodic Timers:** Generating regular time intervals
2. **Clock Division:** Creating slower clock enables
3. **Timeout Generation:** Watchdog timers and timeouts
4. **Sampling Control:** Periodic data sampling triggers
5. **LED Blinking:** Visual indicators and status displays
6. **Protocol Timing:** Communication protocol timeouts

19.12 Verification Considerations

19.12.1 Test Scenarios

1. **Basic Counting:** Verify counting from 0 to MAX
2. **Wrap Behavior:** Confirm return to zero after MAX
3. **Tick Generation:** Verify single-cycle tick pulse
4. **Reset Functionality:** Test asynchronous reset operation
5. **Parameter Sweep:** Test various MAX values

19.12.2 Coverage Points

- Counter reaches MAX value
- Counter wraps from MAX to 0
- Tick pulse generation timing
- Reset during counting operation

19.13 Limitations

1. **No Enable Control:** Always counting when not in reset
2. **Up-Counter Only:** Cannot count down or bidirectionally

3. **Fixed Increment:** Always increments by 1
4. **No Load Capability:** Cannot load arbitrary values
5. **Single Output:** Only provides tick signal, not count value

19.14 Related Modules

- `counter_load_clear`: Adds load and clear functionality
- `counter_bin`: Specialized for FIFO applications
- `counter_freq_invariant`: Frequency-independent timing
- `clock_pulse`: Similar functionality with different interface

19.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

20 Ring Counter Module

20.1 Overview

The `counter_ring` module implements a classic ring counter where a single ‘1’ bit circulates through a shift register. This creates a one-hot encoded sequence that’s ideal for generating sequential enable signals, creating walking LED patterns, or controlling multi-phase operations where only one stage should be active at a time.

20.2 Module Declaration

```
module counter_ring #(
    parameter int WIDTH = 4
) (
    input wire      clk,
    input wire      rst_n,
    input wire      enable,
    output reg [WIDTH-1:0] ring_out
);
```

20.3 Parameters

20.3.1 WIDTH

- **Type:** int
- **Default:** 4
- **Description:** Number of stages in the ring counter
- **Range:** Any positive integer ≥ 1
- **States Generated:** WIDTH unique states
- **Pattern:** Single '1' bit rotates through WIDTH positions

20.4 Ports

20.4.1 Inputs

Port	Width	Type	Description
clk	1	wire	System clock input
rst_n	1	wire	Active-low asynchronous reset
enable	1	wire	Counter enable control

20.4.2 Outputs

Port	Width	Type	Description
ring_out	WIDTH	reg	Ring counter output (one-hot encoded)

20.5 Theory of Operation

20.5.1 Ring Counter Principle

A ring counter is a circular shift register where: - **Initialization:** Single '1' bit in LSB position - **Operation:** The '1' bit rotates right each clock cycle - **Feedback:** MSB connects back to LSB (no inversion) - **States:** WIDTH unique one-hot states

20.5.2 Mathematical Representation

For each clock cycle when enabled:

```
ring_out[i] = ring_out[i-1] for i = 1 to WIDTH-1  
ring_out[0] = ring_out[WIDTH-1]
```

20.6 Implementation Details

20.6.1 Core Logic

```
always_ff @(posedge clk or negedge rst_n) begin  
    if (!rst_n) begin  
        ring_out <= 'b1; // Initialize with LSB set  
    end else if (enable) begin  
        // Right rotate operation  
        ring_out <= {ring_out[0], ring_out[WIDTH-1:1]};  
    end  
end
```

20.6.2 Operation Breakdown

1. **Reset State:** ring_out = 'b1 (only LSB set)
2. **Rotation:** {ring_out[0], ring_out[WIDTH-1:1]}
 - MSB becomes new LSB
 - All other bits shift right
3. **Enable Control:** Only advances when enable is high
4. **Circular:** After WIDTH cycles, returns to initial state

20.7 State Sequences

20.7.1 4-bit Ring Counter (WIDTH=4)

Step	ring_out	Binary	One-Hot Position
0	0001	Reset state	Position 0 active
1	1000	After 1 clock	Position 3 active
2	0100	After 2 clocks	Position 2 active
3	0010	After 3 clocks	Position 1 active

Step	ring_out	Binary	One-Hot Position
			active
4	0001	After 4 clocks	Back to start

20.7.23-bit Ring Counter (WIDTH=3)

Step	ring_out	Active Stage
0	001	Stage 0
1	100	Stage 2
2	010	Stage 1
3	001	Stage 0 (repeat)

20.7.38-bit Ring Counter Example

Step 0: 00000001 (Stage 0)
 Step 1: 10000000 (Stage 7)
 Step 2: 01000000 (Stage 6)
 Step 3: 00100000 (Stage 5)
 Step 4: 00010000 (Stage 4)
 Step 5: 00001000 (Stage 3)
 Step 6: 00000100 (Stage 2)
 Step 7: 00000010 (Stage 1)
 Step 8: 00000001 (Stage 0) - Cycle repeats

20.8 Design Examples

20.8.11. Sequential Enable Generator

```

counter_ring #(
    .WIDTH(4)
) seq_enable (
    .clk(clk),
    .rst_n(rst_n),
    .enable(sequence_active),
    .ring_out(stage_enables)
);

// Use each bit as an enable for different stages
assign stage0_enable = stage_enables[0];
assign stage1_enable = stage_enables[1];
assign stage2_enable = stage_enables[2];
assign stage3_enable = stage_enables[3];

// Example: Pipeline stage control
always_ff @(posedge clk) begin

```

```

    if (stage0_enable) stage0_data <= input_data;
    if (stage1_enable) stage1_data <= process_stage0(stage0_data);
    if (stage2_enable) stage2_data <= process_stage1(stage1_data);
    if (stage3_enable) output_data <= process_stage2(stage2_data);
end

```

20.8.22. LED Chaser Display

```

// Create walking LED pattern
counter_ring #(
    .WIDTH(8)
) led_chaser (
    .clk(slow_clk),           // ~2Hz for visible effect
    .rst_n(rst_n),
    .enable(chase_enabled),
    .ring_out(led_pattern)
);

// Connect directly to LEDs
assign leds[7:0] = led_pattern;

// Optional: Bidirectional chaser
logic direction;
logic [7:0] led_display;

always_comb begin
    if (direction)
        led_display = led_pattern;
    else
        led_display = {led_pattern[0], led_pattern[7:1]}; // Reverse
end

```

20.8.33. Memory Bank Sequencer

```

counter_ring #(
    .WIDTH(4)
) bank_sequencer (
    .clk(clk),
    .rst_n(rst_n),
    .enable(memory_active),
    .ring_out(bank_select)
);

// Memory bank control
logic [3:0] bank_cs_n;
assign bank_cs_n = ~bank_select; // Active low chip selects

// Example: Round-robin memory access

```

```

always_comb begin
    mem_addr = base_addr;
    mem_data_out = 32'h0;

    case (bank_select)
        4'b0001: begin
            mem_addr = bank0_addr;
            mem_data_out = bank0_data;
        end
        4'b0010: begin
            mem_addr = bank1_addr;
            mem_data_out = bank1_data;
        end
        4'b0100: begin
            mem_addr = bank2_addr;
            mem_data_out = bank2_data;
        end
        4'b1000: begin
            mem_addr = bank3_addr;
            mem_data_out = bank3_data;
        end
    endcase
end

```

20.8.4.4. State Machine Controller

// Simple state machine using ring counter

```

counter_ring #(
    .WIDTH(5)
) state_machine (
    .clk(clk),
    .rst_n(rst_n),
    .enable(state_advance),
    .ring_out(current_state)
);

```

// Decode states

```

assign state_idle    = current_state[0];
assign state_request = current_state[1];
assign state_wait    = current_state[2];
assign state_active  = current_state[3];
assign state_done    = current_state[4];

```

// State machine logic

```

always_ff @(posedge clk) begin
    case (1'b1) // One-hot case statement
        state_idle: state_advance <= start_request;
        state_request: state_advance <= request_granted;
    endcase
end

```

```

        state_wait:    state_advance <= ready_signal;
        state_active:  state_advance <= operation_complete;
        state_done:    state_advance <= 1'b1;  // Always advance from
done
        default:       state_advance <= 1'b0;
    endcase
end

```

20.8.55. Clock Domain Multiplexer

```

// Sequentially select different clock domains
counter_ring #(
    .WIDTH(3)
) clk_mux_ctrl (
    .clk(control_clk),
    .rst_n(rst_n),
    .enable(mux_advance),
    .ring_out(clk_select)
);

// Clock multiplexer (use proper clock muxing in real design)
logic selected_clk;
always_comb begin
    case (clk_select)
        3'b001: selected_clk = clk_domain_0;
        3'b010: selected_clk = clk_domain_1;
        3'b100: selected_clk = clk_domain_2;
        default: selected_clk = 1'b0;
    endcase
end

```

20.9 Properties and Characteristics

20.9.1 One-Hot Encoding

Ring counters naturally provide one-hot encoding: - **Exactly One Bit Set**: Always exactly one bit is '1' - **Unique States**: Each state is distinctly different - **Easy Decode**: No additional decoding logic needed - **Glitch-Free**: Clean transitions between states

20.9.2 Self-Correcting Behavior

Ring counters have some self-correcting properties: - **All Zeros**: Will remain stuck (not self-correcting) - **Multiple Ones**: Will maintain multiple bits indefinitely - **Single One**: Will operate correctly

20.9.3 Comparison with Other Counters

Counter Type	States	Decode Logic	One-Hot	Power
Binary	2^N	Complex	No	Low

Counter Type	States	Decode Logic	One-Hot	Power
Gray	2^N	Moderate	No	Low
Johnson	$2 \times N$	Moderate	No	Low
Ring	N	None	Yes	Medium

20.10 Advanced Usage

20.10.1 1. Self-Correcting Ring Counter

```
// Add error correction for stuck states
logic error_detected;
logic [WIDTH-1:0] corrected_ring;

assign error_detected = (ring_out == 'b0) || ($countones(ring_out) !=
1);

always_comb begin
    if (error_detected)
        corrected_ring = 'b1; // Force to valid state
    else
        corrected_ring = ring_out;
end
```

20.10.2 2. Bidirectional Ring Counter

```
logic direction; // 0 = right, 1 = left

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        ring_out <= 'b1;
    end else if (enable) begin
        if (direction)
            // Left rotate
            ring_out <= {ring_out[WIDTH-2:0], ring_out[WIDTH-1]};
        else
            // Right rotate
            ring_out <= {ring_out[0], ring_out[WIDTH-1:1]};
        end
    end
```

20.10.3 3. Programmable Ring Counter

```
logic [WIDTH-1:0] init_pattern;
logic load_pattern;

always_ff @(posedge clk or negedge rst_n) begin
```

```

    if (!rst_n) begin
        ring_out <= 'b1;
    end else if (load_pattern) begin
        ring_out <= init_pattern;
    end else if (enable) begin
        ring_out <= {ring_out[0], ring_out[WIDTH-1:1]};
    end
end
end

```

20.11 Verification Strategy

20.11.1 Test Scenarios

1. **Reset Behavior:** Verify initialization to single '1' bit
2. **Rotation Sequence:** Check complete WIDTH-step cycle
3. **Enable Control:** Test hold behavior when disabled
4. **One-Hot Property:** Ensure exactly one bit set always
5. **Cycle Completion:** Verify return to initial state

20.11.2 Coverage Model

```

covergroup ring_cg @(posedge clk);
    cp_ring_state: coverpoint ring_out {
        bins valid_states[] = {1, 2, 4, 8, 16, 32, 64, 128}; // For
WIDTH=8
        bins invalid_states[] = default;
    }

    cp_enable: coverpoint enable {
        bins enabled = {1};
        bins disabled = {0};
    }

    cp_one_hot: coverpoint $countones(ring_out) {
        bins exactly_one = {1};
        bins zero = {0};
        bins multiple = {[2:WIDTH]};
    }

    // Verify complete cycle
    cp_transitions: coverpoint ring_out {
        bins cycle[] = (1 ==> 8), (8 ==> 4), (4 ==> 2), (2 ==> 1); // For
WIDTH=4
    }

```

```
}  
endgroup
```

20.11.3 Assertions

```
// Exactly one bit should be set (except during reset)  
property one_hot_property;  
    @(posedge clk) disable iff (!rst_n)  
        $countones(ring_out) == 1;  
endproperty  
  
// After reset, should start with LSB set  
property reset_state;  
    @(posedge clk)  
        $rose(rst_n) |-> ring_out == 1;  
endproperty  
  
// When enabled, should rotate right  
property rotation_property;  
    @(posedge clk) disable iff (!rst_n)  
        enable |=> ring_out == {$past(ring_out[0]), $past(ring_out[WIDTH-  
1:1])});  
endproperty  
  
assert property (one_hot_property);  
assert property (reset_state);  
assert property (rotation_property);
```

20.12 Synthesis Considerations

20.12.1 Resource Utilization

- **Flip-Flops:** WIDTH registers
- **LUTs:** Minimal - just shift logic
- **Routing:** Simple interconnections

20.12.2 Optimization Tips

```
// Use SRL (Shift Register LUT) inference when appropriate  
(* SRL_STYLE = "register" *) reg [WIDTH-1:0] ring_out;  
  
// For very wide counters, consider using dedicated shift register  
primitives
```

20.12.3 Timing Considerations

- **Clock Skew:** Ensure clean clock distribution
- **Setup/Hold:** Standard flip-flop requirements
- **Routing Delay:** Minimal due to simple structure

20.13 Performance Characteristics

20.13.1 Maximum Frequency

- **Typical:** 400-600 MHz in modern FPGAs
- **Limitation:** Shift register timing
- **Advantage:** Very simple logic path

20.13.2 Power Consumption

- **Dynamic:** Proportional to switching frequency
- **Optimization:** Higher switching than binary counters
- **Trade-off:** Simplicity vs. power efficiency

20.14 Common Pitfalls

20.14.1 1. Invalid Initial States

```
// Problem: Multiple bits set  
ring_out <= 4'b1001; // Invalid - two bits set
```

```
// Solution: Use proper initialization  
ring_out <= 4'b0001; // Valid - exactly one bit
```

20.14.2 2. Stuck at Zero

```
// Problem: All zeros state  
if (ring_out == 'b0) begin  
    ring_out <= 'b1; // Force recovery  
end
```

20.14.3 3. Width Mismatch

```
// Problem: Initialization doesn't match width  
parameter WIDTH = 8;  
ring_out <= 4'b0001; // Wrong - should be 8'b00000001
```



```
// Solution: Use proper width
ring_out <= 'b1;           // Correct - sized automatically
```

20.15 Related Modules

- counter_johnson: Twisted-ring counter with $2 \times \text{WIDTH}$ states
- counter_bingray: Binary/Gray counter for different applications
- Standard shift register implementations
- One-hot state machines

20.16 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

21 Count Leading Zeros Module

21.1 Overview

The `count_leading_zeros` module implements a leading zero counter that determines how many consecutive zero bits appear at the beginning (MSB side) of a data word. This is a fundamental operation in computer arithmetic, used extensively in floating-point normalization, priority encoding, and bit manipulation algorithms.

21.2 Module Declaration

```
module count_leading_zeros #(
    parameter int WIDTH          = 32,
    parameter      INSTANCE_NAME = "CLZ"
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] clz
);
```

21.3 Parameters

21.3.1 WIDTH

- **Type:** int
- **Default:** 32

- **Description:** Bit width of the input data
- **Range:** Any positive integer ≥ 1
- **Common Values:** 8, 16, 32, 64 for standard data widths
- **Impact:** Determines output width and algorithm complexity

21.3.2 INSTANCE_NAME

- **Type:** string (parameter without explicit storage type)
- **Default:** "CLZ"
- **Description:** Instance identifier for debugging and display
- **Usage:** Appears in debug messages and simulation output

21.4 Ports

21.4.1 Inputs

Port	Width	Type	Description
data	WIDTH	logic	Input data word to analyze

21.4.2 Outputs

Port	Width	Type	Description
clz	$\$clog2(WIDTH)+1$	logic	Count of leading zeros

21.4.3 Output Width Explanation

The output width $\$clog2(WIDTH)+1$ ensures it can represent all possible counts: - **Range:** 0 to WIDTH (inclusive) - **Example:** For WIDTH=32, output is 6 bits (0-32 requires 6 bits) - **Special Case:** All zeros input produces CLZ = WIDTH

21.5 Algorithm Implementation

21.5.1 Function-Based Approach

```
function automatic [ $clog2(WIDTH):0 ] clz_func;
    input [WIDTH-1:0] input_data;
    logic found;
```

```

begin
    clz_func = 0;
    found = 1'b0;
    for (int i = 0; i < WIDTH; i++) begin
        if (!input_data[i] && !found) begin
            clz_func += 1;
        end else begin
            found = 1'b1; // Stop counting when first '1' found
        end
    end
end
endfunction

```

21.5.2 Bit Scanning Process

The algorithm scans from LSB to MSB: 1. **Initialize:** `clz_func = 0`, `found = 0` 2. **Scan Loop:** For each bit position `i` from 0 to `WIDTH-1`: - If `data[i] == 0` AND no '1' found yet: increment count - If `data[i] == 1`: set found flag, stop counting 3. **Result:** Final count represents leading zeros

21.5.3 Why LSB-to-MSB Scanning?

The implementation scans from LSB (bit 0) to MSB (bit `WIDTH-1`): - **Leading Zeros:** Count zeros from MSB side until first '1' - **Bit Order:** `data[0]` is LSB, `data[WIDTH-1]` is MSB - **Count Logic:** Zeros from MSB down = `WIDTH - position_of_first_one`

21.6 Examples and Truth Tables

21.6.1 8-bit Examples (WIDTH=8)

Input (data)	Binary	First '1' at	Leading Zeros	CLZ Output
8'b00000000	00000000	None	8	8
0				
8'b00000001	00000001	Bit 0	7	7
1				
8'b00000010	00000010	Bit 1	6	6
0				
8'b00000100	00000100	Bit 2	5	5
0				
8'b00001000	00001000	Bit 3	4	4
0				
8'b00010000	00010000	Bit 4	3	3
0				

Input (data)	Binary	First '1' at	Leading Zeros	CLZ Output
8'b0010000 0	00100000	Bit 5	2	2
8'b0100000 0	01000000	Bit 6	1	1
8'b1000000 0	10000000	Bit 7	0	0
8'b1111111 1	11111111	Bit 0	0	0
8'b1010101 0	10101010	Bit 1	0	0

21.6.2 32-bit Examples

Input	Hex	Leading Zeros	CLZ
32'h00000000	0x00000000	32	32
32'h00000001	0x00000001	31	31
32'h00000080	0x00000080	24	24
32'h00008000	0x00008000	16	16
32'h00800000	0x00800000	8	8
32'h80000000	0x80000000	0	0
32'hFFFFFFFF	0xFFFFFFFF	0	0

21.7 Applications

21.7.1 1. Floating-Point Normalization

```
// IEEE 754 floating-point normalization
logic [31:0] mantissa_raw;
logic [5:0] shift_amount;
logic [31:0] normalized_mantissa;

count_leading_zeros #(.WIDTH(32)) norm_clz (
    .data(mantissa_raw),
    .clz(shift_amount)
);

// Normalize mantissa by shifting left
```

```
assign normalized_mantissa = mantissa_raw << shift_amount;
```

```
// Adjust exponent accordingly
```

```
logic [7:0] adjusted_exponent;
```

```
assign adjusted_exponent = raw_exponent - shift_amount;
```

21.7.22. Priority Encoder

```
// Find highest priority request
```

```
logic [15:0] request_vector;
```

```
logic [4:0] highest_priority;
```

```
logic any_request;
```

```
count_leading_zeros #(.WIDTH(16)) priority_enc (  
    .data(request_vector),  
    .clz(leading_zeros)  
);
```

```
assign any_request = (leading_zeros != 16);
```

```
assign highest_priority = any_request ? (15 - leading_zeros) : 5'b0;
```

```
// Example: request_vector = 16'b0000_0100_1000_0000
```

```
// CLZ = 10 (ten leading zeros)
```

```
// highest_priority = 15 - 10 = 5 (bit 5 is highest set)
```

21.7.33. Bit Width Calculation

```
// Determine minimum bits needed to represent a number
```

```
logic [31:0] number;
```

```
logic [5:0] min_bits_needed;
```

```
count_leading_zeros #(.WIDTH(32)) bit_width_calc (  
    .data(number),  
    .clz(leading_zeros)  
);
```

```
assign min_bits_needed = (number == 0) ? 1 : (32 - leading_zeros);
```

```
// Examples:
```

```
// number = 0      → min_bits = 1
```

```
// number = 1      → min_bits = 1
```

```
// number = 255    → min_bits = 8
```

```
// number = 256    → min_bits = 9
```

```
// number = 65535  → min_bits = 16
```

21.7.44. Log2 Calculation

```
// Calculate floor(log2(x)) for x > 0
logic [31:0] input_value;
logic [4:0] log2_result;
logic valid;

count_leading_zeros #(.WIDTH(32)) log2_calc (
    .data(input_value),
    .clz(leading_zeros)
);

assign valid = (input_value != 0);
assign log2_result = valid ? (31 - leading_zeros) : 5'b0;

// Examples:
// input = 1    → log2 = 0
// input = 2    → log2 = 1
// input = 4    → log2 = 2
// input = 7    → log2 = 2 (floor)
// input = 8    → log2 = 3
```

21.7.55. Data Compression - Run Length Encoding

```
// Count leading zeros for compression
logic [63:0] data_block;
logic [6:0] zero_run_length;

count_leading_zeros #(.WIDTH(64)) rle_counter (
    .data(data_block),
    .clz(zero_run_length)
);

// Use in compression algorithm
if (zero_run_length > THRESHOLD) begin
    // Encode as zero run
    compressed_data = {RLE_CODE, zero_run_length};
end else begin
    // Encode as literal data
    compressed_data = {LITERAL_CODE, data_block};
end
```

21.8 Advanced Implementations

21.8.11. Pipeline Version for High Speed

```
module count_leading_zeros_pipelined #(
    parameter int WIDTH = 32,
```

```

    parameter int STAGES = 2
) (
    input  logic                clk,
    input  logic                rst_n,
    input  logic [WIDTH-1:0]    data,
    output logic [$clog2(WIDTH):0] clz
);

// Stage 1: Partial counting
logic [WIDTH-1:0] data_s1;
logic [$clog2(WIDTH):0] partial_clz_s1;

always_ff @(posedge clk) begin
    data_s1 <= data;
    // Count leading zeros in upper half
    partial_clz_s1 <= clz_upper_half(data);
end

// Stage 2: Complete counting
always_ff @(posedge clk) begin
    if (partial_clz_s1 == WIDTH/2) begin
        // All upper bits are zero, count in lower half
        clz <= WIDTH/2 + clz_lower_half(data_s1);
    end else begin
        // Found '1' in upper half
        clz <= partial_clz_s1;
    end
end

endmodule

```

21.8.22. Hierarchical Implementation

```

// Divide and conquer approach for large widths
module count_leading_zeros_hierarchical #(
    parameter int WIDTH = 64
) (
    input  logic [WIDTH-1:0]    data,
    output logic [$clog2(WIDTH):0] clz
);

localparam int HALF_WIDTH = WIDTH/2;

logic [$clog2(HALF_WIDTH):0] clz_upper, clz_lower;
logic upper_all_zeros;

// Count in upper half

```

```

count_leading_zeros #(.WIDTH(HALF_WIDTH)) upper_clz (
    .data(data[WIDTH-1:HALF_WIDTH]),
    .clz(clz_upper)
);

// Count in lower half
count_leading_zeros #(.WIDTH(HALF_WIDTH)) lower_clz (
    .data(data[HALF_WIDTH-1:0]),
    .clz(clz_lower)
);

assign upper_all_zeros = (clz_upper == HALF_WIDTH);

// Combine results
assign clz = upper_all_zeros ? (HALF_WIDTH + clz_lower) : clz_upper;

endmodule

```

21.8.33. LUT-Based Implementation (Small Widths)

```

// Optimized for small widths using case statement
module count_leading_zeros_lut #(
    parameter int WIDTH = 8
) (
    input logic [WIDTH-1:0] data,
    output logic [$clog2(WIDTH):0] clz
);

always_comb begin
    case (data)
        8'b00000000: clz = 8;
        8'b00000001: clz = 7;
        8'b0000001?: clz = 6;
        8'b000001?: clz = 5;
        8'b00001?: clz = 4;
        8'b0001?: clz = 3;
        8'b001?: clz = 2;
        8'b01?: clz = 1;
        8'b1?: clz = 0;
        default: clz = 0;
    endcase
end

endmodule

```


21.9 Performance Analysis

21.9.1 Resource Utilization

WIDTH	LUTs (Typical)	Delay Levels	Max Frequency
8	15-20	3-4	500+ MHz
16	30-40	4-5	400+ MHz
32	60-80	5-6	300+ MHz
64	120-150	6-7	250+ MHz

21.9.2 Timing Characteristics

- **Combinational Delay:** $O(\log(\text{WIDTH}))$ for tree implementations
- **Critical Path:** Through the priority encoding logic
- **Scalability:** Linear increase in logic for function implementation

21.10 Verification Strategy

21.10.1 Test Scenarios

1. **Boundary Cases:** All zeros, all ones, single bit patterns
2. **Random Patterns:** Comprehensive random testing
3. **Systematic Sweep:** Test all possible leading zero counts
4. **Corner Cases:** Maximum width values, alternating patterns

21.10.2 Coverage Model

```
covergroup clz_cg;
  cp_leading_zeros: coverpoint clz {
    bins zero_clz = {0};
    bins low_clz = {[1:WIDTH/4]};
    bins mid_clz = {[WIDTH/4+1:3*WIDTH/4]};
    bins high_clz = {[3*WIDTH/4+1:WIDTH-1]};
    bins all_zeros = {WIDTH};
  }

  cp_data_patterns: coverpoint data {
    bins all_zeros = {'0'};
    bins all_ones = {'1'};
    bins single_bit[] = {1, 2, 4, 8, 16, 32}; // For appropriate
WIDTH
```

```

        bins msb_set = {1 << (WIDTH-1)};
        bins mixed[] = {[1:2**(WIDTH-1)-1]};
    }
endgroup

```

21.10.3 Assertions

// CLZ should never exceed WIDTH

```

property clz_bounds;
    clz <= WIDTH;
endproperty

```

// All zeros should give CLZ = WIDTH

```

property all_zeros_case;
    (data == '0) |-> (clz == WIDTH);
endproperty

```

// MSB set should give CLZ = 0

```

property msb_set_case;
    data[WIDTH-1] |-> (clz == 0);
endproperty

```

// Relationship between CLZ and first '1' position

```

property clz_correctness;
    logic [WIDTH-1:0] shifted;
    assign shifted = data << clz;
    (data != '0) |-> shifted[WIDTH-1];
endproperty

```

```

assert property (clz_bounds);
assert property (all_zeros_case);
assert property (msb_set_case);
assert property (clz_correctness);

```

21.11 Synthesis Optimization

21.11.1 Tool-Specific Attributes

// Xilinx: Prevent SRL inference for better timing

```

(* SRL_STYLE = "register" *) logic [WIDTH-1:0] data_reg;

```

// Intel/Altera: Use specific LUT implementation

// synthesis attribute LUT_SIZE of clz_func is 6;

21.11.2 Area vs. Speed Trade-offs

// For area optimization: Use iterative approach
// For speed optimization: Use tree-based or LUT approach
// For power optimization: Add enable signals and clock gating

21.12 Common Use Cases Summary

1. **CPU/DSP Cores:** Instruction implementation (CLZ instruction)
2. **Floating-Point Units:** Mantissa normalization
3. **Network Processors:** Longest prefix matching
4. **Compression Engines:** Run-length encoding optimization
5. **Memory Controllers:** Priority arbitration
6. **Graphics Processors:** Texture coordinate processing
7. **Cryptographic Units:** Bit manipulation operations

21.13 Related Modules and Functions

- Count trailing zeros (CTZ)
- Population count (number of ‘1’ bits)
- Find first set (FFS) / Find last set (FLS)
- Priority encoders
- Barrel shifters

21.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

22 dataint_checksum Module Documentation

22.1 Purpose

The `dataint_checksum` module implements a simple accumulating checksum calculator. It continuously adds incoming data values to maintain a running checksum that can be used for basic data integrity verification.

22.2 Module Declaration

```
module dataint_checksum #(
    parameter int WIDTH = 8
) (
    input logic          clk,
    rst_n,
    input logic          reset,
    input logic          valid,
    input logic [WIDTH-1:0] data,
    output logic [WIDTH-1:0] chksum
);
```

22.3 Parameters

Parameter	Default	Description
WIDTH	8	Width of the data and checksum in bits

22.4 Ports

Port	Direction	Width	Description
clk	Input	1	Clock signal
rst_n	Input	1	Active-low asynchronous reset
reset	Input	1	Synchronous reset signal
valid	Input	1	Data valid signal - when high, data is accumulated
data	Input	WIDTH	Input data to be added to checksum
chksum	Output	WIDTH	Current checksum value

22.5 Functionality

22.5.1 Operation

The module maintains an internal counter (`r_count`) that accumulates incoming data: - On reset (either asynchronous `rst_n` or synchronous reset), the counter is cleared to zero - When `valid` is asserted, the input data is added to the current counter value - The checksum output is simply the current counter value

22.5.2 Reset Behavior

The module supports dual reset mechanisms: 1. **Asynchronous Reset:** `rst_n` (active-low) immediately clears the counter 2. **Synchronous Reset:** `reset` (active-high) clears the counter on the next clock edge

22.6 Implementation Details

22.6.1 Key Features

- **Simple Accumulation:** Uses basic addition for checksum calculation
- **Parameterizable Width:** Supports any data width through the `WIDTH` parameter
- **Dual Reset Support:** Both asynchronous and synchronous reset capabilities
- **Enable Control:** Data accumulation only occurs when `valid` is asserted

22.6.2 Special Considerations

- **Overflow Handling:** The checksum will wrap around when the accumulated value exceeds the maximum value representable in `WIDTH` bits
- **Continuous Output:** The checksum output is always valid and reflects the current accumulated value
- **Reset Priority:** Asynchronous reset (`rst_n`) takes precedence over synchronous reset (`reset`)

22.7 State Machine

This module does not implement a formal FSM but operates as a simple accumulator with the following states: - **RESET:** Counter is zero - **ACCUMULATING:** Counter adds valid data inputs

22.8 Usage Example

```
// 16-bit checksum calculator
dataint_checksum #(
    .WIDTH(16)
) checksum_calc (
    .clk(clk),
    .rst_n(rst_n),
    .reset(soft_reset),
    .valid(data_valid),
    .data(input_data),
    .chksum(calculated_checksum)
);
```

22.9 Applications

- Simple data integrity checking
- Packet checksum calculation
- Basic error detection in data streams
- Accumulation of data values for verification purposes

22.10 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

23 dataint_crc

A generic Cyclic Redundancy Check (CRC) computation engine that provides flexible, high-performance CRC calculation with configurable polynomials, data widths, and processing options.

23.1 Overview

The `dataint_crc` module implements a comprehensive CRC calculation engine suitable for data integrity applications, communication protocols, and storage systems. It supports configurable polynomial widths, data widths, input/output reflection, and cascade processing for high-throughput applications.

23.2 Module Declaration

```
module dataint_crc #(
    parameter string ALGO_NAME = "DEADF1F0", // Algorithm identifier
    parameter int DATA_WIDTH = 64,         // Input data width

```

```

    parameter int CRC_WIDTH = 64,           // CRC polynomial width
    parameter int REF_IN = 1,              // Reflect input data
    parameter int REF_OUT = 1              // Reflect output data
) (
    // Derived localparam (computed internally - not user-settable):
    // localparam int CHUNKS = DATA_WIDTH / 8; // Number of byte
chunks
    // localparam int CW = CRC_WIDTH;        // CRC width alias
    // localparam int DW = DATA_WIDTH;      // Data width alias
    // localparam int CH = CHUNKS;           // Chunks alias
    input logic [CRC_WIDTH-1:0] POLY,        // CRC polynomial
    input logic [CRC_WIDTH-1:0] POLY_INIT,   // Initial CRC
value
    input logic [CRC_WIDTH-1:0] XOROUT,      // Output XOR mask
    input logic clk,                        // Clock
    input logic rst_n,                      // Active-low
reset
    input logic load_crc_start,             // Load initial
CRC
    input logic load_from_cascade,          // Load from
cascade
    input logic [CHUNKS-1:0] cascade_sel,    // Cascade
selection (one-hot)
    input logic [DATA_WIDTH-1:0] data,       // Input data
    output logic [CRC_WIDTH-1:0] crc         // CRC output
);

```

23.3 Parameters

23.3.1 User-Settable Parameters

Parameter	Type	Default	Description
ALGO_NAME	string	"DEADF1F0"	Algorithm identifier for documentation
DATA_WIDTH	int	64	Input data width in bits (must be multiple of 8)
CRC_WIDTH	int	64	CRC polynomial width in bits (8, 16, 32, or 64)
REFIN	int	1	Reflect input data bytes (1=reflect, 0=direct)
REFOUT	int	1	Reflect output CRC (1=reflect, 0=direct)

23.3.2 Derived Localparam (Computed Internally)

Localparam	Computation	Description
CHUNKS (CH)	DATA_WIDTH/8	Number of 8-bit processing chunks per cycle
CW	CRC_WIDTH	Convenience alias for CRC width
DW	DATA_WIDTH	Convenience alias for data width

Note: CHUNKS, CW, DW, and CH are computed internally and cannot be overridden by users.

23.4 Ports

23.4.1 Inputs

Port	Width	Description
POLY	CRC_WIDTH	CRC polynomial coefficient
POLY_INIT	CRC_WIDTH	Initial CRC value (seed)
XOROUT	CRC_WIDTH	Final XOR mask for output
clk	1	System clock
rst_n	1	Active-low asynchronous reset
load_crc_start	1	Load initial CRC value
load_from_cascade	1	Load CRC from cascade output
cascade_sel	CHUNKS	One-hot cascade stage selection
data	DATA_WIDTH	Input data for CRC calculation

23.4.2 Outputs

Port	Width	Description
crc	CRC_WIDTH	Computed CRC value (registered)

23.5 Functionality

23.5.1 CRC Algorithm Implementation

The module implements a parallel CRC calculation using the following stages:

1. **Input Data Conditioning:** Optional bit reflection per REFIN parameter
2. **Cascade Processing:** Parallel processing through multiple XOR-shift stages
3. **Output Conditioning:** Optional bit reflection per REFOUT parameter
4. **Final XOR:** Application of XOROUT mask to final result

23.5.2 Key Features

- **Parallel Processing:** Processes multiple bytes simultaneously for high throughput
- **Configurable Reflection:** Support for MSB-first and LSB-first bit ordering
- **Cascade Architecture:** Enables partial CRC computation and continuation
- **Standards Compliance:** Configurable for standard CRC algorithms (CRC-32, CRC-64, etc.)

23.6 Implementation Details

23.6.1 Input Data Reflection

```
// Input reflection based on REFIN parameter
if (REFIN != 0) begin : gen_reflect_inputs
    for (genvar i = 0; i < CH; i++) begin : gen_ch_reflect
        for (genvar j = 0; j < 8; j++) begin : gen_bit_reflect
            assign w_block_data[i][j] = data[i*8+7-j]; // Bit
reversal
        end
    end
end else begin : gen_direct_assign_inputs
    for (genvar i = 0; i < CH; i++) begin : gen_ch_direct
        assign w_block_data[i] = data[i*8+:8]; // Direct assignment
```

```
end
end
```

23.6.2 Cascade Processing Architecture

```
// Chain of XOR-shift cascade stages
for (i = 0; i < CH; i++) begin : gen_xor_shift_cascade
    if (i == 0) begin : gen_xor_cascade_0
        dataint_crc_xor_shift_cascade #(.CRC_WIDTH(CRC_WIDTH))
        cascade_0 (
            .block_input(r_crc_value),      // Start with current CRC
            .poly(w_poly),
            .data_input(w_block_data[i]),
            .block_output(w_cascade[i])
        );
    end else begin : gen_xor_cascade_N
        dataint_crc_xor_shift_cascade #(.CRC_WIDTH(CRC_WIDTH))
        cascade_N (
            .block_input(w_cascade[i-1]),    // Chain from previous
            .poly(w_poly),
            .data_input(w_block_data[i]),
            .block_output(w_cascade[i])
        );
    end
end
end
```

23.6.3 Cascade Selection Logic

```
// Select appropriate cascade output based on data length
always_comb begin
    w_selected_cascade_output = POLY_INIT; // Default
    for (int i = 0; i < CH; i++) begin
        if (cascade_sel[i]) begin
            w_selected_cascade_output = w_cascade[i];
        end
    end
end
end
```

23.7 CRC Standards Compatibility

23.7.1 Common CRC Configurations

23.7.1.1 CRC-32 (IEEE 802.3)

```
dataint_crc #(
    .ALGO_NAME("CRC32_IEEE"),
    .DATA_WIDTH(32),
    .CRC_WIDTH(32),
    .REFIN(1),

```

```

        .REFOUT(1)
    ) u_crc32 (
        .POLY(32'h04C11DB7),
        .POLY_INIT(32'hFFFFFFFF),
        .XOROUT(32'hFFFFFFFF),
        // ... other connections
    );

```

23.7.1.2 CRC-16 (CCITT)

```

dataint_crc #(
    .ALGO_NAME("CRC16_CCITT"),
    .DATA_WIDTH(16),
    .CRC_WIDTH(16),
    .REFIN(0),
    .REFOUT(0)
) u_crc16 (
    .POLY(16'h1021),
    .POLY_INIT(16'hFFFF),
    .XOROUT(16'h0000),
    // ... other connections
);

```

23.7.1.3 CRC-64 (ECMA-182)

```

dataint_crc #(
    .ALGO_NAME("CRC64_ECMA"),
    .DATA_WIDTH(64),
    .CRC_WIDTH(64),
    .REFIN(1),
    .REFOUT(1)
) u_crc64 (
    .POLY(64'h42F0E1EBA9EA3693),
    .POLY_INIT(64'h0000000000000000),
    .XOROUT(64'hFFFFFFFFFFFFFFFF),
    // ... other connections
);

```

23.8 Usage Examples

23.8.1 Basic CRC Calculation

```

logic [63:0] input_data;
logic [31:0] crc_result;
logic data_valid, crc_ready;

```

```

dataint_crc #(
    .DATA_WIDTH(64),
    .CRC_WIDTH(32),
    .REFIN(1),

```

```

        .REFOUT(1)
    ) u_crc_engine (
        .POLY(32'h04C11DB7),
        .POLY_INIT(32'hFFFFFFFF),
        .XOROUT(32'hFFFFFFFF),
        .clk(clk),
        .rst_n(rst_n),
        .load_crc_start(data_valid),
        .load_from_cascade(1'b0),
        .cascade_sel(8'hFF), // Use all chunks
        .data(input_data),
        .crc(crc_result)
    );

```

23.8.2 Streaming CRC with Cascade

```

// Process variable-length packets using cascade selection
logic [63:0] packet_data;
logic [7:0] byte_count; // Number of valid bytes (1-8)
logic [7:0] cascade_select;

// Convert byte count to one-hot cascade selection
always_comb begin
    cascade_select = '0;
    if (byte_count > 0 && byte_count <= 8) begin
        cascade_select[byte_count-1] = 1'b1;
    end
end

dataint_crc #(
    .DATA_WIDTH(64),
    .CRC_WIDTH(32)
) u_stream_crc (
    .clk(clk),
    .rst_n(rst_n),
    .load_crc_start(packet_start),
    .load_from_cascade(packet_continue),
    .cascade_sel(cascade_select),
    .data(packet_data),
    .crc(streaming_crc),
    // ... other connections
);

```

23.8.3 Multi-Stage CRC Pipeline

```

// Pipeline multiple CRC engines for high throughput
logic [63:0] stage_data [0:3];
logic [31:0] stage_crc [0:3];

```

```

genvar stage;
generate
  for (stage = 0; stage < 4; stage++) begin : gen_crc_pipeline
    dataint_crc #(
      .DATA_WIDTH(64),
      .CRC_WIDTH(32),
      .ALGO_NAME($sprintf("STAGE_%0d", stage))
    ) u_pipeline_crc (
      .clk(clk),
      .rst_n(rst_n),
      .data(stage_data[stage]),
      .crc(stage_crc[stage]),
      // ... configure for pipeline operation
    );
  end
endgenerate

```

23.9 Performance Characteristics

23.9.1 Throughput Analysis

Data Width	CRC Width	Max Frequency	Throughput
32-bit	32-bit	~400 MHz	12.8 Gbps
64-bit	32-bit	~350 MHz	22.4 Gbps
128-bit	64-bit	~300 MHz	38.4 Gbps

23.9.2 Latency Characteristics

- **Pipeline Stages:** 2 cycles (input conditioning + CRC computation)
- **Reset Recovery:** 1 cycle
- **Cascade Selection:** 0 cycles (combinational)

23.10 Timing Considerations

23.10.1 Setup Requirements

```

// Data must be stable before clock edge
timing_spec: assert property (
  @(posedge clk) $stable(data) throughout (load_crc_start)
);

// Cascade selection must be stable
cascade_timing: assert property (

```

```
); @(posedge clk) $stable(cascade_sel) throughout (load_from_cascade)
```

23.10.2 Critical Paths

1. **Cascade Chain:** Data flows through all cascade stages
2. **Output Reflection:** Bit reversal for REFOUT
3. **Register Setup:** Loading CRC state register

23.11 Design Considerations

23.11.1 Area Optimization

- **Polynomial Optimization:** Use polynomials with fewer terms
- **Width Reduction:** Match CRC_WIDTH to actual requirements
- **Cascade Stages:** Reduce CHUNKS for lower throughput applications

23.11.2 Power Optimization

- **Clock Gating:** Gate unused cascade stages
- **Data Gating:** Prevent unnecessary switching when not processing

23.11.3 Verification Notes

- **Polynomial Validation:** Verify against known CRC test vectors
- **Cascade Testing:** Test all cascade selection combinations
- **Reflection Testing:** Verify bit ordering for REFIN/REFOUT modes
- **Standards Compliance:** Validate against published CRC standards

23.12 Related Modules

- **dataint_crc_xor_shift_cascade:** Building block for cascade stages
- **dataint_crc_xor_shift:** Simple XOR-shift CRC implementation
- **dataint_checksum:** Alternative integrity checking method
- **dataint_parity:** Simpler parity-based error detection

The `dataint_crc` module provides a comprehensive, high-performance solution for CRC-based data integrity checking with the flexibility to support a wide range of CRC standards and applications.

23.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

24 dataint_crc_xor_shift_cascade Module Documentation

24.1 Purpose

The `dataint_crc_xor_shift_cascade` module processes 8 bits of data through a cascade of CRC calculation stages. It chains together 8 instances of `dataint_crc_xor_shift` to efficiently process a full byte of data in a single combinational operation.

24.2 Module Declaration

```
module dataint_crc_xor_shift_cascade #(
    parameter int CRC_WIDTH = 32
) (
    input  [CRC_WIDTH-1:0] block_input,
    input  [CRC_WIDTH-1:0] poly,
    input  [7:0] data_input,
    output [CRC_WIDTH-1:0] block_output
);
```

24.3 Parameters

Parameter	Default	Description
CRC_WIDTH	32	Width of the CRC register and polynomial

24.4 Ports

Port	Direction	Width	Description
block_input	Input	CRC_WIDTH	Initial CRC state for the byte
poly	Input	CRC_WIDTH	CRC polynomial
data_input	Input	8	8-bit data byte to process

Port	Direction	Width	Description
t			
block_out put	Output	CRC_WIDTH	Final CRC state after processing 8 bits

24.5 Functionality

24.5.1 Cascade Operation

The module creates a pipeline of 8 CRC calculation stages: 1. **Stage 0**: Processes MSB of data byte with initial CRC state 2. **Stages 1-6**: Each processes next data bit with previous stage output 3. **Stage 7**: Processes LSB and produces final result

24.5.2 Data Bit Ordering

Data bits are processed from MSB to LSB: - data_input[7] is processed first - data_input[0] is processed last - This matches typical serial transmission order

24.6 Implementation Details

24.6.1 Architecture Overview

block_input → [Stage 0] → [Stage 1] → ... → [Stage 7] → block_output
 data[7] data[6] data[0]

24.6.2 Generate Block Structure

The module uses a parameterized generate block to create the cascade:

```
generate
  for (i = 0; i < 8; i++) begin : gen_xor_shift_cascade
    if (i == 0) begin : gen_xor_shift_0
      // First stage uses block_input
      dataint_crc_xor_shift #(.CRC_WIDTH(CRC_WIDTH)) stage (
        .stage_input(block_input),
        .poly(poly),
        .new_bit(data_input[7-i]),
        .stage_output(w_cascade[i])
      );
    end else begin : gen_xor_shift_N
      // Subsequent stages chain previous output
      dataint_crc_xor_shift #(.CRC_WIDTH(CRC_WIDTH)) stage (
        .stage_input(w_cascade[i-1]),
        .poly(poly),
        .new_bit(data_input[7-i]),
        .stage_output(w_cascade[i])
      );
    end
  end
end
```


**end
endgenerate**

24.6.3 Intermediate Signals

- **w_cascade[0:7]**: Array holding intermediate CRC states
- **Fixed Width**: Each element is CRC_WIDTH bits wide
- **Sequential Chain**: Each stage feeds the next

24.7 Key Features

24.7.1 Parallel Byte Processing

- Processes 8 bits in one combinational operation
- No clock required - purely combinational
- Suitable for high-speed applications

24.7.2 Bit Order Handling

- MSB-first processing (network byte order)
- Bit index calculation: `data_input[7-i]`
- Matches standard CRC bit processing order

24.7.3 Scalable Architecture

- Easy to modify for different byte sizes
- Generate blocks enable clean scaling
- Parameterized for different CRC widths

24.8 Timing Characteristics

24.8.1 Combinational Delay

Total delay is the sum of 8 CRC stage delays: - Each stage: ~2-3 gate delays - Total cascade: ~16-24 gate delays - May require pipelining for very high frequencies

24.8.2 Critical Path

The critical path runs through all 8 stages:

`block_input` → Stage0 → Stage1 → ... → Stage7 → `block_output`

24.9 Usage Examples

24.9.1 Basic Usage

```
dataint_crc_xor_shift_cascade #(
    .CRC_WIDTH(16)
) crc_byte_processor (
    .block_input(current_crc_state),
    .poly(16'h1021),           // CRC-16-CCITT
    .data_input(data_byte),
    .block_output(updated_crc_state)
);
```

24.9.2 Multi-Byte Processing

```
// Process 4 bytes (32 bits) of data
wire [CRC_WIDTH-1:0] stage_output[0:3];

genvar i;
generate
    for (i = 0; i < 4; i++) begin : gen_multi_byte
        dataint_crc_xor_shift_cascade #(.CRC_WIDTH(CRC_WIDTH)) cascade
        (
            .block_input(i == 0 ? initial_crc : stage_output[i-1]),
            .poly(polyomial),
            .data_input(data_bytes[i]),
            .block_output(stage_output[i])
        );
    end
endgenerate

assign final_crc = stage_output[3];
```

24.10 Integration with CRC System

24.10.1 Role in CRC Module

This module is used by dataint_crc for: - Processing data in byte-sized chunks - Building wider data processing capability - Maintaining processing efficiency

24.10.2 Interface with Parent Module

- Multiple cascade instances handle wide data
- Cascade selection allows intermediate results
- Enables flexible data width support

24.11 Performance Optimization

24.11.1 Pipelining Considerations

For high-frequency operation: - Consider adding pipeline registers between cascades - Balance latency vs. throughput requirements - May need to pipeline within the 8-stage cascade

24.11.2 Area Optimization

- Each stage requires $\sim \text{CRC_WIDTH}$ XOR gates
- Total area scales with $\text{CRC_WIDTH} \times 8$
- Consider time-multiplexed alternatives for area-critical applications

24.12 Applications

- High-speed packet processing
- Real-time data integrity checking
- Communication protocol implementations
- Storage system error detection
- Building block for wider CRC processors

24.13 Related Modules

- **`dataint_crc_xor_shift`**: Single-bit processing building block
- **`dataint_crc`**: Parent CRC calculation module
- Used together to create flexible CRC calculation systems

24.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

25 dataint_crc_xor_shift Module Documentation

25.1 Purpose

The `dataint_crc_xor_shift` module performs CRC calculation for a single bit of input data. It implements the fundamental CRC operation: shifting the current CRC state and XORing with the polynomial when the feedback bit is set.

25.2 Module Declaration

```
module dataint_crc_xor_shift #(
    parameter int CRC_WIDTH = 32
) (
    input [CRC_WIDTH-1:0] stage_input,
    input [CRC_WIDTH-1:0] poly,
    input new_bit,
    output [CRC_WIDTH-1:0] stage_output
);
```

25.3 Parameters

Parameter	Default	Description
CRC_WIDTH	32	Width of the CRC register and polynomial

25.4 Ports

Port	Direction	Width	Description
stage_input	Input	CRC_WIDTH	Current CRC state
poly	Input	CRC_WIDTH	CRC polynomial
new_bit	Input	1	New data bit to process
stage_output	Output	CRC_WIDTH	Updated CRC state

25.5 Functionality

25.5.1 Core CRC Operation

The module implements the basic CRC shift-and-XOR operation: 1. **Feedback Calculation:** XOR the new input bit with the MSB of current CRC 2. **Shift Operation:** Shift CRC register left by one position 3. **Polynomial XOR:** If feedback bit is 1, XOR with polynomial

25.5.2 Mathematical Foundation

The CRC operation follows this algorithm:

```
feedback = new_bit ⊕ CRC[MSB]
CRC_new[0] = feedback
CRC_new[MSB:1] = CRC[MSB-1:0] ⊕ (polynomial[MSB:1] & {WIDTH-1{feedback}})
```

25.6 Implementation Details

25.6.1 Key Features

- **Single Bit Processing:** Processes exactly one bit per operation
- **Combinational Logic:** Pure combinational implementation (no clock)
- **Parameterizable Width:** Supports any CRC width
- **Optimized Design:** Breaks circular dependency with intermediate signal

25.6.2 Circular Dependency Solution

The original implementation could have a circular dependency. This is solved by:

```
logic feedback_bit;
```

```
// Calculate feedback first
```

```
assign feedback_bit = new_bit ^ stage_input[CRC_WIDTH-1];
```

```
// Then use it for output calculation
```

```
assign stage_output[0] = feedback_bit;
```

```
assign stage_output[CRC_WIDTH-1:1] = stage_input[CRC_WIDTH-2:0] ^  
    ({CRC_WIDTH-1{feedback_bit}} & poly[CRC_WIDTH-1:1]);
```

25.6.3 Bit-wise Operations

- **LSB Assignment:** Always gets the feedback bit
- **MSBs Assignment:** Shifted previous bits XORed with polynomial when feedback is active

- **Conditional XOR:** Uses bit replication to create conditional XOR mask

25.7 Special Implementation Features

25.7.1 Feedback Bit Logic

The feedback bit is critical for CRC operation: - Determines whether polynomial is applied - Calculated as XOR of input bit and current MSB - Used to gate polynomial application

25.7.2 Polynomial Application

The polynomial is applied conditionally: - Only bits [MSB:1] of polynomial are used (bit 0 is implicit) - Applied via AND mask created by replicating feedback bit - XORED with shifted CRC bits

25.7.3 No State Elements

This module is purely combinational: - No clock or reset signals needed - Instantaneous response to input changes - Can be cascaded for multi-bit processing

25.8 Usage in CRC Chain

This module is typically used as a building block:

25.8.1 Single Bit Processing

```
dataint_crc_xor_shift #(.CRC_WIDTH(16)) crc_stage (
    .stage_input(current_crc),
    .poly(crc_polynomial),
    .new_bit(data_bit),
    .stage_output(next_crc)
);
```

25.8.2 Cascaded for Multiple Bits

Multiple instances can be chained to process multiple bits:

```
// Process 8 bits
wire [CRC_WIDTH-1:0] stage[0:7];
assign stage[0] = initial_crc;

genvar i;
generate
    for (i = 0; i < 8; i++) begin
        dataint_crc_xor_shift #(.CRC_WIDTH(CRC_WIDTH)) stage_inst (
            .stage_input(i == 0 ? initial_crc : stage[i-1]),
            .poly(polynomial),
            .new_bit(data_byte[7-i]),
            .stage_output(stage[i])
        );
    end
end
```

endgenerate

```
assign final_crc = stage[7];
```

25.9 Mathematical Verification

For a polynomial $P(x)$ and data bit d : - If feedback = 0: Simple left shift - If feedback = 1: Left shift XOR polynomial

This implements the mathematical CRC division in hardware.

25.10 Applications

- Building block for CRC calculation engines
- Serial CRC processors
- Component in parallel CRC architectures
- Educational CRC implementations
- Custom protocol CRC calculators

25.11 Performance Characteristics

- **Latency:** Combinational (zero clock cycles)
- **Throughput:** One bit per clock when used in sequential system
- **Area:** Minimal - few XOR gates and multiplexers
- **Power:** Low - simple combinational logic

25.12 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

26 dataint_ecc_hamming_decode_secded Module Documentation

26.1 Purpose

The dataint_ecc_hamming_decode_secded module implements a Hamming decoder with Single Error Correction, Double Error Detection (SECDDED) capability. It decodes Hamming-encoded data, detects and corrects single-bit errors, and flags double-bit errors.

26.2 Module Declaration

```
module dataint_ecc_hamming_decode_secded #(
    parameter int WIDTH = 4,
    parameter int DEBUG = 0
) (
    input logic clk,
    input logic rst_n,
    input logic enable,
    input logic [WIDTH+ParityBits:0] hamming_data,
    output logic [WIDTH-1:0] data,
    output logic error_detected,
    output logic double_error_detected
);
```

26.3 Parameters

Parameter	Default	Description
WIDTH	4	Width of original data in bits
DEBUG	0	Enable debug output (1=enabled, 0=disabled)

26.4 Calculated Parameters

Parameter	Formula	Description
ParityBits	$\$clog2(WIDTH + \$clog2(WIDTH) + 1)$	Number of Hamming parity bits
TotalWidth	$WIDTH + ParityBits + 1$	Total encoded data width

26.5 Ports

Port	Direction	Width	Description
clk	Input	1	Clock signal
rst_n	Input	1	Active-low asynchronous reset
enable	Input	1	Enable decoding operation
hamming_data	Input	TotalWidth	Encoded data with ECC bits
data	Output	WIDTH	Decoded original data
error_detected	Output	1	Single or double error detected
double_error_detected	Output	1	Double error specifically detected

26.6 Functionality

26.6.1 Decoding Process

1. **Syndrome Generation:** Calculate parity check syndrome
2. **Overall Parity Check:** Verify SECDED bit
3. **Error Classification:** Determine error type and location
4. **Error Correction:** Fix single-bit errors
5. **Data Extraction:** Extract corrected original data

26.6.2 Error Detection Logic

Overall Parity	Syndrome	Error Type	Action
Match	Zero	No Error	Pass data through
Mismatch	Non-zero	Single Error (Hamming)	Correct using syndrome
Mismatch	Zero	Single Error (SECDED)	No correction needed
Match	Non-zero	Double Error	Flag error, no

Overall Parity	Syndrome	Error Type	Action
			correction

26.7 Implementation Details

26.7.1 Key Functions

26.7.1.1 *bit_position(k)* Function

Calculates the position of data bit *k* in the encoded stream:

```
function automatic integer bit_position(input integer k);
    integer j, pos;
    begin
        pos = k + 1; // Account for parity bit at position 0
        for (j = 0; j < ParityBits; j++) begin
            if (pos >= (2 ** j)) pos = pos + 1; // Skip parity
positions
        end
        bit_position = pos - 1; // Convert to 0-based index
    end
endfunction
```

26.7.1.2 *get_covered_bits(parity_bit)* Function

Returns bitmask of positions covered by a specific parity bit:

```
function automatic [TotalWidth-1:0] get_covered_bits(input integer
parity_bit);
    integer j;
    begin
        get_covered_bits = 0;
        // FIXED: Exclude SECDED bit from Hamming parity calculations
        for (j = 0; j < TotalWidth - 1; j++) begin
            if (((j + 1) >> parity_bit) & 1)
                get_covered_bits[j] = 1'b1;
        end
    end
endfunction
```

26.7.2 Syndrome Generation

```
always_comb begin : create_syndrome_covered_bits
    if (enable) begin
        for (i = 0; i < ParityBits; i++) begin
            parity_pos = (2 ** i) - 1; // Parity bit positions
            w_syndrome_in[i] = hamming_data[parity_pos]; // Stored
parity
            w_syndrome[i] = 1'b0;
        end
    end
end
```

```

        w_covered_bits = get_covered_bits(i);
        for (bit_index = 0; bit_index < TotalWidth; bit_index++)
begin
    // Calculate parity over covered bits (excluding
    parity bit itself)
    if (w_covered_bits[bit_index] && (bit_index !=
    parity_pos))
        w_syndrome[i] = w_syndrome[i] ^
        hamming_data[bit_index];
    end

    // Compare with stored parity to get syndrome
    w_syndrome[i] = w_syndrome[i] ^ w_syndrome_in[i];
end
end
end

```

26.7.3 Overall Parity Check

```

always_comb begin : check_overall_parity
    w_overall_parity_in = hamming_data[TotalWidth-1]; // SECEDED bit
    if (enable) begin
        w_overall_parity = ^hamming_data[TotalWidth-2:0]; //
        Calculated parity
    end else begin
        w_overall_parity = 'b0;
    end
end
end

```

26.8 State Machine Operation

26.8.1 Control States

The module operates under enable control with these logical states:

1. **IDLE**: enable = 0 - No processing
2. **DECODE**: enable = 1 - Active decoding and error correction

26.8.2 Error Correction Logic

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_data_with_parity <= 'b0;
        error_detected <= 'b0;
        double_error_detected <= 'b0;
    end else if (enable) begin
        r_data_with_parity <= hamming_data; // Default assignment
    end
end

```

```

error_detected <= 'b0;
double_error_detected <= 'b0;

// SECEDED error detection logic
if (w_overall_parity != w_overall_parity_in) begin
    error_detected <= 1'b1;

    if (w_syndrome != {ParityBits{1'b0}}) begin
        // Single-bit error in Hamming data - correct it
        r_data_with_parity[w_syndrome_0_based] <=
            ~hamming_data[w_syndrome_0_based];
    end
    // Single-bit error in SECEDED bit - no correction needed

end else if (w_syndrome != {ParityBits{1'b0}}) begin
    // Double-bit error detected
    error_detected <= 1'b1;
    double_error_detected <= 1'b1;
end
end
end

```

26.9 Key Features

26.9.1 Error Correction Capabilities

- **Single Error Correction:** Automatically fixes single-bit errors
- **Double Error Detection:** Flags but cannot correct double-bit errors
- **Error Location:** Syndrome points to exact error location
- **SECEDED Protection:** Additional parity bit improves error detection

26.9.2 Performance Characteristics

- **Latency:** One clock cycle for error correction
- **Throughput:** One word per clock cycle
- **Detection Rate:** 100% for single and double errors

26.9.3 Debug Support

When `DEBUG != 0`: - Displays parity calculations - Shows syndrome values - Reports error detection details - Useful for verification and debugging

26.10 Error Handling Details

26.10.1 Syndrome Interpretation

- **Syndrome = 0:** No error in Hamming bits
- **Syndrome $\neq 0$:** Points to error bit position (1-based)
- **Zero-based Syndrome:** $w_syndrome_0_based = w_syndrome - 1$

26.10.2 SECEDED Bit Function

The SECEDED bit enables distinguishing between: - Single-bit errors (correctable) - Double-bit errors (detectable but uncorrectable) - No errors

26.10.3 Error Correction Process

1. Calculate syndrome from received parity bits
2. Check overall parity (SECEDED bit)
3. If single error detected, flip bit at syndrome position
4. If double error detected, flag but don't correct

26.11 Usage Examples

26.11.1 Basic Decoder

```
dataint_ecc_hamming_decode_secDED #(
    .WIDTH(8),
    .DEBUG(0)
) hamming_decoder (
    .clk(clk),
    .rst_n(rst_n),
    .enable(decode_enable),
    .hamming_data(encoded_input),
    .data(decoded_data),
    .error_detected(single_or_double_error),
    .double_error_detected(uncorrectable_error)
);
```

26.11.2 Error Handling System

```
always_ff @(posedge clk) begin
    if (error_detected) begin
        if (double_error_detected) begin
            // Log uncorrectable error
            error_log <= error_log + 1;
        end
    end
end
```

```

        // Request retransmission
        request_retransmit <= 1'b1;
    end else begin
        // Log corrected error
        corrected_errors <= corrected_errors + 1;
        // Continue with corrected data
    end
end
end
end

```

26.12 Data Extraction

26.12.1 Generate Block for Data Output

```

genvar j;
generate
    for (j = 0; j < WIDTH; j++) begin : gen_block
        assign data[j] = r_data_with_parity[bit_position(j)];
    end
endgenerate

```

This extracts only the original data bits from the corrected encoded word.

26.13 Applications

- ECC memory systems
- Communication error correction
- Storage system integrity
- Real-time data processing
- Safety-critical systems
- Network protocol implementation

26.14 Related Modules

- **dataint_ecc_hamming_encode_secded**: Corresponding encoder
- Used together for complete ECC system implementation

26.15 Performance Considerations

- **Critical Path**: Syndrome calculation and error correction

- **Pipeline Stages:** Consider pipelining for high-frequency operation
- **Area:** Scales with data width and parity requirements
- **Power:** Moderate due to combinational parity calculations

26.16 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

27 dataint_ecc_hamming_encode_secded Module Documentation

27.1 Purpose

The dataint_ecc_hamming_encode_secded module implements a Hamming encoder with Single Error Correction, Double Error Detection (SECDDED) capability. It generates error correction codes that can detect up to 2-bit errors and correct single-bit errors in data.

27.2 Module Declaration

```
module dataint_ecc_hamming_encode_secded #(
    parameter int WIDTH = 4,
    parameter int DEBUG = 0
) (
    input logic [WIDTH-1:0] data,
    output logic [TotalWidth-1:0] encoded_data
);
```

27.3 Parameters

Parameter	Default	Description
WIDTH	4	Width of input data in bits
DEBUG	0	Enable debug output (1=enabled, 0=disabled)

27.4 Calculated Parameters

Parameter	Formula	Description
ParityBits	$\$clog2(WIDTH + \$clog2(WIDTH) + 1)$	Number of Hamming parity bits
TotalWidth	$WIDTH + ParityBits + 1$	Total encoded data width (includes SECDED bit)

27.5 Ports

Port	Direction	Width	Description
data	Input	WIDTH	Original data to be encoded
encoded_data	Output	TotalWidth	Encoded data with parity and SECDED bits

27.6 Functionality

27.6.1 Hamming Code Structure

The encoded data contains: 1. **Data Bits**: Original data placed at specific positions 2. **Parity Bits**: Hamming parity bits at power-of-2 positions (1, 2, 4, 8, ...) 3. **SECDED Bit**: Additional parity bit for double error detection

27.6.2 Bit Position Layout

Position:	1	2	3	4	5	6	7	8	9	10	11	12	...
Content:	P1	P2	D1	P4	D2	D3	D4	P8	D5	D6	D7	D8	...

Where: P = Parity bit, D = Data bit

27.7 Implementation Details

27.7.1 Key Functions

27.7.1.1 *bit_position(k) Function*

Calculates where data bit k should be placed in the encoded stream:

```
function automatic integer bit_position(input integer k);
    integer j, pos;
    begin
        pos = k + 1; // Start at k+1 to account for parity at
                    // position 0
```



```

        for (j = 0; j < ParityBits; j++) begin
            if (pos >= (2 ** j)) pos = pos + 1; // Skip parity
positions
        end
        bit_position = pos - 1; // Convert to 0-based index
    end
endfunction

```

27.7.1.2 get_covered_bits(parity_bit) Function

Returns a bitmask of positions covered by a specific parity bit:

```

function automatic [TotalWidth-1:0] get_covered_bits(input integer
parity_bit);
    integer j;
    begin
        get_covered_bits = 'b0;
        for (j = 0; j < TotalWidth; j++) begin
            // Check if position j+1 has parity_bit set in its binary
representation
            if (((j + 1) >> parity_bit) & 1)
                get_covered_bits[j] = 1'b1;
        end
    end
endfunction

```

27.7.2 Encoding Algorithm

27.7.2.1 Step 1: Data Placement

```

// Initialize with zeros
w_data_with_parity = {TotalWidth{1'b0}};

// Insert data bits into correct positions
for (i = 0; i < WIDTH; i++) begin
    w_data_with_parity[bit_position(i)] = data[i];
end

```

27.7.2.2 Step 2: Parity Calculation

```

// Calculate each Hamming parity bit
for (i = 0; i < ParityBits; i++) begin
    parity_pos = (2 ** i) - 1; // Positions 0, 1, 3, 7, 15, ...
    w_data_with_parity[parity_pos] = 1'b0; // Initialize

    // XOR all covered positions
    w_covered_bits = get_covered_bits(i);
    for (bit_index = 0; bit_index < TotalWidth; bit_index++) begin
        if (w_covered_bits[bit_index]) begin
            w_data_with_parity[parity_pos] =
                w_data_with_parity[parity_pos] ^

```

```
w_data_with_parity[bit_index];
    end
end
end
```

27.7.2.3 Step 3: SECDED Bit Calculation

```
// Calculate overall parity for SECDED
w_data_with_parity[TotalWidth-1] = ^w_data_with_parity[TotalWidth-2:0];
```

27.8 Key Features

27.8.1 Combinational Design

- **No Clock Required:** Purely combinational logic
- **Immediate Output:** Encoded data available immediately after input change
- **Stateless Operation:** No internal state or memory

27.8.2 Scalable Architecture

- **Parameterizable Width:** Supports any data width
- **Automatic Sizing:** Parity bits calculated automatically
- **Efficient Layout:** Optimal bit positioning for Hamming code

27.8.3 Debug Support

When `DEBUG != 0`: - Function calls display bit position calculations - Covered bits masks are displayed - Useful for verification and understanding

27.9 Example Configurations

27.9.1 4-bit Data (Common Example)

- Data bits: 4
- Parity bits: 3 (positions 0, 1, 3)
- SECDED bit: 1 (position 7)
- Total width: 8 bits

27.9.28-bit Data

- Data bits: 8
- Parity bits: 4 (positions 0, 1, 3, 7)
- SECDED bit: 1 (position 12)
- Total width: 13 bits

27.9.3 16-bit Data

- Data bits: 16
- Parity bits: 5 (positions 0, 1, 3, 7, 15)
- SECDED bit: 1 (position 21)
- Total width: 22 bits

27.10 Usage Example

27.10.1 Basic Encoding

```
// 8-bit data encoder
dataint_ecc_hamming_encode_secDED #(
    .WIDTH(8),
    .DEBUG(0)
) hamming_encoder (
    .data(input_data),
    .encoded_data(ecc_encoded_data)
);
```

27.10.2 With Debug

```
// Debug-enabled encoder for verification
dataint_ecc_hamming_encode_secDED #(
    .WIDTH(4),
    .DEBUG(1)
) debug_encoder (
    .data(test_data),
    .encoded_data(encoded_output)
);
```

27.11 Error Correction Capabilities

27.11.1 Single Error Correction

- Can correct any single-bit error in the encoded data

- Uses Hamming parity bits to identify error position
- Decoder can flip the erroneous bit

27.11.2 Double Error Detection

- SECEDED bit enables detection of double-bit errors
- Cannot correct double errors, but can flag them
- Prevents silent data corruption

27.11.3 Error Detection Matrix

Error Type	Detection	Correction
No Error	✓	N/A
Single Bit Error	✓	✓
Double Bit Error	✓	✗
Triple+ Bit Error	Partial	✗

27.12 Applications

- Memory systems (ECC RAM)
- Communication protocols
- Storage systems
- Critical data processing
- Aerospace and automotive systems
- Any application requiring data integrity

27.13 Related Modules

- **dataint_ecc_hamming_decode_secDED**: Corresponding decoder module
- Used together for complete SECEDED error correction system

27.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

28 dataint_parity Module Documentation

28.1 Purpose

The dataint_parity module implements a generic parity generator and checker. It can generate parity bits for data chunks and verify existing parity bits, supporting both even and odd parity schemes across multiple data segments.

28.2 Module Declaration

```
module dataint_parity #(
    parameter int CHUNKS = 4, // Number of chunks to check parity
    parameter int WIDTH  = 32 // Total width of the data
) (
    input logic [WIDTH-1:0] data_in, // Data input
    input logic [CHUNKS-1:0] parity_in, // Parity input for
checking
    input logic parity_type, // 1=even, 0=odd
    output logic [CHUNKS-1:0] parity, // Generated parity bits
    output logic [CHUNKS-1:0] parity_err // Error indicators
);
```

28.3 Parameters

Parameter	Default	Description
CHUNKS	4	Number of data chunks to process
WIDTH	32	Total width of input data in bits

28.4 Calculated Parameters

Parameter	Formula	Description
ChunkSize	WIDTH / CHUNKS	Size of each chunk in bits
ExtraBits	WIDTH % CHUNKS	Remaining bits if WIDTH not evenly divisible

28.5 Ports

Port	Direction	Width	Description
data_in	Input	WIDTH	Input data to calculate/check

Port	Direction	Width	Description
parity_in	Input	CHUNKS	parity Expected parity bits for verification
parity_type	Input	1	Parity type (1=even, 0=odd)
parity	Output	CHUNKS	Calculated parity bits
parity_err	Output	CHUNKS	Error flags (1=parity mismatch)

28.6 Functionality

28.6.1 Dual Operation Modes

The module simultaneously: 1. **Generates Parity**: Calculates parity for each data chunk 2. **Checks Parity**: Compares calculated parity with input parity

28.6.2 Parity Types

- **Even Parity** (parity_type = 1): Parity bit makes total 1s even
- **Odd Parity** (parity_type = 0): Parity bit makes total 1s odd

28.6.3 Data Chunking

Data is divided into chunks with special handling for non-even divisions: - Most chunks: ChunkSize bits each - Last chunk: May include extra bits if $WIDTH \% CHUNKS \neq 0$

28.7 Implementation Details

28.7.1 Generate Block Architecture

```

genvar i;
generate
  for (i = 0; i < CHUNKS; i++) begin : gen_parity
    // Calculate bounds for each chunk
    localparam int LowerBound = i * ChunkSize;
    localparam int UpperBound = (i < CHUNKS - 1) ?
      ((i + 1) * ChunkSize) - 1 : WIDTH - 1;
  end
endgenerate

```

```

// Calculate parity for this chunk
wire calculated_parity = parity_type ?
    ^data_in[UpperBound:LowerBound] :    // Even parity
    ~^data_in[UpperBound:LowerBound];    // Odd parity

assign parity[i] = calculated_parity;
assign parity_err[i] = (calculated_parity != parity_in[i]);
end
endgenerate

```

28.7.2 Chunk Boundary Calculation

Each chunk's boundaries are calculated statically: - **Regular Chunks** ($i < \text{CHUNKS}-1$): Fixed size chunks - **Last Chunk** ($i = \text{CHUNKS}-1$): Includes any remaining bits

28.7.3 Parity Calculation Logic

For each chunk: 1. **XOR Reduction**: $\text{^data_in}[\text{UpperBound}:\text{LowerBound}]$ 2. **Parity Type Application**: - Even parity: Use XOR result directly - Odd parity: Invert XOR result

28.7.4 Error Detection

Error detection compares calculated vs. expected parity:

```
assign parity_err[i] = (calculated_parity != parity_in[i]);
```

28.8 Key Features

28.8.1 Scalable Architecture

- **Parameterizable Chunks**: Any number of data segments
- **Flexible Data Width**: Supports any total data width
- **Automatic Sizing**: Handles non-even chunk divisions

28.8.2 Simultaneous Operation

- **Generation and Checking**: Both operations happen concurrently
- **Independent Chunks**: Each chunk operates independently
- **Parallel Processing**: All chunks processed simultaneously

28.8.3 Parity Flexibility

- **Runtime Configurable**: Parity type can change during operation
- **Per-Module Setting**: All chunks use same parity type

- **Standard Support:** Supports both common parity schemes

28.9 Example Configurations

28.9.1 32-bit Data, 4 Chunks (8 bits each)

```

Chunk 0: data_in[7:0]    → parity[0], parity_err[0]
Chunk 1: data_in[15:8]   → parity[1], parity_err[1]
Chunk 2: data_in[23:16]  → parity[2], parity_err[2]
Chunk 3: data_in[31:24]  → parity[3], parity_err[3]

```

28.9.2 30-bit Data, 4 Chunks (uneven division)

```

Chunk 0: data_in[6:0]    (7 bits) → parity[0], parity_err[0]
Chunk 1: data_in[13:7]   (7 bits) → parity[1], parity_err[1]
Chunk 2: data_in[20:14]  (7 bits) → parity[2], parity_err[2]
Chunk 3: data_in[29:21]  (9 bits) → parity[3], parity_err[3]

```

28.10 Usage Examples

28.10.1 Basic Parity Generation

```

dataint_parity #(
    .CHUNKS(8),
    .WIDTH(64)
) parity_gen (
    .data_in(input_data),
    .parity_in(8'b0),           // Not used for generation
    .parity_type(1'b1),        // Even parity
    .parity(generated_parity),
    .parity_err()              // Not used for generation
);

```

28.10.2 Parity Checking

```

dataint_parity #(
    .CHUNKS(4),
    .WIDTH(32)
) parity_check (
    .data_in(received_data),
    .parity_in(received_parity),
    .parity_type(1'b0),        // Odd parity
    .parity(recalc_parity),
    .parity_err(error_flags)
);

```

```

// Check for any errors
wire any_parity_error = |error_flags;

```


28.10.3 Error Correction System

```
logic [CHUNKS-1:0] error_location;
logic [WIDTH-1:0] corrected_data;

dataint_parity #(.CHUNKS(CHUNKS), .WIDTH(WIDTH)) parity_checker (
    .data_in(received_data),
    .parity_in(received_parity),
    .parity_type(even_parity),
    .parity(recalc_parity),
    .parity_err(error_location)
);

// Simple error correction (single-bit errors only)
always_comb begin
    corrected_data = received_data;
    for (int i = 0; i < CHUNKS; i++) begin
        if (error_location[i]) begin
            // Flip bits in the erroneous chunk
            // (Simplified - real correction needs more sophisticated
            logic)
        end
    end
end
```

28.11 Performance Characteristics

28.11.1 Timing

- **Combinational Logic:** Zero clock delay
- **Critical Path:** XOR tree depth $\approx \log_2(\text{ChunkSize})$
- **Propagation Delay:** Minimal for typical chunk sizes

28.11.2 Area

- **Linear Scaling:** Area increases with CHUNKS and WIDTH
- **Efficient Implementation:** Simple XOR trees
- **Low Overhead:** Minimal logic per chunk

28.11.3 Power

- **Low Power:** Simple combinational logic
- **Activity Dependent:** Power scales with data switching

28.12 Applications

28.12.1 Error Detection

- Memory parity checking
- Communication protocol verification
- Data bus integrity monitoring
- Storage system error detection

28.12.2 Data Integrity

- Real-time parity generation
- Concurrent error checking
- Multi-segment data validation
- Protocol compliance verification

28.12.3 System Integration

- Building block for ECC systems
- Component in larger error correction schemes
- Interface protection
- Debug and validation tools

28.13 Design Considerations

28.13.1 Chunk Size Selection

- **Power of 2:** Often preferred for alignment
- **Protocol Requirements:** Match system requirements
- **Error Granularity:** Smaller chunks = finer error localization

28.13.2 Parity Type Selection

- **Even Parity:** More common in digital systems
- **Odd Parity:** Provides all-zeros detection

- **System Compatibility:** Match existing standards

28.13.3 Width Considerations

- **Alignment:** Consider data bus alignment
- **Extra Bits:** Handle remainder bits appropriately
- **Performance:** Balance chunk count vs. granularity

28.14 Related Applications

- **ECC Systems:** Building block for Hamming codes
- **Network Protocols:** Ethernet, UART parity
- **Memory Systems:** DRAM/SRAM parity protection
- **Communication:** RS-232, SPI, I2C error detection

28.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

29 Debounce Module (debounce . sv)

29.1 Purpose

Eliminates mechanical switch bounce by sampling button inputs at regular intervals and requiring a stable state for multiple consecutive samples before considering the button state as valid.

29.2 Ports

29.2.1 Input Ports

- **clk** - System clock signal
- **rst_n** - Active-low reset signal
- **long_tick** - ~10ms sampling tick signal (controls sampling rate)

- **button_in[N-1:0]** - Raw button input signals to be debounced

29.2.2 Output Ports

- **button_out[N-1:0]** - Debounced button output signals

29.2.3 Parameters

- **N** - Number of buttons/input signals (default: 4)
- **DEBOUNCE_DELAY** - Number of consecutive stable samples required (default: 4)
- **PRESSED_STATE** - Logic level when button is pressed (1 for normally open, 0 for normally closed)

29.3 Implementation Details

29.3.1 Core Algorithm

The module uses shift registers to track the history of each button's state:

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        for (int i = 0; i < N; i++) begin
            r_shift_regs[i] <= {DEBOUNCE_DELAY{1'b0}};
        end
    end else if (long_tick) begin
        for (int i = 0; i < N; i++) begin
            r_shift_regs[i] <= {
                r_shift_regs[i][DEBOUNCE_DELAY-2:0],
                PRESSED_STATE ? button_in[i] : ~button_in[i]
            };
        end
    end
end
```

29.3.2 Key Features

29.3.2.1 Sampling Control

- Only samples inputs when **long_tick** is asserted
- Prevents over-sampling which could cause false triggering
- Typical **long_tick** period: ~10ms

29.3.2.2 Button Type Support

- **Normally Open (NO):** PRESSED_STATE = 1
 - Button reads '0' when not pressed, '1' when pressed
- **Normally Closed (NC):** PRESSED_STATE = 0
 - Button reads '1' when not pressed, '0' when pressed
- Input inversion handles NC buttons transparently

29.3.2.3 Debounce Logic

- Maintains DEBOUNCE_DELAY samples for each button
- Output goes high only when all samples in shift register are '1'
- This ensures the button has been stable in pressed state for required duration

```
always_comb begin
    for (int i = 0; i < N; i++) begin
        // Output 1 when shift register shows stable pressed state
        // (all 1s)
        w_debounced_signals[i] = &r_shift_regs[i];
    end
end
```

29.3.2.4 Output Registration

- Final output is registered to provide clean, glitch-free transitions
- Resets to all zeros on system reset

29.4 Timing Characteristics

29.4.1 Debounce Delay

- **Total delay** = DEBOUNCE_DELAY × long_tick period
- **Default:** 4 × 10ms = 40ms debounce time
- **Minimum stable time:** Button must remain stable for full delay period

29.4.2 Response Time

- **Press detection:** DEBOUNCE_DELAY ticks after button stabilizes

- **Release detection:** DEBOUNCE_DELAY ticks after button releases
- **Asymmetric:** Same delay for both press and release

29.5 Use Cases

- Mechanical pushbuttons and switches
- Rotary encoder inputs (with appropriate delay settings)
- Any digital input subject to contact bounce
- Both normally open and normally closed switch types

29.6 Design Considerations

- **Delay tuning:** Adjust DEBOUNCE_DELAY based on switch characteristics
- **Tick frequency:** long_tick should be much slower than bounce duration
- **Multiple buttons:** All buttons share same debounce parameters
- **Reset behavior:** All outputs go low on reset, regardless of button states

29.7 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

30 Decoder Module (decoder.sv)

30.1 Purpose

Converts an M-bit binary encoded input to a one-hot N-bit output where $N = 2^M$. This is a standard binary-to-one-hot decoder implementation.

30.2 Ports

30.2.1 Input Ports

- **encoded[M-1:0]** - Binary encoded input value

30.2.2 Output Ports

- **data[N-1:0]** - One-hot decoded output vector

30.2.3 Parameters

- **M** - Width of binary input (default: 4)
- **N** - Width of one-hot output (default: $2^M = 16$)

30.3 Implementation Details

30.3.1 Core Logic

The decoder uses a generate loop to create comparison logic for each output bit:

```
assign data = 0; // Initialize all outputs to 0

genvar i;
generate
    for (i = 0; i < N; i++) begin : gen_DECODER_LOOP
        assign data[i] = (encoded == i) ? 1'b1 : 1'b0;
    end
endgenerate
```

30.3.2 Operation Principle

- Each output bit corresponds to one possible input value
- Only one output bit is high at any time (one-hot encoding)
- Output bit *i* is high when encoded input equals *i*

30.4 Functional Behavior

30.4.1 Truth Table Example (M=2, N=4)

encoded[1:0]	data[3]	data[2]	data[1]	data[0]
00	0	0	0	1
01	0	0	1	0
10	0	1	0	0
11	1	0	0	0

30.4.2 Key Characteristics

- **Combinational logic:** No clock dependency, immediate response
- **One-hot output:** Exactly one bit high for valid inputs
- **Complete decoding:** All possible input combinations decoded

- **Default initialization:** All outputs start at 0

30.5 Design Features

30.5.1 Parameterization

- **Scalable width:** M parameter controls input width
- **Automatic sizing:** N automatically calculated as 2^M
- **Generate loops:** Synthesizes efficiently for any size

30.5.2 Synthesis Considerations

- **Resource usage:** Typically implements as LUT-based logic
- **Propagation delay:** Single LUT delay in most FPGA architectures
- **Fan-out:** Each input bit fans out to multiple output comparisons

30.6 Use Cases

- **Address decoding:** Memory or register select signals
- **State machine outputs:** Decode binary state to control signals
- **Multiplexer control:** Generate select signals for data routing
- **Interrupt controllers:** Decode interrupt vectors
- **Bus decoding:** Generate chip select signals

30.7 Common Applications

30.7.1 Memory Address Decoding

```
decoder #(.M(2), .N(4)) addr_decoder (
    .encoded(address[1:0]),
    .data(chip_select) // 4 chip selects
);
```

30.7.2 State Machine Output Decoding

```
decoder #(.M(3), .N(8)) state_decoder (
    .encoded(current_state),
    .data(state_outputs) // 8 control signals
);
```


30.8 Timing Characteristics

- **Propagation delay:** Typically 1 LUT delay
- **Setup/hold:** None (purely combinational)
- **Output changes:** Immediately follow input changes

30.9 Related Modules

- **Encoder:** Performs inverse operation (one-hot to binary)
- **Priority Encoder:** Handles multiple simultaneous inputs
- **Multiplexer:** Often used together for data routing

30.10 Design Notes

- No error checking for invalid inputs (though all combinations are valid)
- Output initialization ensures clean power-up behavior
- Generate loop structure scales efficiently to any required size

30.11 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

31 Encoder Module (encoder .sv)

31.1 Purpose

Converts a one-hot N-bit input to a binary encoded output. Performs the inverse operation of a decoder, encoding the position of the asserted bit into binary format.

31.2 Ports

31.2.1 Input Ports

- **decoded[N-1:0]** - One-hot input vector

31.2.2 Output Ports

- **data[\$clog2(N)-1:0]** - Binary encoded output representing the position of the set bit

31.2.3 Parameters

- **N** - Width of one-hot input vector (default: 8)

31.3 Implementation Details

31.3.1 Core Algorithm

The encoder uses a combinational always block with a priority search:

```
always_comb begin
    data = 0; // Default output when no bits set
    for (int i = 0; i < N; i++) begin
        if (decoded[i]) data = $clog2(N)'(i);
    end
end
```

31.3.2 Operation Principle

- Searches through input vector from LSB to MSB
- Outputs the binary representation of the highest indexed set bit
- Acts as a **priority encoder** when multiple bits are set (higher index wins)

31.4 Functional Behavior

31.4.1 Truth Table Example (N=8, output width=3)

decoded[7:0]	data[2:0]	Notes
00000001	000	Bit 0 set
00000010	001	Bit 1 set
00000100	010	Bit 2 set
00001000	011	Bit 3 set
00010000	100	Bit 4 set
00100000	101	Bit 5 set
01000000	110	Bit 6 set
10000000	111	Bit 7 set
00000000	000	No bits set

decoded[7:0]	data[2:0]	Notes
10000001	111	Multiple bits - highest wins

31.4.2 Key Characteristics

- **Priority encoding:** Higher-indexed bits take precedence
- **Zero default:** Outputs 0 when no input bits are set
- **Combinational:** Immediate response to input changes
- **Self-sizing:** Output width automatically calculated using $\lceil \log_2(N) \rceil$

31.5 Design Features

31.5.1 Automatic Width Calculation

- **Output width:** $\lceil \log_2(N) \rceil$ bits
- **Example:** $N=8 \rightarrow 3$ -bit output, $N=16 \rightarrow 4$ -bit output
- **Optimal sizing:** No wasted bits in output

31.5.2 Priority Behavior

The for-loop structure creates implicit priority: - Later iterations overwrite earlier results - Highest-indexed set bit determines final output - Behaves as priority encoder for invalid one-hot inputs

31.6 Use Cases

31.6.1 Valid One-Hot Inputs

- **Interrupt acknowledgment:** Convert interrupt vector to binary ID
- **Resource arbitration:** Encode granted request to requester ID
- **State machine encoding:** Convert one-hot state to binary
- **Position encoding:** Find position of active element

31.6.2 Priority Encoding Applications

- **Multiple interrupt handling:** Encode highest priority interrupt
- **Error reporting:** Report highest severity error condition
- **Resource allocation:** Grant to highest priority requester

31.7 Timing Characteristics

- **Propagation delay:** Depends on input width and synthesis
- **Critical path:** Through the for-loop comparisons
- **Setup/hold:** None (purely combinational)

31.8 Design Considerations

31.8.1 Input Validation

- **Assumes one-hot input** for standard encoder behavior
- **Handles multiple bits** gracefully (acts as priority encoder)
- **Zero input** produces zero output (may or may not be desired)

31.8.2 Synthesis Implications

- **Resource usage:** Typically synthesizes to multiplexer logic
- **Optimization:** Modern synthesizers optimize the for-loop efficiently
- **Scalability:** Performance degrades gradually with increasing N

31.9 Common Usage Patterns

31.9.1 Interrupt Controller

```
encoder #(.N(8)) int_encoder (  
    .decoded(interrupt_pending),  
    .data(interrupt_id)  
);
```

31.9.2 Arbiter Output Encoding

```
encoder #(.N(16)) grant_encoder (  
    .decoded(grant_vector),
```

```
        .data(granted_id)
    );
```

31.10 Related Modules

- **Decoder:** Performs inverse operation (binary to one-hot)
- **Priority Encoder with Enable:** Enhanced version with enable control
- **Find First Set:** Similar functionality with different search order

31.11 Behavioral Notes

- **For-loop order:** Creates LSB-to-MSB priority (higher index wins)
- **Default assignment:** Ensures output has known value for all inputs
- **Type casting:** `$clog2(N) ' (i)` ensures proper bit width matching

31.12 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

32 Priority Encoder with Enable (encoder_priority_enable.sv)

32.1 Purpose

Encodes the highest priority (highest index) asserted bit in the input vector, with an enable control signal. Unlike the basic encoder, this module explicitly searches from MSB to LSB for true priority encoding.

32.2 Ports

32.2.1 Input Ports

- **priority_in[WIDTH-1:0]** - Priority input vector (higher index = higher priority)
- **enable** - Enable signal (active high)

32.2.2 Output Ports

- **encode[\$clog2(WIDTH)-1:0]** - Binary encoded output of highest priority bit position

32.2.3 Parameters

- **WIDTH** - Input vector width (default: 8)

32.3 Implementation Details

32.3.1 Core Algorithm

logic found;

always_comb begin

// Default assignments to prevent latches

encode = '0;

found = 1'b0;

if (enable == 1'b1) **begin**

// Find the highest priority bit using found flag to prevent overwrites

for (int i = WIDTH-1; i >= 0; i--) **begin**

if (priority_in[i] == 1'b1 && !found) **begin**

encode = \$clog2(WIDTH)'(i);

found = 1'b1;

end

end

end

end

32.3.2 Operation Principle

- **True priority encoding:** Searches from MSB (highest priority) to LSB
- **Enable control:** Only operates when enable is asserted
- **Found flag:** Prevents multiple assignments, ensuring highest priority wins
- **Clean disable:** Outputs zero when disabled

32.4 Functional Behavior

32.4.1 Priority Resolution Example (WIDTH=8)

priority_in[7:0]	enable	encode[2:0]	Notes
00000001	1	000	Only bit 0 set
10000001	1	111	Bit 7 has highest priority
01010101	1	110	Bit 6 is highest set
11111111	1	111	Bit 7 wins among all
10000001	0	000	Disabled - output zero
00000000	1	000	No bits set

32.4.2 Key Differences from Basic Encoder

Feature	Basic Encoder	Priority Encoder with Enable
Search Direction	LSB to MSB	MSB to LSB
Priority	Higher index overwrites	True highest priority
Enable Control	None	Enable signal required
Found Flag	Implicit	Explicit prevention of overwrites

32.5 Design Features

32.5.1 True Priority Encoding

- **MSB-first search:** for (int i = WIDTH-1; i >= 0; i--)
- **Single assignment:** Found flag prevents multiple writes
- **Highest wins:** First match (highest index) determines output

32.5.2 Enable Control

- **Conditional operation:** Only active when enable = 1
- **Clean disable:** Output forced to zero when disabled
- **Power savings:** Can gate operation when not needed

32.5.3 Latch Prevention

// Default assignments to prevent latches

```
encode = '0;  
found = 1'b0;
```

- Ensures all signal paths have defined values
- Prevents synthesis warnings about incomplete assignments

32.6 Use Cases

32.6.1 Interrupt Controllers

```
encoder_priority_enable #(.WIDTH(8)) irq_encoder (  
    .priority_in(interrupt_requests),  
    .enable(interrupt_enable),  
    .encode(highest_priority_irq)  
);
```

32.6.2 Resource Arbitration

```
encoder_priority_enable #(.WIDTH(16)) arbiter (  
    .priority_in(request_vector),  
    .enable(arbitration_enable),  
    .encode(granted_requestor)  
);
```

32.6.3 Error Reporting

```
encoder_priority_enable #(.WIDTH(4)) error_encoder (  
    .priority_in({fatal_error, severe_error, warning, info}),  
    .enable(error_reporting_on),  
    .encode(error_level)  
);
```

32.7 Timing Characteristics

- **Propagation delay:** Depends on WIDTH and synthesis optimization
- **Critical path:** Through the priority resolution loop

- **Enable response:** Immediate when enable changes
- **Setup/hold:** None (purely combinational)

32.8 Design Considerations

32.8.1 Priority Assignment

- **Bit assignment:** Higher index = higher priority
- **Planning:** Assign critical functions to higher-indexed bits
- **Documentation:** Clear priority hierarchy essential

32.8.2 Performance Scaling

- **Linear complexity:** Search time increases with WIDTH
- **Synthesis optimization:** Modern tools optimize priority encoding well
- **Resource usage:** Typically maps to priority encoder primitives

32.8.3 Enable Usage Patterns

- **Power management:** Disable when priority encoding not needed
- **Conditional operation:** Enable based on system state
- **Test/debug:** Disable during certain test modes

32.9 Common Applications

32.9.1 Multi-Level Interrupt Controllers

- CPU interrupt prioritization
- Nested interrupt handling
- Real-time system scheduling

32.9.2 Bus Arbitration

- Multi-master bus systems
- DMA channel arbitration
- Memory access prioritization

32.9.3 Error and Exception Handling

- Exception priority resolution
- Multi-source error reporting
- Diagnostic priority encoding

32.10 Synthesis Considerations

- **Resource efficient:** Maps well to FPGA priority encoder resources
- **Timing closure:** Generally meets timing with proper constraints
- **Area scaling:** Grows approximately linearly with WIDTH

32.11 Related Modules

- **Basic Encoder:** Simpler version without enable or true priority
- **Find Last Set:** Similar MSB-first search functionality
- **Arbiter:** Often uses priority encoders for grant generation

32.12 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

33 Asynchronous FIFO - Divisible by 2 (fifo_async_div2.sv)

33.1 Purpose

Implements an asynchronous FIFO for clock domain crossing that **works with any even depth** (not restricted to powers of 2). Uses Johnson counters instead of traditional Gray codes for pointer management.

33.2 Ports

33.2.1 Write Domain

- **wr_clk** - Write domain clock

- **wr_rst_n** - Write domain active-low reset
- **write** - Write enable signal
- **wr_data[DATA_WIDTH-1:0]** - Data to write
- **wr_full** - Write domain full flag
- **wr_almost_full** - Write domain almost full flag

33.2.2 Read Domain

- **rd_clk** - Read domain clock
- **rd_rst_n** - Read domain active-low reset
- **read** - Read enable signal
- **rd_data[DATA_WIDTH-1:0]** - Data read from FIFO
- **rd_empty** - Read domain empty flag
- **rd_almost_empty** - Read domain almost empty flag

33.2.3 Parameters

- **REGISTERED** - 0=mux mode, 1=flop mode for read output
- **DATA_WIDTH** - Width of data (default: 8)
- **DEPTH** - FIFO depth - **any even number** (default: 10)
- **N_FLOP_CROSS** - Number of synchronizer stages (default: 2)
- **ALMOST_WR_MARGIN** - Almost full threshold (default: 1)
- **ALMOST_RD_MARGIN** - Almost empty threshold (default: 1)

33.3 Architecture Overview

The key innovation is the **hybrid pointer system**:

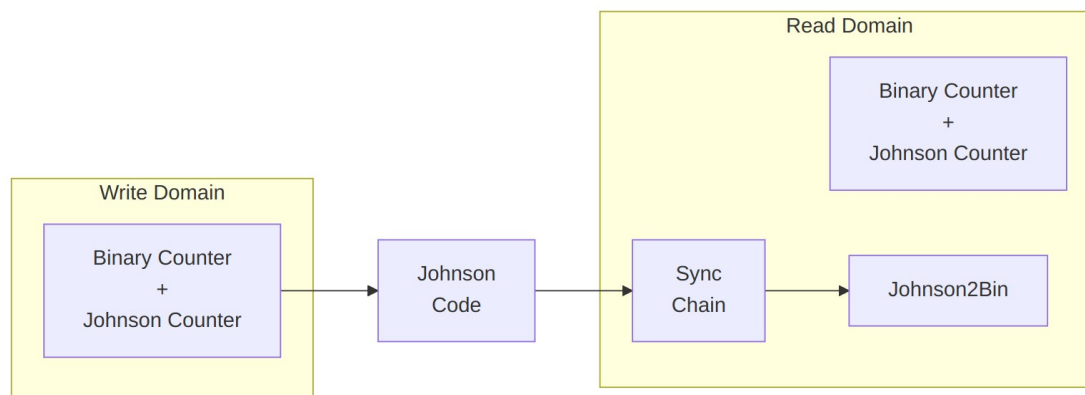


Diagram 2

33.4 Johnson Counter Deep Dive

33.4.1 What are Johnson Counters?

Johnson counters are **shift registers with inverted feedback**:

For DEPTH=6:

State 0: 000000

State 1: 100000 ← shift left, insert 1

State 2: 110000

State 3: 111000

State 4: 111100

State 5: 111110

State 6: 111111

State 7: 011111 ← shift left, insert ~MSB (0)

State 8: 001111

State 9: 000111

State 10: 000011

State 11: 000001

State 12: 000000 ← back to start

33.4.2 Johnson vs. Gray Code Comparison

Aspect	Gray Code (fifo_async)	Johnson Counter (fifo_async_div2)
Counter Width	$\lceil \log_2(\text{DEPTH}) \rceil + 1$	DEPTH bits
Depth Restriction	Powers of 2 only	Any even number
State	Binary Gray pattern	One-hot rotation

Aspect	Gray Code (fifo_async)	Johnson Counter (fifo_async_div2)
Sequence		
Memory Efficiency	Logarithmic growth	Linear growth
CDC Safety	Single bit transitions	Single bit transitions
Conversion Complexity	Simple XOR tree	Complex position detection

33.5 Implementation Details

33.5.1 Dual Counter Architecture

33.5.1.1 Binary Counters for Control

```
counter_bin #(
    .MAX(D),
    .WIDTH(AW + 1)
) wr_ptr_counter_bin(
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(write && !wr_full),
    .counter_bin_next(w_wr_ptr_bin_next),
    .counter_bin_curr(r_wr_ptr_bin)
);
```

33.5.1.2 Johnson Counters for CDC

```
counter_johnson #(
    .WIDTH(JCW) // JCW = DEPTH
) wr_ptr_counter_gray(
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(write && !wr_full),
    .counter_gray(r_wr_ptr_gray) // Johnson code output
);
```

33.5.2 Why Both Counter Types?

33.5.2.1 Binary Counters Used For:

- **Memory addressing:** Direct binary-to-address mapping
- **Occupancy calculation:** Standard arithmetic operations
- **Almost full/empty logic:** Numerical comparisons

33.5.2.2 Johnson Counters Used For:

- **Clock domain crossing:** Safe single-bit transitions
- **Synchronization:** Metastability-resistant transitions
- **Wrap detection:** Clear sequence boundaries

33.6 Johnson-to-Binary Conversion

33.6.1 Conversion Challenge

Converting Johnson counter states back to binary addresses requires **position detection**:

```
// Johnson state: 111100000 (for DEPTH=10)
// Need to find: position of transition (leading 1s to trailing 0s)
// Result: Binary address for memory access
```

33.6.2 grayj2bin Module Implementation

```
// Key conversion logic
if (w_all_zeroes || w_all_ones) begin
    w_binary = {WIDTH{1'b0}};
end else if (gray[JCW-1]) begin
    // Second half: use leading one position directly
    w_binary = {{(WIDTH-N){1'b0}}, w_trailing_one};
end else begin
    // First half: use trailing one + 1
    w_binary = {{(WIDTH-N){1'b0}}, (w_leading_one + 1'b1)};
end
```

33.6.3 Leading/Trailing One Detection

The conversion relies on `leading_one_trailing_one` module: - **Leading one:** Position of leftmost '1' bit - **Trailing one:** Position of rightmost '1' bit
- **All zeros/ones:** Special case handling - **Valid flag:** Ensures proper Johnson sequence

33.7 Flexible Depth Support

33.7.1 Even Number Requirement

Johnson counters naturally support even depths because: - **Sequence length**: $2 \times \text{DEPTH}$ states in full cycle - **Half cycles**: Each half contains DEPTH unique states - **Symmetry**: Clean wraparound behavior

33.7.2 Example Depth Flexibility

```
// Valid depths for fifo_async_div2:
parameter int DEPTH = 6;    // ✓ (even)
parameter int DEPTH = 10;   // ✓ (even)
parameter int DEPTH = 14;   // ✓ (even)
parameter int DEPTH = 100;  // ✓ (even)
```

```
// Invalid depths:
parameter int DEPTH = 7;    // ✗ (odd)
parameter int DEPTH = 15;   // ✗ (odd)
```

33.7.3 vs. Power-of-2 Restriction

```
// fifo_async valid depths: 2, 4, 8, 16, 32, 64, ...
// fifo_async_div2 valid depths: 2, 4, 6, 8, 10, 12, 14, 16, 18, ...
```

33.8 Memory Efficiency Trade-offs

33.8.1 Resource Usage Comparison

FIFO Depth	Gray Code Width	Johnson Width	Efficiency Ratio
4	3 bits	4 bits	1.33×
8	4 bits	8 bits	2.00×
16	5 bits	16 bits	3.20×
32	6 bits	32 bits	5.33×
64	6 bits	64 bits	10.67×

33.8.2 When to Choose Each Approach

33.8.2.1 Choose *fifo_async* (Gray Code) When:

- **Large depths**: >32 words (better resource efficiency)
- **Standard depths**: Powers of 2 are acceptable
- **Resource constrained**: Minimizing logic usage critical

33.8.2.2 Choose `fifo_async_div2` (Johnson) When:

- **Specific depths:** Need exact non-power-of-2 sizes
- **Small to medium depths:** <32 words (manageable overhead)
- **System requirements:** Exact depth matching critical

33.9 Clock Domain Crossing Details

33.9.1 Synchronization Strategy

Same multi-stage synchronizer approach as Gray code version:

```
glitch_free_n_dff_arn #(
    .FLOP_COUNT(N_FLOP_CROSS),
    .WIDTH(JCW) // Note: wider than Gray code version
) rd_ptr_gray_cross_inst(
    .q(r_wdom_rd_ptr_gray),
    .d(r_rd_ptr_gray),
    .clk(wr_clk),
    .rst_n(wr_rst_n)
);
```

33.9.2 Metastability Protection

- **Single bit transitions:** Johnson counters guarantee this
- **Multi-stage sync:** Standard 2-3 stage synchronizers
- **Conversion timing:** Johnson-to-binary happens after synchronization

33.10 Use Cases

33.10.1 Ideal Applications

- **Video buffers:** Need specific line/frame buffer sizes
- **Audio processing:** Exact sample count requirements
- **Protocol buffers:** Match specific packet/frame sizes
- **Memory interfaces:** Align with specific burst lengths

33.10.2 Application Examples

```
// Video line buffer (1920 pixels)
fifo_async_div2 #(.DEPTH(1920)) line_buffer (...);
```



```
// Audio sample buffer (48 samples)
fifo_async_div2 #(.DEPTH(48)) audio_buffer (...);

// Packet buffer (1500 bytes)
fifo_async_div2 #(.DEPTH(1500)) packet_buffer (...);
```

33.11 Performance Characteristics

33.11.1 Throughput

- **Same as Gray version:** 1 operation per clock per domain
- **No performance penalty:** CDC latency identical

33.11.2 Resource Usage

- **Higher logic usage:** Linear scaling with depth
- **Same memory usage:** Identical memory array
- **Conversion overhead:** Additional logic for Johnson-to-binary

33.11.3 Timing

- **CDC latency:** Same 2-3 cycle synchronizer delay
- **Conversion delay:** Additional combinational path
- **Critical paths:** May be longer due to conversion logic

33.12 Design Considerations

33.12.1 Depth Selection Guidelines

- **Small depths (<32):** Consider div2 for exact sizing
- **Large depths (>64):** Prefer power-of-2 for efficiency
- **Medium depths:** Evaluate specific size vs. efficiency needs

33.12.2 Resource Planning

- **Logic overhead:** Budget for Johnson counter width
- **Timing closure:** Account for conversion logic delay
- **Power consumption:** Wider counters consume more power

33.13 WaveDrom Visualization

High-quality waveforms showcasing Johnson counter CDC mechanism are available!

Run the WaveDrom test to generate detailed timing diagrams:

```
# Generate Johnson counter waveforms (showcases unique CDC implementation)
pytest val/common/test_fifo_async_div2_wavedrom.py -v
```

Waveform Scenarios Generated:

1. Write-Fill-Read-Empty Cycle

- Complete FIFO lifecycle
- Johnson counter state progression
- Flag transitions (wr_full, rd_empty)
- Demonstrates end-to-end operation

2. Cross-Domain Synchronization ★ KEY SCENARIO

- Johnson counter transitions (single-bit changes only)
- Write pointer sync from wr_clk to rd_clk domain
- Read pointer sync from rd_clk to wr_clk domain
- CDC latency visualization
- **THIS IS THE UNIQUE FEATURE** distinguishing from Gray code

3. Back-to-Back Operations

- Dual pointer architecture in action
- Binary pointers for memory addressing
- Johnson pointers for CDC safety
- Simultaneous read/write capability

4. Almost-Full/Almost-Empty Flags

- Flow control signaling

- Threshold-based flag assertion
- Proactive backpressure indication

What Makes These Waveforms Special:

The test specifically highlights the **Johnson counter CDC mechanism** which is the unique differentiator of `fifo_async_div2`:

- **Johnson Counter States:** Shows the one-hot-like rotation pattern
- **Single-Bit Transitions:** Demonstrates CDC safety (only one bit changes at a time)
- **Even-Depth Flexibility:** Works with depths like 6, 10, 14 (not just powers of 2)
- **Dual Pointer System:** Binary for addressing, Johnson for CDC

Comparison with Standard Async FIFO:

For comparison, also see: - `test_fifo_async_wavedrom.py` - Standard Gray code CDC (power-of-2 depths) - `test_fifo_sync_wavedrom.py` - Synchronous FIFO (single clock, no CDC)

33.14 Error Detection

Same simulation-based error checking as other FIFO variants: - Write-while-full detection - Read-while-empty detection - Timing-aware error reporting

33.15 Related Modules

- **fifo_async:** Power-of-2 version using Gray codes
- **counter_johnson:** Johnson counter implementation
- **grayj2bin:** Johnson-to-binary converter
- **leading_one_trailing_one:** Position detection helper

33.16 Test and Verification

Comprehensive Test Suite: - `val/common/test_fifo_buffer_async_div2.py` - Full functional verification - `val/common/test_fifo_async_div2_wavedrom.py` - WaveDrom timing diagrams ★

Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_fifo_buffer_async_div2.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_fifo_async_div2_wavedrom.py -v
```

33.17 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

34 Asynchronous FIFO - Power of 2 (fifo_async.sv)

34.1 Purpose

Implements an asynchronous FIFO for safe clock domain crossing between different clock domains. **Restricted to power-of-2 depths only** due to Gray code pointer implementation.

34.2 Ports

34.2.1 Write Domain

- **wr_clk** - Write domain clock
- **wr_rst_n** - Write domain active-low reset
- **write** - Write enable signal
- **wr_data[DATA_WIDTH-1:0]** - Data to write
- **wr_full** - Write domain full flag
- **wr_almost_full** - Write domain almost full flag

34.2.2 Read Domain

- **rd_clk** - Read domain clock
- **rd_rst_n** - Read domain active-low reset
- **read** - Read enable signal
- **rd_data[DATA_WIDTH-1:0]** - Data read from FIFO

- **rd_empty** - Read domain empty flag
- **rd_almost_empty** - Read domain almost empty flag

34.2.3 Parameters

- **REGISTERED** - 0=mux mode, 1=flop mode for read output
- **DATA_WIDTH** - Width of data (default: 8)
- **DEPTH** - FIFO depth - **must be power of 2** (default: 16)
- **N_FLOP_CROSS** - Number of synchronizer stages (default: 2)
- **ALMOST_WR_MARGIN** - Almost full threshold (default: 1)
- **ALMOST_RD_MARGIN** - Almost empty threshold (default: 1)

34.3 Architecture Overview

34.3.1 Clock Domain Crossing Strategy

The FIFO uses **Gray code pointers** for safe clock domain crossing:

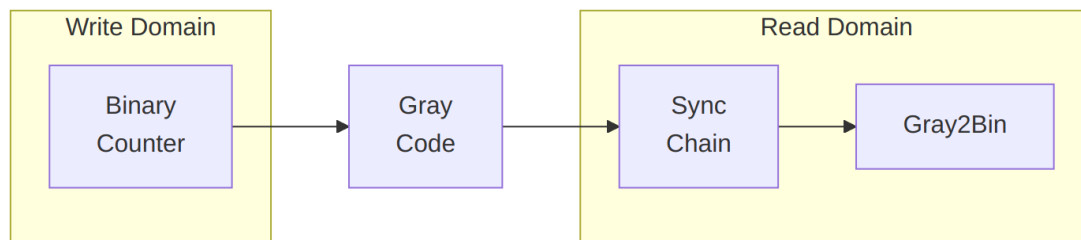


Diagram 3

34.3.2 Core Components

1. **Binary-Gray counters** (counter_bingray) for pointer generation
2. **Multi-stage synchronizers** (glitch_free_n_dff_arn) for CDC
3. **Gray-to-binary converters** (gray2bin) for pointer comparison
4. **Shared memory array** accessible from both domains
5. **FIFO control logic** for status flag generation

34.4 Implementation Deep Dive

34.4.1 Gray Code Pointer System

34.4.1.1 Why Gray Codes?

Gray codes ensure **only one bit changes** per increment: - **Binary**: 011 → 100 (3 bits change simultaneously) - **Gray**: 010 → 110 (only 1 bit changes) - **Benefit**: Eliminates metastability from multi-bit transitions

34.4.1.2 Pointer Architecture

```
// Write domain Gray counter
counter_bingray #(.WIDTH(AW + 1)) wr_ptr_counter_gray (
    .clk(wr_clk),
    .rst_n(wr_rst_n),
    .enable(write && !wr_full),
    .counter_bin(r_wr_ptr_bin),           // Binary for addressing
    .counter_bin_next(w_wr_ptr_bin_next), // Next binary value
    .counter_gray(r_wr_ptr_gray)         // Gray for CDC
);
```

34.4.1.3 Pointer Width Calculation

- **Address width**: $AW = \lceil \log_2(DEPTH) \rceil$
- **Pointer width**: $AW + 1$ (extra bit for wrap detection)
- **Example**: $DEPTH=16 \rightarrow AW=4$, Pointer=5 bits

34.4.2 Clock Domain Crossing

34.4.2.1 Synchronizer Chains

```
// Cross read pointer to write domain
glitch_free_n_dff_arn #(
    .FLOP_COUNT(N_FLOP_CROSS),
    .WIDTH(AW + 1)
) rd_ptr_gray_cross_inst (
    .q(r_wdom_rd_ptr_gray), // Synchronized output
    .d(r_rd_ptr_gray),      // Gray pointer input
    .clk(wr_clk),           // Destination clock
    .rst_n(wr_rst_n)        // Destination reset
);
```

34.4.2.2 Multi-Stage Synchronization

- **Default stages**: 2 flip-flops ($N_FLOP_CROSS=2$)
- **MTBF improvement**: Each stage reduces metastability probability

- **Latency trade-off:** More stages = better MTBF but higher latency

34.4.3 Memory Organization

34.4.3.1 Dual-Port Memory

```

logic [DW-1:0] r_mem[0:((1<<AW)-1)]; // Memory array

// Write port (write domain)
always_ff @(posedge wr_clk) begin
    if (write) begin
        r_mem[r_wr_addr] <= wr_data;
    end
end

// Read port (read domain)
assign w_rd_data = r_mem[r_rd_addr];

```

34.4.3.2 Address Generation

- **Write address:** r_wr_addr = r_wr_ptr_bin[AW-1:0]
- **Read address:** r_rd_addr = r_rd_ptr_bin[AW-1:0]
- **Truncation:** Uses only lower bits for memory indexing

34.4.4 Full/Empty Detection

34.4.4.1 Full Detection Logic

```

// In write domain
assign wr_full = (w_wdom_ptr_xor &&
    (wr_ptr_bin[AW-1:0] == wdom_rd_ptr_bin[AW-1:0]));

// Where:
assign w_wdom_ptr_xor = wr_ptr_bin[AW] ^ wdom_rd_ptr_bin[AW];

```

34.4.4.2 Empty Detection Logic

```

// In read domain
assign rd_empty = (!w_rdom_ptr_xor_for_empty &&
    (rd_ptr_bin[AW:0] == w_wr_ptr_for_empty[AW:0]));

```

34.4.4.3 Full/Empty Algorithm

- **Full condition:** MSBs differ AND LSBs equal (write caught up after wrap)
- **Empty condition:** All bits equal (pointers at same location)
- **MSB significance:** Indicates which pointer has wrapped around

34.5 Power-of-2 Requirement

34.5.1 Why Power-of-2 Only?

1. **Gray code properties:** Natural binary-Gray relationship
2. **Wraparound behavior:** Clean modulo- 2^n arithmetic
3. **Address truncation:** Simple bit slicing for memory addressing
4. **Pointer comparison:** Efficient full/empty detection

34.5.2 Limitation Impact

```
// Valid depths: 2, 4, 8, 16, 32, 64, 128, 256, ...  
// Invalid depths: 3, 5, 6, 7, 9, 10, 12, 15, ...
```

34.6 Timing Considerations

34.6.1 Clock Domain Crossing Latency

- **Pointer propagation:** 2-3 clock cycles (depending on N_FLOP_CROSS)
- **Status flag delay:** Flags reflect state with synchronizer latency
- **Conservative design:** Prevents overflow/underflow despite delay

34.6.2 Metastability Protection

- **Gray code transitions:** Single bit changes only
- **Multi-stage sync:** Reduces MTBF exponentially
- **Setup/hold margins:** Proper timing constraints essential

34.7 Use Cases

34.7.1 Typical Applications

- **Video processing:** Different pixel and memory clock domains
- **Networking:** Packet buffers between different rate domains
- **Audio systems:** Sample rate conversion buffers
- **Microprocessor interfaces:** CPU and peripheral clock domains

34.7.2 When to Use vs. Alternatives

- **Use async FIFO when:** Clock domains are truly independent
- **Use sync FIFO when:** Single clock domain sufficient
- **Use async_div2 when:** Need non-power-of-2 depth

34.8 Design Guidelines

34.8.1 Depth Sizing

```
// Calculate minimum depth for burst handling
// DEPTH ≥ burst_size + synchronizer_latency + margin
parameter int MIN_DEPTH = 16; // Typical minimum for async FIFOs
```

34.8.2 Clock Relationship

- **Asynchronous clocks:** No phase relationship assumed
- **Clock gating:** Avoid gating clocks used by FIFO
- **Reset deassertion:** Ensure proper reset release sequencing

34.8.3 Almost Full/Empty Settings

- **Almost full:** DEPTH - ALMOST_WR_MARGIN
- **Almost empty:** ALMOST_RD_MARGIN
- **Guideline:** Set margins > synchronizer latency

34.9 Performance Characteristics

- **Throughput:** Up to 1 operation per clock per domain
- **Latency:** 0-1 cycles (depending on REGISTERED mode)
- **CDC latency:** 2-3 cycles for status propagation
- **Efficiency:** ~100% utilization possible

34.10 Error Detection Features

```
// Simulation-only error checking
always_ff @(posedge wr_clk) begin
    if (!wr_rst_n && (write && wr_full) == 1'b1) begin
```

```

        $display("Error: %s write while fifo full, %t", INSTANCE_NAME,
$time);
    end
end

```

34.11 WaveDrom Visualization

High-quality waveforms showcasing Gray code CDC mechanism are available!

Run the WaveDrom test to generate detailed timing diagrams:

```

# Generate Gray code waveforms (standard async FIFO implementation)
pytest val/common/test_fifo_async_wavedrom.py -v

```

Waveform Scenarios Generated:

1. Write-Fill-Read-Empty Cycle

- Standard async FIFO operation
- Gray code pointer progression
- Power-of-2 depth utilization

2. Gray Code Synchronization

- Efficient Gray code CDC
- Logarithmic pointer width (vs. linear for Johnson)
- Cross-domain pointer transitions

3. Power-of-2 Depth Utilization

- Efficient addressing with power-of-2 depths
- Full depth wraparound behavior
- Resource-efficient pointer management

Key Differences from Johnson Counter Variant:

- **Pointer Width:** Logarithmic ($\lceil \log_2(\text{DEPTH}) \rceil + 1$) vs. linear (DEPTH) for Johnson
- **Depth Restriction:** Power-of-2 only vs. any even number for Johnson

- **Resource Efficiency:** Better for large depths (>32) vs. Johnson's flexibility for small depths

Comparison Tests:

- `test_fifo_async_div2_wavedrom.py` - Johnson counter CDC (flexible even depths)
- `test_fifo_sync_wavedrom.py` - Synchronous FIFO (single clock, no CDC)

34.12 Related Modules

- **`fifo_async_div2`:** For non-power-of-2 depths using Johnson counters
- **`fifo_sync`:** For single clock domain applications
- **`counter_bingray`:** Binary-Gray counter implementation
- **`glitch_free_n_dff_arn`:** Multi-stage synchronizer

34.13 Test and Verification

Comprehensive Test Suite: - `val/common/test_fifo_buffer_async.py` - Full functional verification - `val/common/test_fifo_async_wavedrom.py` - WaveDrom timing diagrams

Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_fifo_buffer_async.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_fifo_async_wavedrom.py -v
```

34.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

35 FIFO Control Logic (`fifo_control.sv`)

35.1 Purpose

Centralized control logic module that generates full/empty status flags for all FIFO variants (sync, async, async_div2). Handles the complex pointer arithmetic and mode-aware timing considerations.

35.2 Ports

35.2.1 Clock and Reset

- `wr_clk` - Write domain clock
- `wr_rst_n` - Write domain active-low reset
- `rd_clk` - Read domain clock
- `rd_rst_n` - Read domain active-low reset

35.2.2 Pointer Inputs

- `wr_ptr_bin[ADDR_WIDTH:0]` - Write pointer (binary, next value)
- `wdom_rd_ptr_bin[ADDR_WIDTH:0]` - Read pointer synchronized to write domain
- `rd_ptr_bin[ADDR_WIDTH:0]` - Read pointer (binary, next value)
- `rdom_wr_ptr_bin[ADDR_WIDTH:0]` - Write pointer synchronized to read domain

35.2.3 Status Outputs

- `count[ADDR_WIDTH:0]` - Current FIFO occupancy count
- `wr_full` - Write domain full flag
- `wr_almost_full` - Write domain almost full flag
- `rd_empty` - Read domain empty flag
- `rd_almost_empty` - Read domain almost empty flag

35.2.4 Parameters

- `ADDR_WIDTH` - Address width (default: 3)

- **DEPTH** - FIFO depth (default: 16)
- **ALMOST_WR_MARGIN** - Almost full threshold (default: 1)
- **ALMOST_RD_MARGIN** - Almost empty threshold (default: 1)
- **REGISTERED** - Output mode: 0=mux, 1=flop (default: 0)

35.3 Architecture Overview

35.3.1 Dual-Domain Design

The module operates across two clock domains: - **Write domain**: Generates full and almost_full flags - **Read domain**: Generates empty and almost_empty flags

35.3.2 Pointer Arithmetic Foundation

All status generation relies on **pointer comparison with wraparound detection**:

```
// XOR the MSBs to detect wraparound condition
assign w_wdom_ptr_xor = wr_ptr_bin[AW] ^ wdom_rd_ptr_bin[AW];
assign w_rdom_ptr_xor = rd_ptr_bin[AW] ^ rdom_wr_ptr_bin[AW];
```

35.4 Full Detection Logic

35.4.1 Basic Full Condition

```
assign w_wr_full_d = (w_wdom_ptr_xor &&
                     (wr_ptr_bin[AW-1:0] == wdom_rd_ptr_bin[AW-1:0]));
```

35.4.2 Full Detection Algorithm

- **Condition 1**: MSBs must differ ($w_wdom_ptr_xor = 1$)
- **Condition 2**: LSBs must be equal
- **Meaning**: Write pointer has “lapped” the read pointer

35.4.3 Visual Example (DEPTH=8, ADDR_WIDTH=3)

Pointers: [wrap_bit][address_bits]

Case 1: Not Full

```
wr_ptr = 0_101 (address 5, no wrap)
rd_ptr = 0_010 (address 2, no wrap)
→ MSBs same, not full
```

Case 2: Full

```
wr_ptr = 1_010 (address 2, wrapped once)
```

rd_ptr = 0_010 (address 2, not wrapped)
→ MSBs differ, addresses same = FULL

35.5 Almost Full Logic

35.5.1 Occupancy Calculation

```
assign w_almost_full_count = (w_wdom_ptr_xor) ?  
    (AW'(D) - wdom_rd_ptr_bin[AW-1:0] + wr_ptr_bin[AW-1:0]) :  
    (wr_ptr_bin[AW-1:0] - wdom_rd_ptr_bin[AW-1:0]);
```

35.5.2 Two Cases Handled

1. **No wraparound** ($w_wdom_ptr_xor = 0$):
 - Count = $wr_ptr - rd_ptr$
 - Simple subtraction
2. **Wraparound** ($w_wdom_ptr_xor = 1$):
 - Count = $DEPTH - rd_ptr + wr_ptr$
 - Accounts for circular buffer wraparound

35.5.3 Almost Full Threshold

```
assign w_wr_almost_full_d = w_almost_full_count >= AW'(AFT);  
// Where AFT = DEPTH - ALMOST_WR_MARGIN
```

35.6 Empty Detection Logic - Mode Aware

35.6.1 Critical Innovation: Mode-Aware Write Pointer Selection

```
generate  
    if (REGISTERED == 1) begin : gen_flop_mode  
        // FLOP mode: Use previous cycle's write pointer  
        logic [ADDR_WIDTH:0] r_rdom_wr_ptr_bin_delayed;  
  
        always_ff @(posedge rd_clk or negedge rd_rst_n) begin  
            if (!rd_rst_n) begin  
                r_rdom_wr_ptr_bin_delayed <= '0;  
            end else begin  
                r_rdom_wr_ptr_bin_delayed <= rdom_wr_ptr_bin;  
            end  
        end  
  
        assign w_wr_ptr_for_empty = r_rdom_wr_ptr_bin_delayed;  
    end else begin : gen_mux_mode
```

```

        // MUX mode: Use current write pointer
        assign w_wr_ptr_for_empty = rdom_wr_ptr_bin;
    end
endgenerate

```

35.6.2 Why Mode-Aware Detection?

35.6.2.1 MUX Mode (REGISTERED = 0)

- **Data availability:** Immediate (combinational read)
- **Write pointer:** Use current value
- **Reasoning:** Data is immediately available when written

35.6.2.2 FLOP Mode (REGISTERED = 1)

- **Data availability:** Delayed by 1 cycle (registered read)
- **Write pointer:** Use delayed value
- **Reasoning:** Data isn't available until next clock cycle

35.6.3 Empty Detection Algorithm

```

assign w_rd_empty_d = (!w_rdom_ptr_xor_for_empty &&
    (rd_ptr_bin[AW:0] == w_wr_ptr_for_empty[AW:0]));

```

- **Condition 1:** MSBs must be same (no wrap difference)
- **Condition 2:** All bits must be equal
- **Meaning:** Read pointer has caught up to write pointer

35.7 Almost Empty Logic

35.7.1 Standard Timing (Mode-Independent)

Almost empty calculation uses standard timing regardless of FIFO mode:

```

assign w_almost_empty_count = (w_rdom_ptr_xor) ?
    (AW'(D) - rd_ptr_bin[AW-1:0] + rdom_wr_ptr_bin[AW-1:0]) :
    (rdom_wr_ptr_bin[AW-1:0] - rd_ptr_bin[AW-1:0]);

assign w_rd_almost_empty_d = w_almost_empty_count <= AW'(AET);
// Where AET = ALMOST_RD_MARGIN

```

35.8 Count Generation

35.8.1 Occupancy Count Logic

```
assign count = (w_rdom_ptr_xor) ?  
    (rdom_wr_ptr_bin[AW-1:0] - rd_ptr_bin[AW-1:0] + AW'(D)) :  
    (rdom_wr_ptr_bin[AW-1:0] - rd_ptr_bin[AW-1:0]);
```

35.8.2 Count Interpretation

- **Range:** 0 to DEPTH
- **Zero:** FIFO empty
- **DEPTH:** FIFO full
- **Uses:** Flow control, occupancy monitoring

35.9 Flag Registration

35.9.1 Write Domain Flags

```
always_ff @(posedge wr_clk, negedge wr_rst_n) begin  
    if (!wr_rst_n) begin  
        wr_full <= 'b0;  
        wr_almost_full <= 'b0;  
    end else begin  
        wr_full <= w_wr_full_d;  
        wr_almost_full <= w_wr_almost_full_d;  
    end  
end
```

35.9.2 Read Domain Flags

```
always_ff @(posedge rd_clk, negedge rd_rst_n) begin  
    if (!rd_rst_n) begin  
        rd_empty <= 'b1;           // Reset to empty  
        rd_almost_empty <= 'b0;  
    end else begin  
        rd_empty <= w_rd_empty_d;  
        rd_almost_empty <= w_rd_almost_empty_d;  
    end  
end
```

35.9.3 Reset Behavior

- **Full flags:** Reset to 0 (not full)
- **Empty flags:** Reset to 1 (empty)

- **Almost flags:** Reset to 0

35.10 Width Casting Fix

35.10.1 Type Width Matching

```
// Fixed: Cast D to AW-bit width to match other operands
assign w_almost_full_count = (w_wdom_ptr_xor) ?
    (AW'(D) - wdom_rd_ptr_bin[AW-1:0] + wr_ptr_bin[AW-1:0]) :
    (wr_ptr_bin[AW-1:0] - wdom_rd_ptr_bin[AW-1:0]);
```

The AW'(D) casting ensures all operands have matching bit widths, preventing synthesis warnings.

35.11 Key Design Insights

35.11.1 Wraparound Handling

- **MSB significance:** Extra bit detects wraparound events
- **Circular arithmetic:** Proper modulo DEPTH calculations
- **Comparison logic:** XOR-based wraparound detection

35.11.2 Timing Considerations

- **Synchronizer delay:** Accounted for in conservative design
- **Mode awareness:** Different timing for different output modes
- **Flag updates:** Registered for clean transitions

35.11.3 Conservative Design

- **Over-reporting:** Flags may assert slightly early
- **Safety margin:** Prevents overflow/underflow
- **Synchronizer latency:** Built into safety margins

35.12 Use Cases

- **All FIFO variants:** Shared by sync, async, and async_div2
- **Status monitoring:** Provides comprehensive FIFO state
- **Flow control:** Enables back-pressure and rate matching

- **Debug/verification:** Count output aids in debugging

35.13 Performance Impact

- **Minimal overhead:** Efficient pointer arithmetic
- **Low latency:** Single cycle flag updates
- **Resource efficient:** Shared across multiple FIFO types

35.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

36 Synchronous FIFO (`fifo_sync.sv`)

36.1 Purpose

Implements a synchronous First-In-First-Out buffer for single clock domain applications. Provides configurable depth, data width, and output modes with comprehensive status signaling.

36.2 Ports

36.2.1 Input Ports

- **clk** - System clock
- **rst_n** - Active-low reset
- **write** - Write enable signal
- **wr_data[DATA_WIDTH-1:0]** - Data to write into FIFO
- **read** - Read enable signal

36.2.2 Output Ports

- **wr_full** - Write domain full flag
- **wr_almost_full** - Write domain almost full flag
- **rd_data[DATA_WIDTH-1:0]** - Data read from FIFO
- **rd_empty** - Read domain empty flag

- **rd_almost_empty** - Read domain almost empty flag

36.2.3 Parameters

- **REGISTERED** - Output mode: 0=mux mode (combinational), 1=flop mode (registered)
- **DATA_WIDTH** - Width of data bus (default: 4)
- **DEPTH** - FIFO depth in words (default: 4)
- **ALMOST_WR_MARGIN** - Almost full threshold (default: 1)
- **ALMOST_RD_MARGIN** - Almost empty threshold (default: 1)
- **INSTANCE_NAME** - Debug identifier (default: "DEADF1F0")

36.3 Architecture Overview

36.3.1 Core Components

1. **Binary counters** for read/write pointers
2. **Memory array** for data storage
3. **FIFO control logic** for status generation
4. **Output multiplexing** based on REGISTERED parameter

36.3.2 Memory Organization

```
logic [DW-1:0] r_mem[0:((1<<AW)-1)]; // Memory array
assign r_wr_addr = r_wr_ptr_bin[AW-1:0]; // Write address
assign r_rd_addr = r_rd_ptr_bin[AW-1:0]; // Read address
```

36.4 Implementation Details

36.4.1 Pointer Management

Uses binary counters for both read and write pointers:

```
counter_bin #(.WIDTH(AW + 1), .MAX(D)) write_pointer_inst (
    .clk(clk),
    .rst_n(rst_n),
    .enable(write && !wr_full),
    .counter_bin_curr(r_wr_ptr_bin),
    .counter_bin_next(w_wr_ptr_bin_next)
);
```

36.4.1.1 *Pointer Characteristics*

- **Width:** $\$clog2(DEPTH) + 1$ bits (extra bit for wrap detection)
- **Increment:** Only when operation is valid (write && !full, read && !empty)
- **Wraparound:** Automatic modulo DEPTH counting

36.4.2 Memory Operations

36.4.2.1 *Write Operation*

```
always_ff @(posedge clk) begin
    if (write) begin
        r_mem[r_wr_addr] <= wr_data;
    end
end
```

36.4.2.2 *Read Operation - Dual Mode*

```
generate
    if (REGISTERED != 0) begin : gen_flop_mode
        // Flop mode - registered output
        always_ff @(posedge clk or negedge rst_n) begin
            if (!rst_n)
                rd_data <= 'b0;
            else
                rd_data <= w_rd_data;
            end
        end else begin : gen_mux_mode
            // Mux mode - non-registered output
            assign rd_data = w_rd_data;
        end
    end
endgenerate
```

36.4.3 Status Flag Generation

Utilizes shared `fifo_control` module: - **Full detection:** Based on pointer comparison with wraparound handling - **Almost full:** When remaining space $\leq$ ALMOST_WR_MARGIN - **Empty detection:** When read pointer equals write pointer - **Almost empty:** When available data $\leq$ ALMOST_RD_MARGIN

36.5 Operating Modes

36.5.1 Mux Mode (REGISTERED = 0)

- **Read latency:** 0 cycles (combinational output)
- **Data availability:** Immediate after write

- **Use case:** Low-latency applications
- **Timing:** Read data changes combinatorially with address

36.5.2 Flop Mode (REGISTERED = 1)

- **Read latency:** 1 cycle (registered output)
- **Data availability:** 1 cycle after read enable
- **Use case:** High-speed designs, timing closure
- **Timing:** Read data stable for full clock cycle

36.6 Functional Behavior

36.6.1 Write Operations

- **Condition:** write && !wr_full
- **Action:** Store data at write pointer, increment pointer
- **Blocking:** Writes ignored when FIFO is full
- **Status:** Full flags update based on new occupancy

36.6.2 Read Operations

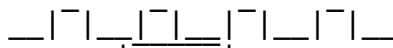
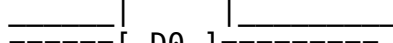
- **Condition:** read && !rd_empty
- **Action:** Advance read pointer (data handling depends on mode)
- **Blocking:** Reads ignored when FIFO is empty
- **Status:** Empty flags update based on new occupancy

36.6.3 Reset Behavior

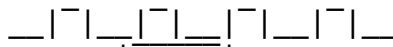
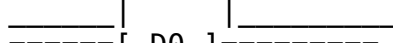
- **Pointers:** Reset to 0
- **Flags:** Full flags → 0, Empty flags → 1
- **Data:** Read data cleared in flop mode
- **Memory:** Contents undefined but not critical

36.7 Timing Diagrams

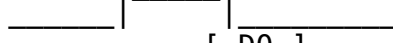
36.7.1 Write Sequence

clk 
write 
wr_data =====[D0]=====
wr_full _____|⁻ (when FIFO becomes full)

36.7.2 Read Sequence - Mux Mode

clk 
read 
rd_data =====[D0]===== (immediate)

36.7.3 Read Sequence - Flop Mode

clk 
read 
rd_data =====[D0]===== (1 cycle delay)

36.8 Use Cases

36.8.1 Low Latency Applications (Mux Mode)

- Data streaming with minimal delay
- Real-time processing pipelines
- Clock domain buffers

36.8.2 High Speed Applications (Flop Mode)

- High-frequency designs
- Timing-critical paths
- Pipelined architectures

36.9 Design Considerations

36.9.1 Mode Selection Guidelines

- **Choose Mux Mode when:** Latency is critical, moderate clock speeds
- **Choose Flop Mode when:** High clock speeds, timing closure issues
- **Performance impact:** Flop mode adds 1 cycle latency but improves fmax

36.9.2 Sizing Considerations

- **Depth:** Must accommodate worst-case burst sizes
- **Almost flags:** Set margins based on producer/consumer response times
- **Data width:** Should match datapath requirements

36.9.3 Error Detection

Built-in simulation checks:

```
always_ff @(posedge clk) begin
    if ((write && wr_full) == 1'b1) begin
        $display("Error: %s write while fifo full, %t", INSTANCE_NAME,
$time);
    end
end
```

36.10 Performance Characteristics

- **Throughput:** 1 operation per clock cycle (when not full/empty)
- **Latency:** 0 cycles (mux) or 1 cycle (flop) for read
- **Occupancy:** 0 to DEPTH words
- **Efficiency:** 100% bandwidth utilization possible

36.11 WaveDrom Visualization

High-quality waveforms showcasing synchronous FIFO operation are available!

Run the WaveDrom test to generate detailed timing diagrams:

```
# Generate synchronous FIFO waveforms (single clock domain)
pytest val/common/test_fifo_sync_wavedrom.py -v
```

Waveform Scenarios Generated:

1. Write-Fill-Read-Empty Cycle

- Basic synchronous FIFO operation
- Simple binary pointer management
- No CDC complexity (single clock domain)

2. Back-to-Back Operations

- Maximum throughput demonstration
- Minimal inter-transaction delays
- Sequential write and read bursts

3. Simultaneous Write-Read (Ping-Pong)

- Read and write in same cycle
- Steady-state FIFO operation
- Demonstrating simultaneous access capability

4. Flag Transitions

- Full/empty flag behavior at boundaries
- Almost-full/almost-empty thresholds
- Flow control signaling

Key Characteristics vs. Async FIFOs:

- **Single Clock Domain:** No CDC complexity, simpler design
- **Binary Pointers:** Direct addressing, no Gray code conversion
- **Zero CDC Latency:** Flags update immediately (no synchronization delay)
- **Simultaneous Access:** Can read and write in same cycle

Comparison Tests:

- `test_fifo_async_wavedrom.py` - Gray code CDC (power-of-2 depths)
- `test_fifo_async_div2_wavedrom.py` - Johnson counter CDC (flexible even depths)

36.12 Related Modules

- **fifo_async:** For clock domain crossing applications
- **fifo_control:** Shared status flag generation logic
- **counter_bin:** Binary counter implementation

36.13 Test and Verification

Comprehensive Test Suite: - val/common/test_fifo_buffer.py - Full functional verification - val/common/test_fifo_sync_wavedrom.py - WaveDrom timing diagrams

Run Tests:

```
# Full functional test (basic/medium/full levels)
pytest val/common/test_fifo_buffer.py -v
```

```
# WaveDrom waveform generation
pytest val/common/test_fifo_sync_wavedrom.py -v
```

36.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

37 Multi-Field Synchronous FIFO

Module: fifo_sync_multi.sv **Location:** rtl/common/ **Status:** Production Ready

37.1 Overview

fifo_sync_multi is a thin convenience wrapper around fifo_sync that lets a caller push and pop several **named fields** through one synchronous FIFO without hand-packing them into a single wide bus. It concatenates an address field, a control field, and two data words into one fifo_sync payload on the write side, and splits the payload back out into the same named fields on the read side.

The storage, pointer management, and full/empty/almost flag generation are entirely provided by the underlying fifo_sync instance. This wrapper adds only the field-packing wiring.

37.1.1 Key Features

- **Field-oriented interface:** Separate wr_addr / wr_ctrl / wr_data0 / wr_data1 write ports and matching read ports
- **Single storage instance:** All fields ride one fifo_sync with a combined AW + CW + DW + DW payload
- **Inherited status flags:** wr_full, wr_almost_full, rd_empty, rd_almost_empty come straight from fifo_sync

- **Mux / flop output modes:** REGISTERED selects combinational (mux) or registered (flop) read output
- **Any depth:** Depth is passed through to `fifo_sync`

37.2 Module Purpose

Datapaths frequently move a small bundle of related fields together (for example an address plus a control tag plus a pair of data beats). Rather than making every caller concatenate and slice those fields by hand around a plain `fifo_sync`, this wrapper exposes the fields as individual ports and does the packing internally, keeping call sites readable and consistent.

Use Cases:

- Buffering an {address, control, data0, data1} bundle between two pipeline stages
- Queuing descriptor-like records where fields have distinct widths
- Any place a plain `fifo_sync` would be used but the payload is naturally several fields

Key Benefit: Callers work with named fields of distinct widths while still using a single, well-tested `fifo_sync` for all storage and flow control.

37.3 Parameters

Parameter	Type	Default	Description
REGISTERED	int	0	Read output mode: 0 = mux mode (combinational), 1 = flop mode (registered)
ADDR_WIDTH	int	4	Width of the address field (<code>wr_addr</code> / <code>rd_addr</code>)
CTRL_WIDTH	int	4	Width of the control field (<code>wr_ctrl</code> / <code>rd_ctrl</code>)
DATA_WIDTH	int	4	Width of each data word (<code>wr_data0/1</code> , <code>rd_data0/1</code>)
DEPTH	int	4	FIFO depth in entries
ALMOST_WR_MARGIN	int	1	Almost-full threshold,

Parameter	Type	Default	Description
ALMOST_RD_MARGIN	int	1	passed to fifo_sync Almost-empty threshold, passed to fifo_sync
AW	int	ADDR_WIDTH	Derived alias for ADDR_WIDTH (do not override)
CW	int	CTRL_WIDTH	Derived alias for CTRL_WIDTH (do not override)
DW	int	DATA_WIDTH	Derived alias for DATA_WIDTH (do not override)
D	int	DEPTH	Derived alias for DEPTH (do not override)

37.4 Port Groups

37.4.1 Clock and Reset

Port	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low reset

37.4.2 Write Interface

Port	Direction	Width	Description
write	Input	1	Write enable
wr_addr	Input	AW	Address field to enqueue
wr_ctrl	Input	CW	Control field to enqueue
wr_data0	Input	DW	Data word 0 to enqueue
wr_data1	Input	DW	Data word 1 to enqueue

Port	Direction	Width	Description
wr_full	Output	1	Full flag
wr_almost_full	Output	1	Almost-full flag

37.4.3 Read Interface

Port	Direction	Width	Description
read	Input	1	Read enable
rd_addr	Output	AW	Dequeued address field
rd_ctrl	Output	CW	Dequeued control field
rd_data0	Output	DW	Dequeued data word 0
rd_data1	Output	DW	Dequeued data word 1
rd_empty	Output	1	Empty flag
rd_almost_empty	Output	1	Almost-empty flag

37.5 Functional Description

37.5.1 Field Packing

On the write side the four fields are concatenated into the single `fifo_sync` payload. The concatenation order places `wr_addr` in the most significant bits, then `wr_ctrl`, then `wr_data1`, then `wr_data0` in the least significant bits:

```
.wr_data ({wr_addr, wr_ctrl, wr_data1, wr_data0})
```

The `fifo_sync` payload width is therefore $AW + CW + DW + DW$.

37.5.2 Field Unpacking

On the read side the same concatenation order is used to split the payload back into named outputs, so each field re-emerges on its own port:

```
.rd_data ({rd_addr, rd_ctrl, rd_data1, rd_data0})
```

37.5.3 Storage and Flow Control

Everything else, binary read/write pointers, the memory array, and full/almost-full/empty/almost-empty flag generation, is delegated to the single `fifo_sync` instance. See `fifo_sync` for the pointer, memory, and flag details, including the mux-vs-flop read timing controlled by `REGISTERED`.

37.6 Usage Example

```
fifo_sync_multi #(
    .REGISTERED (0),      // 0 = mux (combinational read), 1 = flop
    (registered)
    .ADDR_WIDTH (8),
    .CTRL_WIDTH (4),
    .DATA_WIDTH (32),
    .DEPTH      (16)
) u_bundle_fifo (
    .clk          (clk),
    .rst_n        (rst_n),
    // write side
    .write        (wr_en),
    .wr_addr      (req_addr),
    .wr_ctrl      (req_tag),
    .wr_data0     (req_data_lo),
    .wr_data1     (req_data_hi),
    .wr_full      (fifo_full),
    .wr_almost_full (fifo_afull),
    // read side
    .read         (rd_en),
    .rd_addr      (out_addr),
    .rd_ctrl      (out_tag),
    .rd_data0     (out_data_lo),
    .rd_data1     (out_data_hi),
    .rd_empty     (fifo_empty),
    .rd_almost_empty (fifo_aempty)
);
```

37.7 Design Notes

- **Pure wiring wrapper.** The module contains no storage or control logic of its own; it only concatenates and slices fields around one `fifo_sync`. All timing, flag, and depth behavior is that of `fifo_sync`.
- **Concatenation order matters.** Write and read use the same {addr, ctrl, data1, data0} order, so the fields map back correctly. Do not reorder one side without the other.

- **Two data words.** The wrapper is fixed at exactly two data words (data0, data1) plus an address and control field. For a different bundle shape, see the signal-map variant below or use `fifo_sync` directly.
- **REGISTERED read latency.** As with `fifo_sync`, mux mode gives 0-cycle combinational read and flop mode gives 1-cycle registered read.

37.8 Related Modules

37.8.1 Used By

- Pipeline stages that buffer an {address, control, data0, data1} bundle

37.8.2 Uses

- [fifo_sync](#) - The underlying single-payload synchronous FIFO providing storage and flow control

37.8.3 See Also

- [fifo_sync_multi_sigmap](#) - Same structure with a generic signal-map naming style (siga..sigh)
- [fifo_control](#) - Shared full/empty flag generation used by `fifo_sync`
- [fifo_async](#) - Clock-domain-crossing FIFO variant

37.9 References

37.9.1 Source Code

- `rtl/common/testcode/fifo_sync_multi.sv`
- `rtl/common/fifo_sync.sv` (instantiated submodule)

37.9.2 Documentation

- `rtl/common/PRD.md`

Last Updated: 2026-07-15

37.10 Navigation

- [Back to RTLCommon Index](#)

38 Multi-Field Synchronous FIFO (Signal-Map Variant)

Module: `fifo_sync_multi_sigmap.sv` **Location:** `rtl/common/` **Status:** Production Ready

38.1 Overview

`fifo_sync_multi_sigmap` is a variant of `fifo_sync_multi` that carries the same four-field payload through one `fifo_sync` instance, but exposes the fields under **generic, position-oriented signal names** (`sig_a`, `sig_b`, `sig_c`, `sig_d` in, `sig_e`, `sig_f`, `sig_g`, `sig_h` out) instead of role-specific names like `addr / ctrl / data`. This makes it a convenient building block when the fields do not have fixed semantic roles, or when a generator maps an array of arbitrary signals onto FIFO slots by position.

As with the base wrapper, all storage, pointers, and status-flag generation come from the single underlying `fifo_sync`; this module only supplies the field-packing wiring.

38.1.1 Key Features

- **Generic field names:** Four write fields `wr_sig_a..wr_sig_d` and four read fields `rd_sig_e..rd_sig_h`
- **Single storage instance:** All fields packed into one `fifo_sync` payload of $AW + CW + DW + DW$ bits
- **Inherited status flags:** `wr_full`, `wr_almost_full`, `rd_empty`, `rd_almost_empty` from `fifo_sync`
- **Mux / flop output modes:** `REGISTERED` selects combinational (mux) or registered (flop) read output
- **Any depth:** Depth passed through to `fifo_sync`

38.2 Module Purpose

This module serves the same buffering role as `fifo_sync_multi`, but its port names are deliberately semantic-free so it can be dropped in wherever a bundle of same-shaped fields is mapped by position rather than by meaning. It is especially handy for generated or templated instantiations that assign an ordered list of signals to FIFO slots.

Use Cases:

- Buffering a bundle of positionally-mapped fields whose roles are assigned by the caller
- Generated / templated wiring that connects an ordered signal list to FIFO slots

- A neutral-named alternative to `fifo_sync_multi` when `addr / ctrl / data` labels do not fit

Key Benefit: Provides the same single-`fifo_sync` field bundling as `fifo_sync_multi` while keeping port names generic, so callers assign meaning by position.

38.3 Parameters

Parameter	Type	Default	Description
REGISTERED	int	0	Read output mode: 0 = mux mode (combinational), 1 = flop mode (registered)
ADDR_WIDTH	int	4	Width of the first field (<code>wr_siga / rd_sige</code>)
CTRL_WIDTH	int	4	Width of the second field (<code>wr_sigb / rd_sigf</code>)
DATA_WIDTH	int	4	Width of each of the two data-sized fields (<code>sigc/sigd, sigg/sigh</code>)
DEPTH	int	4	FIFO depth in entries
ALMOST_WR_MARGIN	int	1	Almost-full threshold, passed to <code>fifo_sync</code>
ALMOST_RD_MARGIN	int	1	Almost-empty threshold, passed to <code>fifo_sync</code>
AW	int	ADDR_WIDTH	Derived alias for ADDR_WIDTH (do not override)
CW	int	CTRL_WIDTH	Derived alias for CTRL_WIDTH (do not override)
DW	int	DATA_WIDTH	Derived alias for DATA_WIDTH (do not override)
D	int	DEPTH	Derived alias for DEPTH (do not override)

38.4 Port Groups

38.4.1 Clock and Reset

Port	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low reset

38.4.2 Write Interface

Port	Direction	Width	Description
write	Input	1	Write enable
wr_siga	Input	AW	First field to enqueue
wr_sigb	Input	CW	Second field to enqueue
wr_sigc	Input	DW	Third (data-sized) field to enqueue
wr_sigd	Input	DW	Fourth (data-sized) field to enqueue
wr_full	Output	1	Full flag
wr_almost_full	Output	1	Almost-full flag

38.4.3 Read Interface

Port	Direction	Width	Description
read	Input	1	Read enable
rd_sige	Output	AW	Dequeued first field (maps from wr_siga)
rd_sigf	Output	CW	Dequeued second field (maps from wr_sigb)
rd_sigg	Output	DW	Dequeued third field (maps from wr_sigc)
rd_sigh	Output	DW	Dequeued fourth field

Port	Direction	Width	Description
			(maps from wr_sigd)
rd_empty	Output	1	Empty flag
rd_almost_empty	Output	1	Almost-empty flag

38.5 Functional Description

38.5.1 Field Packing

The four write fields are concatenated into the single `fifo_sync` payload with `wr_siga` in the most significant bits, then `wr_sigb`, then `wr_sigd`, then `wr_sigg` in the least significant bits:

```
.wr_data ({wr_siga, wr_sigb, wr_sigd, wr_sigg})
```

The payload width is `AW + CW + DW + DW`.

38.5.2 Field Unpacking

The read side splits the payload back out using the matching order, so each field re-emerges on its own output. `sigg/sigh` map to `sigg/sigh` respectively:

```
.rd_data ({rd_sige, rd_sigf, rd_sigh, rd_sigg})
```

The packing order pairs the two data-sized fields identically on both sides (`sigd/sigc` in, `sigh/sigg` out), preserving the positional mapping through the FIFO.

38.5.3 Storage and Flow Control

As with `fifo_sync_multi`, all pointer management, memory, and full/almost-full/empty/almost-empty flag generation is provided by the single `fifo_sync` instance. See `fifo_sync` for the underlying storage and the mux-vs-flop read-timing behavior selected by `REGISTERED`.

38.6 Usage Example

```
fifo_sync_multi_sigmap #(
    .REGISTERED (1),           // 1 = registered (flop) read output
    .ADDR_WIDTH (10),
    .CTRL_WIDTH (2),
    .DATA_WIDTH (16),
    .DEPTH      (8)
) u_sigmap_fifo (
    .clk          (clk),
    .rst_n        (rst_n),
    // write side (positional fields)
    .write        (wr_en),
    .wr_siga      (field_a),
    .wr_sigb      (field_b),
```

```

.wr_sigc      (field_c),
.wr_sigd      (field_d),
.wr_full      (fifo_full),
.wr_almost_full (fifo_afull),
// read side (positional fields)
.read         (rd_en),
.rd_sige      (out_a),
.rd_sigf      (out_b),
.rd_sigg      (out_c),
.rd_sigh      (out_d),
.rd_empty     (fifo_empty),
.rd_almost_empty (fifo_aempty)
);

```

38.7 Design Notes

- **Naming is the only difference.** Structurally this is identical to `fifo_sync_multi`; only the port names change from role-based (`addr/ctrl/data`) to positional (`sigc..sigh`).
- **Positional mapping.** `sigc` -> `sige`, `sigb` -> `sigf`, `sigc` -> `sigg`, `sigd` -> `sigh`. The two data-sized fields keep their pairing because the write and read concatenations use the same order.
- **Pure wiring wrapper.** No storage or control logic lives here; all timing, flags, and depth behavior come from `fifo_sync`.

38.8 Related Modules

38.8.1 Used By

- Generated or templated instantiations that map an ordered signal list onto FIFO slots

38.8.2 Uses

- [fifo_sync](#) - The underlying single-payload synchronous FIFO providing storage and flow control

38.8.3 See Also

- [fifo_sync_multi](#) - The role-named (`addr / ctrl / data0 / data1`) equivalent
- [fifo_control](#) - Shared full/empty flag generation used by `fifo_sync`
- [fifo_async](#) - Clock-domain-crossing FIFO variant

38.9 References

38.9.1 Source Code

- `rtl/common/testcode/fifo_sync_multi_sigmap.sv`
- `rtl/common/fifo_sync.sv` (instantiated submodule)

38.9.2 Documentation

- `rtl/common/PRD.md`

Last Updated: 2026-07-15

38.10 Navigation

- [Back to RTLCommon Index](#)

39 Find First Set (`find_first_set.sv`)

39.1 Purpose

Finds the index of the least significant bit (LSB) that is set to ‘1’ in the input vector. Implements a priority encoder that favors lower-indexed bits.

39.2 Ports

39.2.1 Input Ports

- **data[WIDTH-1:0]** - Input data vector to search

39.2.2 Output Ports

- **index[\$clog2(WIDTH)-1:0]** - Index of the first (lowest) set bit

39.2.3 Parameters

- **WIDTH** - Width of input data vector (default: 32)
- **INSTANCE_NAME** - String identifier for debug (default: “FFS”)

39.3 Implementation Details

39.3.1 Core Algorithm

```
localparam int N = $clog2(WIDTH);  
logic w_found;
```

```

always_comb begin
    index = {N{1'b0}}; // Default value if no bit is set
    w_found = 1'b0;

    for (int i = 0; i < WIDTH; i++) begin
        if (data[i] && !w_found) begin
            index = i[N-1:0]; // Ensure correct bit width
            w_found = 1'b1;
        end
    end
end

```

39.3.2 Search Strategy

- **Direction:** LSB to MSB (left to right, ascending index)
- **Priority:** Lower indices have higher priority
- **First match:** Stops at first '1' bit encountered
- **Found flag:** Prevents multiple assignments

39.4 Functional Behavior

39.4.1 Truth Table Examples (WIDTH=8)

data[7:0]	index[2:0]	Description
00000001	000	Bit 0 is first set
00000010	001	Bit 1 is first set
00000100	010	Bit 2 is first set
00001000	011	Bit 3 is first set
00010000	100	Bit 4 is first set
00100000	101	Bit 5 is first set
01000000	110	Bit 6 is first set
10000000	111	Bit 7 is first set
00000000	000	No bits set - default
10000001	000	Multiple bits - LSB wins
11111111	000	All bits set - bit 0 wins

39.4.2 Key Characteristics

- **LSB priority:** Always returns lowest index when multiple bits set
- **Zero default:** Returns 0 when no bits are set
- **Deterministic:** Same input always produces same output
- **Combinational:** Immediate response to input changes

39.5 Design Features

39.5.1 Width Parameterization

- **Automatic sizing:** Output width calculated as $\lceil \log_2(\text{WIDTH}) \rceil$
- **Examples:**
 - WIDTH=8 → index[2:0] (3 bits)
 - WIDTH=16 → index[3:0] (4 bits)
 - WIDTH=32 → index[4:0] (5 bits)

39.5.2 Found Flag Logic

```
if (data[i] && !w_found) begin
    index = i[N-1:0];
    w_found = 1'b1;
end
```

- **Purpose:** Prevents overwriting the first found result
- **Mechanism:** Once w_found is set, no further assignments occur
- **Result:** Only the lowest index is captured

39.5.3 Bit Width Handling

```
index = i[N-1:0]; // Ensure correct bit width
```

- **Type safety:** Explicit width matching prevents synthesis warnings
- **Truncation:** Safely handles loop variable width vs. output width

39.6 Use Cases

39.6.1 Interrupt Processing

```
find_first_set #(.WIDTH(8)) irq_ffs (  
    .data(interrupt_pending),  
    .index(lowest_priority_irq)  
);
```

- Process lowest-numbered interrupt first
- Fair servicing of interrupt sources

39.6.2 Resource Allocation

```
find_first_set #(.WIDTH(16)) resource_ffs (  
    .data(available_resources),  
    .index(allocated_resource_id)  
);
```

- Allocate lowest-numbered available resource
- Round-robin allocation when combined with masking

39.6.3 Error Detection

```
find_first_set #(.WIDTH(4)) error_ffs (  
    .data(error_status),  
    .index(first_error_type)  
);
```

- Report first error condition detected
- Prioritize error handling by position

39.6.4 Bit Manipulation

```
find_first_set #(.WIDTH(32)) bit_scan (  
    .data(search_pattern),  
    .index(first_set_position)  
);
```

- Bit scanning operations
- Pattern analysis and parsing

39.7 Timing Characteristics

39.7.1 Propagation Delay

- **Best case:** 1 LUT delay (bit 0 set)
- **Worst case:** Multiple LUT delays (higher bits set)
- **Average case:** Depends on typical input patterns

39.7.2 Critical Path

- **Path length:** Increases with WIDTH
- **Optimization:** Modern synthesizers create efficient priority encoders
- **Scaling:** Generally logarithmic complexity in hardware

39.8 Algorithm Comparison

39.8.1 vs. Find Last Set

Aspect	Find First Set	Find Last Set
Search direction	LSB → MSB	MSB → LSB
Priority	Lower index wins	Higher index wins
Use case	Fair allocation	Priority allocation

39.8.2 vs. Priority Encoder

Aspect	Find First Set	Priority Encoder
Priority direction	LSB has priority	MSB has priority
Application	Round-robin systems	Hierarchical systems

39.9 Design Considerations

39.9.1 Input Validation

- **All zeros:** Returns 0 (may need validation in application)
- **Valid range:** All input patterns handled gracefully
- **No error conditions:** Always produces a result

39.9.2 Performance Optimization

- **Synthesis:** Let tools optimize the priority encoding
- **Pipelining:** Consider registering for high-speed applications
- **Parallel:** Can process multiple vectors simultaneously

39.9.3 Width Selection

- **Power of 2:** Often most efficient for synthesis
- **Arbitrary width:** Supported but may have overhead
- **Large widths:** Consider hierarchical implementation

39.10 Common Usage Patterns

39.10.1 Round-Robin Arbitration

```
// Mask off already-serviced requests
assign masked_requests = pending_requests & ~service_mask;

find_first_set #(.WIDTH(N)) arbiter_ffs (
    .data(masked_requests),
    .index(next_grant)
);

// Update mask to exclude granted request
assign next_mask = service_mask | (1 << next_grant);
```

39.10.2 Free List Management

```
find_first_set #(.WIDTH(P00L_SIZE)) free_finder (
    .data(free_pool_bitmap),
    .index(allocated_entry)
);

// Mark entry as allocated
assign updated_bitmap = free_pool_bitmap & ~(1 << allocated_entry);
```

39.10.3 Packet Classification

```
find_first_set #(.WIDTH(NUM_RULES)) rule_matcher (
    .data(rule_match_vector),
    .index(matched_rule_id)
);
```

39.11 Synthesis Considerations

39.11.1 Resource Utilization

- **LUT usage:** Typically maps to priority encoder primitives
- **Efficiency:** Good utilization for most FPGA architectures
- **Scaling:** Linear increase in resources with WIDTH

39.11.2 Timing Optimization

- **Critical paths:** May need pipeline registers for large WIDTH
- **Clock frequency:** Generally achieves high fmax
- **Constraints:** Standard combinational timing constraints apply

39.12 Verification Considerations

39.12.1 Test Scenarios

- **Single bit:** Each bit position individually
- **Multiple bits:** Various combinations
- **All zeros:** Edge case handling
- **All ones:** Maximum case behavior

39.12.2 Coverage Metrics

- **Bit coverage:** Each input bit position
- **Pattern coverage:** Representative bit combinations
- **Boundary cases:** Min/max index values

39.13 Related Modules

- **find_last_set:** MSB-first search variant
- **encoder:** Basic priority encoder
- **encoder_priority_enable:** Enhanced priority encoder with enable

39.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

40 Find Last Set (find_last_set.sv)

40.1 Purpose

Finds the index of the most significant bit (MSB) that is set to '1' in the input vector. Implements a priority encoder that favors higher-indexed bits.

40.2 Ports

40.2.1 Input Ports

- **data[WIDTH-1:0]** - Input data vector to search

40.2.2 Output Ports

- **index[\$clog2(WIDTH)-1:0]** - Index of the last (highest) set bit

40.2.3 Parameters

- **WIDTH** - Width of input data vector (default: 32)
- **INSTANCE_NAME** - String identifier for debug (default: "FLS")

40.3 Implementation Details

40.3.1 Core Algorithm

```
localparam int N = $clog2(WIDTH);
logic w_found;

always_comb begin
    index = {N{1'b0}}; // Default value if no bit is set
    w_found = 1'b0;

    for (int i = WIDTH - 1; i >= 0; i--) begin
        if (data[i] && !w_found) begin
            index = i[N-1:0]; // Ensure correct bit width
            w_found = 1'b1;
        end
    end
end
```

40.3.2 Search Strategy

- **Direction:** MSB to LSB (right to left, descending index)
- **Priority:** Higher indices have higher priority
- **First match:** Stops at first '1' bit encountered from MSB
- **Found flag:** Prevents multiple assignments

40.4 Functional Behavior

40.4.1 Truth Table Examples (WIDTH=8)

data[7:0]	index[2:0]	Description
10000000	111	Bit 7 is last set
01000000	110	Bit 6 is last set
00100000	101	Bit 5 is last set
00010000	100	Bit 4 is last set
00001000	011	Bit 3 is last set
00000100	010	Bit 2 is last set
00000010	001	Bit 1 is last set
00000001	000	Bit 0 is last set
00000000	000	No bits set - default
10000001	111	Multiple bits - MSB wins
11111111	111	All bits set - bit 7 wins

40.4.2 Key Characteristics

- **MSB priority:** Always returns highest index when multiple bits set
- **Zero default:** Returns 0 when no bits are set (same as find_first_set)
- **Deterministic:** Same input always produces same output
- **Combinational:** Immediate response to input changes

40.5 Design Features

40.5.1 Reverse Loop Implementation

```
for (int i = WIDTH - 1; i >= 0; i--) begin
```

- **MSB-first:** Starts from highest bit index
- **Decrementing:** Moves toward LSB
- **Early termination:** Stops at first match found

40.5.2 Width Parameterization

Same auto-sizing as `find_first_set`: - **Output width:** $\$clog2(WIDTH)$ bits - **Examples:** -
WIDTH=8 → `index[2:0]` (3 bits) - WIDTH=16 → `index[3:0]` (4 bits)
- WIDTH=32 → `index[4:0]` (5 bits)

40.5.3 Found Flag Logic

```
if (data[i] && !w_found) begin  
    index = i[N-1:0];  
    w_found = 1'b1;  
end
```

- **Single assignment:** Prevents overwriting the highest found bit
- **Priority enforcement:** Only the first match (highest index) is captured

40.6 Use Cases

40.6.1 Interrupt Priority Handling

```
find_last_set #(.WIDTH(8)) irq_priority (  
    .data(interrupt_pending),  
    .index(highest_priority_irq)  
);
```

- Service highest-priority interrupt first
- Preemptive interrupt handling

40.6.2 Leading Zero Count

```
// Find position of leading 1, then calculate leading zeros  
find_last_set #(.WIDTH(32)) leading_one (  
    .data(input_value),  
    .index(msb_position)  
);
```

```
assign leading_zero_count = (input_value == 0) ? 32 : (31 -  
msb_position);
```

40.6.3 Bit Width Calculation

```
find_last_set #(.WIDTH(64)) width_calc (  
    .data(value),  
    .index(msb_pos)  
);
```

```
assign required_bits = msb_pos + 1; // Minimum bits needed to  
represent value
```

40.6.4 Cache Line Selection

```
find_last_set #(.WIDTH(16)) cache_priority (  
    .data(cache_line_priorities),  
    .index(selected_cache_line)  
);
```

- Select highest priority cache line for replacement
- LRU policy implementation

40.7 Timing Characteristics

40.7.1 Propagation Delay

- **Best case:** 1 LUT delay (MSB set)
- **Worst case:** Multiple LUT delays (LSB only set)
- **Average case:** Depends on typical bit patterns

40.7.2 Critical Path Analysis

MSB check → MSB-1 check → ... → LSB check → Output

- **Path length:** Linear with WIDTH in worst case
- **Typical case:** Shorter path when higher bits are set
- **Optimization:** Synthesis tools create efficient implementations

40.8 Algorithm Comparison

40.8.1 Find Last Set vs. Find First Set

Aspect	Find Last Set	Find First Set
Search Direction	MSB → LSB	LSB → MSB
Priority	Higher index wins	Lower index wins
Loop Direction	i - - (decrementing)	i++ (incrementing)
Use Case	Priority systems	Fair allocation
Default Output	0 (same)	0 (same)

40.8.2 Typical Use Case Patterns

Application Type	Preferred Algorithm
Interrupt Controllers	Find Last Set (priority)
Round-Robin Arbiters	Find First Set (fairness)
Error Reporting	Find Last Set (severity)
Resource Allocation	Find First Set (efficiency)
Bit Manipulation	Either (depends on need)

40.9 Design Considerations

40.9.1 Zero Input Handling

Both modules return 0 for zero input:

```
// data = 8'b00000000  
// Both find_first_set and find_last_set return 3'b000
```

Application consideration: May need explicit zero detection:

```
assign valid_result = |data; // OR-reduce to detect any bits set  
assign result_index = valid_result ? index : INVALID_INDEX;
```

40.9.2 Performance Scaling

- **Small WIDTH (<16):** Excellent performance
- **Medium WIDTH (16-64):** Good performance, may need pipelining
- **Large WIDTH (>64):** Consider hierarchical implementation

40.9.3 Synthesis Optimization

Modern synthesis tools recognize these patterns: - **Priority encoder primitives**: Often map to dedicated hardware - **LUT optimization**: Efficient resource utilization - **Timing optimization**: Automatic critical path optimization

40.10 Advanced Usage Patterns

40.10.1 Hierarchical Priority Encoding

```
// For very wide inputs, use hierarchical approach
find_last_set #(.WIDTH(8)) level0_fls [7:0] (
    .data(data_chunks), // 8 chunks of 8 bits each
    .index(chunk_indices)
);

find_last_set #(.WIDTH(8)) level1_fls (
    .data(chunk_valid), // Which chunks have set bits
    .index(active_chunk)
);

// Combine results
assign final_index = {active_chunk, chunk_indices[active_chunk]};
```

40.10.2 Conditional Priority Selection

```
// Select between different priority schemes
assign priority_vector = use_high_priority ? high_pri_requests :
    use_fair_mode ? fair_requests :
    normal_requests;

find_last_set #(.WIDTH(N)) flexible_priority (
    .data(priority_vector),
    .index(selected_request)
);
```

40.10.3 Dynamic Width Processing

```
// Process variable-width data
logic [MAX_WIDTH-1:0] padded_data;
assign padded_data = {(MAX_WIDTH-actual_width){1'b0}}, input_data;

find_last_set #(.WIDTH(MAX_WIDTH)) variable_fls (
    .data(padded_data),
    .index(raw_index)
);
```



```
// Adjust result for actual width  
assign adjusted_index = (raw_index >= actual_width) ? 0 : raw_index;
```

40.11 Verification Strategies

40.11.1 Comprehensive Test Coverage

```
// Test single bit at each position  
for (int i = 0; i < WIDTH; i++) begin  
    test_vector = 1 << i;  
    expected_result = i;  
    // Verify find_last_set returns i  
end
```

```
// Test multiple bits with known MSB  
test_vector = 8'b10101010;  
expected_result = 7; // MSB position
```

```
// Test edge cases  
test_vector = 8'b00000000; // All zeros  
expected_result = 0; // Default
```

40.11.2 Random Testing

```
// Random bit patterns  
repeat (1000) begin  
    random_vector = $random();  
    expected = calculate_msb_position(random_vector);  
    // Verify against reference function  
end
```

40.12 Related Modules

- **find_first_set**: LSB-priority search
- **encoder_priority_enable**: Enhanced priority encoder
- **leading_one_trailing_one**: Position detection for Johnson counters

40.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

41 Glitch-Free N-DFF Synchronizer (glitch_free_n_dff_arn.sv)

41.1 Purpose

Provides a parameterized multi-stage synchronizer for safe clock domain crossing (CDC). Reduces metastability risk through configurable stages of flip-flops while maintaining data integrity across asynchronous clock domains.

41.2 Ports

41.2.1 Input Ports

- **clk** - Destination domain clock
- **rst_n** - Destination domain active-low reset
- **d[WIDTH-1:0]** - Input data from source clock domain

41.2.2 Output Ports

- **q[WIDTH-1:0]** - Synchronized output data in destination domain

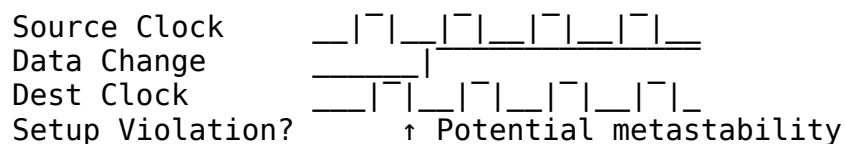
41.2.3 Parameters

- **FLOP_COUNT** - Number of synchronizer flip-flop stages (default: 3)
- **WIDTH** - Data width in bits (default: 4)

41.3 Clock Domain Crossing Theory

41.3.1 Metastability Problem

When signals cross between asynchronous clock domains:

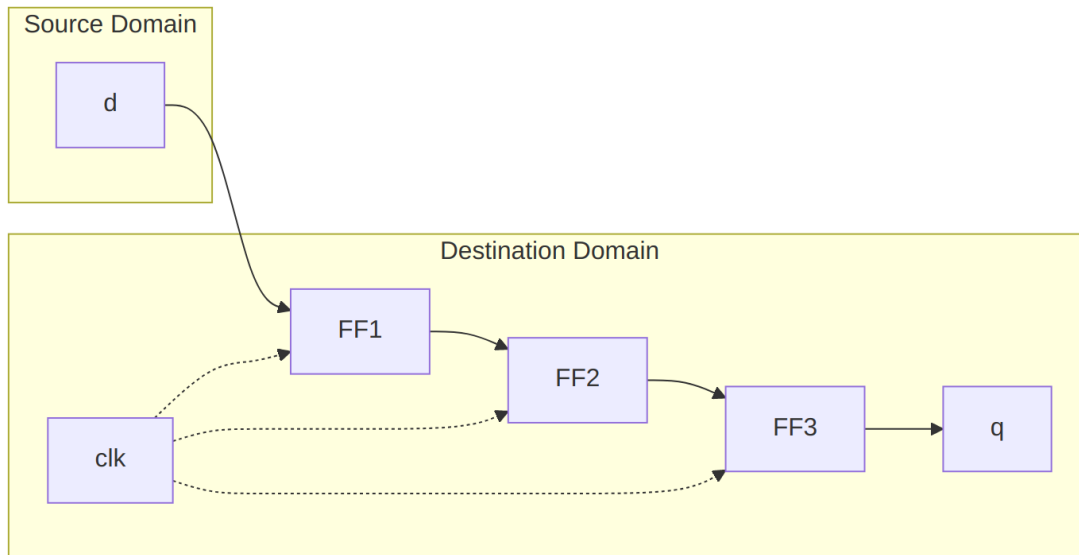


41.3.2 Metastability Effects

- **Undefined output**: Neither 0 nor 1, but intermediate voltage
- **Extended resolution**: Takes longer than normal clock period
- **Propagation**: Can corrupt downstream logic

- **System failure:** Can cause entire design malfunction

41.4 Multi-Stage Synchronizer Solution



Architecture Overview

41.4.1 Stage-by-Stage Analysis

1. **Stage 1 (FF1):** Captures source data, may go metastable
2. **Stage 2 (FF2):** Allows metastability to resolve, low probability of propagation
3. **Stage 3+ (FFN):** Further reduces MTBF (Mean Time Between Failures)

41.5 Implementation Details

41.5.1 Packed Array Structure

logic [FC-1:0][WIDTH-1:0] r_q_array;

- **Efficient storage:** Single declaration for all stages
- **Clear indexing:** r_q_array[0] is first stage, r_q_array[FC-1] is last
- **Synthesis friendly:** Maps well to FPGA/ASIC resources

41.5.2 Shift Register Behavior

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        // Reset all flip-flops to zero
        r_q_array <= {FC{{DW{1'b0}}}};
    end else begin
        // Load new data into first stage
        r_q_array[0] <= d;
        // Shift existing data through remaining stages
        for (int i = 1; i < FC; i++) begin
            r_q_array[i] <= r_q_array[i-1];
        end
    end
end
```

41.5.3 Reset Behavior

- **Synchronous reset:** Uses destination domain reset
- **Complete reset:** All stages reset to zero
- **Nested replication:** {FC{{DW{1'b0}}}} creates FC copies of DW zeros

41.6 MTBF (Mean Time Between Failures) Analysis

41.6.1 MTBF Calculation

$$\text{MTBF} = (e^{(t_{\text{res}}/\tau)}) / (f_{\text{clk}} \times f_{\text{data}} \times \tau)$$

Where:

- t_{res} = Resolution time available (clock period - setup - routing)
- τ = Flip-flop metastability time constant (~200ps typical)
- f_{clk} = Destination clock frequency
- f_{data} = Data transition frequency

41.6.2 Stage Count Impact

Stages	Relative MTBF	Typical MTBF
1	1× (baseline)	Minutes
2	~1000×	Hours
3	~1,000,000×	Years
4	~1,000,000,000×	Millennia

41.6.3 Practical Guidelines

- **2 stages:** Minimum for most applications

- **3 stages:** Recommended for high-reliability systems
- **4+ stages:** Extreme reliability requirements (aerospace, medical)

41.7 Latency vs. Reliability Trade-off

41.7.1 Synchronizer Latency

- **Minimum latency:** 2 clock cycles (2-stage synchronizer)
- **General latency:** FLOP_COUNT clock cycles
- **Fixed delay:** Independent of data pattern

41.7.2 Performance Impact

```
// 2-stage: 2 cycle latency, good MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(2)) fast_sync (...);

// 3-stage: 3 cycle latency, excellent MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(3)) safe_sync (...);

// 4-stage: 4 cycle latency, extreme MTBF
glitch_free_n_dff_arn #(.FLOP_COUNT(4)) ultra_safe_sync (...);
```

41.8 Usage Guidelines

41.8.1 Appropriate Use Cases

- **Control signals:** Reset, enable, mode switches
- **Slow status signals:** Error flags, configuration bits
- **Gray code pointers:** FIFO read/write pointers
- **Single-bit signals:** Generally safe for multi-bit if Gray coded

41.8.2 Inappropriate Use Cases

- **High-speed data buses:** Use proper CDC techniques (handshaking, FIFOs)
- **Binary counters:** Multiple bits can transition simultaneously
- **Address buses:** Binary addresses are not CDC-safe
- **Clock signals:** Never synchronize clocks this way

41.8.3 Multi-Bit Considerations

```
// SAFE: Gray-coded FIFO pointers
glitch_free_n_dff_arn #(.WIDTH(5)) gray_sync (
    .d(gray_pointer),           // Gray code - only 1 bit changes
    .q(sync_gray_pointer)
);

// UNSAFE: Binary counter synchronization
// Don't do this - multiple bits can change simultaneously
glitch_free_n_dff_arn #(.WIDTH(8)) bad_sync (
    .d(binary_counter),        // Binary - multiple bit transitions
    .q(sync_counter)           // May capture inconsistent values
);
```

41.9 Advanced Applications

41.9.1 Reset Synchronizer Pattern

```
// Asynchronous assert, synchronous deassert
logic reset_sync;
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(1)) reset_sync_inst (
    .clk(dest_clk),
    .rst_n(1'b1),           // No reset feedback
    .d(~async_reset),       // Invert for active-low
    .q(reset_sync)
);
```

```
assign sync_reset_n = reset_sync & ~async_reset;
```

41.9.2 Enable Signal Synchronization

```
glitch_free_n_dff_arn #(.FLOP_COUNT(3), .WIDTH(1)) enable_sync (
    .clk(fast_clk),
    .rst_n(rst_n),
    .d(slow_enable),
    .q(fast_enable_sync)
);
```

41.9.3 Status Flag Collection

```
// Synchronize multiple independent status signals
glitch_free_n_dff_arn #(.FLOP_COUNT(2), .WIDTH(4)) status_sync (
    .clk(dest_clk),
    .rst_n(rst_n),
    .d({error_flag, done_flag, busy_flag, ready_flag}),
    .q({sync_error, sync_done, sync_busy, sync_ready})
);
```

41.10 Synthesis Considerations

41.10.1 FPGA Implementation

- **Dedicated resources:** Maps to flip-flop primitives
- **Placement:** Tools may place stages in different locations
- **Timing constraints:** Require proper CDC constraints

41.10.2 Timing Constraints

```
# Constrain the synchronizer input as asynchronous
set_false_path -to [get_pins sync_inst/r_q_array[0]/D]

# Or set maximum delay if there's some relationship
set_max_delay -to [get_pins sync_inst/r_q_array[0]/D] 2.0
```

41.10.3 Optimization Prevention

```
// Synthesis attributes to prevent optimization
(* ASYNC_REG = "TRUE" *) logic [FC-1:0][WIDTH-1:0] r_q_array;
(* DONT_TOUCH = "TRUE" *) logic [FC-1:0][WIDTH-1:0] r_q_array;
```

41.11 Verification Strategies

41.11.1 Functional Verification

- **Data integrity:** Verify output eventually matches input
- **Reset behavior:** Check proper reset operation
- **Latency verification:** Confirm expected delay

41.11.2 Metastability Testing

```
// Stress test with rapid data changes
initial begin
    forever begin
        @(posedge source_clk);
        d <= $random();
        #($random() % 100); // Random timing
    end
end
```

41.11.3 Formal Verification

- **Eventually properties:** Data will eventually propagate

- **Reset properties:** Reset behavior verification
- **No X propagation:** Ensure no unknown values propagate

41.12 Debug Features

41.12.1 Waveform Analysis

```
// Flatten array for waveform viewing
wire [(DW*FC)-1:0] flat_r_q;
genvar i;
generate
    for (i = 0; i < FC; i++) begin : gen_flatten_memory
        assign flat_r_q[i*DW+:DW] = r_q_array[i];
    end
endgenerate
```

41.12.2 Simulation Visibility

- **All stages visible:** Can observe data propagation through stages
- **Metastability injection:** Can inject X values for testing
- **Timing analysis:** Observe setup/hold violations

41.13 Related Modules

- **FIFO CDC modules:** Use this for pointer synchronization
- **Reset synchronizers:** Specialized CDC for reset signals
- **Handshake synchronizers:** For complex data CDC
- **Pulse synchronizers:** For single-cycle pulse CDC

41.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

42 Gray-to-Binary Converter (gray2bin.sv)

42.1 Purpose

Converts Gray code (reflected binary code) to standard binary representation using XOR reduction. Essential component for asynchronous FIFO pointer comparison and other CDC applications.

42.2 Ports

42.2.1 Input Ports

- **gray[WIDTH-1:0]** - Gray code input

42.2.2 Output Ports

- **binary[WIDTH-1:0]** - Binary code output

42.2.3 Parameters

- **WIDTH** - Data width in bits (default: 4)

42.3 Gray Code Theory

42.3.1 What is Gray Code?

Gray code (also called reflected binary code) is a binary numeral system where **only one bit changes** between consecutive values:

Binary	Gray	Transitions
000	→ 000	
001	→ 001	1 bit change
010	→ 011	1 bit change
011	→ 010	1 bit change
100	→ 110	1 bit change
101	→ 111	1 bit change
110	→ 101	1 bit change
111	→ 100	1 bit change

42.3.2 Why Gray Codes Matter for CDC

- **Metastability protection:** Only one bit transition eliminates multi-bit race conditions
- **Safe sampling:** Intermediate values during transition are still valid Gray codes
- **FIFO pointers:** Essential for async FIFO full/empty detection

42.4 Conversion Algorithm

42.4.1 Mathematical Foundation

For Gray-to-binary conversion, each binary bit is the XOR of all Gray bits from that position to the MSB:

$$\text{binary}[i] = \text{gray}[\text{MSB}] \oplus \text{gray}[\text{MSB}-1] \oplus \dots \oplus \text{gray}[i]$$

42.4.2 Implementation

```
genvar i;
generate
  for (i = 0; i < WIDTH; i++) begin : gen_gray_to_bin
    assign binary[i] = ^(gray >> i);
  end
endgenerate
```

42.4.3 Bit-by-Bit Breakdown

```
// For 4-bit example:
assign binary[3] = gray[3];           // MSB unchanged
assign binary[2] = gray[3] ^ gray[2]; // XOR from MSB
down
assign binary[1] = gray[3] ^ gray[2] ^ gray[1]; // XOR from MSB
down
assign binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0]; // XOR all
bits
```

42.5 Functional Examples

42.5.1 4-Bit Conversion Table

Gray[3:0]	Binary[3:0]	Calculation
0000	0000	0⊕0⊕0⊕0=0, 0⊕0⊕0=0, 0⊕0=0, 0=0
0001	0001	0⊕0⊕0⊕1=1, 0⊕0⊕0=0, 0⊕0=0, 0=0
0011	0010	0⊕0⊕1⊕1=0, 0⊕0⊕1=1, 0⊕0=0, 0=0
0010	0011	0⊕0⊕1⊕0=1, 0⊕0⊕1=1, 0⊕0=0,

Gray[3:0]	Binary[3:0]	Calculation
		$0=0$
0110	0100	$0\oplus 1\oplus 1\oplus 0=0,$ $0\oplus 1\oplus 1=0, 0\oplus 1=1,$ $0=0$
0111	0101	$0\oplus 1\oplus 1\oplus 1=1,$ $0\oplus 1\oplus 1=0, 0\oplus 1=1,$ $0=0$
0101	0110	$0\oplus 1\oplus 0\oplus 1=0,$ $0\oplus 1\oplus 0=1, 0\oplus 1=1,$ $0=0$
0100	0111	$0\oplus 1\oplus 0\oplus 0=1,$ $0\oplus 1\oplus 0=1, 0\oplus 1=1,$ $0=0$

42.5.2 Step-by-Step Example (Gray 0110 → Binary)

Gray input: 0110

```

binary[3] = gray[3] = 0
binary[2] = gray[3] ^ gray[2] = 0 ^ 1 = 1
binary[1] = gray[3] ^ gray[2] ^ gray[1] = 0 ^ 1 ^ 1 = 0
binary[0] = gray[3] ^ gray[2] ^ gray[1] ^ gray[0] = 0 ^ 1 ^ 1 ^ 0 = 0

```

Result: binary = 0100 (decimal 4)

42.6 Implementation Analysis

42.6.1 XOR Reduction Technique

```
assign binary[i] = ^(gray >> i);
```

How it works: - gray >> i: Right-shift gray by i positions - ^(...): XOR-reduce all bits in the shifted value - **Effect:** XORs all bits from position i to MSB

42.6.2 Generate Loop Benefits

- **Parameterizable:** Works for any WIDTH
- **Synthesis friendly:** Tools recognize and optimize pattern
- **Parallel computation:** All bits calculated simultaneously

- **Resource efficient:** Maps to XOR tree structures

42.7 Use Cases in Digital Design

42.7.1 Asynchronous FIFO Pointers

```
// Convert synchronized Gray pointers back to binary for comparison
gray2bin #(.WIDTH(5)) wr_ptr_conv (
    .gray(sync_wr_ptr_gray),
    .binary(sync_wr_ptr_bin)
);
```

```
gray2bin #(.WIDTH(5)) rd_ptr_conv (
    .gray(sync_rd_ptr_gray),
    .binary(sync_rd_ptr_bin)
);
```

```
// Now can perform binary arithmetic for occupancy calculation
assign occupancy = sync_wr_ptr_bin - sync_rd_ptr_bin;
```

42.7.2 Clock Domain Crossing Counters

```
// Convert Gray counter back to binary for address generation
gray2bin #(.WIDTH(AW)) addr_converter (
    .gray(gray_counter),
    .binary(memory_address)
);
```

42.7.3 Position Encoding

```
// Convert Gray-encoded position to binary for processing
gray2bin #(.WIDTH(8)) position_decoder (
    .gray(gray_position),
    .binary(bin_position)
);
```

42.8 Timing Characteristics

42.8.1 Propagation Delay

- **XOR tree depth:** $\log_2(\text{WIDTH})$ levels
- **Typical delay:** 1-2 LUT delays for most widths
- **Critical path:** From MSB input to LSB output

42.8.2 Delay Analysis by Width

WIDTH	XOR Levels	Typical Delay
4	2	1 LUT delay
8	3	2 LUT delays
16	4	2 LUT delays
32	5	3 LUT delays

42.8.3 Synthesis Optimization

Modern synthesis tools: - **Recognize pattern**: Optimize XOR reduction automatically - **Balance trees**: Create balanced XOR structures - **Resource sharing**: Reuse XOR gates where possible

42.9 Design Considerations

42.9.1 Width Scaling

- **Linear resource growth**: XOR gates increase with WIDTH
- **Logarithmic delay growth**: Delay scales with $\log(\text{WIDTH})$
- **Practical limits**: Works well up to 64+ bits

42.9.2 Input Validation

```
// Gray codes have no invalid states - all inputs are valid
// However, may want to validate source of Gray code
always_comb begin
    if (gray_code_valid) begin
        binary_out = converted_binary;
    end else begin
        binary_out = '0; // Default when invalid
    end
end
```

42.9.3 Pipeline Considerations

For very high-speed applications:

```
// Optional pipeline stage for timing closure
always_ff @(posedge clk) begin
    if (!rst_n) begin
        binary_reg <= '0;
    end else begin
        binary_reg <= binary_comb;
    end
end
```

42.10 Verification Strategies

42.10.1 Exhaustive Testing

```
// For reasonable widths, test all possible inputs
for (int i = 0; i < (1 << WIDTH); i++) begin
    gray_input = int_to_gray(i);           // Convert integer to Gray
    expected_binary = i;                   // Expected result
    #1;
    assert(binary_output == expected_binary);
end
```

42.10.2 Property-Based Verification

```
// Verify round-trip conversion
property round_trip;
    @(posedge clk)
        binary_output == original_binary;
endproperty

// Apply after binary→Gray→binary conversion
assert property (round_trip);
```

42.10.3 Random Testing

```
repeat (10000) begin
    random_gray = $random();
    expected_result = reference_gray2bin(random_gray);
    #1;
    assert(binary_output == expected_result);
end
```

42.11 Common Mistakes and Pitfalls

42.11.1 Bit Order Confusion

```
// WRONG: Bit order matters in Gray codes
assign binary[i] = ^(gray << i); // Left shift instead of right

// CORRECT:
assign binary[i] = ^(gray >> i); // Right shift
```

42.11.2 Width Mismatches

```
// WRONG: Input/output width mismatch
gray2bin #(.WIDTH(4)) converter (
    .gray(gray_5bit),           // 5-bit input
    .binary(binary_4bit)       // 4-bit output
);
```

```
// CORRECT: Match widths
gray2bin #(.WIDTH(5)) converter (
    .gray(gray_5bit),
    .binary(binary_5bit)
);
```

42.11.3 Timing Assumptions

```
// WRONG: Assuming zero delay
gray_in <= new_value;
binary_out <= converted_value; // May not be updated yet
```

```
// CORRECT: Account for combinational delay
gray_in <= new_value;
#1; // Wait for conversion
binary_out <= converted_value;
```

42.12 Related Modules

- **bin2gray**: Performs inverse conversion (binary to Gray)
- **counter_bingray**: Combined binary/Gray counter
- **fifo_async**: Uses Gray codes for CDC
- **grayj2bin**: Johnson counter to binary converter (different algorithm)

42.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

43 Johnson-to-Binary Converter (grayj2bin.sv)

43.1 Purpose

Converts Johnson counter codes to binary representation for use in asynchronous FIFOs with non-power-of-2 depths. Unlike standard Gray-to-binary conversion, this module handles the unique properties of Johnson counter sequences.

43.2 Ports

43.2.1 Input Ports

- **clk** - Clock signal (required for leading/trailing one detection)
- **rst_n** - Active-low reset
- **gray[JCW-1:0]** - Johnson counter input (misleadingly named “gray”)

43.2.2 Output Ports

- **binary[WIDTH-1:0]** - Binary output representing position in sequence

43.2.3 Parameters

- **JCW** - Johnson Counter Width (equals FIFO DEPTH)
- **WIDTH** - Binary output width (typically $\lceil \log_2(\text{DEPTH}) \rceil + 1$)
- **INSTANCE_NAME** - String identifier for debug

43.3 Johnson Counter Sequence Review

43.3.1 Johnson Counter Characteristics

Johnson counters are **shift registers with inverted feedback**:

For JCW=6 (DEPTH=6):

```
State 0: 000000 ← Initial state
State 1: 100000 ← Shift left, insert 1
State 2: 110000
State 3: 111000
State 4: 111100
State 5: 111110
State 6: 111111 ← All 1s (special case)
State 7: 011111 ← Shift left, insert ~MSB (0)
State 8: 001111
State 9: 000111
State 10: 000011
State 11: 000001
State 0: 000000 ← Cycle complete (12 states total)
```

43.3.2 Key Properties

- **Single bit transitions:** Only one bit changes per state
- **Two phases:**

- **First half** (0 to DEPTH-1): Filling with 1s from left
- **Second half** (DEPTH to 2×DEPTH-1): Emptying 1s from left
- **Wrap indicator:** MSB indicates which half of cycle

43.4 Conversion Algorithm

43.4.1 Strategy Overview

The conversion uses **position detection** of the transition between 1s and 0s:

```

if (w_all_zeroes || w_all_ones) begin
    w_binary = {WIDTH{1'b0}};
end else if (gray[JCW-1]) begin
    // Second half: use leading one position directly
    w_binary = {{(WIDTH-N){1'b0}}, w_trailing_one};
end else begin
    // First half: use trailing one + 1
    w_binary = {{(WIDTH-N){1'b0}}, (w_leading_one + 1'b1)};
end

```

43.4.2 Position Detection Module

```

leading_one_trailing_one #(
    .WIDTH(JCW),
    .INSTANCE_NAME(INSTANCE_NAME)
) u_leading_one_trailing_one (
    .data(gray),
    .leadingone(w_leading_one),
    .trailingone(w_trailing_one),
    .all_zeroes(w_all_zeroes),
    .all_ones(w_all_ones),
    .valid(w_valid)
);

```

43.5 Detailed Conversion Examples

43.5.1 First Half Conversion (MSB = 0)

Johnson: 001111 (w_leading_one = 5, w_trailing_one = 2)

Logic: First half, so position = w_leading_one + 1 = 5 + 1 = 6

Binary: 000110 (with MSB=0 indicating first half)

43.5.2 Second Half Conversion (MSB = 1)

Johnson: 111000 (w_leading_one = 0, w_trailing_one = 2)

Logic: Second half, so position = w_trailing_one = 2

Binary: 100010 (with MSB=1 indicating second half)

43.5.3 Special Cases

Johnson: 000000 → Binary: 000000 (all zeros case)

Johnson: 111111 → Binary: 000000 (all ones case - same as zeros)

43.6 Implementation Deep Dive

43.6.1 Three-Part Binary Construction

```
assign binary[WIDTH-1] = gray[JCW-1];           // MSB = wrap
indicator
assign binary[WIDTH-2:0] = w_binary[WIDTH-2:0]; // Lower bits
= position
```

43.6.2 Width Calculations

```
localparam int N = $clog2(JCW);                 // Address
bits needed
localparam int PAD_WIDTH = (WIDTH > N+1) ? WIDTH-N-1 : 0; // Padding
if needed
```

43.6.3 Why This Algorithm Works

43.6.3.1 First Half Logic (MSB = 0)

- **Pattern:** 000...0111...1 (0s followed by 1s)
- **Leading one:** Position of leftmost 1
- **Conversion:** Position = leading_one + 1
- **Reasoning:** We've filled (leading_one + 1) positions with 1s

43.6.3.2 Second Half Logic (MSB = 1)

- **Pattern:** 111...1000...0 (1s followed by 0s)
- **Trailing one:** Position of rightmost 1
- **Conversion:** Position = trailing_one
- **Reasoning:** We're emptying from the left, trailing_one shows how far

43.7 Use in Asynchronous FIFO

43.7.1 Context in FIFO Operation

```
// fifo_async_div2.sv usage:
gray2bin #(  

```

```

        .JCW(JCW),                // = DEPTH
        .WIDTH(AW + 1),          // Address width + wrap bit
        .INSTANCE_NAME("rd_ptr_gray2bin_inst")
    ) rd_ptr_gray2bin_inst(
        .binary(w_wdom_rd_ptr_bin), // Binary for arithmetic
        .gray(r_wdom_rd_ptr_gray),  // Johnson counter from CDC
        .clk(wr_clk),
        .rst_n(wr_rst_n)
    );

```

43.7.2 Why Clock is Required

Unlike pure combinational Gray-to-binary conversion, Johnson-to-binary needs the `leading_one_trailing_one` helper, which may use sequential logic for complex position detection.

43.8 Comparison with Standard Gray2Bin

Aspect	Standard Gray2Bin	Johnson2Bin (grayj2bin)
Input type	Traditional Gray code	Johnson counter sequence
Algorithm	XOR reduction	Position detection
Complexity	Simple combinational	Complex position logic
Clock requirement	None	Required for helper modules
Width scaling	Logarithmic	Linear with JCW
Use case	Power-of-2 sequences	Any even sequences

43.9 Performance Characteristics

43.9.1 Timing Analysis

- **Critical path:** Through position detection logic
- **Delay components:**
 - Leading/trailing one detection
 - Binary arithmetic (addition)

- Output multiplexing
- **Synthesis complexity:** Higher than standard Gray conversion

43.9.2 Resource Utilization

Resources scale with JCW (Johnson Counter Width):

- Small FIFOs (JCW ≤ 16): Reasonable overhead
- Medium FIFOs (JCW = 32-64): Significant resources
- Large FIFOs (JCW > 64): May become limiting factor

43.10 Design Considerations

43.10.1 When to Use Johnson vs. Gray

// Use Johnson counter approach when:

```
parameter int DEPTH = 10; // Non-power-of-2 required
parameter int DEPTH = 6;  // Specific size needed
parameter int DEPTH = 14; // Standard Gray won't work
```

// Use standard Gray approach when:

```
parameter int DEPTH = 8;  // Power-of-2 is acceptable
parameter int DEPTH = 16; // Efficiency more important
parameter int DEPTH = 64; // Large depth, resource conscious
```

43.10.2 Resource Planning

// Resource overhead estimation:

// Standard Gray: $\sim \log_2(\text{DEPTH})$ XOR gates

// Johnson: $\sim \text{DEPTH}$ comparators + position logic

// Break-even point typically around DEPTH = 16-32

43.10.3 Verification Challenges

// More complex verification due to:

// 1. Two-phase sequence behavior

// 2. Position detection correctness

// 3. Special case handling (all 0s, all 1s)

// 4. Wraparound boundary conditions

43.11 Error Conditions and Debug

43.11.1 Invalid Johnson Sequences

The `leading_one_trailing_one` module provides validation:

```
.valid(w_valid) // Indicates valid Johnson pattern
```

Valid Johnson patterns have exactly one transition from 1s to 0s (or vice versa).

43.11.2 Debug Visibility

```
// Monitor internal state for debugging:  
// - w_leading_one: Position of leftmost 1  
// - w_trailing_one: Position of rightmost 1  
// - w_all_zeroes: Special case flag  
// - w_all_ones: Special case flag  
// - w_valid: Sequence validity
```

43.11.3 Common Issues

1. **Invalid sequences:** Non-Johnson patterns on input
2. **Width mismatches:** JCW vs. actual Johnson counter width
3. **Phase confusion:** Misunderstanding first vs. second half logic
4. **Boundary conditions:** All-zeros and all-ones handling

43.12 Related Modules

- **counter_johnson:** Generates Johnson counter sequences
- **leading_one_trailing_one:** Position detection helper
- **fifo_async_div2:** Primary user of this conversion
- **gray2bin:** Standard Gray-to-binary for comparison
- **fifo_control:** Uses converted binary values for status generation

43.13 Advanced Topics

43.13.1 Hierarchical Johnson Counters

For very large depths, consider hierarchical approach:

```
// Break large Johnson counter into smaller segments  
// Use multiple grayj2bin instances with higher-level arbitration
```

43.13.2 Alternative Position Detection

Some implementations use different position detection algorithms: - **Priority encoders:** Find first/last set bit - **Thermometer decoders:** Convert 1-hot positions - **LUT-based:** For small, fixed widths

43.14 Navigation

- [← Back to RTLCommon Index](#)

- [← Back to Main Documentation Index](#)

44 Hex to 7-Segment Display (hex_to_7seg.sv)

44.1 Purpose

Converts 4-bit hexadecimal values (0-F) to 7-segment display patterns for common anode displays. Provides human-readable display of digital values for debugging, user interfaces, and embedded systems.

44.2 Ports

44.2.1 Input Ports

- **hex[3:0]** - 4-bit hexadecimal input (0x0 to 0xF)

44.2.2 Output Ports

- **seg[6:0]** - 7-segment display pattern

44.2.3 Parameters

None - fixed functionality for standard 7-segment displays.

44.3 7-Segment Display Layout

44.3.1 Segment Naming Convention

```

aaa
f   b
f   b
ggg
e   c
e   c
ddd

```

44.3.2 Bit Assignment

- **seg[6:0]** = {g, f, e, d, c, b, a}
- **Common Anode:** 0 = segment ON, 1 = segment OFF
- **Active Low:** LED segments illuminate when driven low

44.4 Character Patterns

44.4.1 Complete Truth Table

Hex	Display	seg[6:0]	Segments ON	Pattern
0x0	0	1000000	abcdef	Classic 0
0x1	1	1111001	bc	Right side
0x2	2	0100100	abged	Snake pattern
0x3	3	0110000	abgcd	Right-heavy
0x4	4	0011001	fgbc	Left + right
0x5	5	0010010	afgcd	S-curve
0x6	6	0000010	afgdce	Almost all
0x7	7	1111000	abc	Top + right
0x8	8	0000000	all	All segments
0x9	9	0011000	abgcdf	Top + middle + right
0xA	A	0001000	abcefg	Letter A
0xB	b	0000011	fgedc	Lowercase b
0xC	C	1000110	afed	Letter C
0xD	d	0100001	bgedc	Lowercase d
0xE	E	0000110	afged	Letter E
0xF	F	0001110	afge	Letter F

44.4.2 Visual Representation

```

###  #  ###  ###  #  #  ###  ###  ###  ###  ###
#  #  #  #  #  #  #  #  #  #  #  #  #  #  #
#  #  #  ###  ###  ###  ###  ###  #  ###  ###
#  #  #  #  #  #  #  #  #  #  #  #  #
###  #  ###  ###  #  ###  ###  #  ###  #
0    1    2    3    4    5    6    7    8    9

###  ###  ##  ###  ###  ###

```

```

#   # #   #   #   # #   #
###   ###   #   #   # ###   ###
#   # #   # #   #   # #   #
#   # ###   ##   ###   ###   #
    A       b       C       d       E       F

```

44.5 Implementation Details

44.5.1 Case Statement Structure

```

always_comb begin
  casez (hex)
    4'h0: seg = 7'b1000000; // 0: abcdef (not g)
    4'h1: seg = 7'b1111001; // 1: bc
    4'h2: seg = 7'b0100100; // 2: abged
    4'h3: seg = 7'b0110000; // 3: abgcd
    4'h4: seg = 7'b0011001; // 4: fgbc
    4'h5: seg = 7'b0010010; // 5: afgcd
    4'h6: seg = 7'b0000010; // 6: afgdce
    4'h7: seg = 7'b1111000; // 7: abc
    4'h8: seg = 7'b0000000; // 8: all segments
    4'h9: seg = 7'b0011000; // 9: abgcdf
    4'ha: seg = 7'b0001000; // A: abcefg (not d)
    4'hb: seg = 7'b0000011; // b: fgedc (not ab)
    4'hc: seg = 7'b1000110; // C: afed (not bcd)
    4'hd: seg = 7'b0100001; // d: bgedc (not af)
    4'he: seg = 7'b0000110; // E: afged (not bc)
    4'hf: seg = 7'b0001110; // F: afge (not bcd)
    default: seg = 7'bxxxxxxx; // Invalid - undefined output
  endcase
end

```

44.5.2 Design Choices

44.5.2.1 Lowercase vs. Uppercase

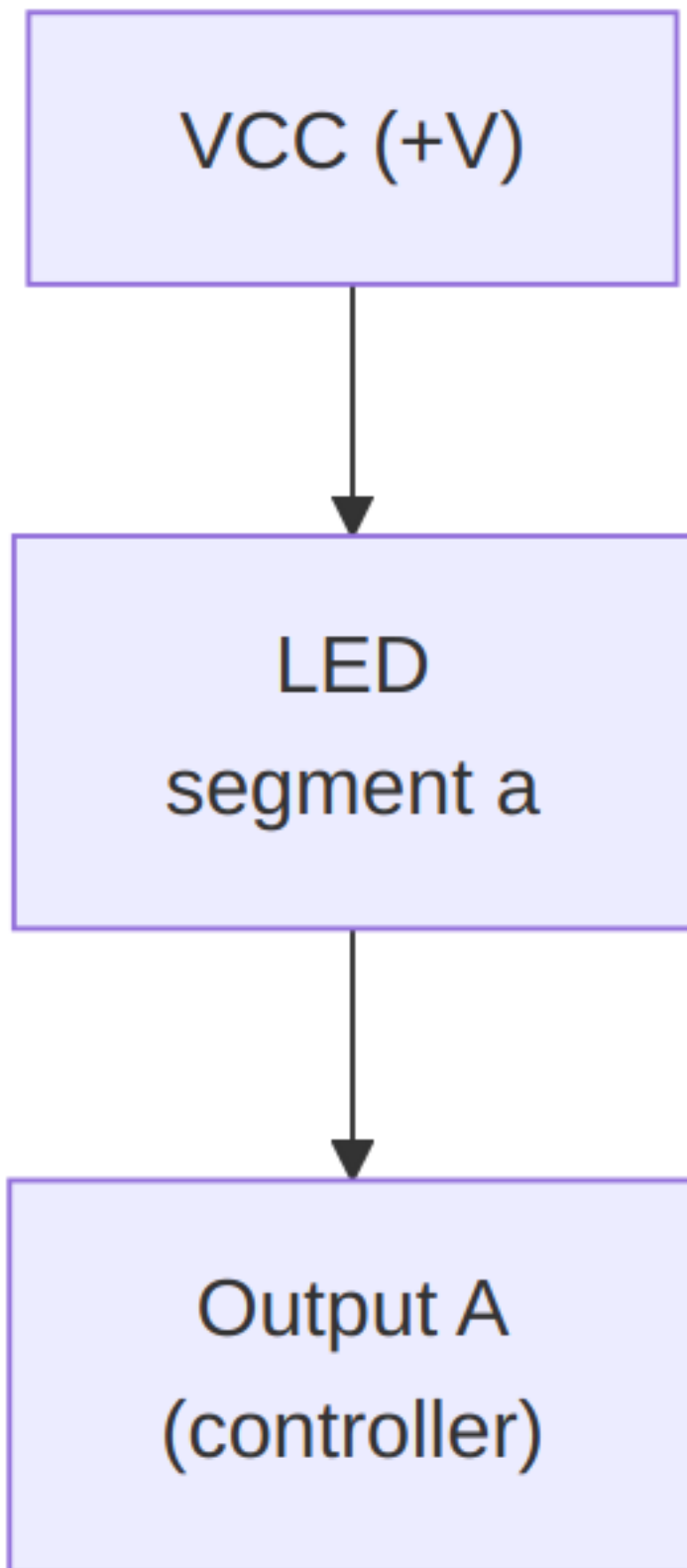
- A: Uppercase (more readable)
- b, d: Lowercase (distinguishable from 8, 0)
- C: Uppercase (standard representation)
- E, F: Uppercase (clear representation)

44.5.2.2 Common Anode Configuration

- **Active low outputs:** 0 = LED on, 1 = LED off
- **Current sourcing:** Display provides +V, controller sinks current

- **Logic inversion:** Many controllers expect active-high, requiring external inversion

44.6 Display Technology Integration



Common Anode Connection

44.6.1 Common Cathode Alternative

For common cathode displays, invert all outputs:

```
// For common cathode displays:  
assign seg_common_cathode = ~seg;
```

44.6.2 Current Limiting

Controller → [Resistor] → LED Segment → Common Anode (+V)

Typical resistor values: 220Ω - 1kΩ depending on LED specifications.

44.7 Use Cases and Applications

44.7.1 Debug Displays

```
// Display processor status  
hex_to_7seg status_display (  
    .hex(processor_status[3:0]),  
    .seg(status_segments)  
);
```

44.7.2 Multi-Digit Displays

```
// 4-digit hex display  
hex_to_7seg digit0 (.hex(value[3:0]), .seg(seg0));  
hex_to_7seg digit1 (.hex(value[7:4]), .seg(seg1));  
hex_to_7seg digit2 (.hex(value[11:8]), .seg(seg2));  
hex_to_7seg digit3 (.hex(value[15:12]), .seg(seg3));
```

44.7.3 Memory/Register Viewers

```
// Display memory contents  
hex_to_7seg mem_display (  
    .hex(memory_data[address]),  
    .seg(memory_segments)  
);
```

44.7.4 Error Code Display

```
// Show error codes  
hex_to_7seg error_display (  
    .hex(error_code),  
    .seg(error_segments)  
);
```

44.8 Timing Characteristics

44.8.1 Propagation Delay

- **Combinational logic:** Case statement maps to LUT
- **Typical delay:** 1 LUT delay (sub-nanosecond)
- **No clock dependency:** Immediate response to input changes

44.8.2 Display Update Rate

- **Electronic response:** Immediate (propagation delay)
- **Visual response:** Limited by LED rise/fall times (~microseconds)
- **Human perception:** >30Hz refresh rate for flicker-free display

44.9 Design Variations

44.9.1 Extended Character Set

Some implementations add additional characters:

```
// Extended characters (implementation dependent)
4'h10: seg = 7'b1111111; // Blank/space
4'h11: seg = 7'b0111111; // Dash/minus
4'h12: seg = 7'b1110111; // Underscore
```

44.9.2 Alternative Fonts

Different character representations for better readability:

```
// Alternative 'b' (uppercase-like)
4'hb: seg = 7'b0000111; // More symmetric b
```

44.9.3 Brightness Control

```
// PWM brightness control
logic [6:0] seg_raw;
hex_to_7seg decoder (.hex(hex_in), .seg(seg_raw));

// Apply PWM for brightness
assign seg_out = brightness_enable ? seg_raw : 7'b1111111;
```

44.10 Multiplexed Display Support

44.10.1 Time-Division Multiplexing

```
// 4-digit multiplexed display
logic [1:0] digit_select;
logic [3:0] current_hex;

// Select current digit data
always_comb begin
    case (digit_select)
        2'b00: current_hex = value[3:0];    // Digit 0
        2'b01: current_hex = value[7:4];    // Digit 1
        2'b10: current_hex = value[11:8];   // Digit 2
        2'b11: current_hex = value[15:12];  // Digit 3
    endcase
end

hex_to_7seg display_decoder (
    .hex(current_hex),
    .seg(segments)
);

// Digit enable signals (one active at a time)
assign digit_enable = ~(4'b0001 << digit_select);
```

44.10.2 Refresh Rate Calculation

```
// For 4-digit display at 1kHz total refresh:
// Each digit displayed for 250µs
// Switching frequency: 4kHz

localparam REFRESH_DIVIDER = CLOCK_FREQ / 4000;
```

44.11 Verification and Testing

44.11.1 Exhaustive Testing

```
// Test all 16 hex values
for (int i = 0; i < 16; i++) begin
    hex_input = i[3:0];
    #1;
    expected_segments = reference_table[i];
    assert(seg_output == expected_segments);
end
```

44.11.2 Visual Verification

- **Physical testing:** Connect to actual 7-segment display
- **Simulation:** Use testbench with visual output
- **Reference comparison:** Compare with known-good implementations

44.11.3 Pattern Validation

```
// Verify no segments are permanently on/off
logic [6:0] all_segments_used = '0;
for (int i = 0; i < 16; i++) begin
    all_segments_used |= ~reference_table[i]; // Accumulate active
segments
end
assert(all_segments_used == 7'b1111111); // All segments used
somewhere
```

44.12 Common Issues and Debugging

44.12.1 Segment Mapping Errors

- **Bit order:** Ensure {g,f,e,d,c,b,a} ordering matches hardware
- **Active level:** Verify common anode vs. common cathode
- **Pin assignment:** Check physical connections match bit assignments

44.12.2 Display Quality Issues

- **Ghost segments:** Check current limiting and multiplexing timing
- **Dim display:** Verify current levels and refresh rates
- **Flicker:** Ensure adequate refresh rate (>30Hz visible, >60Hz preferred)

44.12.3 Character Readability

- **Ambiguous characters:** 6 vs b, 8 vs B, 0 vs O
- **Context:** Use surrounding digits/letters for disambiguation
- **Alternative fonts:** Consider modified patterns for better distinction

44.13 Related Modules

- **BCD to 7-segment:** For decimal-only displays

- **ASCII to 7-segment:** Extended character set support
- **Display multiplexer:** Time-division multiplexing controller
- **PWM brightness controller:** Dynamic brightness adjustment

44.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

45 icg

An Integrated Clock Gating (ICG) module that provides controlled clock gating for power optimization in digital designs.

45.1 Overview

The `icg` module implements a latch-based clock gating cell that allows selective disabling of clock signals to reduce dynamic power consumption. This is a fundamental power management technique used extensively in modern ASIC and FPGA designs.

45.2 Module Declaration

```
module icg(
    input  logic en,
    input  logic clk,
    output logic gclk
);
```

45.3 Ports

45.3.1 Inputs

Port	Width	Description
en	1	Enable signal - when high, allows clock to pass through
clk	1	Input clock signal to be gated

45.3.2 Outputs

Port	Width	Description
gclk	1	Gated clock output - clk when en=1, low when en=0

45.4 Functionality

45.4.1 Clock Gating Operation

The ICG module implements a safe clock gating scheme that:

1. **Latches Enable Signal:** Uses a negative-edge triggered latch to capture the enable signal
2. **Prevents Glitches:** Ensures the enable signal is stable during clock high periods
3. **Generates Gated Clock:** ANDs the latched enable with the input clock

45.4.2 Implementation Details

```
logic en_out;
```

```
// Latch-based approach - enable is latched on clock low
always_latch begin
    if (!clk) begin
        en_out = en;
    end
end
```

```
// Generate gated clock
assign gclk = en_out && clk;
```

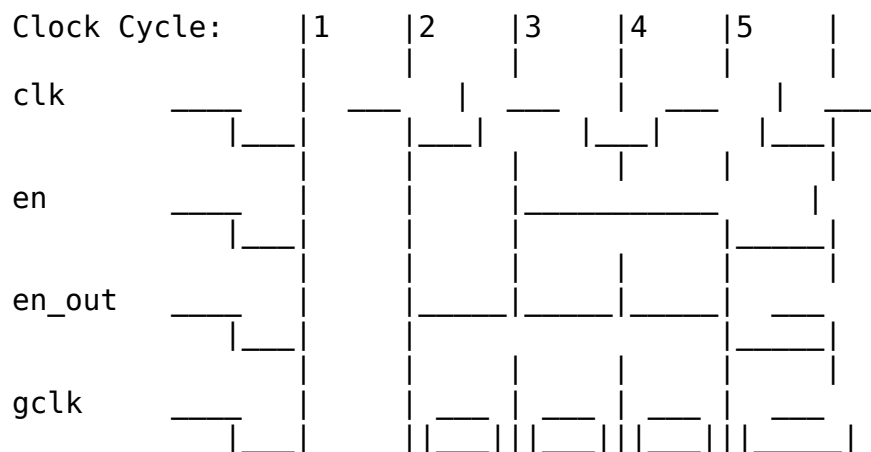
45.5 Timing Characteristics

45.5.1 Critical Timing Requirements

Characteristic	Description
Setup Time	en must be stable before clk rising edge
Hold Time	en must remain stable after clk rising edge

Characteristic	Description
Propagation Delay	Delay from clk to gclk through AND gate
Enable Latching	en is latched when clk is low

45.5.2 Timing Diagram



45.6 Power Optimization Benefits

45.6.1 Dynamic Power Reduction

The ICG module reduces dynamic power consumption by:

- **Clock Tree Power:** Eliminates switching activity in downstream clock networks
- **Register Power:** Prevents unnecessary flip-flop transitions when data is not changing
- **Combinational Power:** Reduces spurious switching in logic fed by gated registers

45.6.2 Power Savings Calculation

Typical power savings achievable: - **Clock Network:** 20-40% reduction in clock tree power - **Sequential Logic:** 30-60% reduction in flip-flop power - **Overall Dynamic Power:** 10-30% depending on gating efficiency

45.7 Design Considerations

45.7.1 Safety Requirements

- **Glitch-Free Operation:** Latch-based design prevents glitches on gated clock
- **Setup/Hold Timing:** Enable signal must meet timing requirements relative to clock
- **Clock Domain:** All logic using gclk must be in the same clock domain

45.7.2 Enable Signal Guidelines

```
// Good: Enable changes during clock low period
always_ff @(posedge clk) begin
    if (rst_n) begin
        enable_reg <= new_enable_value; // Changes on clock edge
    end
end

// Caution: Ensure enable is stable during clock high
assign icg_enable = enable_reg && additional_condition;
```

45.8 Usage Examples

45.8.1 Basic Register Bank Gating

```
// Clock gating for a register bank
logic bank_enable;
logic gated_clk;

icg u_bank_icg (
    .en    (bank_enable),
    .clk   (sys_clk),
    .gclk  (gated_clk)
);

// Register bank using gated clock
always_ff @(posedge gated_clk or negedge rst_n) begin
    if (!rst_n) begin
        register_bank <= '0;
    end else begin
        register_bank <= next_register_values;
    end
end
```

45.8.2 CPU Pipeline Stage Gating

```
// Pipeline stage clock gating
logic pipe_stage_valid;
logic stage_clk;

icg u_pipe_icg (
    .en    (pipe_stage_valid),
    .clk   (cpu_clk),
    .gclk  (stage_clk)
);

// Pipeline stage registers
always_ff @(posedge stage_clk or negedge rst_n) begin
    if (!rst_n) begin
        pipe_stage_regs <= '0;
    end else begin
        pipe_stage_regs <= pipe_stage_inputs;
    end
end
```

45.8.3 Memory Interface Gating

```
// Memory controller clock gating
logic mem_access_active;
logic mem_clk;

icg u_mem_icg (
    .en    (mem_access_active),
    .clk   (system_clk),
    .gclk  (mem_clk)
);

assign mem_access_active = mem_read_req || mem_write_req || mem_busy;
```

45.9 Implementation Alternatives

45.9.1 Standard Cell ICG

Many ASIC libraries provide optimized ICG cells:

```
// Using library ICG cell (example)
CLOCK_GATE_ICG u_lib_icg (
    .CLK    (clk),
    .EN     (en),
    .GCLK   (gclk)
);
```

45.9.2 FPGA Implementation

For FPGA targets, use dedicated clock control primitives:

```
// Xilinx BUFGCE example
BUFGCE u_bufgce (
    .I   (clk),
    .CE  (en),
    .O   (gclk)
);
```

45.10 Verification Considerations

45.10.1 Functional Tests

- **Enable/Disable:** Verify correct gating behavior for enable transitions
- **Clock Integrity:** Ensure gated clock maintains proper duty cycle when enabled
- **Reset Behavior:** Verify proper operation during reset sequences

45.10.2 Timing Tests

```
// Timing verification example
initial begin
    // Test enable setup/hold timing
    @(negedge clk);
    en = 1'b1; // Enable during clock low
    @(posedge clk);
    // Verify gclk follows clk

    @(negedge clk);
    en = 1'b0; // Disable during clock low
    @(posedge clk);
    // Verify gclk remains low
end
```

45.10.3 Power Analysis

- **Activity Monitoring:** Track switching activity reduction in gated domains
- **Power Estimation:** Measure power savings using EDA power analysis tools
- **Efficiency Metrics:** Calculate gating efficiency (% time clock is disabled)

45.11 Synthesis Guidelines

45.11.1 Technology Mapping

- **ASIC:** Use library-specific ICG cells for optimal area/power/timing
- **FPGA:** Map to dedicated clock control primitives (BUFGCE, etc.)
- **Timing Closure:** Account for additional setup/hold requirements

45.11.2 Clock Tree Synthesis

```
# Example synthesis constraints for ICG
set_case_analysis 0 [get_pins u_icg/en] ;# For power analysis
set_clock_gating_check -setup 0.1 [get_pins u_icg/gclk]
set_clock_gating_check -hold 0.1 [get_pins u_icg/gclk]
```

45.12 Design Integration

45.12.1 Clock Tree Integration

The ICG should be placed strategically in the clock tree:

Main Clock → ICG → Local Clock Tree → Registers
 ↑
 Enable Logic

45.12.2 Power Domain Considerations

- **Retention:** Ensure enable logic remains powered during clock gating
- **Isolation:** Properly isolate gated domains in multi-voltage designs
- **Power Sequencing:** Coordinate with power management units

45.13 Related Modules

- `clock_gate_ctrl`: Enhanced clock gating controller with additional features
- `clock_divider`: Clock frequency division functionality
- `reset_sync`: Reset synchronization for gated clock domains

The `icg` module provides a fundamental building block for power-efficient digital design, enabling significant power savings through controlled clock gating while maintaining design safety and functionality.

45.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

46 Leading One Trailing One Module

46.1 Purpose

The `leading_one_trailing_one` module identifies the positions of the most significant bit (leading one) and least significant bit (trailing one) in an input data vector. It provides both index outputs and one-hot vector representations of these positions, along with status flags for edge cases.

46.2 Key Features

- Finds the highest set bit position (leading one)
- Finds the lowest set bit position (trailing one)
- Provides one-hot vector outputs for both positions
- Includes validity and edge case detection
- Parameterizable width with automatic index sizing

46.3 Port Description

46.3.1 Parameters

- **WIDTH:** Width of input data vector (default: 8)
- **INSTANCE_NAME:** String identifier for debugging (default: "")

46.3.2 Inputs

Port	Width	Description
data	WIDTH	Input data vector to analyze

46.3.3 Outputs

Port	Width	Description
leadingone	$\lceil \log_2(\text{WIDTH}) \rceil$	Index of the most significant set bit

Port	Width	Description
leadingone_vector	WIDTH	One-hot vector with leading one position marked
trailingone	$\$clog2(WIDTH)$	Index of the least significant set bit
trailingone_vector	WIDTH	One-hot vector with trailing one position marked
all_zeroes	1	High when input data is all zeros
all_ones	1	High when input data is all ones
valid	1	High when input data contains at least one set bit

46.4 Implementation Details

46.4.1 Submodules

The module instantiates two specialized finder modules:

1. **find_last_set**: Locates the position of the most significant set bit
 - Connected to leadingone output
 - Implements priority encoding from MSB to LSB
2. **find_first_set**: Locates the position of the least significant set bit
 - Connected to trailingone output
 - Implements priority encoding from LSB to MSB

46.4.2 One-Hot Vector Generation

```

always_comb begin
    leadingone_vector = '0;
    trailingone_vector = '0;

    // Only set vector bits if there is valid data
    if (!data) begin
        if (int'(leadingone) < WIDTH) begin
            leadingone_vector[leadingone] = 1'b1;
        end

        if (int'(trailingone) < WIDTH) begin

```



```

        trailingone_vector[trailingone] = 1'b1;
    end
end
end

```

46.4.3 Status Flag Logic

- **all_ones:** Uses reduction AND (&data) to detect when all bits are set
- **all_zeroes:** Uses reduction OR negation (~(|data)) to detect when no bits are set
- **valid:** Uses reduction OR (|data) to indicate presence of at least one set bit

46.5 Special Implementation Notes

1. **Index Width Calculation:** Uses `$clog2(WIDTH)` to automatically size index outputs for any input width
2. **Bounds Checking:** Includes explicit bounds checking (`int'(leadingone) < WIDTH`) to prevent out-of-range indexing
3. **Zero Input Handling:** When input is all zeros:
 - One-hot vectors remain all zeros
 - Index values are undefined but bounded
 - `valid` flag indicates invalid state
4. **Type Casting:** Uses `int'()` casting for index comparisons to ensure proper integer arithmetic

46.6 Usage Examples

46.6.1 8-bit Example

```

Input:  data = 8'b00101000
Output: leadingone = 3'd5
        leadingone_vector = 8'b00100000
        trailingone = 3'd3
        trailingone_vector = 8'b00001000
        all_zeroes = 1'b0
        all_ones = 1'b0
        valid = 1'b1

```

46.6.2 Edge Cases

Input: data = 8'b00000000
Output: leadingone = undefined
leadingone_vector = 8'b00000000
trailingone = undefined
trailingone_vector = 8'b00000000
all_zeroes = 1'b1
all_ones = 1'b0
valid = 1'b0

Input: data = 8'b11111111
Output: leadingone = 3'd7
leadingone_vector = 8'b10000000
trailingone = 3'd0
trailingone_vector = 8'b00000001
all_zeroes = 1'b0
all_ones = 1'b1
valid = 1'b1

46.7 Applications

- Priority arbitration systems
- Bit manipulation operations
- Data compression algorithms
- Network packet processing
- Cache management systems

46.8 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

47 Modulo-3 Compressor

Module: mod_3_compress.sv **Location:** rtl/common/ **Status:** Production Ready

47.1 Overview

mod_3_compress is a purely combinational block that computes $d_{in} \bmod 3$ for a 16-bit operand, returning the 2-bit remainder (0..2). It is built in the same carry-save-compressor style as div_by_15_ceil_32compress.sv, a 3:2 compressor tree feeding a final carry-propagate add and

a small fold, but it emits only the remainder, which is exactly what the monbus burst-writer needs to round a beat count **down** to a whole number of 3-beat records: $\text{rounded} = X - (X \bmod 3)$.

It uses no $*$ or $/$ operator, so it infers no DSP block and no iterative divider, just a shallow tree of carry-save adders and a couple of small adds.

47.1.1 Key Features

- **Combinational, single-cycle:** No clock, no state; pure logic from `d_in` to `rem_out`
- **No multiply / divide:** Avoids inferred DSP and iterative dividers
- **Carry-save tree:** Eight base-4 digits reduced with 3:2 compressors instead of a ripple-add chain
- **Remainder only:** Returns just $d_{in} \bmod 3$ (0..2), the quantity the monbus record rounding needs
- **Shared style:** Same construction as `div_by_15_ceil_32compress.sv`, reusing `math_adder_carry_save_nbit`

47.2 Module Purpose

The monbus compressor packs monitor packets into fixed 3-beat records. To decide how many whole records a beat count covers, it needs $X - (X \bmod 3)$, which requires the remainder $X \bmod 3$. This module computes that remainder combinationaly without a divider, so the burst-writer can round the count down in a single cycle of logic.

Use Cases:

- Rounding a monbus beat count down to a whole number of 3-beat records ($X - (X \bmod 3)$)
- Any combinational $\bmod 3$ of a 16-bit value where a DSP-free, divider-free implementation is desired
- A worked reference for the base-4 digit-sum residue technique with carry-save reduction

Key Benefit: Produces $d_{in} \bmod 3$ in one combinational stage with a shallow carry-save tree, no multiplier, divider, or DSP block required.

47.3 Parameters

This module has no parameters. The operand width is fixed at 16 bits and the result at 2 bits.

(Internally, `localparam int BITS = 6` sizes the carry-save datapath so the weight-2 carry left-shifts have headroom.)

47.4 Port Groups

47.4.1 Data

Port	Direction	Width	Description
d_in	Input	16	Operand whose remainder mod 3 is computed
rem_out	Output	2	d_in mod 3, in the range 0..2

47.5 Functional Description

47.5.1 Base-4 Digit-Sum Method

The core identity is that $4^k \equiv 1 \pmod{3}$. If d_in is split into 2-bit groups, each group is a base-4 digit with weight 4^k , and since every weight is $1 \pmod{3}$, the **sum of the eight digits is congruent to d_in (mod 3)**. The problem therefore reduces to summing eight small digits and taking that sum mod 3.

The eight 2-bit groups are zero-extended to the 6-bit carry-save width:

```
assign w_g0 = {{(BITS-2){1'b0}}, d_in[1:0]};
assign w_g1 = {{(BITS-2){1'b0}}, d_in[3:2]};
// ... through ...
assign w_g7 = {{(BITS-2){1'b0}}, d_in[15:14]};
```

47.5.23:2 Compressor Tree

Rather than a ripple-add chain, the eight digits are reduced with a tree of `math_adder_carry_save_nbit 3:2` compressors. Each carry output has weight 2, so wherever a carry feeds the next level it is left-shifted by one (`{carry[BITS-2:0], 1'b0}`); the always-zero top carry bit is dropped, exactly as the div-by-15 reference does:

- **Level 1:** three compressors reduce the eight digits three at a time (the last triple padded with 0)

- **Level 2:** two compressors combine the level-1 sums and shifted carries
- **Level 3:** one compressor, holding `w_carryE2` for the next level
- **Level 4:** one compressor folds in the held carry

A final carry-propagate add collapses the last sum/carry pair into the digit sum ($0..24$, still $\equiv d_in \pmod{3}$):

```
assign w_grp_sum = w_sumG4 + {w_carryG4[BITS-2:0], 1'b0};
```

47.5.3 Final Fold and Conditional Subtract

The digit sum `w_grp_sum` is at most 24, so it is folded once more onto its own base-4 digits (same residue mod 3), giving a value in $0..7$, and then a conditional subtract brings it into the final $0..2$ range:

```
assign w_fold = {2'b0, w_grp_sum[1:0]} + {2'b0, w_grp_sum[3:2]} +
{3'b0, w_grp_sum[4]};
assign rem_out = 2'((w_fold >= 4'd6) ? (w_fold - 4'd6)
                  : (w_fold >= 4'd3) ? (w_fold - 4'd3)
                  : w_fold);
```

The two-branch subtract handles `w_fold` values up to 7 (subtract 6, subtract 3, or pass through), yielding the exact remainder 0, 1, or 2.

47.6 Usage Example

```
// Round a beat count down to a whole number of 3-beat monbus records.
logic [15:0] beat_count;
logic [1:0] beat_rem;
logic [15:0] rounded_count;

mod_3_compress u_mod3 (
    .d_in      (beat_count),
    .rem_out   (beat_rem)
);

// rounded = X - (X mod 3)
assign rounded_count = beat_count - {14'b0, beat_rem};
```

47.7 Design Notes

- **Why remainder only?** The div-by-15 reference computes a quotient; here only the residue is needed, so the quotient machinery is dropped, keeping the logic shallow.

- **Datapath width headroom.** BITS = 6 (rather than the minimum 5 needed for a 0..24 sum) gives the weight-2 carry left-shifts room so the dropped top carry is always 0, matching the div-by-15 construction.
- **No inferred arithmetic primitives.** Because there is no * or /, synthesis produces adders and LUTs only, no DSP slices, no multi-cycle divider.
- **Purely combinational.** rem_out tracks d_in with no clock; register the result externally if a pipeline stage is desired.

47.8 Related Modules

47.8.1 Used By

- [monbus_compressor](#) - Rounds a beat count down to whole 3-beat records when packing monitor packets
- [monbus_group_core](#) - Shared filter/FIFO core of the monbus group wrappers

47.8.2 Uses

- [math_adder_carry_save](#) - The N-bit carry-save (3:2) compressor instantiated throughout the tree

47.8.3 See Also

- `div_by_15_ceil_32compress.sv` - The reference implementation this module's compressor style follows

47.9 References

47.9.1 Source Code

- `rtl/common/mod_3_compress.sv`
- `rtl/common/math_adder_carry_save.sv` (instantiated submodule)
- `rtl/common/div_by_15_ceil_32compress.sv` (style reference)

47.9.2 Documentation

- `rtl/common/PRD.md`

Last Updated: 2026-07-15

47.10 Navigation

- [Back to RTLCommon Index](#)

48 PWM (Pulse Width Modulation) Module

48.1 Purpose

The PWM module generates multiple independent pulse width modulation signals with configurable duty cycles, periods, and repeat counts. Each channel operates as an independent state machine with precise timing control and completion detection.

48.2 Key Features

- Multiple independent PWM channels (parameterizable)
- Individual duty cycle, period, and repeat count control per channel
- Edge-triggered start control with automatic edge detection
- Completion status reporting per channel
- Support for continuous operation (infinite repeat)
- Precise timing with configurable counter width

48.3 Port Description

48.3.1 Parameters

- **WIDTH:** Bit width of counters and timing parameters (default: 8)
- **CHANNELS:** Number of independent PWM channels (default: 4)

48.3.2 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
start	CHANNELS	Start trigger for each channel (edge-triggered)
duty	CHANNELS*WIDTH	Concatenated duty cycle values for all channels

Port	Width	Description
period	CHANNELS*WIDTH	Concatenated period values for all channels
repeat_count	CHANNELS*WIDTH	Concatenated repeat counts (0 = infinite)

48.3.3 Outputs

Port	Width	Description
done	CHANNELS	Completion status for each channel
pwm_out	CHANNELS	PWM output signals for each channels

48.4 State Machine

Each PWM channel implements a 3-state finite state machine:

48.4.1 States

```
typedef enum logic [1:0] {
    IDLE = 2'b00,    // Waiting for start trigger
    RUNNING = 2'b01, // Actively generating PWM
    DONE = 2'b10     // Completed all repeats
} state_t;
```

48.4.2 State Transitions

48.4.2.1 IDLE → RUNNING

- **Condition:** `w_start_edge && local_period > 0`
- **Action:** Reset counters and begin PWM generation

48.4.2.2 RUNNING → DONE

- **Condition:** `w_period_complete && w_all_repeats_done`
- **Action:** Stop PWM generation and assert done flag

48.4.2.3 DONE → RUNNING

- **Condition:** `w_start_edge`
- **Action:** Restart PWM generation with reset counters

48.4.2.4 Any State → IDLE

- **Condition:** System reset
- **Action:** Initialize all registers

48.5 Implementation Details

48.5.1 Per-Channel Parameter Extraction

```
localparam int EndIdx = (i + 1) * WIDTH - 1;
assign local_duty = duty[EndIdx:WIDTH];
assign local_period = period[EndIdx:WIDTH];
assign local_repeat = repeat_count[EndIdx:WIDTH];
```

48.5.2 Edge Detection for Start Signal

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) r_start_prev <= 1'b0;
    else r_start_prev <= start[i];
end
assign w_start_edge = start[i] && !r_start_prev;
```

48.5.3 PWM Output Generation Logic

```
always_comb begin
    case (r_state)
        RUNNING: begin
            if (local_duty == 0) begin
                // 0% duty cycle
                pwm_out[i] = 1'b0;
            end else if (local_duty >= local_period) begin
                // 100% duty cycle (duty >= period)
                pwm_out[i] = 1'b1;
            end else begin
                // Normal case: PWM high for first 'duty' cycles
                pwm_out[i] = (r_count < local_duty);
            end
        end
        default: begin
            pwm_out[i] = 1'b0;
        end
    endcase
end
```

48.5.4 Counter Management

48.5.4.1 Main Counter (Period Timing)

- Increments each clock cycle during RUNNING state

- Resets to 0 when reaching `local_period - 1`
- Controls PWM frequency

48.5.4.2 *Repeat Counter*

- Increments each time a period completes
- Compared against `repeat_count` to determine completion
- Special case: `repeat_count == 0` means infinite operation

48.5.5 Critical Control Signals

48.5.5.1 *Period Completion Detection*

```
assign w_period_complete = (r_count == local_period - 1) && (r_state == RUNNING);
```

48.5.5.2 *All Repeats Done Detection*

```
assign w_all_repeats_done = (local_repeat == 0) ? 1'b0 : // 0 means infinite
                             (r_repeat_value >= local_repeat);
```

48.6 Special Implementation Notes

48.6.1 1. Edge-Triggered Start Control

- Prevents continuous restart while start signal is held high
- Enables precise control of PWM initiation
- Each channel has independent edge detection

48.6.2 2. Duty Cycle Edge Cases

- **0% Duty:** Output permanently low during RUNNING state
- **100% Duty:** Output permanently high when `duty >= period`
- **Normal Operation:** High for first duty clock cycles of each period

48.6.3 3. Infinite Repeat Mode

- `repeat_count = 0` enables continuous operation
- PWM runs indefinitely until new start trigger
- Useful for continuous motor control or LED dimming

48.6.44. Simultaneous Multi-Channel Operation

- All channels operate independently
- No inter-channel dependencies or conflicts
- Scalable to any number of channels via parameters

48.6.55. Reset Behavior

- Asynchronous reset immediately stops all PWM outputs
- All internal counters and state machines reset to IDLE
- Clean startup guaranteed after reset deassertion

48.7 Timing Examples

48.7.1 Single Period Example (WIDTH=8, duty=3, period=8)

```
Clock:  0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 ...
Count:  0 1 2 3 4 5 6 7 0 1 2 3 4 5 6 7 ...
PWM:    1 1 1 0 0 0 0 0 1 1 1 0 0 0 0 0 ...
```

48.7.2 Multi-Repeat Example (repeat_count=2)

```
Period: 1      2      DONE
PWM:    [Solid][Hatched][Solid][Hatched]
```

48.8 Usage Scenarios

48.8.1 Motor Speed Control

```
// 50% duty cycle, 1kHz frequency (assuming 1MHz clock)
duty = 500;
period = 1000;
repeat_count = 0; // Continuous operation
```

48.8.2 LED Dimming Sequence

```
// Fade effect with 10 brightness levels
duty = brightness_level * 10;
period = 100;
repeat_count = fade_duration;
```

48.8.3 Servo Motor Control

```
// 20ms period with 1-2ms pulse width
duty = servo_position; // 1000-2000 for 1-2ms
```

```
period = 20000;           // 20ms at 1MHz clock
repeat_count = 1;         // Single pulse
```

48.9 Applications

- Motor speed control
- LED brightness control
- Servo motor positioning
- Audio signal generation
- Power supply switching
- Heating element control

48.10 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

49 Reset Synchronizer Module

49.1 Purpose

The `reset_sync` module implements a parameterized reset synchronizer that provides asynchronous assertion and synchronous deassertion of reset signals. This is a critical circuit for proper reset distribution in synchronous digital systems, ensuring that reset release occurs synchronously with the clock to prevent metastability and timing violations.

49.2 Key Features

- Asynchronous reset assertion (immediate)
- Synchronous reset deassertion (clocked)
- Parameterizable synchronization depth
- Metastability resolution
- Standard reset synchronizer topology
- Clean reset release timing

49.3 Port Description

49.3.1 Parameters

- N: Number of synchronization stages (default: 3)

49.3.2 Inputs

Port	Width	Description
clk	1	System clock for synchronization
rst_n	1	Asynchronous input reset (active-low)

49.3.3 Outputs

Port	Width	Description
sync_rst_n	1	Synchronized reset output (active-low)

49.4 Reset Synchronizer Theory

49.4.1 The Reset Problem

In synchronous digital systems, reset signals must be carefully managed to avoid: - **Metastability**: When reset is released asynchronously relative to clock - **Timing Violations**: Setup/hold violations during reset release - **Race Conditions**: Different flip-flops releasing from reset at different times

49.4.2 Asynchronous Assert, Synchronous Deassert

The ideal reset behavior is: - **Assert**: Immediate (asynchronous) when reset input becomes active - **Deassert**: Synchronized to clock edge to ensure clean timing

49.5 Implementation Details

49.5.1 Synchronization Register Chain

```
logic [N-1:0] r_sync_reg = {N{1'b0}};
```

An N-stage shift register initialized to all zeros provides the synchronization delay.

49.5.2 Reset Logic (Corrected Implementation)

```
always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin                                // When rst_n is LOW
        (active reset)
```

```

        r_sync_reg <= {N{1'b0}};           // Clear register
    (reset asserted)
    end else begin                           // When rst_n is
HIGH (inactive)
        r_sync_reg <= {r_sync_reg[N-2:0], 1'b1}; // Shift in 1's
    (releasing reset)
    end
end

```

Critical Bug Fix (2025-10-14): - **Original bug:** Logic was inverted (if (rst_n) instead of if (!rst_n)) - **Impact:** Module was completely non-functional - reset never worked correctly - **Discovered by:** Comprehensive test suite (val/common/test_reset_sync.py) - **Fix:** Corrected reset polarity check - **Verification:** All 4 test configurations passing (N=2, 3, 4, 5)

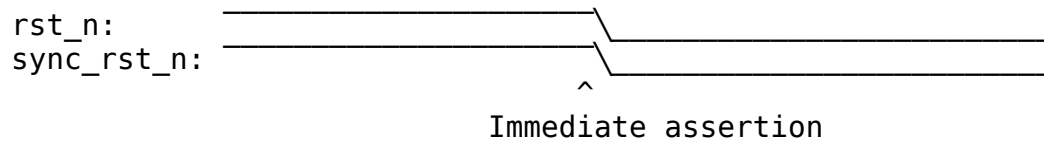
49.5.3 Output Assignment

```
assign sync_rst_n = r_sync_reg[N-1];
```

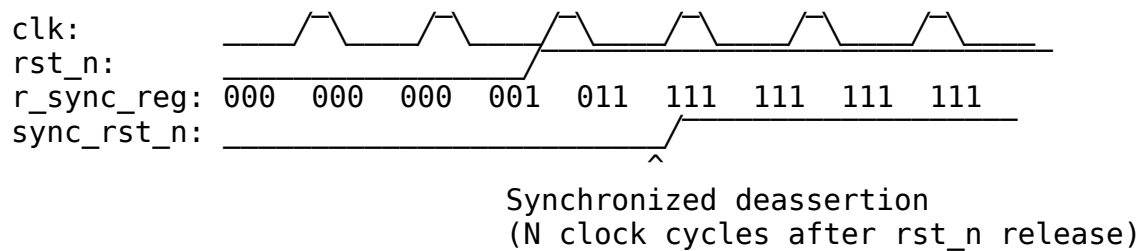
The synchronized reset output comes from the final stage of the shift register.

49.6 Timing Behavior

49.6.1 Reset Assertion (Asynchronous)



49.6.2 Reset Deassertion (Synchronous, N=3)



49.6.3 State Sequence for N=3

Clock	rst_n	r_sync_reg	sync_rst_n	Description
-	0	000	0	Reset active, output low
1	0	001	0	First 1 shifted in
2	0	011	0	Second 1

Clock	rst_n	r_sync_reg	sync_rst_n	Description
3	0	111	1	shifted in Third 1 reaches output
4+	0	111	1	Reset deasserted, stable

49.7 Special Implementation Notes

49.7.1 1. Inverted Logic Explanation

The seemingly counterintuitive reset condition logic serves a specific purpose: - During reset: shift register accumulates 1's - After reset release: shift register is cleared, then accumulates 1's again - This ensures N clock cycles of delay before output goes high

49.7.22. Metastability Resolution

The N-stage design provides multiple clock cycles for metastability resolution: - Each flip-flop stage resolves potential metastability - Probability of metastability propagation: $(MTBF)^{(-N)}$ - N=2 typically sufficient, N=3 provides extra margin

49.7.33. Initialization

logic [N-1:0] r_sync_reg = {N{1'b0}};

Explicit initialization ensures predictable power-on behavior.

49.7.44. Parameterization Benefits

Configurable N allows optimization for different requirements: - N=2: Minimum recommended, fast reset release - N=3: Standard choice, good metastability margin - N≥4: Conservative, slower but very robust

49.8 Parameter Selection Guidelines

49.8.1 Choosing N Value

49.8.1.1 N=2 (Minimum)

```
reset_sync #(N(2)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when:** Fast reset release required
- **Risk:** Minimal metastability margin
- **Applications:** High-speed designs with good clock quality

49.8.1.2 $N=3$ (Recommended)

```
reset_sync #(.N(3)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when:** Standard applications
- **Benefit:** Good balance of speed vs. robustness
- **Applications:** Most digital designs

49.8.1.3 $N \geq 4$ (Conservative)

```
reset_sync #(.N(4)) rst_sync_inst (
    .clk(sys_clk),
    .rst_n(external_rst_n),
    .sync_rst_n(internal_rst_n)
);
```

- **Use when:** Critical applications, poor clock quality
- **Benefit:** Maximum metastability protection
- **Trade-off:** Slower reset release

49.9 Applications

49.9.1 Global Reset Distribution

// Synchronize external reset for internal use

```
reset_sync #(.N(3)) global_rst_sync (
    .clk(system_clock),
    .rst_n(external_reset_n),
    .sync_rst_n(global_reset_n)
);
```

// Use synchronized reset throughout design

```
always_ff @(posedge system_clock or negedge global_reset_n) begin
    if (!global_reset_n) begin
        // Reset state
    end
end
```



```

        end else begin
            // Normal operation
        end
    end
end

```

49.9.2 Clock Domain Crossing

```

// Separate reset synchronizers for each clock domain
reset_sync #(.N(3)) clk_a_rst_sync (
    .clk(clk_a),
    .rst_n(system_rst_n),
    .sync_rst_n(clk_a_rst_n)
);

reset_sync #(.N(3)) clk_b_rst_sync (
    .clk(clk_b),
    .rst_n(system_rst_n),
    .sync_rst_n(clk_b_rst_n)
);

```

49.9.3 Power-On Reset

```

// Synchronize power-on reset
reset_sync #(.N(3)) por_sync (
    .clk(main_clk),
    .rst_n(power_on_rst_n),
    .sync_rst_n(system_rst_n)
);

```

49.9.4 Reset Controller Integration

```

module reset_controller (
    input logic sys_clk,
    input logic external_rst_n,
    input logic watchdog_rst_n,
    input logic sw_rst,
    output logic sync_rst_n
);

    logic combined_rst_n;

    // Combine all reset sources
    assign combined_rst_n = external_rst_n & watchdog_rst_n & ~sw_rst;

    // Synchronize the combined reset
    reset_sync #(.N(3)) rst_sync_inst (
        .clk(sys_clk),
        .rst_n(combined_rst_n),
        .sync_rst_n(sync_rst_n)
    );

```

endmodule

49.10 Design Considerations

49.10.1 Reset Tree Planning

```
// Plan reset distribution hierarchy
// Level 1: Global reset synchronizer
// Level 2: Domain-specific reset synchronizers
// Level 3: Module-level reset distribution
```

49.10.2 Clock Quality Requirements

- **Clean Clock:** Stable, low-jitter clock required
- **Clock Frequency:** Must be running during reset release
- **Clock Gating:** Avoid gating clock used for reset synchronization

49.10.3 Reset Source Management

```
// Multiple reset sources need careful combination
wire combined_reset = por_n & external_rst_n & watchdog_rst_n &
~soft_reset;
```

49.11 Common Design Mistakes

49.11.1 Incorrect Sensitivity List

```
// WRONG: Missing async reset in sensitivity list
always_ff @(posedge clk) begin
    if (!rst_n) r_sync_reg <= '0;
    else r_sync_reg <= {r_sync_reg[N-2:0], 1'b1};
end
```

```
// CORRECT: Include async reset
always_ff @(posedge clk or negedge rst_n) begin
    // ... correct implementation
end
```

49.11.2 Wrong Reset Polarity

```
// Ensure consistent reset polarity throughout design
// This module uses active-low reset (rst_n)
```

49.11.3 Clock Domain Issues

```
// Each clock domain needs its own reset synchronizer
// Don't share synchronized reset across clock domains
```

49.12 Verification Considerations

49.12.1 Test Scenarios

- Reset assertion during various clock phases
- Reset deassertion timing verification
- Multiple reset assertion/deassertion cycles
- Clock jitter during reset release
- Power-on behavior

49.12.2 Assertions

```
// Verify synchronous deassertion
property sync_deassert;
    @(posedge clk)
        $rose(rst_n) |-> ##N $rose(sync_rst_n);
endproperty
```

```
// Verify asynchronous assertion
property async_assert;
    $fell(rst_n) |-> $fell(sync_rst_n);
endproperty
```

49.12.3 Coverage Points

- All reset assertion scenarios
- Reset release with different clock phases
- Multiple consecutive reset cycles
- Parameter variation coverage (different N values)

49.13 Test Verification

49.13.1 Test Coverage

The reset_sync module has comprehensive test coverage via: - **Testbench Class:** bin/TBClasses/reset_sync_tb.py - **Test Runner:** val/common/test_reset_sync.py

49.13.2 Test Scenarios

✓ **Basic Reset Synchronization** - Verifies N-cycle synchronization delay ✓ **Immediate Reset Assertion** - Verifies asynchronous assertion behavior ✓ **Multiple Reset Cycles** - Tests repeated reset/release sequences ✓ **Reset Glitch Filtering** - Validates recovery from short reset pulses

49.13.3 Running Tests

Run all reset_sync tests (4 configurations: N=2,3,4,5)

```
pytest val/common/test_reset_sync.py -v
```

Run specific configuration

```
pytest val/common/test_reset_sync.py::test_reset_sync[2-min] -v
```

Run with waveform generation

```
pytest val/common/test_reset_sync.py -v -s
```

49.13.4 Test Results

All 4 parameter configurations passing (100% success rate): - N=2 (min) - ✓ PASSED - N=3 (typical) - ✓ PASSED - N=4 (safe) - ✓ PASSED - N=5 (max) - ✓ PASSED

Bug Discovery: The comprehensive test suite discovered a critical RTL bug during initial test run (inverted reset polarity), demonstrating the value of thorough verification.

49.14 Related Modules

49.14.1 Reset Synchronizer Variants

- **Positive-edge reset:** For active-high reset systems
- **Multi-domain reset:** For complex clock domain systems
- **Reset pulse generator:** For generating reset pulses
- **Reset debouncer:** For mechanical switch inputs

49.14.2 Integration with Other Modules

- Clock domain crossing modules
- PLL/clock management units
- Power management controllers
- System controllers and reset managers

49.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

50 Reverse Vector Module

50.1 Purpose

The `reverse_vector` module performs bit-order reversal on an input vector, swapping the most significant bit with the least significant bit, second MSB with second LSB, and so on. This is a purely combinational operation that reverses the bit ordering of the input data.

50.2 Key Features

- Combinational bit reversal operation
- Parameterizable vector width
- Zero propagation delay (combinational logic only)
- Simple and efficient implementation
- No clock or reset required

50.3 Port Description

50.3.1 Parameters

- **WIDTH:** Width of the input/output vectors (default: 32)

50.3.2 Inputs

Port	Width	Description
<code>vector_in</code>	WIDTH	Input vector to be reversed

50.3.3 Outputs

Port	Width	Description
<code>vector_rev</code>	WIDTH	Bit-reversed output vector

50.4 Implementation Details

50.4.1 Bit Reversal Logic

```
always_comb begin
    for (integer i = 0; i < WIDTH; i++) begin
        vector_rev[(WIDTH-1)-i] = vector_in[i];
    end
end
```

end
end

The implementation uses a simple for-loop to map each bit from the input to its corresponding reversed position in the output: - Input bit 0 → Output bit (WIDTH-1) - Input bit 1 → Output bit (WIDTH-2) - Input bit i → Output bit (WIDTH-1-i) - Input bit (WIDTH-1) → Output bit 0

50.5 Operation Examples

50.5.1 8-bit Vector Reversal

Input: `vector_in = 8'b10110001`
Output: `vector_rev = 8'b10001101`

Bit mapping:

```
vector_in[7] = 1 → vector_rev[0] = 1
vector_in[6] = 0 → vector_rev[1] = 0
vector_in[5] = 1 → vector_rev[2] = 1
vector_in[4] = 1 → vector_rev[3] = 1
vector_in[3] = 0 → vector_rev[4] = 0
vector_in[2] = 0 → vector_rev[5] = 0
vector_in[1] = 0 → vector_rev[6] = 0
vector_in[0] = 1 → vector_rev[7] = 1
```

50.5.2 4-bit Vector Reversal

Input: `vector_in = 4'b1011`
Output: `vector_rev = 4'b1101`

Bit mapping:

```
vector_in[3] = 1 → vector_rev[0] = 1
vector_in[2] = 0 → vector_rev[1] = 0
vector_in[1] = 1 → vector_rev[2] = 1
vector_in[0] = 1 → vector_rev[3] = 1
```

50.6 Special Implementation Notes

50.6.1 1. Combinational Operation

- No clock signal required
- No sequential logic elements
- Output changes immediately when input changes
- Zero propagation delay (limited only by gate delays)

50.6.22. Parameterizable Width

The module works with any vector width specified by the WIDTH parameter: - Automatically scales to any desired width - Synthesis tools optimize for the specific width - No hardcoded bit indices

50.6.33. For-Loop Synthesis

The for-loop is completely unrolled during synthesis, creating individual wire assignments for each bit position. This results in: - Parallel bit assignments (no sequential operation) - Optimal timing characteristics - Efficient hardware implementation

50.6.44. Bidirectional Property

Applying the reverse operation twice returns the original vector:

```
original_vector == reverse(reverse(original_vector))
```

50.7 Synthesis Considerations

50.7.1 Hardware Implementation

The synthesized hardware consists of: - Direct wire connections from input bits to output bits - No logic gates required (just routing) - Minimal area overhead - Excellent timing characteristics

50.7.2 Optimization

```
// Synthesis tools will optimize this to simple wire assignments:
assign vector_rev[WIDTH-1] = vector_in[0];
assign vector_rev[WIDTH-2] = vector_in[1];
// ...
assign vector_rev[1] = vector_in[WIDTH-2];
assign vector_rev[0] = vector_in[WIDTH-1];
```

50.8 Applications

50.8.1 Endianness Conversion

```
// Convert between big-endian and little-endian bit ordering
little_endian_data = reverse_vector(big_endian_data);
```

50.8.2 Serial Communication

```
// Reverse bit order for serial transmission protocols
// that send LSB first instead of MSB first
serial_data = reverse_vector(parallel_data);
```

50.8.3 Cryptographic Operations

```
// Bit permutation operations in encryption algorithms
permuted_key = reverse_vector(encryption_key);
```

50.8.4 Signal Processing

```
// Bit-reversed addressing for FFT algorithms
reversed_address = reverse_vector(sequential_address);
```

50.8.5 Data Format Conversion

```
// Convert between different data representation formats
converted_data = reverse_vector(input_format);
```

50.8.6 Mirror Pattern Generation

```
// Create mirror patterns for display or test applications
mirror_pattern = reverse_vector(original_pattern);
```

50.9 Usage Examples

50.9.1 Instantiation with Different Widths

```
// 16-bit reversal
reverse_vector #(.WIDTH(16)) rev16 (
    .vector_in(data_16bit),
    .vector_rev(reversed_16bit)
);
```

```
// 64-bit reversal
reverse_vector #(.WIDTH(64)) rev64 (
    .vector_in(data_64bit),
    .vector_rev(reversed_64bit)
);
```

50.9.2 Pipeline Integration

```
// Use in data processing pipeline
always_ff @(posedge clk) begin
    stage1_data <= input_data;
    stage2_data <= reverse_vector_inst.vector_rev;
    stage3_data <= process(stage2_data);
end
```

50.9.3 Conditional Reversal

```
// Conditional bit reversal based on control signal
assign output_data = reverse_enable ?
    reverse_vector_inst.vector_rev :
    vector_in;
```


50.10 Design Considerations

50.10.1 Timing Analysis

- **Propagation Delay:** Minimal (wire delay only)
- **Critical Path:** Direct wire connections
- **Setup/Hold:** Not applicable (combinational)
- **Clock Frequency:** Unlimited (no sequential logic)

50.10.2 Power Consumption

- **Static Power:** Minimal (no storage elements)
- **Dynamic Power:** Only during input transitions
- **Switching Activity:** Proportional to input bit toggles

50.10.3 Area Utilization

- **Logic Elements:** Zero (pure routing)
- **Routing Resources:** WIDTH wire connections
- **Memory:** None required
- **Scalability:** Linear with WIDTH

50.11 Verification Considerations

50.11.1 Test Cases

```
// Test all zeros
vector_in = '0;
assert(vector_rev == '0);

// Test all ones
vector_in = '1;
assert(vector_rev == '1);

// Test alternating pattern
vector_in = 8'b10101010;
assert(vector_rev == 8'b01010101);

// Test double reversal
```

```
temp = reverse_vector_inst1.vector_rev;  
assert(reverse_vector_inst2.vector_rev == vector_in);
```

50.11.2 Coverage Points

- All bit positions exercised
- Boundary conditions (all 0s, all 1s)
- Random patterns
- Double reversal verification

50.12 Related Modules and Alternatives

50.12.1 Byte Reversal

For byte-level endianness conversion, a separate module might be more appropriate:

```
// Reverse byte order instead of bit order  
assign byte_reversed = {data[7:0], data[15:8], data[23:16],  
data[31:24]};
```

50.12.2 Configurable Reversal

An enhanced version could include enable control:

```
assign vector_out = enable ? vector_rev : vector_in;
```

50.12.3 Multi-Width Reversal

Support multiple data widths in a single module:

```
generate  
  case (WIDTH)  
    8: assign vector_rev = {vector_in[0], vector_in[1], ...};  
    16: assign vector_rev = {vector_in[0], vector_in[1], ...};  
    32: assign vector_rev = {vector_in[0], vector_in[1], ...};  
  endcase  
endgenerate
```

50.13 Performance Metrics

50.13.1 Speed

- **Combinational Delay:** < 1ns (typical)
- **Maximum Frequency:** Limited by surrounding logic
- **Throughput:** One operation per clock cycle (if clocked)

50.13.2 Area

- **Gate Count:** 0 (wire-only implementation)
- **Equivalent Gates:** WIDTH (for routing complexity)
- **Memory Bits:** 0

50.13.3 Power

- **Leakage:** Negligible
- **Dynamic:** Proportional to toggle rate
- **Peak Power:** During simultaneous bit transitions

50.14 Common Pitfalls and Solutions

50.14.1 Simulation Warnings

```
// Use 'integer' instead of 'int' for compatibility  
for (integer i = 0; i < WIDTH; i++) begin
```

50.14.2 Synthesis Issues

```
// Ensure WIDTH is properly parameterized  
localparam int W = WIDTH; // Local parameter for clarity
```

50.14.3 Timing Closure

```
// Add pipeline stage if needed for timing  
always_ff @(posedge clk) begin  
    vector_rev_reg <= vector_rev;  
end
```

50.15 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

51 Barrel Shifter Module

51.1 Purpose

The `shifter_barrel` module implements a combinational barrel shifter that can perform multiple types of shift operations in a single clock cycle. It supports logical left/right shifts with

and without wrap-around, arithmetic right shifts, and no-shift operations, all controlled by a 3-bit control signal.

51.2 Key Features

- Single-cycle shift operations for any shift amount
- Multiple shift modes: logical, arithmetic, and rotational
- Configurable shift amount up to data width
- Wrap-around (rotation) capabilities
- Purely combinational logic for maximum speed

51.3 Port Description

51.3.1 Parameters

- **WIDTH:** Width of the data bus (default: 8)

51.3.2 Inputs

Port	Width	Description
data	WIDTH	Input data to be shifted
ctrl	3	Control signal for shift operation mode
shift_amount	$\$clog2(WIDTH)+1$	Number of positions to shift

51.3.3 Outputs

Port	Width	Description
data_out	WIDTH	Shifted output data

51.4 Control Signal Encoding

The 3-bit ctrl signal determines the shift operation:

ctrl	Operation	Description
3'b000	No shift	Output equals input
3'b001	Logical Right Shift	Fill with zeros from

ctrl	Operation	Description
	(no wrap)	left
3'b010	Arithmetic Right Shift	Preserve sign bit (MSB)
3'b011	Logical Right Shift (wrap)	Rotate right
3'b100	Logical Left Shift (no wrap)	Fill with zeros from right
3'b110	Logical Left Shift (wrap)	Rotate left
Others	No shift	Default case

51.5 Implementation Details

51.5.1 Shift Amount Modulation

```
logic [$clog2(WIDTH)-1:0] shift_amount_mod;
assign shift_amount_mod = shift_amount[$clog2(WIDTH)-1:0];
```

The shift amount is automatically reduced modulo WIDTH to prevent out-of-bounds operations.

51.5.2 Wrap-Around Implementation

```
logic [WIDTH*2-1:0] w_data_double;
assign w_data_double = {data, data};

// Generate lookup values for rotating shifts
genvar i;
generate
    for (i = 0; i < WIDTH; i++) begin : gen_unrolled_shifts
        assign w_array_rs[i] = w_data_double[WIDTH-1+i:i]; //
    Right shift
        assign w_array_ls[i] = w_data_double[WIDTH*2-1-i:WIDTH-i]; //
    Left shift
    end
endgenerate
```

51.5.3 Pre-computed Rotation Arrays

The module uses two lookup arrays for efficient rotation: - **w_array_rs**[WIDTH](#): Pre-computed right rotation results for all possible shift amounts - **w_array_ls**[WIDTH](#): Pre-computed left rotation results for all possible shift amounts

This approach eliminates the need for complex barrel shifting logic and provides constant-time access to any rotation amount.

51.5.4 Main Shift Logic

```
always_comb begin
    case (ctrl)
        3'b000: // No shift
            data_out = data;

        3'b001: // Logical Right Shift (no wrap)
            data_out = (shift_amount_mod == 0) ? data : data >>
shift_amount;

        3'b010: // Arithmetic Right Shift (preserve sign)
            data_out = $signed(data) >>> shift_amount_mod;

        3'b011: // Logical Right Shift with wrap
            data_out = w_array_rs[shift_amount_mod];

        3'b100: // Logical Left Shift (no wrap)
            data_out = (shift_amount_mod == 0) ? data : data <<
shift_amount;

        3'b110: // Logical Left Shift with wrap
            data_out = w_array_ls[shift_amount_mod];

        default:
            data_out = data;
    endcase
end
```

51.6 Special Implementation Notes

51.6.1 1. Combinational Design

- All operations complete in zero clock cycles
- No state machines or sequential logic
- Suitable for high-frequency operations

51.6.2.2. Arithmetic Right Shift

Uses SystemVerilog's >>> operator with \$signed() casting to properly handle sign extension for two's complement numbers.

51.6.33. Zero Shift Optimization

Explicit checks for `shift_amount_mod == 0` optimize the common case of no shifting, avoiding unnecessary computation.

51.6.44. Double-Width Concatenation

The `{data, data}` concatenation creates a seamless circular buffer for rotation operations, enabling simple slice extraction for any rotation amount.

51.6.55. Generate Block Optimization

Pre-computing all possible rotation results at compile time eliminates runtime computation, trading minimal area for maximum speed.

51.7 Timing Examples

51.7.1 8-bit Examples (WIDTH=8)

51.7.1.1 *Logical Right Shift (no wrap)*

Input: `data = 8'b11010110`, `shift_amount = 3`, `ctrl = 3'b001`
Output: `8'b00011010`

51.7.1.2 *Arithmetic Right Shift*

Input: `data = 8'b11010110`, `shift_amount = 3`, `ctrl = 3'b010`
Output: `8'b11111010` (sign extended)

51.7.1.3 *Right Rotation*

Input: `data = 8'b11010110`, `shift_amount = 3`, `ctrl = 3'b011`
Output: `8'b11011010` (bits wrap around)

51.7.1.4 *Logical Left Shift (no wrap)*

Input: `data = 8'b11010110`, `shift_amount = 3`, `ctrl = 3'b100`
Output: `8'b10110000`

51.7.1.5 *Left Rotation*

Input: `data = 8'b11010110`, `shift_amount = 3`, `ctrl = 3'b110`
Output: `8'b10110110` (bits wrap around)

51.8 Resource Utilization

51.8.1 Area Considerations

- **Lookup Arrays:** Require $\text{WIDTH} \times \text{WIDTH}$ bits of combinational logic
- **Multiplexers:** Large mux structures for final output selection

- **Total:** Approximately $O(\text{WIDTH}^2)$ area complexity

51.8.2 Speed Characteristics

- **Propagation Delay:** Constant regardless of shift amount
- **Critical Path:** Through lookup array and final output mux
- **Frequency:** Limited by combinational delay, not shift complexity

51.9 Applications

- ALU implementations
- Cryptographic operations
- Signal processing
- Bit manipulation engines
- Network packet processing
- Data alignment circuits

51.10 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

52 Beat-Packing Shifter

Module: shifter_beat_pack.sv **Location:** rtl/common/ **Status:** Production Ready

52.1 Overview

shifter_beat_pack is a bit-granular packing/aligning shifter. It accepts fixed-width `CHUNK_BITS` entries on a push handshake, accumulates them in a multi-chunk staging register, and drains **configurable-width “beats”** out the low end on a pop handshake. The beat width is a runtime input (bytes minus one), so the same instance can emit different beat sizes across bursts.

All of the shift/load/mux logic lives inside this one module so that callers, such as an aligner that needs to repack DFI cycles into DRAM beats, do not have to invent their own shift-and-compensate scheme. Push and pop can occur in the same cycle; the next-state logic applies the pop first and the push second, and a single non-blocking assignment commits the result so no internal last-assignment race is possible.

52.1.1 Key Features

- **Bit-granular staging:** A `DEPTH_CHUNKS × CHUNK_BITS` register packs chunks and drains beats at bit resolution
- **Runtime beat width:** `cfg_beat_bytes_m1` selects the beat size (bytes – 1) per burst without re-elaboration
- **Independent ingress / egress sizing:** Chunk width and beat cap are separate parameters
- **Same-cycle push + pop:** Pop-then-push ordering with one NBA eliminates internal races
- **Elaboration-time contract checks:** `$error` guards enforce `DEPTH_CHUNKS >= 2` and `MAX_BEAT_BITS < STORAGE_BITS`
- **Status outputs:** `empty` and `count_bits_o` expose occupancy to the wrapper

52.2 Module Purpose

The module solves the recurring problem of repacking a stream of one fixed width into beats of another (often runtime-selected) width. Data arrives one whole chunk at a time and leaves as beats whose width is chosen at run time; the module holds the partial-beat residue between cycles and shifts it down as beats drain, so the caller sees clean valid/ready handshakes on both sides.

Use Cases:

- Packing DFI cycles into DRAM beats in a memory-controller aligner
- Width adaptation where ingress chunk width and egress beat width differ and the beat width is runtime-configurable
- Any packer that must hold partial-beat residue while new fixed-width data lands

Key Benefit: Centralizes the shift, load, and residue-compensation logic behind two simple handshakes, so callers get runtime-variable beat repacking without hand-rolling their own barrel shifter.

52.3 Parameters

Parameter	Type	Default	Description
<code>CHUNK_BITS</code>	<code>int</code>	128	Width of one push entry (ingress granularity)

Parameter	Type	Default	Description
MAX_BEAT_BYTES	int	16	Maximum beat width in bytes; caps what reaches pop_data
CFG_BITS	int	8	Width of the runtime cfg_beat_bytes_m1 field. 8 bits gives beat_bytes $\in$ 1..256; 4 bits gives 1..16, etc.
DEPTH_CHUNKS	int	2	Staging-register depth in chunks. Default 2 = “current + prefetched-next”. Must be $\geq$ 2. Larger depths absorb more before draining but widen the pop-side barrel shifter (timing cost).
MAX_BEAT_BITS	int	MAX_BEAT_BYTES * 8	Derived (do not override): maximum beat width in bits
STORAGE_BITS	int	DEPTH_CHUNKS * CHUNK_BITS	Derived (do not override): total staging-register width
COUNT_BITS	int	$\$clog2(STORAGE_BITS + 1)$	Derived (do not override): width of the bit-occupancy counter
IDX_BITS	int	$\$clog2(STORAGE_BITS)$	Derived (do not override): index width for r_data part-selects

52.4 Port Groups

52.4.1 Clock and Reset

Port	Direction	Width	Description
clk	Input	1	Clock

Port	Direction	Width	Description
rst_n	Input	1	Active-low reset (via reset_defs.svh macros)

52.4.2 Configuration

Port	Direction	Width	Description
cfg_beat_bytes_m1	Input	CFG_BITS	Beat width in bytes – 1 (0 → 1 byte). Held stable per burst.

52.4.3 Push Interface (ingress)

Port	Direction	Width	Description
push_valid	Input	1	New chunk valid
push_ready	Output	1	Room for another whole chunk (post-push count <= STORAGE_BITS)
push_data	Input	CHUNK_BITS	The chunk to pack

52.4.4 Pop Interface (egress)

Port	Direction	Width	Description
pop_valid	Output	1	At least one whole beat is available
pop_ready	Input	1	Consumer accepts a beat
pop_data	Output	MAX_BEAT_BITS	Low MAX_BEAT_BITS of the staging register; consumer reads only the low (cfg_beat_bytes_m1+1)*8 bits

52.4.5 Status

Port	Direction	Width	Description
empty	Output	1	Staging register holds no bits (r_count == 0)

Port	Direction	Width	Description
count_bits_0	Output	COUNT_BITS	Current occupancy in bits

52.5 Functional Description

52.5.1 Runtime Beat Width

The runtime beat width in bits is derived from the byte-minus-one config field, widened by 4 bits so the counter math can represent the largest beat any CFG_BITS-wide value can encode:

```
localparam int BEAT_BITS_W = CFG_BITS + 4;
assign w_beat_bits = ({..., cfg_beat_bytes_m1} + 1) << 3; //
(bytes_m1 + 1) * 8
```

MAX_BEAT_BYTES caps what actually appears on pop_data; w_beat_bits is used only for the occupancy accounting.

52.5.2 Handshake and Status

The combinational handshakes are driven directly from the bit-occupancy counter r_count:

- empty = (r_count == 0)
- push_ready = there is room for another whole chunk, i.e. r_count <= STORAGE_BITS - CHUNK_BITS
- pop_valid = the register holds at least one whole beat, i.e. r_count >= w_beat_bits and non-zero
- pop_data = the low MAX_BEAT_BITS of r_data (a plain slice, well-defined because a beat is always strictly narrower than the staging window)

When the configured beat is narrower than MAX_BEAT_BITS, the upper bits of pop_data are stale bytes belonging to the next beat; the consumer already knows the width from cfg and reads only the low valid bits.

52.5.3 Next-State: Pop First, Push Second

Each cycle the combinational block computes next-state v_data / v_count in two ordered steps:

1. **Pop** (when pop_valid && pop_ready): shift the register down by w_beat_bits and subtract that many bits from the count, draining the low beat.
2. **Push** (when push_valid && push_ready): land the new chunk at [v_count +: CHUNK_BITS] using the *post-pop* occupancy, so the chunk sits right

above the still-valid bits whether or not a pop fired, then add `CHUNK_BITS` to the count.

```
if (pop_valid && pop_ready) begin
    v_data = v_data >> w_beat_bits;
    v_count = v_count - COUNT_BITS'(w_beat_bits);
end
if (push_valid && push_ready) begin
    v_data[v_count[IDX_BITS-1:0] +: CHUNK_BITS] = push_data;
    v_count = v_count + COUNT_BITS'(CHUNK_BITS);
end
```

A single non-blocking assignment then writes `r_data / r_count` from `v_data / v_count`, so same-cycle push + pop has no internal last-assignment race.

52.5.4 Elaboration-Time Contract

Two \$error guards enforce the sizing rules that keep forward progress guaranteed:

- `DEPTH_CHUNKS >= 2`: a single-chunk register cannot hold partial-beat residue while a new chunk lands.
- `MAX_BEAT_BITS < STORAGE_BITS`: a beat must be strictly narrower than the staging window.

52.5.5 Sizing Guidance

Pick `CHUNK_BITS` and `MAX_BEAT_BYTES` so any runtime beat width fits in the 2N (or deeper) storage, i.e. `cfg_beat_bytes × 8 ≤ DEPTH_CHUNKS × CHUNK_BITS`. Ingress chunk width and egress beat cap are independent, so callers can right-size both.

52.6 Usage Example

```
// Repack 128-bit DFI cycles into runtime-sized DRAM beats (up to 16 bytes).
```

```
shifter_beat_pack #(
    .CHUNK_BITS      (128),
    .MAX_BEAT_BYTES  (16),
    .CFG_BITS        (8),
    .DEPTH_CHUNKS    (2)
) u_beat_pack (
    .clk              (clk),
    .rst_n            (rst_n),
    .cfg_beat_bytes_m1 (beat_bytes - 8'd1),    // e.g. 8 bytes -> 7
```

```
// ingress: one 128-bit chunk per handshake
```

```
.push_valid    (dfi_valid),
.push_ready    (dfi_ready),
```

```

        .push_data          (dfi_cycle),

        // egress: one beat per handshake
        .pop_valid          (beat_valid),
        .pop_ready          (beat_ready),
        .pop_data           (beat_data),          // read low
        (beat_bytes*8) bits

        // status
        .empty              (pack_empty),
        .count_bits_o       (pack_bits)
    );

```

52.7 Design Notes

- **Bit occupancy, not entry count.** `r_count` tracks bits, not entries, because chunks land whole but beats drain at a runtime-variable bit width.
- **Pop-then-push ordering is deliberate.** Draining first frees the low bits so the incoming chunk lands at the correct post-pop position, which is what makes same-cycle push + pop safe.
- **pop_data upper bits may be stale.** Only the low $(\text{cfg_beat_bytes_m1}+1)*8$ bits are meaningful; the consumer must mask to the configured width.
- **Depth vs. timing.** Increasing `DEPTH_CHUNKS` lets the packer absorb more data before draining but widens the pop-side barrel shifter, degrading timing.
- **Reset macros.** The module uses the project `reset_defs.svh` `ALWAYS_FF_RST` / `RST_ASSERTED` macros for the registered state.

52.8 Related Modules

52.8.1 Used By

- Memory-controller aligners that repack DFI cycles into DRAM beats

52.8.2 Uses

- None (self-contained; relies only on the `reset_defs.svh` reset macros)

52.8.3 See Also

- [fifo_sync](#) - Whole-entry synchronous FIFO (contrast: this module repacks at bit granularity)

52.9 References

52.9.1 Source Code

- `rtl/common/shifter_beat_pack.sv`

52.9.2 Documentation

- `rtl/common/PRD.md`

Last Updated: 2026-07-15

52.10 Navigation

- [Back to RTLCommon Index](#)

53 Fibonacci LFSR Module

53.1 Purpose

The `shifter_lfsr_fibonacci` module implements a Fibonacci Linear Feedback Shift Register, which is a specific type of LFSR where the feedback is applied only to the most significant bit position. This design is complementary to Galois LFSRs and offers different implementation trade-offs.

53.2 Key Features

- Fibonacci (external XOR) LFSR architecture
- Configurable tap positions and polynomial
- Right-shift operation with feedback to MSB
- Seed loading and cycle detection
- Non-zero state enforcement
- Parameterizable width and tap configuration

53.3 Port Description

53.3.1 Parameters

- **WIDTH:** Width of the LFSR register (default: 8)
- **TAP_INDEX_WIDTH:** Width of each tap index (default: 12)
- **TAP_COUNT:** Number of feedback taps (default: 4)
- **TIW:** Shorthand for TAP_INDEX_WIDTH

53.3.2 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
enable	1	Enable LFSR operation
seed_load	1	Load seed value into LFSR
seed_data	WIDTH	Seed value for LFSR initialization
taps	TAP_COUNT*TIW	Concatenated tap positions

53.3.3 Outputs

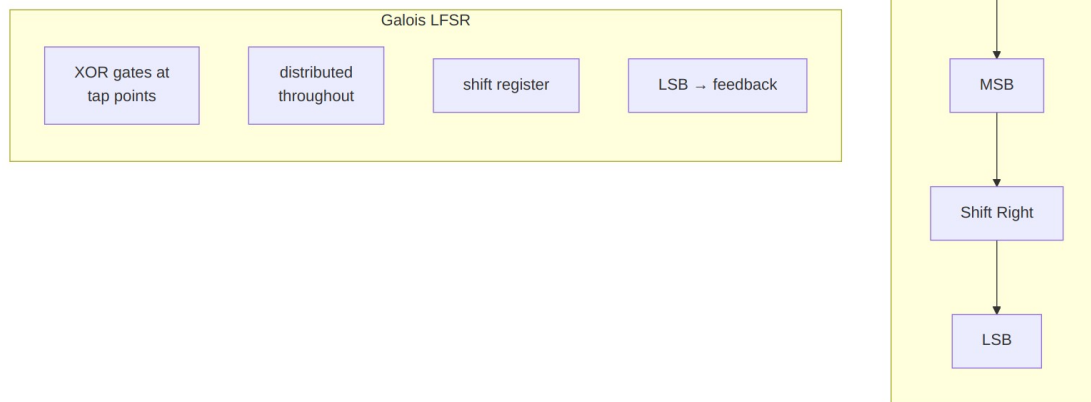
Port	Width	Description
lfsr_out	WIDTH	Current LFSR value
lfsr_done	1	High when LFSR returns to seed value

53.4 Fibonacci vs Galois LFSR Architecture

53.4.1 Fibonacci LFSR Characteristics

- **Feedback Location:** Single XOR gate feeding MSB
- **Data Path:** Simple right-shift with external feedback

- **XOR Gate Count:** One multi-input XOR gate
- **Critical Path:** Through feedback XOR to MSB



Implementation Comparison

53.5 Implementation Details

53.5.1 Tap Processing (Same as Standard LFSR)

```
// Split concatenated tap positions
always_comb begin
    for (int i = 0; i < TAP_COUNT; i++)
        w_tap_positions[i] = taps[i*TIW+:TIW];
end

// Convert to bit mask
always_comb begin
    w_taps = 'b0;
    for (int i = 0; i < TAP_COUNT; i++)
        if (w_tap_positions[i] > 0)
            w_taps[w_tap_positions[i]-1'b1] = 1'b1;
end
```

53.5.2 Fibonacci Feedback Calculation

```
assign w_feedback = ^(r_lfsr & w_taps);
```

Key Difference: Uses XOR (^) instead of XNOR (~^) compared to the standard LFSR module.

53.5.3 Right-Shift with MSB Feedback

```
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        r_lfsr <= 'b0; // initialization to all 0's
    end else begin
        if (enable) begin
            if (seed_load) begin
                r_lfsr <= seed_data; // Load seed
            end else if (!r_lfsr) begin // Only shift if we have non-
zero value
                // Fibonacci LFSR: Shift right, feedback to MSB
                r_lfsr <= {w_feedback, r_lfsr[WIDTH-1:1]};
            end
        end
    end
end
```

53.6 Special Implementation Notes

53.6.1 1. XOR vs XNOR Feedback

- **Fibonacci LFSR:** Uses XOR (^) for feedback calculation
- **Standard LFSR:** Uses XNOR (~^) for feedback calculation
- Both produce maximal-length sequences with appropriate polynomials

53.6.22. Non-Zero State Protection

```
end else if (!r_lfsr) begin // Only shift if we have non-zero value
```

Prevents shifting when LFSR contains all zeros, which would lock the sequence in the zero state permanently.

53.6.33. Right-Shift Architecture

```
r_lfsr <= {w_feedback, r_lfsr[WIDTH-1:1]};
```

- Feedback enters at MSB position
- Data shifts right (towards LSB)
- LSB data is discarded each cycle

53.6.44. Polynomial Compatibility

The tap positions should correspond to primitive polynomials appropriate for Fibonacci LFSR implementation. The polynomial form is:

$$P(x) = x^n + x^{(\text{tap1})} + x^{(\text{tap2})} + \dots + x^{(\text{tapk})} + 1$$

53.6.55. Reset to All Zeros

Unlike some LFSR implementations that reset to all ones, this module resets to all zeros and relies on seed loading for proper initialization.

53.7 Timing Example (4-bit Fibonacci LFSR)

53.7.1 Configuration

- WIDTH = 4
- Polynomial: $x^4 + x^3 + 1$ (taps at positions 4,3)
- Seed: 4'b0001

53.7.2 Sequence Generation

Cycle	LFSR	Tap Bits	Feedback	Next LFSR
-----	-----	-----	-----	-----
0	0001	00	0	0000 (shift right)
1	0000	00	0	0000 (locked - need seed)

After seed reload with 4'b1001:

0	1001	10	$1^0=1$	1100
1	1100	11	$1^1=0$	0110
2	0110	01	$0^1=1$	1011
3	1011	10	$1^0=1$	1101
4	1101	11	$1^1=0$	0110
...				

53.8 Comparison with Galois LFSR

53.8.1 Advantages of Fibonacci LFSR

- **Simpler Data Path:** Only one shift register with external feedback
- **Easy to Understand:** Classical LFSR textbook implementation
- **Single XOR Gate:** All feedback logic concentrated in one location

53.8.2 Disadvantages of Fibonacci LFSR

- **Fan-out:** Feedback XOR gate may have high fan-out for many taps
- **Critical Path:** All tapped bits must route to single XOR gate
- **Timing:** May be slower than distributed Galois implementation

53.8.3 When to Use Fibonacci LFSR

- **Educational Purposes:** Easier to understand and debug
- **Few Taps:** When polynomial has only 2-3 taps
- **Area Constraints:** When minimizing XOR gate count is critical
- **Legacy Compatibility:** When interfacing with existing Fibonacci LFSR systems

53.9 Applications

- Pseudo-random sequence generation (identical to Galois LFSR output)
- CRC calculation (with appropriate polynomial)
- Data scrambling and encryption
- Test pattern generation
- Spread spectrum communications
- Error detection and correction

53.10 Polynomial Examples for Fibonacci LFSR

53.10.1 Common Primitive Polynomials

Width	Polynomial	Tap Positions	Period
3	x^3+x^2+1	[3,2]	7
4	x^4+x^3+1	[4,3]	15
5	x^5+x^3+1	[5,3]	31
8	$x^8+x^6+x^5+x^4+1$	[8,6,5,4]	255
16	$x^{16}+x^{15}+x^{13}+x^4+1$	[16,15,13,4]	65535

53.10.2 Usage Note

The same polynomial can be used for both Fibonacci and Galois LFSRs, but they will produce different (but equivalent) pseudo-random sequences.

53.11 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

54 Galois LFSR Module

54.1 Purpose

The `shifter_lfsr_galois` module implements a Galois Linear Feedback Shift Register, which features distributed XOR gates at tap positions rather than a single feedback point. This architecture offers better timing characteristics and higher operating frequencies compared to Fibonacci LFSRs.

54.2 Key Features

- Galois (internal XOR) LFSR architecture
- Distributed feedback XOR gates at tap positions
- Right-shift operation with LSB feedback
- Superior timing characteristics for high-speed operation
- Configurable tap positions and polynomial
- Seed loading and cycle detection

54.3 Port Description

54.3.1 Parameters

- **WIDTH:** Width of the LFSR register (default: 8)
- **TAP_INDEX_WIDTH:** Width of each tap index (default: 12)
- **TAP_COUNT:** Number of feedback taps (default: 4)
- **TIW:** Shorthand for TAP_INDEX_WIDTH

54.3.2 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
enable	1	Enable LFSR operation
seed_load	1	Load seed value into LFSR
seed_data	WIDTH	Seed value for LFSR initialization
taps	TAP_COUNT*TIW	Concatenated tap positions

54.3.3 Outputs

Port	Width	Description
lfsr_out	WIDTH	Current LFSR value
lfsr_done	1	High when LFSR returns to seed value

54.4 Galois LFSR Architecture

54.4.1 Key Architectural Differences

Fibonacci LFSR:

MSB $\leftarrow$ [XOR of all taps] $\leftarrow$ LSB

↓
Shift Right

↓
LSB

Galois LFSR:

MSB $\rightarrow$ [XOR] $\rightarrow$ [XOR] $\rightarrow$ [XOR] $\rightarrow$ LSB

↑ ↑ ↑ ↓
tap1 tap2 tap3 feedback

54.4.2 Advantages of Galois Architecture

- **Distributed Logic:** XOR gates spread throughout the shift register
- **Better Timing:** Shorter critical paths between flip-flops
- **Higher Frequency:** Can operate at higher clock speeds

- **Localized Routing:** Reduced interconnect complexity

54.5 Implementation Details

54.5.1 Tap Position Processing

```
// Split concatenated tap positions
always_comb begin
    for (int i = 0; i < TAP_COUNT; i++) begin
        w_tap_positions[i] = taps[i*TIW+:TIW];
    end
end
```

54.5.2 LSB Feedback Generation

```
assign w_feedback = r_lfsr[0];
```

The feedback signal comes directly from the LSB (bit 0) of the current LFSR state.

54.5.3 Galois Next State Calculation

```
always_comb begin
    // Start with right shift (include 0 in MSB)
    next_lfsr = {1'b0, r_lfsr[WIDTH-1:1]};

    // Apply Galois feedback taps if LSB is 1
    if (w_feedback) begin
        for (int j = 0; j < TAP_COUNT; j++) begin
            if (w_tap_positions[j] > 0 && w_tap_positions[j] <= WIDTH)
                // Apply XOR to the tap positions
                next_lfsr[w_tap_positions[j]-1] =
next_lfsr[w_tap_positions[j]-1] ^ 1'b1;
            end
        end
    end
end
```

54.5.4 Sequential Update

```
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        r_lfsr <= {WIDTH{1'b1}}; // initialization to all 1's
    end else if (enable) begin
        if (seed_load) begin
            r_lfsr <= seed_data;
        end else begin
            r_lfsr <= next_lfsr;
        end
    end
end
```

```
end
end
```

54.6 Special Implementation Notes

54.6.11. Reset to All Ones

```
r_lfsr <= {WIDTH{1'b1}}; // initialization to all 1's
```

Unlike the Fibonacci version, this module resets to all ones, which is a valid non-zero state for immediate operation.

54.6.22. Combinational Next State Logic

The next LFSR state is calculated combinatorially in the `always_comb` block, then registered in the `always_ff` block. This separates timing concerns from logic complexity.

54.6.33. Conditional XOR Application

```
if (w_feedback) begin
    // Only apply tap XORs when LSB is 1
end
```

XOR operations at tap positions only occur when the LSB feedback bit is 1, implementing the proper Galois LFSR behavior.

54.6.44. Bounds Checking

```
if (w_tap_positions[j] > 0 && w_tap_positions[j] <= WIDTH) begin
```

Explicit bounds checking prevents out-of-range array access and handles unused tap positions (value 0).

54.6.55. Right-Shift with Distributed Feedback

The basic operation is a right shift, but with XOR gates inserted at tap positions that conditionally invert bits based on the LSB feedback.

54.7 Timing Example (4-bit Galois LFSR)

54.7.1 Configuration

- WIDTH = 4
- Polynomial: $x^4 + x^3 + 1$ (taps at positions 4,3)
- Seed: 4'b1001

54.7.2 Sequence Generation

Cycle	LFSR	LSB	Shift	XOR@pos4	XOR@pos3	Next LFSR
-----	-----	-----	-----	-----	-----	-----

0		1001		1		0100		0100^1		0000^1		1110
1		1110		0		0111		no XOR		no XOR		0111
2		0111		1		0011		0011^1		0001^1		1010
3		1010		0		0101		no XOR		no XOR		0101
4		0101		1		0010		0010^1		0000^1		1011
...												

54.8 Performance Characteristics

54.8.1 Critical Path Analysis

Fibonacci LFSR Critical Path:

All tapped bits → Multi-input XOR → MSB flip-flop

Galois LFSR Critical Path:

Previous bit → XOR gate → Current bit flip-flop

54.8.2 Timing Advantages

- **Shorter Logic Depth:** Each bit only sees local XOR gate
- **Parallel Processing:** All XOR operations occur simultaneously
- **Reduced Fan-out:** No single XOR gate with multiple inputs
- **Better Scaling:** Critical path doesn't grow with tap count

54.9 Polynomial Considerations

54.9.1 Tap Position Mapping

For a polynomial $P(x) = x^n + x^a + x^b + \dots + 1$: - Galois LFSR taps: [n, a, b, ...] - Same polynomial works for both Fibonacci and Galois - Different sequences produced but same mathematical properties

54.9.2 Example Polynomials

Width	Polynomial	Galois Taps	Max Period
4	$x^4 + x^3 + 1$	[4,3]	15
8	$x^8 + x^6 + x^5 + x^4 + 1$	[8,6,5,4]	255
16	$x^{16} + x^{15} + x^{13} + x^4 + 1$	[16,15,13,4]	65535
32	$x^{32} + x^{22} + x^2 + x + 1$	[32,22,2,1]	$2^{32} - 1$

54.10 Design Trade-offs

54.10.1 Galois LFSR Advantages

- **Speed:** Higher maximum clock frequency
- **Timing:** Better setup/hold characteristics
- **Scalability:** Performance doesn't degrade with tap count
- **Power:** Potentially lower dynamic power

54.10.2 Galois LFSR Disadvantages

- **Complexity:** More complex combinational logic
- **Area:** May require more logic gates for many taps
- **Debug:** Harder to trace intermediate states
- **Understanding:** Less intuitive than classical Fibonacci

54.11 Applications

54.11.1 High-Speed Systems

*// Use Galois LFSR for high-frequency applications
// Clock rates > 500MHz where timing is critical*

54.11.2 Communications

*// Spread spectrum systems requiring fast sequence generation
// Real-time scrambling/descrambling applications*

54.11.3 Cryptography

*// Stream cipher implementations
// Key sequence generation*

54.11.4 Testing

*// High-speed BIST pattern generation
// Fast pseudo-random test vectors*

54.12 When to Choose Galois over Fibonacci

54.12.1 Use Galois LFSR When:

- High clock frequency is required (>100MHz)

- Timing closure is challenging
- Multiple taps are needed (>3 taps)
- Power consumption is critical
- Synthesis tools favor distributed logic

54.12.2 Use Fibonacci LFSR When:

- Educational/demonstration purposes
- Legacy system compatibility required
- Very few taps (2 taps or less)
- Area is more critical than speed
- Debugging capabilities are important

54.13 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

55 LFSR (Linear Feedback Shift Register) Module

55.1 Purpose

The `shifter_lfsr` module implements a generic Linear Feedback Shift Register (LFSR) with configurable tap positions for pseudo-random number generation. It supports various LFSR polynomials and provides cycle completion detection for deterministic sequence generation.

55.2 Key Features

- Configurable tap positions for different polynomial implementations
- Seed loading capability for sequence initialization
- Cycle completion detection (sequence wrap-around)
- Enable control for conditional operation
- Support for various LFSR widths and tap configurations

55.3 Port Description

55.3.1 Parameters

- **WIDTH:** Width of the LFSR register (default: 8)
- **TAP_INDEX_WIDTH:** Width of each tap index (default: 12)
- **TAP_COUNT:** Number of feedback taps (default: 4)
- **TIW:** Shorthand for TAP_INDEX_WIDTH

55.3.2 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
enable	1	Enable LFSR operation
seed_load	1	Load seed value into LFSR
seed_data	WIDTH	Seed value for LFSR initialization
taps	TAP_COUNT*TIW	Concatenated tap positions

55.3.3 Outputs

Port	Width	Description
lfsr_out	WIDTH	Current LFSR value
lfsr_done	1	High when LFSR returns to seed value

55.4 LFSR Theory and Implementation

55.4.1 Feedback Polynomial

The LFSR implements a linear feedback shift register based on a primitive polynomial. The feedback is calculated by XORing bits at specified tap positions.

55.4.2 Tap Position Processing

```
// Split concatenated tap positions into separate groups
always_comb begin
    for (int i = 0; i < TAP_COUNT; i++)
        w_tap_positions[i] = taps[i*TIW+:TIW];
end

// Convert tap positions to bit mask
always_comb begin
    w_taps = 'b0;
    for (int i = 0; i < TAP_COUNT; i++)
        if (w_tap_positions[i] > 0)
            w_taps[w_tap_positions[i]-1'b1] = 1'b1;
end
```

55.4.3 Feedback Calculation

```
assign w_feedback = ~(r_lfsr[WIDTH-1:0] & w_taps[WIDTH-1:0]);
```

The feedback uses XNOR reduction (inverted XOR) of tapped bits, which is characteristic of maximal-length LFSR sequences.

55.4.4 Shift Register Operation

```
always_ff @(posedge clk or negedge rst_n) begin
    if (~rst_n) begin
        r_lfsr <= 'b0;
    end else begin
        if (enable) begin
            if (seed_load)
                r_lfsr <= seed_data;
            else
                r_lfsr <= {r_lfsr[WIDTH-2:0], w_feedback};
        end
    end
end
```

55.5 LFSR Polynomial Reference

The module supports various standard LFSR polynomials. Here are some common configurations:

55.5.1 Popular LFSR Polynomials (XNOR taps)

Width	Tap Positions	Max Period
3	[3,2]	7
4	[4,3]	15
5	[5,3]	31

Width	Tap Positions	Max Period
8	[8,6,5,4]	255
16	[16,15,13,4]	65535
32	[32,22,2,1]	$2^{32}-1$

55.5.2 Example Tap Configuration for 8-bit LFSR

```
// For WIDTH=8, polynomial  $x^8 + x^6 + x^5 + x^4 + 1$ 
// Taps at positions 8,6,5,4 (1-indexed)
parameter TAP_COUNT = 4;
parameter TIW = 4; // 4 bits enough for tap positions up to 8

// Concatenated tap positions: {8,6,5,4}
wire [TAP_COUNT*TIW-1:0] taps = {4'd8, 4'd6, 4'd5, 4'd4};
```

55.6 Special Implementation Notes

55.6.1 1. Tap Position Encoding

- Tap positions are 1-indexed (position 1 = bit 0)
- Zero tap position is ignored (allows unused taps)
- Multiple tap positions concatenated into single bus

55.6.2.2. XNOR Feedback

- Uses XNOR ($\sim^{\wedge}$) instead of XOR for feedback
- Produces maximal-length sequences for primitive polynomials
- Different from some LFSR implementations that use XOR

55.6.3.3. Cycle Detection

```
assign lfsr_done = (lfsr_out == seed_data) ? 1'b1 : 1'b0;
```

Detects when LFSR returns to its initial seed value, indicating completion of full pseudo-random sequence.

55.6.4.4. Left-Shift Architecture

The implementation uses left-shift with feedback to LSB, which is equivalent to traditional right-shift LFSRs but with different bit ordering.

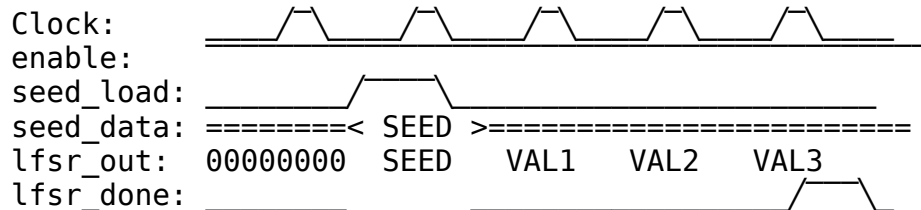
55.6.5.5. Zero State Handling

- Reset initializes LFSR to all zeros

- Seed loading required for proper operation
- All-zero state is typically avoided in LFSR operation

55.7 Timing Diagrams

55.7.1 Seed Loading and Operation



55.7.2 Sequence Example (3-bit LFSR, taps=[3,2])

Step	LFSR	Feedback	Next
0	001	$\sim(1^0)=1$	011
1	011	$\sim(1^1)=0$	110
2	110	$\sim(0^1)=1$	101
3	101	$\sim(1^0)=1$	011
4	011	$\sim(1^1)=0$	110
...	...	...	...

55.8 Applications

55.8.1 Pseudo-Random Number Generation

```
// Initialize with non-zero seed
seed_data = 8'hA5;
seed_load = 1'b1; // Load seed
// Then enable normal operation
enable = 1'b1;
seed_load = 1'b0;
// lfsr_out provides pseudo-random sequence
```

55.8.2 Built-In Self-Test (BIST)

```
// Use lfsr_done to detect test completion
if (lfsr_done) begin
    // Full test sequence completed
    test_complete = 1'b1;
end
```

55.8.3 Scrambling/Descrambling

```
// XOR data with LFSR output for scrambling
scrambled_data = input_data ^ lfsr_out;
```

55.9 Common Use Cases

- Pseudo-random number generation
- Data scrambling and encryption
- Built-in self-test pattern generation
- CRC calculation
- Spread spectrum communications
- Test vector generation
- Noise generation for audio applications

55.10 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

56 Universal Shifter Module

56.1 Purpose

The `shifter_universal` module implements a versatile shift register that can perform multiple operations including left shift, right shift, parallel load, and hold operations. It supports both serial and parallel data input/output, making it suitable for a wide range of digital system applications.

56.2 Key Features

- Four distinct operations: hold, right shift, left shift, parallel load
- Bidirectional serial data input/output
- Parallel data load capability
- Serial data capture during shift operations
- Synchronous operation with asynchronous reset
- Configurable width

56.3 Port Description

56.3.1 Parameters

- **WIDTH:** Width of the shift register (default: 4)

56.3.2 Inputs

Port	Width	Description
clk	1	System clock
rst_n	1	Active-low asynchronous reset
select	2	Operation select control
i_pdata	WIDTH	Parallel data input
i_sdata_lt	1	Serial data input (left side)
i_sdata_rt	1	Serial data input (right side)

56.3.3 Outputs

Port	Width	Description
o_pdata	WIDTH	Parallel data output
o_sdata_lt	1	Serial data output (left side)
o_sdata_rt	1	Serial data output (right side)

56.4 Operation Control

The 2-bit select signal controls the shifter operation:

select	Operation	Description
2'b00	Hold	Maintain current state
2'b01	Right Shift	Shift data right, input from left
2'b10	Left Shift	Shift data left, input

select	Operation	Description
		from right
2'b11	Parallel Load	Load parallel input data

56.5 Implementation Details

56.5.1 Combinational Logic for Next State

```

always_comb begin
    // Default values - hold current state
    w_pdata = o_pdata;
    w_sdata_lt = 1'b0;
    w_sdata_rt = 1'b0;

    casez (select)
        2'b00: begin // Hold
            w_pdata = o_pdata;
            w_sdata_lt = 1'b0;
            w_sdata_rt = 1'b0;
        end

        2'b01: begin // Right Shift
            w_pdata = {i_sdata_rt, o_pdata[WIDTH-1:1]};
            w_sdata_lt = 1'b0;
            w_sdata_rt = o_pdata[0];
        end

        2'b10: begin // Left Shift
            w_pdata = {o_pdata[WIDTH-2:0], i_sdata_lt};
            w_sdata_lt = o_pdata[WIDTH-1];
            w_sdata_rt = 1'b0;
        end

        2'b11: begin // Parallel Load
            w_pdata = i_pdata;
            w_sdata_lt = 1'b0;
            w_sdata_rt = 1'b0;
        end
    endcase
end

```

56.5.2 Sequential Logic for State Update

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        o_pdata <= '0;
    end
end

```

```

        o_sdata_lt <= 1'b0;
        o_sdata_rt <= 1'b0;
    end else begin
        o_pdata      <= w_pdata;
        o_sdata_lt <= w_sdata_lt;
        o_sdata_rt <= w_sdata_rt;
    end
end

```

56.6 Operation Details

56.6.1 Hold Operation (select = 2'b00)

- **Function:** Maintains current register state
- **Data Flow:** No data movement
- **Serial Outputs:** Both remain low (no shift occurring)
- **Use Case:** Pause operation while maintaining data integrity

56.6.2 Right Shift Operation (select = 2'b01)

- **Function:** Shifts data toward LSB
- **Data Flow:** i_sdata_rt → MSB, LSB → o_sdata_rt
- **Serial Input:** New data enters from i_sdata_rt
- **Serial Output:** Shifted-out data exits via o_sdata_rt
- **Use Case:** Serial-to-parallel conversion, delay lines

56.6.3 Left Shift Operation (select = 2'b10)

- **Function:** Shifts data toward MSB
- **Data Flow:** i_sdata_lt → LSB, MSB → o_sdata_lt
- **Serial Input:** New data enters from i_sdata_lt
- **Serial Output:** Shifted-out data exits via o_sdata_lt
- **Use Case:** Multiplication by 2, bit-serial processing

56.6.4 Parallel Load Operation (select = 2'b11)

- **Function:** Loads entire parallel word
- **Data Flow:** i_pdata → o_pdata
- **Serial Outputs:** Both remain low (no shift occurring)
- **Use Case:** Initialization, data injection

56.7 Special Implementation Notes

56.7.1 1. Bidirectional Serial Interface

The module provides separate serial inputs and outputs for both directions: - **Left side:** i_sdata_lt (input), o_sdata_lt (output) - **Right side:** i_sdata_rt (input), o_sdata_rt (output)

This allows chaining multiple shifters or connecting to bidirectional serial systems.

56.7.22. Clean Serial Output Management

Serial outputs are explicitly set to zero when not actively shifting in that direction, preventing glitches and providing well-defined states.

56.7.33. Asynchronous Reset

Reset immediately clears all outputs to zero, providing deterministic startup conditions.

56.7.44. Combinational/Sequential Separation

The design cleanly separates: - **Combinational logic:** Calculates next state - **Sequential logic:** Registers the state on clock edge

This approach improves timing predictability and synthesis results.

56.8 Timing Diagrams

56.8.1 Right Shift Operation Example (WIDTH=4)

Clock:										
select:	01	01	01	01	01					
i_sdata_rt:	1	0	1	1	0					
o_pdata:	0000	1000	0100	1010	1101					
o_sdata_rt:	0	0	0	0	1					

56.8.2 Parallel Load followed by Left Shift

Clock:										
select:	11	10	10	10						
i_pdata:	1010	----	----	----						
i_sdata_lt:	----	1	0	1						

```
o_pdata:    1010 0101 1010 0101
o_sdata_lt:---- 1    0    1
```

56.9 Applications

56.9.1 Serial Communication Interface

```
// Receive shift register for UART
// Right shift to collect incoming serial data
select = 2'b01;
i_sdata_rt = uart_rx;
// When 8 bits received, parallel data available
received_byte = o_pdata;
```

56.9.2 Multiplication/Division by Powers of 2

```
// Multiply by 2: left shift
select = 2'b10;
i_sdata_lt = 1'b0; // Fill with zeros

// Divide by 2: right shift
select = 2'b01;
i_sdata_rt = 1'b0; // Fill with zeros (unsigned)
// or i_sdata_rt = o_pdata[WIDTH-1] for arithmetic shift
```

56.9.3 Data Delay Line

```
// Create N-cycle delay using N shifters
// Right shift with data input
select = 2'b01;
i_sdata_rt = data_input;
delayed_data = o_sdata_rt; // N cycles later
```

56.9.4 Pattern Generator

```
// Load pattern then circulate
// 1. Load initial pattern
select = 2'b11;
i_pdata = initial_pattern;

// 2. Circulate pattern
select = 2'b01; // Right shift
i_sdata_rt = o_sdata_rt; // Feedback for circulation
```

56.9.5 Scan Chain for Testing

```
// JTAG-style scan chain
// Load test data
select = 2'b11;
i_pdata = test_vector;
```

```
// Shift out for observation
select = 2'b01;
i_sdata_rt = next_scan_in;
scan_out = o_sdata_rt;
```

56.10 Design Considerations

56.10.1 Chaining Multiple Shifters

```
// Connect shifters in series
shifter1.i_sdata_rt = serial_input;
shifter2.i_sdata_rt = shifter1.o_sdata_rt;
shifter3.i_sdata_rt = shifter2.o_sdata_rt;
```

56.10.2 Bidirectional Operation

```
// Support both shift directions
if (shift_direction == RIGHT) begin
    select = 2'b01;
    i_sdata_rt = serial_data;
end else begin
    select = 2'b10;
    i_sdata_lt = serial_data;
end
```

56.10.3 Control Logic Integration

```
// State machine control
case (state)
    LOAD: select = 2'b11;
    SHIFT_RIGHT: select = 2'b01;
    SHIFT_LEFT: select = 2'b10;
    HOLD: select = 2'b00;
endcase
```

56.11 Common Use Cases

- Serial-to-parallel conversion
- Parallel-to-serial conversion
- Data delay and timing adjustment
- Arithmetic operations ($\times 2$, $\div 2$)
- Pattern generation and circulation

- Scan chain testing
- FIFO buffer implementation
- Digital signal processing pipelines

56.12 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

57 Sort Module RTL Documentation

57.1 Purpose

The Sort module is a pipelined hardware sorting engine that implements the odd-even sort algorithm to sort arrays of configurable size in descending order. The module is designed for high-throughput applications where new data can be accepted every clock cycle, with predictable latency equal to the number of elements being sorted.

57.2 Module Overview

```
module sort #(
    parameter int NUM_VALS = 5,    // Number of values to sort
    parameter int SIZE     = 16    // Size of each value in bits
) (
    input  logic          clk,      // Clock signal
    input  logic          rst_n,    // Active-low reset
    input  logic [NUM_VALS*SIZE-1:0] data, // Packed input data
    input  logic          valid_in, // Start sorting when
asserted
    output logic [NUM_VALS*SIZE-1:0] sorted, // Packed sorted
output
    output logic          done      // Asserted when
sorting complete
);
```

57.3 Port Descriptions

57.3.1 Input Ports

Port	Width	Type	Description
clk	1	Clock	System clock signal. All operations are synchronous to

Port	Width	Type	Description
rst_n	1	Reset	the rising edge. Active-low asynchronous reset. Clears all pipeline stages when deasserted.
data	NUM_VALS*SIZE	Data	Packed input array. Elements are packed as data[i*SIZE + : SIZE] for element i.
valid_in	1	Control	When asserted (high) for one cycle, starts the sorting process for the current data input.

57.3.2 Output Ports

Port	Width	Type	Description
sorted	NUM_VALS*SIZE	Data	Packed output array sorted in descending order. Same packing format as input.
done	1	Status	Asserted (high) when the sorting operation is complete and sorted output is valid.

57.3.3 Parameters

Parameter	Default	Range	Description
NUM_VALS	5	2-32	Number of values in the array to be sorted. Determines pipeline depth.
SIZE	16	1-64	Bit width of each individual value. All values must be unsigned integers.

57.4 Data Format

Input/Output Packing: Arrays are packed into a single wide bus where: - Element 0 occupies bits [SIZE-1:0] - Element 1 occupies bits [2*SIZE-1:SIZE] - Element i occupies bits [(i+1)*SIZE-1:i*SIZE]

Sort Order: Output is sorted in **descending order** (largest to smallest).

57.5 Algorithm: Odd-Even Sort

57.5.1 Overview

The module implements the **Odd-Even Sort** (also known as **Brick Sort**) algorithm, which is particularly well-suited for pipelined hardware implementation.

57.5.2 Algorithm Description

Odd-Even Sort works by alternating between two types of comparison passes:

1. **Odd Passes:** Compare and swap adjacent pairs at positions (0,1), (2,3), (4,5), etc.
2. **Even Passes:** Compare and swap adjacent pairs at positions (1,2), (3,4), (5,6), etc.

The algorithm requires exactly NUM_VALS passes to guarantee complete sorting, regardless of input data.

57.5.3 Example Execution

For input array [5, 3, 8, 1, 9] (5 elements):

Initial: [5, 3, 8, 1, 9]

Pass 1 (Odd): Compare (0,1), (2,3), (4,X)
[5, 3] → [5, 3] (no swap)
[8, 1] → [8, 1] (no swap)
[9, X] → [9, X] (no comparison)
Result: [5, 3, 8, 1, 9]

Pass 2 (Even): Compare (1,2), (3,4)
[3, 8] → [8, 3] (swap)
[1, 9] → [9, 1] (swap)
Result: [5, 8, 3, 9, 1]

Pass 3 (Odd): Compare (0,1), (2,3), (4,X)
[5, 8] → [8, 5] (swap)
[3, 9] → [9, 3] (swap)
[1, X] → [1, X] (no comparison)
Result: [8, 5, 9, 3, 1]

Pass 4 (Even): Compare (1,2), (3,4)
[5, 9] → [9, 5] (swap)
[3, 1] → [3, 1] (no swap)

Result: [8, 9, 5, 3, 1]

Pass 5 (Odd): Compare (0,1), (2,3), (4,X)
[8, 9] → [9, 8] (swap)
[5, 3] → [5, 3] (no swap)
[1, X] → [1, X] (no comparison)
Final: [9, 8, 5, 3, 1] ✓ Sorted!

57.5.4 Algorithm Comparison

Algorithm	Time Complexity	Space Complexity	Hardware Efficiency	Pipeline Stages	Best Use Case
Odd-Even Sort	$O(n^2)$	$O(1)$	★★★★★ Excellent	n	Small arrays, predictable timing
Bubble Sort	$O(n^2)$	$O(1)$	★★★ Poor	Variable	Sequential processors only
Bitonic Sort	$O(n \log^2 n)$	$O(1)$	★★★★★ Very Good	$\log^2 n$	Power-of-2 sizes, larger arrays
Merge Sort	$O(n \log n)$	$O(n)$	★★★★ Good	$\log n$	Large arrays, general purpose
Insertion Sort	$O(n^2)$	$O(1)$	★★★★ Good	Variable	Nearly sorted data

Why Odd-Even Sort for Hardware?

1. **Regular Structure:** Each pipeline stage is identical, making synthesis predictable
2. **Parallel Operations:** All comparisons in each pass happen simultaneously
3. **Fixed Latency:** Always takes exactly NUM_VALS cycles regardless of input data
4. **Simple Control:** No complex indexing or variable loop bounds

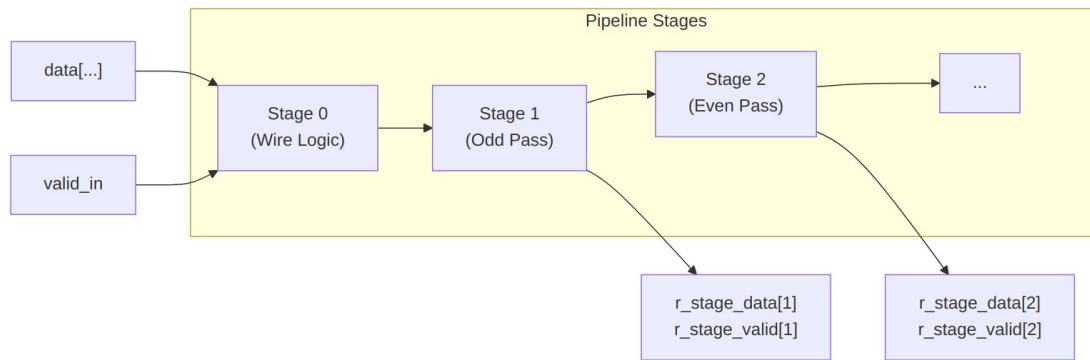
5. **Scalable:** Pipeline depth scales linearly with array size

57.6 Architecture

57.6.1 Pipeline Structure

The module implements a NUM_VALS-stage pipeline where: - **Stage 0:** Input capture (combinational) - **Stages 1 to NUM_VALS:** Pipeline registers with compare-swap logic

Stage: 0 -----> 1 -----> 2 -----> 3 -----> ... -----> NUM_VALS
 Input Odd Pass Even Pass Odd Pass Final Output
 (Comb) (FF) (FF) (FF) (FF)



Signal Flow

57.6.2 Naming Conventions

The module follows strict naming conventions for signal clarity:

- **Wire signals** (combinational): Prefixed with w_
 - w_values[stage][element]: Unpacked arrays for easy manipulation
 - w_next_values[stage][element]: Computed next values
 - w_stage_data_0: Input stage data wire
 - w_stage_valid_0: Input stage valid wire
- **Flopped signals** (sequential): Prefixed with r_
 - r_stage_data[stage]: Pipeline data registers
 - r_stage_valid[stage]: Pipeline valid registers

57.7 Implementation Details

57.7.1 Compare-Swap Logic

Each pipeline stage implements compare-swap operations based on the stage number:

```
// Determine if this is an odd or even pass
localparam bit IS_ODD_PASS = ((stage-1) % 2) == 0;

always_comb begin
    // Default: pass through unchanged
    for (int i = 0; i < NUM_VALS; i++) begin
        w_next_values[stage][i] = w_values[stage-1][i];
    end

    if (IS_ODD_PASS) begin
        // Odd pass: compare (0,1), (2,3), (4,5), etc.
        for (int i = 0; i < NUM_VALS-1; i += 2) begin
            if (w_values[stage-1][i] < w_values[stage-1][i+1]) begin
                w_next_values[stage][i] = w_values[stage-1][i+1];
                w_next_values[stage][i+1] = w_values[stage-1][i];
            end
        end
    end else begin
        // Even pass: compare (1,2), (3,4), (5,6), etc.
        for (int i = 1; i < NUM_VALS-1; i += 2) begin
            if (w_values[stage-1][i] < w_values[stage-1][i+1]) begin
                w_next_values[stage][i] = w_values[stage-1][i+1];
                w_next_values[stage][i+1] = w_values[stage-1][i];
            end
        end
    end
end
```

57.7.2 Reset Behavior

- **Asynchronous Reset:** All pipeline stages are cleared when `rst_n` is deasserted
- **Data Reset:** All `r_stage_data` registers are cleared to zero
- **Valid Reset:** All `r_stage_valid` registers are cleared to zero
- **Output:** Both sorted and done outputs go to zero during reset

57.7.3 Timing Characteristics

- **Latency:** `NUM_VALS` clock cycles from `valid_in` assertion to done assertion

- **Throughput:** 1 array per clock cycle (once pipeline is full)
- **Critical Path:** One compare-swap operation (typically 1-2 gate delays)
- **Pipeline Depth:** NUM_VALS stages

57.8 State Machines

The Sort module does **not** use explicit Finite State Machines (FSMs). Instead, it uses a **pipeline-based control scheme**:

57.8.1 Control Flow

1. Input Stage (Combinational):

- Accepts new data when `valid_in` is asserted
- No state storage at input stage

2. Pipeline Progression (Sequential):

- Valid signal propagates through pipeline automatically
- Each stage processes data independently
- No centralized control or state machine needed

3. Output Indication:

- done signal is simply the valid signal from the final pipeline stage
- No complex output state management required

57.8.2 Valid Signal Flow

Cycle:	1	2	3	4	5	6	
<code>valid_in</code> :	1	0	0	0	0	0	
<code>r_stage_valid[1]</code> :	-		1	0	0	0	0
<code>r_stage_valid[2]</code> :	-		-	1	0	0	0
<code>r_stage_valid[3]</code> :	-		-	-	1	0	0
<code>r_stage_valid[4]</code> :	-		-	-	-	1	0
<code>r_stage_valid[5]</code> :	-		-	-	-	-	1
<code>done</code> :	0	0	0	0	0	1	

57.9 Special Features

57.9.1 1. Parameterizable Design

- **Flexible Array Size:** NUM_VALS can be any reasonable value (tested 3-32)
- **Configurable Data Width:** SIZE supports 1-64 bits per element
- **Automatic Pipeline Scaling:** Pipeline depth automatically adjusts to NUM_VALS

57.9.22. High Throughput Pipeline

- **Back-to-Back Operations:** New data can be accepted every cycle
- **Overlapped Processing:** Multiple arrays can be in the pipeline simultaneously
- **Predictable Performance:** Fixed latency regardless of data patterns

57.9.33. Synthesis-Friendly Design

- **Regular Structure:** All pipeline stages are nearly identical
- **Local Connectivity:** No long combinational paths across stages
- **Inference-Friendly:** Uses standard SystemVerilog constructs for reliable synthesis

57.9.44. Robust Data Handling

- **Packed Buses:** Efficient use of bus resources
- **Unpacked Internal Arrays:** Easy element-wise manipulation
- **Automatic Packing/Unpacking:** Transparent data format conversion

57.10 Verification Strategy

The module comes with a comprehensive CocoTB testbench that includes:

57.10.1 Test Levels

1. **Basic Tests:** Core functionality verification
2. **Random Tests:** Extensive random data patterns
3. **Boundary Tests:** Edge cases and limit conditions

4. **Pipeline Tests:** Throughput and overlapped operation verification
5. **Reset Tests:** Proper reset behavior validation

57.10.2 Test Coverage

- **Functional:** All possible input combinations for small arrays
- **Parametric:** Multiple NUM_VALS and SIZE combinations
- **Temporal:** Pipeline timing and valid signal propagation
- **Stress:** Back-to-back operations and maximum throughput

57.11 Usage Guidelines

57.11.1 Typical Use Cases

1. **Packet Processing:** Sorting priority fields or timestamps
2. **Signal Processing:** Median filtering, statistical operations
3. **Graphics:** Z-buffer sorting, polygon vertex ordering
4. **Control Systems:** Sorting sensor readings or control parameters

57.11.2 Integration Considerations

1. **Clock Domain:** Ensure clk is stable and meets timing requirements
2. **Reset Strategy:** Include proper reset sequencing in system design
3. **Data Formatting:** Pack input arrays correctly according to element ordering
4. **Flow Control:** Monitor done signal for output validity
5. **Resource Planning:** Consider pipeline depth impact on latency-sensitive applications

57.11.3 Performance Optimization

1. **Pipeline Depth:** Use smallest practical NUM_VALS for lowest latency
2. **Data Width:** Use smallest practical SIZE for best area efficiency
3. **Clock Frequency:** Critical path is typically one comparison operation

4. **Resource Sharing:** Multiple instances can share input/output buses if properly arbitrated

57.12 Synthesis Results

Typical synthesis results for common configurations:

Configuration	Pipeline Stages	Critical Path	Area (LUTs)	Frequency
3×8-bit	3	~1.2ns	~150	>800 MHz
5×16-bit	5	~1.5ns	~400	>650 MHz
8×32-bit	8	~2.0ns	~1200	>500 MHz

Results vary by target FPGA family and synthesis tool settings

57.13 Future Enhancements

Potential improvements for future versions:

1. **Bidirectional Sorting:** Parameter to select ascending vs descending order
2. **Early Termination:** Detect when array is already sorted and terminate early
3. **Variable Width Elements:** Support for mixed-width data elements
4. **Priority Encoding:** Include secondary sort keys for equal elements
5. **Power Optimization:** Clock gating for unused pipeline stages

This documentation describes the Sort module implementation. For the latest updates and additional examples, refer to the source code and testbench files.

57.14 Navigation

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)

58 Pulse Synchronizer for Clock Domain Crossing

A safe pulse synchronizer that transfers single-cycle pulses between asynchronous clock domains using a toggle-based handshake with metastability filtering.

58.1 Overview

The `sync_pulse` module provides metastability-safe pulse synchronization across clock domains. It converts a single-cycle pulse in the source clock domain into a single-cycle pulse in the destination clock domain, handling arbitrary clock frequency ratios. The module uses a toggle-register approach with multi-stage synchronization to guarantee no pulse loss when minimum spacing requirements are met.

58.2 Module Declaration

```
module sync_pulse #(
    parameter int SYNC_STAGES = 3    // Synchronizer depth (range: 2-4)
) (
    // Source clock domain
    input logic i_src_clk,           // Source clock
    input logic i_src_rst_n,        // Source reset (active-low)
    input logic i_pulse,            // Input pulse (single-cycle high)

    // Destination clock domain
    input logic i_dst_clk,           // Destination clock
    input logic i_dst_rst_n,        // Destination reset (active-low)
    output logic o_pulse            // Output pulse (single-cycle high)
);
```

58.3 Parameters

58.3.1 User-Settable Parameters

Parameter	Type	Default	Description
SYNC_STAGES	int	3	Number of synchronizer stages for metastability filtering (range: 2-4)

SYNC_STAGES Guidelines: - **2 stages:** Minimum CDC-safe, basic reliability - **3 stages:** Recommended for most applications (>10¹² hours MTBF) - **4 stages:** Ultra-high reliability (aerospace, medical)

58.4 Ports

58.4.1 Source Clock Domain Ports

Port	Direction	Width	Description
i_src_clk	Input	1	Source clock (where input pulse originates)

Port	Direction	Width	Description
i_src_rst_n	Input	1	Source domain active-low asynchronous reset
i_pulse	Input	1	Input pulse (must be single-cycle high)

58.4.2 Destination Clock Domain Ports

Port	Direction	Width	Description
i_dst_clk	Input	1	Destination clock (where output pulse appears)
i_dst_rst_n	Input	1	Destination domain active-low asynchronous reset
o_pulse	Output	1	Synchronized output pulse (single-cycle high)

58.5 Functionality

58.5.1 Toggle-Based Pulse Synchronization

The module uses a four-stage process to safely transfer pulses:

1. **Source Toggle:** Input pulse toggles r_src_toggle register
2. **Synchronization:** Toggle synchronized across clock domains via multi-stage synchronizer
3. **Edge Detection:** XOR detects toggle transitions to generate destination pulse
4. **Single-Cycle Output:** Output pulse is exactly one destination clock cycle wide

58.5.2 Timing Characteristics

- **Latency:** (SYNC_STAGES + 2) destination clock cycles
- **Minimum Pulse Spacing:** $3 \times T_{dst} + 2 \times T_{src}$ clock periods
- **MTBF:** $>10^{12}$ hours @ SYNC_STAGES=3, 100MHz

Where: - T_{dst} = Destination clock period - T_{src} = Source clock period

58.6 Implementation Details

58.6.1 Source Domain: Toggle Register

```
(* ASYNC_REG = "TRUE" *) logic r_src_toggle;

always_ff @(posedge i_src_clk or negedge i_src_rst_n) begin
    if (!i_src_rst_n) begin
        r_src_toggle <= 1'b0;
    end else if (i_pulse) begin
        r_src_toggle <= ~r_src_toggle; // Toggle on each pulse
    end
end
```

Key Points: - Toggle persists across clock domains (quasi-static) - Each pulse inverts toggle state (0→1 or 1→0) - Toggle visible to destination synchronizer for multiple cycles

58.6.2 Destination Domain: Multi-Stage Synchronizer

```
(* ASYNC_REG = "TRUE" *) logic [SYNC_STAGES-1:0] r_sync;

always_ff @(posedge i_dst_clk or negedge i_dst_rst_n) begin
    if (!i_dst_rst_n) begin
        r_sync <= '0;
    end else begin
        r_sync <= {r_sync[SYNC_STAGES-2:0], r_src_toggle}; // Shift
register
    end
end
```

Metastability Filtering: - Stage 0: Captures source toggle (may go metastable) - Stages 1 to N-1: Filter metastability exponentially - Stage N-1: Clean synchronized toggle output

58.6.3 Destination Domain: Edge Detection

```
logic r_sync_prev;

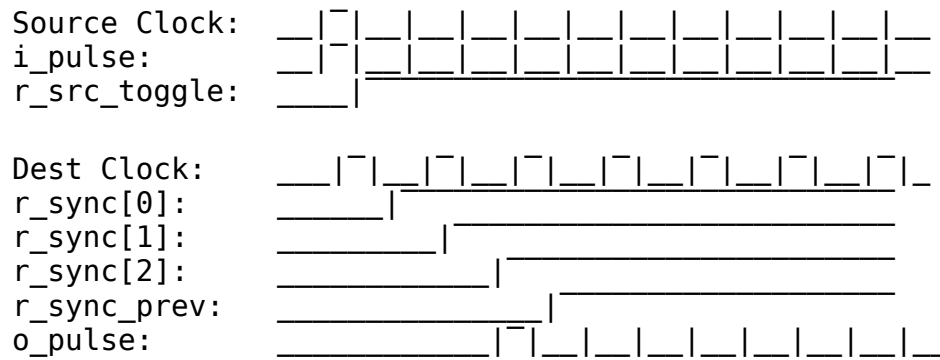
// Delay synchronized toggle by one cycle
always_ff @(posedge i_dst_clk or negedge i_dst_rst_n) begin
    if (!i_dst_rst_n) begin
        r_sync_prev <= 1'b0;
    end else begin
        r_sync_prev <= r_sync[SYNC_STAGES-1];
    end
end

// XOR detects toggle edge (both rising and falling)
assign o_pulse = r_sync[SYNC_STAGES-1] ^ r_sync_prev;
```

Edge Detection Logic: - Compares current toggle with previous toggle - XOR: 0→1 or 1→0 transition generates pulse - Output: Single-cycle high pulse in destination domain

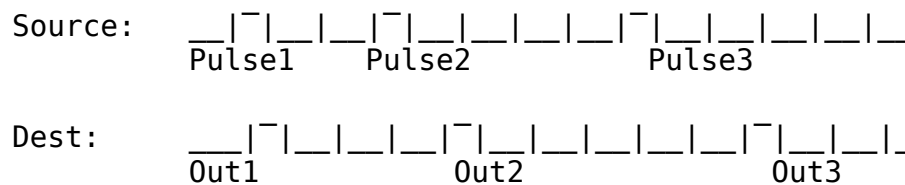
58.7 Protocol Timing

58.7.1 Single Pulse Transfer



Latency: 5 destination clocks (SYNC_STAGES=3 + 2)

58.7.2 Multiple Pulse Transfer



Note: Minimum 3 destination clock spacing maintained

58.8 Usage Examples

58.8.1 Basic Pulse Synchronization

```
// Synchronize button press from slow to fast clock
sync_pulse #(
    .SYNC_STAGES(3)
) u_button_sync (
    .i_src_clk    (button_clk),           // Slow clock (debounced button)
    .i_src_rst_n (sys_rst_n),
    .i_pulse      (button_press),         // Single-cycle pulse
    .i_dst_clk    (core_clk),             // Fast core clock
    .i_dst_rst_n  (sys_rst_n),
    .o_pulse      (button_press_sync)
);
```

58.8.2 Interrupt Synchronization

```
// Synchronize peripheral interrupt to CPU clock
sync_pulse #(
    .SYNC_STAGES(3)
) u_irq_sync (
    .i_src_clk    (periph_clk),
    .i_src_rst_n (periph_rst_n),
    .i_pulse      (periph_irq_pulse), // Interrupt pulse
    .i_dst_clk    (cpu_clk),
    .i_dst_rst_n  (cpu_rst_n),
    .o_pulse      (cpu_irq)
);

// Use synchronized interrupt in CPU
always_ff @(posedge cpu_clk) begin
    if (cpu_irq) begin
        // Handle interrupt
        irq_pending <= 1'b1;
    end
end
```

58.8.3 Event Counter Synchronization

```
// Count events across clock domains
logic [15:0] event_count;

sync_pulse #(
    .SYNC_STAGES(3)
) u_event_sync (
    .i_src_clk    (sensor_clk),
    .i_src_rst_n  (sys_rst_n),
    .i_pulse      (sensor_event),
    .i_dst_clk    (sys_clk),
    .i_dst_rst_n  (sys_rst_n),
    .o_pulse      (event_sync)
);

// Count synchronized events
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        event_count <= 16'h0;
    end else if (event_sync) begin
        event_count <= event_count + 16'h1;
    end
end
```

58.8.4 With Edge Detector for Multi-Cycle Signals

```
// Convert multi-cycle signal to single-cycle pulse first
logic multi_cycle_sig, single_cycle_pulse;

edge_detect u_edge_det (
    .clk      (src_clk),
    .rst_n    (src_rst_n),
    .sig_in   (multi_cycle_sig),
    .pulse_out(single_cycle_pulse) // Rising edge → pulse
);

// Then synchronize the pulse
sync_pulse #(
    .SYNC_STAGES(3)
) u_sync (
    .i_src_clk   (src_clk),
    .i_src_rst_n (src_rst_n),
    .i_pulse     (single_cycle_pulse),
    .i_dst_clk   (dst_clk),
    .i_dst_rst_n (dst_rst_n),
    .o_pulse     (pulse_sync)
);
```

58.9 Design Considerations

58.9.1 When to Use sync_pulse

✓ **Appropriate Use Cases:** - Single-cycle pulse events (interrupts, triggers, strobes) - Event counters across clock domains - Handshake acknowledgments - Sporadic control signals (button presses, sensor events) - Timeout/watchdog pulses

✗ **Inappropriate Use Cases:** - High-frequency continuous pulses (use async FIFO) - Data buses (use handshake or FIFO) - Multi-bit values (use glitch_free_n_dff_arn with Gray code) - Clocks themselves (use dedicated clock crossing primitives)

58.9.2 Minimum Pulse Spacing

To avoid pulse loss, ensure minimum spacing between source pulses:

Fast Source → Slow Destination:

$$\text{Min spacing} = 3 \times T_{\text{dst}} + 2 \times T_{\text{src}}$$

Example: 100MHz src → 25MHz dst

$T_{\text{src}} = 10\text{ns}$, $T_{\text{dst}} = 40\text{ns}$

Min spacing = $3 \times 40\text{ns} + 2 \times 10\text{ns} = 140\text{ns} = 14$ source clocks

Slow Source → Fast Destination:

Min spacing = $3 \times T_{dst} + 2 \times T_{src}$

Example: 10MHz src → 100MHz dst

$T_{src} = 100\text{ns}$, $T_{dst} = 10\text{ns}$

Min spacing = $3 \times 10\text{ns} + 2 \times 100\text{ns} = 230\text{ns} = 2.3$ source clocks

(Round up: 3 source clocks minimum)

58.9.3 Reset Synchronization

Both resets should be properly synchronized to their respective clock domains:

// Proper reset synchronization

```
reset_sync u_src_rst_sync (  
    .clk          (i_src_clk),  
    .async_rst_n(global_rst_n),  
    .sync_rst_n  (i_src_rst_n)  
);
```

```
reset_sync u_dst_rst_sync (  
    .clk          (i_dst_clk),  
    .async_rst_n(global_rst_n),  
    .sync_rst_n  (i_dst_rst_n)  
);
```

58.10 Synthesis and Constraints

58.10.1 Synthesis Attributes

The module includes ASYNC_REG attributes to prevent optimization:

```
(* ASYNC_REG = "TRUE" *) logic r_src_toggle;  
(* ASYNC_REG = "TRUE" *) logic [SYNC_STAGES-1:0] r_sync;
```

These attributes: - Prevent register duplication - Ensure synchronizer stages stay together - Help tools recognize CDC paths

58.10.2 Timing Constraints (Xilinx XDC)

Mark clock domain crossing paths as false paths

```
set_false_path -from [get_clocks i_src_clk] -to [get_pins -  
hierarchical *r_sync_reg[0]/D]
```

Or set maximum delay for CDC analysis

```
set_max_delay -from [get_clocks i_src_clk] -to [get_pins -hierarchical  
*r_sync_reg[0]/D] -datapath_only [get_property PERIOD [get_clocks  
i_dst_clk]]
```

Set ASYNC_REG property (if not in RTL)

```
set_property ASYNC_REG TRUE [get_cells -hierarchical *r_sync_reg*]
```

58.10.3 Timing Constraints (Intel SDC)

```
# Set false path for CDC
set_false_path -from [get_clocks i_src_clk] -to [get_registers
*r_sync[0]]

# Mark synchronizer chain
set_instance_assignment -name SYNCHRONIZER_IDENTIFICATION "FORCED IF
ASYNCHRONOUS" -to *r_sync*
```

58.11 Verification and Assertions

58.11.1 Formal Assertions (Built-in)

The module includes formal assertions for verification:

```
// Assert: Input pulse is single-cycle
property p_single_cycle_pulse;
    @(posedge i_src_clk) disable iff (!i_src_rst_n)
        i_pulse |=> !i_pulse;
endproperty

// Assert: Output pulse is single-cycle
property p_output_single_cycle;
    @(posedge i_dst_clk) disable iff (!i_dst_rst_n)
        o_pulse |=> !o_pulse;
endproperty
```

58.11.2 Simulation Checks

```
// Check input pulse width
always @(posedge i_src_clk) begin
    if (i_pulse) begin
        @(posedge i_src_clk);
        assert (!i_pulse) else $error("Input pulse must be single-
cycle");
    end
end

// Check minimum pulse spacing
int last_pulse_time = 0;
always @(posedge i_dst_clk) begin
    if (o_pulse) begin
        int time_diff = $time - last_pulse_time;
        assert (time_diff >= 3*T_dst)
            else $warning("Pulse spacing too close: %0d ns",
time_diff);
        last_pulse_time = $time;
    end
end
```


end
end

58.12 Performance Analysis

58.12.1 Latency Breakdown

For SYNC_STAGES=3: 1. Source toggle register: 1 source clock 2. Synchronizer stages: 3 destination clocks 3. Edge detection: 1 destination clock 4. Output pulse generation: 1 destination clock

Total: 5 destination clocks + 1 source clock

58.12.2 Resource Utilization

Component	FFs	LUTs	Description
r_src_toggle	1	1	Source toggle register
r_sync	SYNC_STAGES	0	Synchronizer chain
r_sync_prev	1	0	Previous toggle storage
o_pulse	0	1	XOR edge detector
Total	SYNC_STAGES +2	2	Minimal footprint

58.13 Common Pitfalls

✗ **Anti-Pattern 1:** Multi-cycle input pulses

```
// WRONG: Input pulse held for multiple cycles
```

```
i_pulse <= 1'b1;  
#(2*CLK_PERIOD);  
i_pulse <= 1'b0;
```

```
// Result: May generate multiple output pulses
```

```
// RIGHT: Single-cycle pulse
```

```
i_pulse <= 1'b1;  
#CLK_PERIOD;  
i_pulse <= 1'b0;
```

✗ **Anti-Pattern 2:** Pulses too close together

```

// WRONG: Back-to-back pulses in source domain
forever begin
    i_pulse <= 1'b1;
    #CLK_PERIOD;
    i_pulse <= 1'b0;
    #CLK_PERIOD; // Only 1 clock gap!
end

// Result: Pulse loss

// RIGHT: Adequate spacing
forever begin
    i_pulse <= 1'b1;
    #CLK_PERIOD;
    i_pulse <= 1'b0;
    #(MIN_SPACING*CLK_PERIOD); // Meet minimum spacing
end

x Anti-Pattern 3: Using for data transfer

// WRONG: Trying to transfer data with pulse
logic [7:0] data_to_sync;
always @(posedge src_clk) begin
    if (send_data) begin
        data_to_sync <= new_data; // Will be corrupted!
        i_pulse <= 1'b1;
    end
end

// RIGHT: Use handshake or async FIFO for data

```

58.14 Related Modules

- **glitch_free_n_dff_arn.sv** - Multi-bit signal synchronization (quasi-static)
- **reset_sync.sv** - Specialized reset synchronization
- **edge_detect.sv** - Edge-to-pulse conversion (single clock domain)
- **async_fifo.sv** - For high-throughput data CDC
- **handshake_sync.sv** - Multi-bit data CDC with handshake

58.15 References

- Cummings, Clifford E. “Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog.” SNUG 2008.

- “Synthesis and Scripting Techniques for Designing Multi-Asynchronous Clock Designs.” Xilinx WP275.
- IEEE 1800-2017 SystemVerilog LRM - Clock Domain Crossing

58.16 **Navigation**

- [← Back to RTLCommon Index](#)
- [← Back to Main Documentation Index](#)